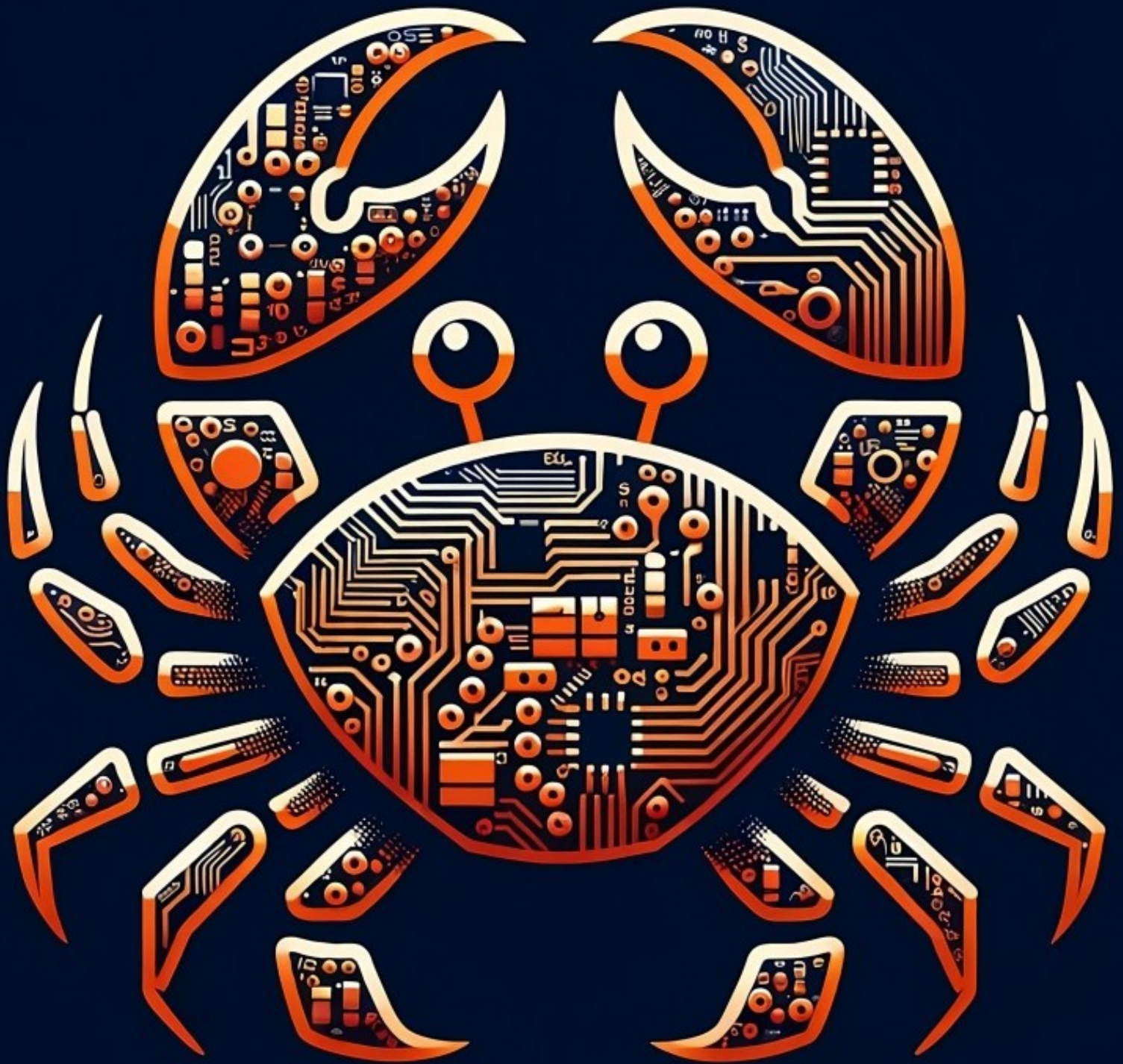


Rust Under the Hood

A deep dive into Rust internals and generated assembly



Sandeep Ahluwalia • Deepa Ahluwalia

Rust Under the Hood

A deep dive into Rust internals and generated assembly

Sandeep Ahluwalia
Deepa Ahluwalia



EventHelix.com Press

2024

© 2024 by Sandeep Ahluwalia and Deepa Ahluwalia

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. For permission requests, write to the publisher at the email address below.

support@eventhelix.com

<https://www.EventHelix.com/>

Names: Sandeep Ahluwalia and Deepa Ahluwalia

Title: Rust Under the Hood / by Sandeep Ahluwalia and Deepa Ahluwalia.

Description: EventHelix.com Press, 2024

Subjects: LCSH: Rust (Programming Language) | Systems programming (Computer science). | Computer programming.

First Edition

2024

Disclaimer from Sandeep: The views and content presented in this book are solely my own and are not related to or representative of my work in Amazon's Project Kuiper.

Dedication

To our parents,

Your teachings guide us every day, and you are forever in our thoughts. This book is dedicated to your enduring wisdom and love.

With enduring love,

Sandeep and Deepa

Acknowledgements

We want to thank:

- Saagar Ahluwalia, our son, for introducing us to Rust. His deep knowledge of Rust and its type system helped us write the book.
- Carl M. Kadie (<https://medium.com/@carlmkadie>) for his suggestions on restructuring the book to reach a wider audience and improve clarity.
- Jack O'Conner (<https://www.jacko.io/>) for his assistance in understanding Rust's closures and move semantics, which improved our explanations.
- John Estess (<https://www.linkedin.com/in/johnestess/>) for his thorough review and valuable insights on Rust and x86-64 assembly, greatly enhancing the book's quality.

Table of Contents

Table of Contents vii

Preface xi

Who Should Read This Book? xi

Structure of This Book xii

 Experiments and Exercises xiii

Part A Introduction to Assembly and Basic Constructs 1

Chapter 1 Assembly Basics: Integers, Floats, and Tuples 3

 Getting Started with Assembly 3

Incrementing an Integer 3

Incrementing a Float 5

Working with Tuples 6

 Experiment: Tuple Element Multiplications 10

 Key Takeaways 11

Chapter 2 Fundamentals of x86-64 Assembly 13

x86-64 Registers 13

x86-64 Calling Convention 15

x86-64 Addressing Modes 16

x86-64 Instruction Set 17

Part B Control Structures and Enums in Assembly 23

Chapter 3 Deconstructing Match and If-Else Expressions 25

Match Expression 25

Match Expression with Ranges 29

If-Else Expression 32

 Experiment: Reorder Match Ranges and If-Else vs Match 34

 Key Takeaways 37

Chapter 4 Deep Dive into Enums: Layout and Patterns 39

Pattern Matching an Enum 39

Using Null Pointers as Discriminators 43

Discriminator Size in a Small Enum 46

 Experiment: Impact of Enum Size and Type on Discriminator Size 48

 Key Takeaways 49

Part C Data Structures and Memory Management 51

Chapter 5 Arguments: Value, Reference, Box, Rc, and Arc 53

Method Receives Self by Value 54

Method Receives Self Reference &T 55

Self Refers to Object on Heap via Box 57

Method Receives Reference-Counted Smart Pointer Rc to Self 59

Method Receives Atomic Reference-Counted Shared Reference Arc to Self 63

 Experiment: Cow Parameter, Change Power, and Quaternions 67

 Key Takeaways 71

Chapter 6 Structs: Alignment and Patterns 73

Field Ordering and Alignment 73

Pattern Matching a Struct 76

 Experiment: Structs, Alignment, and Pattern-Matching 78

 Key Takeaways 80

Chapter 7 Arrays, Tuples, Option, and Box Internals 83

Code Example: Arrays, Tuples, Option, and Box 83

`make_quad_coordinates`: Memory Layout and Code Generation 84

`cross_lines_from_quad_coordinates`: Inlining and deep optimization 89

 Experiment: Changing the Type of Tuple Elements and Enabling AVX-512 93

 Key Takeaways 94

Part D Iteration and Optimization 95

Chapter 8 Traditional vs. Functional Array Iteration 97

Iterator and Functional Style 97

Generated Assembly: Loop Unrolling, Inlining, and Conditional Moves 97

 Experiment: Change Array Size, Loop Code Generation, and Power of x 100

 Key Takeaways 101

Chapter 9 Optimizing Array Iteration with SIMD 103

Functional Style Loop 103

AVX-512 SIMD Instructions 103

AVX-512: Vectorized Assembly Overview 104

Annotated Assembly: SIMD in Array Iteration 105

 Experiment: Impact of Array Size and Power of x on Generated Code 108

 Key Takeaways 109

Chapter 10 In-Depth: Array Slices and Pattern Matching 111

Array Slices 111

Array Slice Pattern Matching Example 111

Assembly Flowchart: Array Slice Processing with Pattern Matching 112

Array Slice Pattern-Matching Assembly Code 112

 Experiment: Array Match Patterns and If-Else Comparisons 114

 Key Takeaways 117

Chapter 11 Vector Iteration in Assembly 119

Vector Memory Layout 119

Sample Vector Handling Code 120

Assembly Insights: Optimizations for Varying Vector Lengths 120

Inlining, Loop Unrolling, and Memory Optimizations 128

 Experiment: Array Length Impact; AVX-512 Optimizations 130

 Key Takeaways 131

Chapter 12 Advanced Iterations: Mapping and Folding 133

Exploring Inner and Outer Loops in Function Application 133

Example: Applying Functions to an Array of Integers 134

Optimizing an Array of Functions into a Single Polynomial 135

Visualizing the Inner Loop Unrolling and Outer Loop SIMD Optimizations 136

Polynomial Folding and SIMD Instructions 138

 Experiment: Array Size, Type, and Inner Loop Optimizations 140

 Key Takeaways 143

Part E Strings, Dispatch, and Recursion 145

Chapter 13 Strings and Slices: Under the Surface 147

String and String Slice Representation in Memory 147

Combining Two Strings: `String` and `&str` in Action 148

Builder Pattern: Optimize Away the Builder Struct 154

Emoji Arithmetic: Overloading the `+` Operator 158

 Experiment: String Manipulation and Optimization 165

 Key Takeaways 167

Chapter 14 Static vs. Dynamic Dispatch 169

Example: Static vs. Dynamic Dispatch 169

- Static Dispatch: Assembly of Trait-Based Generics and Inlining 171
- Dynamic Dispatch with Trait Objects: Dissecting the Assembly and Vtables 173
-  Experiment: Trait Objects in a Box and Optimize Away Dynamic Dispatch 175
-  Key Takeaways 180

Chapter 15 Traits, Vtables, and Tail Calls 183

- Trait Object Fat Pointer and the Vtable 184
- Dynamic Dispatch and Tail Call Optimization 184
- Vtable Method Dispatch and Tail Calls 186
- Boxed Trait Object Memory Management 187
- Rectangle - A Concrete Type Implementing the Draw Trait 188
-  Experiment: Impact of Call Order on TCO; Dynamic Dispatch Inlined 191
-  Key Takeaways 192

Chapter 16 Optimizing Recursive Functions 193

- Tree Structure 193
- Tree Representation in Memory 193
- Flowchart of Partially Tail Call Optimized sum Function 195
- Annotated Assembly Code for Transforming Recursive Tail Calls into Loops 196
-  Experiment: TCO for factorial and fibonacci Functions 197
-  Key Takeaways 198

Part F Closures and Async/Await 199

Chapter 17 Closures: From Environment to Execution 201

- Rust Closures Traits - Fn, FnMut, and FnOnce 201
- Stack and Heap-Allocated Closures 203
- Return an impl Fn from a Function 203
- Returning a Boxed Closure Box<dyn Fn> from a Function 205
- Passing a Dynamic Closure dyn Fn to a Function 209
-  Experiment: Return impl Fn and Box<dyn Fn> with No Capture 213
-  Key Takeaways 214

Chapter 18 Async/Await: Internal State Machine Dynamics 215

- Async Functions in Rust 215
- Async Function goto: Await Reaches the Target Position 216
- Desugaring the Async Example into a State Machine 217
- Relationship between Closures, Futures, and Async Functions 219
- Visualizing the Generated Assembly Code for the Async State Machine 223
- Generated Assembly Code for the Async State Machine 224
-  Experiment: Equivalent Async and Non-Async Functions 230
-  Key Takeaways 232

Chapter 19 Nested Async/Await: Desugaring to Loops 233

- patrol Function Awaits goto Function in an Infinite Loop 233
- patrol and goto Functions: Nested Async/Await Constructs 233
- Desugaring the Patrol Function into a State Machine 234
- Understanding Async: Closures, State Machines, and Async Runtime 237
- Flowchart of the patrol Async Function State Machine 241
- Understand Nested Async/Await Generated Assembly 243
-  Experiment: Executing Async Functions on .await 251
-  Key Takeaways 252

Chapter 20 Async Executors: Task Scheduling Insights 253

- Async Executor Overview 253
- goto and patrol Functions 254
- Async Executors: Scheduling and Task Management 255
- Scheduling and Execution of goto and patrol Tasks in the Async Executor 257
-  Experiment: Spawning Two Async Tasks; Infinite Loops in Async Functions 262
-  Key Takeaways 265

Part G Transformations and Takeaways 267

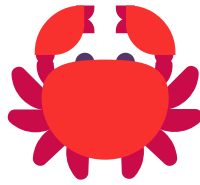
Chapter 21 Transforming into String Slice Vector 269

Example Code: Map a Vector of Bools to a Vector of String Slices 269
 Visualizing the Input and Output Vectors 269
 Assembly Code Flowchart: Bool Vector to Vector of String Slices 270
 Overview of the Assembly Code 271
 Annotated Assembly: Bool Vector to a Vector of String Slices 272
 Experiment: Try Out Different Vector Transformations 279
 Key Takeaways 280

Chapter 22 Transforming into Owned String Vector 281

Example Code: Map a Vector of bools to a Vector of Owned Strings 281
 String Representation in Rust 281
 Visualizing the Input and Output Vectors 282
 String Vector Generation Overview 283
 Annotated Assembly: Bool Vector to a Vector of Owned Strings 285
 Memory Cleanup Code 293
 Experiment: Returning a Vec<char> and String Manipulation 296
 Key Takeaways 298

Chapter 23 Key Insights 301



Preface

In “Rust Under the Hood”, we explore the Rust programming language’s inner workings by examining the assembly code its compiler produces. We will investigate the performance implications, memory representations, and optimizations performed by the Rust compiler and the LLVM backend.

Who Should Read This Book?

This book serves as a practical reference for those interested in Rust programming. You can jump directly to chapters that are relevant to your needs. When reading a chapter, you can select the level of detail that suits your current understanding.

If you are new to Rust, this book will provide a unique perspective on the language’s inner workings. Following the flowcharts and memory representation diagrams, you can understand how Rust’s high-level constructs translate into low-level assembly code. You can skip the assembly code and focus on the high-level explanations to understand Rust’s performance and memory management.

For experienced Rust programmers, this book will provide a deeper understanding of the language’s core mechanics. You can first get an overview of the generated assembly code from the flowcharts and memory representation diagrams. Then, you can delve into the assembly code to understand the performance implications and memory representations. The comments in the assembly code will help you comprehend the assembly instructions in the context of the Rust code they represent.

For developers transitioning from C++ to Rust, this book will provide a unique perspective on Rust’s performance and memory management. The annotated assembly will help you see how the Rust code allocates and deallocates memory. Rust’s unique approach to dynamic dispatch, trait inheritance, and asynchronous programming will become apparent from the generated assembly.

Embedded C++ developers transitioning to Rust will develop a deeper understanding of the generated assembly that will help them optimize their code for operating in constrained environments. The book provides insights into Rust’s performance and memory management, essential for embedded systems development. You will find that Rust lets you write high-level code using traits and functional programming constructs while generating efficient assembly code suitable for embedded systems.

This book will help algorithmic trading developers optimize performance and reduce latency by providing a deep understanding of Rust’s internals and generated assembly code. This knowledge enables fine-tuning of trading algorithms for maximum efficiency and minimal delay.

Structure of This Book

This book comprises seven parts, each centering on a different aspect of Rust's inner workings. Here is a brief overview of the chapters in each part:

Part A: Introduction to Assembly and Basic Constructs

1. **Assembly Basics: Integers, Floats, and Tuples:** We begin by examining simple functions, their assembly representations, and how they interact with immutable values, mutable references, and tuples.
2. **Fundamentals of x86-64 Assembly:** A brief introduction to x86-64 assembly, highlighting the most common instructions and their operands.

Part B: Control Structures and Enums in Assembly

3. **Deconstructing Match and If-Else Expressions:** This chapter dives into the assembly output of match and if-else expressions, highlighting compiler optimizations.
4. **Deep Dive into Enums: Layout and Patterns:** An exploration of how enums reside in memory, paired with insights into the code generated for pattern matching.

Part C: Data Structures and Memory Management

5. **Arguments: Value, Reference, Box, Rc, and Arc:** Investigating the performance nuances of passing `self` by various means, including value, reference, and smart pointers.
6. **Structs: Alignment and Patterns:** A dive into struct memory representation and the compiler's approach to struct field pattern matching.
7. **Arrays, Tuples, Option, and Box Internals:** A consolidated view of how Rust manages these elements at the assembly level.

Part D: Iteration and Optimization

8. **Traditional vs. Functional Array Iteration:** A comparison of assembly code outcomes between for-loop and functional-style array iterations, emphasizing array size's influence.
9. **Optimizing Array Iteration with SIMD:** Enhancing our understanding of array iteration, focusing on the compiler's capability to utilize SIMD (Single Instruction Multiple Data) instructions.
10. **In-Depth: Array Slices and Pattern Matching:** A focused look at the assembly for array slices and pattern matching.
11. **Vector Iteration in Assembly:** An under-the-hood look at vectors, their iteration, and the compiler's selection of loop constructs based on vector length.
12. **Advanced Iterations: Mapping and Folding:** We analyze complex two-tiered iteration structures and the assembly code they generate.

Part E: Strings, Dispatch, and Recursion

13. **Strings and Slices: Under the Surface:** Exploring Rust's representation and manipulation of strings and string slices.

14. **Static vs. Dynamic Dispatch:** We look at how the Rust compiler handles static and dynamic dispatch.
15. **Traits, Vtables, and Tail Calls:** An advanced discussion on trait inheritance, memory management, and dynamic dispatch overhead optimization.
16. **Optimizing Recursive Functions:** Understanding the compiler’s approach to optimizing tail call recursive functions.

Part F: Closures and Async/Await

17. **Closures: From Environment to Execution:** A deep dive into the mechanics of Rust closures, emphasizing variable storage and dynamic dispatch.
18. **Async/Await: Internal State Machine Dynamics:** Introduction to Rust’s asynchronous function mechanics and its translation to assembly.
19. **Nested Async/Await: Desugaring to Loops:** A continuation from the previous chapter, focusing on nested asynchronous constructs and their behavior.
20. **Async Executors: Task Scheduling Insights:** We explore the scheduling of async tasks and delve into the workings of a simple async executor.

Part G: Transformations and Takeaways

21. **Transforming into String Slice Vector:** We go under the hood to understand the transformation of a `bool` vector into a vector of static string slices.
22. **Transforming into Owned String Vector:** We focus on vectors of owned strings, their heap allocations, and memory management techniques.
23. **Key Insights:** A summary of the key takeaways from our journey through Rust’s inner workings.

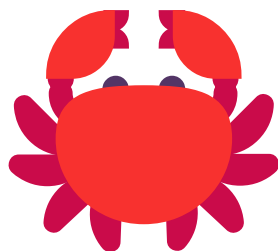


Experiments and Exercises

You can cement your understanding of Rust’s inner workings by exploring the examples and exercises at the end of most chapters. You can experiment with the code using the provided links to [Compiler Explorer](https://godbolt.org/) (<https://godbolt.org/>) or the [Rust Playground](https://play.rust-lang.org/) (<https://play.rust-lang.org/>). The links are also accessible via the book’s companion website, [Rust Under the Hood Companion](https://eventhelix.com/ruth/) (<https://eventhelix.com/ruth/>).



We hope you enjoy reading “Rust Under the Hood” and find it a valuable resource for understanding Rust’s inner workings.



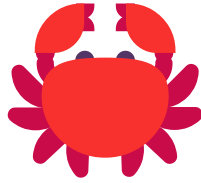
Part A

Introduction to Assembly and Basic Constructs



Rust Under the Hood Companion (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews



Chapter 1

Assembly Basics: Integers, Floats, and Tuples

Let us start our journey into the Rust internals by looking at simple functions and the assembly code they generate. We will look at functions that work with immutable values and mutable references. We will also look at functions that work with tuples.

Getting Started with Assembly

In this introductory chapter, we will introduce assembly code. If you are new to assembly, don't worry. We will use comments to help simplify and clarify the code. Additionally, we employ intuitive symbols to improve understanding of assembly operations and values. For example,  represents a load operation, and  indicates a store operation.

In the assembly code, we will encounter registers such as `rax`, `rdi`, `rsi`, `xmm0`, and `ymm0`. The compiler uses these registers to pass parameters and return values. In the next chapter, we will learn more about these registers and briefly tour the x86-64 assembly language.

As part of our learning process, we will utilize Compiler Explorer links like <https://godbolt.org/z/xahcT9oqc> to understand the generated assembly better. Visit this link for examples in this chapter. In the Compiler Explorer window, the left pane contains the Rust code, while the right pane displays the generated assembly code with instruction opcodes and optional operands. Hovering over an opcode will describe the instruction, which is valuable for understanding the instruction and its context in the generated code.

Now that we have set the stage, let us dive into our first example.

Incrementing an Integer

Integer Passed by Value

Let us start with a simple function that returns the passed argument after adding 1.

Part A Introduction to Assembly and Basic Constructs

```
pub fn increment_i64(num: i64) -> i64 {
    num + 1
}
```

The generated assembly code follows.

-  Input: rdi = num
-  Output: rax = num + 1

```
example::increment_i64:
```

```
; + Add 1 to the value in rdi and store the result in rax
; Here, load effective address (lea) performs the addition
; as lea is more efficient for adding a constant to a register.
lea    rax, [rdi + 1]

;  Return to the caller. The return value is in rax.
ret
```

Right off the bat, we can make some interesting observations.

- The compiler uses registers to pass parameters and return values. The rdi register passes the num parameter, and the rax register returns the value.
- The compiler uses the lea instruction to perform the addition. It seems odd to use lea, as typically, one uses this instruction to calculate addresses. However, adding a constant to a register is more efficient than the add instruction.

Passing a Mutable Reference to an Integer

Let us look at a function that increments the passed number by 1. The caller passes a mutable reference to a number.

```
pub fn increment_i64_mut(num: &mut i64) {
    *num += 1;
}
```

Now, let us look at the generated assembly code.

-  Input: rdi = address of num
 - Side effect: The function increments the value at the address in rdi by 1.

```
example::increment_i64_mut:
```

```
;  Fetch num from the address in rdi
; + Increment it by 1
;  Store the result back to the i64 at the address contained in rdi.
inc    qword ptr [rdi]

;  Return to the caller
ret
```

The compiler generates code that increments the value at the address passed in the rdi register. The inc instruction reads the value from memory, increments it, and then writes it back.

Incrementing a Float

Now, let us repeat the same exercises with floating-point numbers.

Passing Float by Value

Here is a simple function that returns the passed argument after adding 1.0. The function takes a floating-point number as input and returns a floating-point number as output.

```
pub fn increment_f64(num: f64) -> f64 {
    num + 1.0
}
```

The assembly for the floating-point version of the function is more complex than the integer version.

-  Input: `xmm0 = num`
-  Output: `xmm0 = num + 1.0`

```
.LCPI2_0:
.quad    0x3ff0000000000000 ; IEEE 754 double float 1.0

example::increment_f64:
; + Load 1.0 using an address relative to the instruction pointer
; and add it to the value in xmm0
addsd    xmm0, qword ptr [rip + .LCPI2_0]

; ← BACK Return to the caller
; The return value is in xmm0
ret
```

Unlike the integer version, the floating-point version requires memory access to load the constant 1.0 into a register before adding it to the value in `xmm0`. The code loads the constant using an address relative to the instruction pointer (more on this in the next chapter). The compiler represents the constant in the IEEE 754 double-precision floating-point format. Finally, the function returns the result in the `xmm0` register.

Learn More: IEEE 754 Double-Precision Floating-Point Format

The IEEE 754 double-precision floating-point format is a binary format that represents floating-point numbers. It is 64 bits wide and organized as follows:

Sign bit	Exponent bits	Significand bits
1 bit	11 bits	52 bits

The sign bit determines whether the number is positive or negative, with 0 indicating positive and 1 indicating negative.

The exponent bits represent the floating-point number's biased exponent. Subtracting the bias value 1023 from the biased exponent yields the actual exponent. This bias adjustment permits a range of exponents from -1022 to 1023.

The significand bits represent the fractional part of the floating-point number. The first bit is an implicit leading 1 (not stored in memory). The remaining 52 bits represent the fractional part of the significand.

Passing a Mutable Reference to a Float

Let us look at a function that increments the passed floating-point number by 1. The caller passes a mutable reference to a f64.

```
pub fn increment_f64_mut(num: &mut f64) {
    *num += 1.0;
}
```

Here is the assembly generated for the function.

-  Input: rdi = address of num

```
.LCPI3_0:
.quad    0x3ff0000000000000    ; IEEE 754 double float 1.0

example::increment_f64_mut:

;  Load the value from the address in rdi into xmm0.
movsd    xmm0, qword ptr [rdi]

;  Load the constant 1.0 and add the fetched value to xmm0.
addsd    xmm0, qword ptr [rip + .LCPI3_0]

;  Store the result back to the address pointed to by rdi.
movsd    qword ptr [rdi], xmm0

;  Return to the caller.
ret
```

The floating-point version of the function is less compact than its integer counterpart. The first instruction loads the value from the address in rdi into xmm0. The second instruction loads the constant 1.0 and adds it to the value in xmm0. The third instruction stores the result in xmm0 back to the address pointed to by rdi.

Working with Tuples

What Is a Tuple?

In Rust, a tuple is a collection that can hold multiple values of various types. The individual values within a tuple are known as elements. Accessing these elements is typically done via pattern matching. An example demonstrates how to construct tuples and retrieve their elements.

```
let alice = ("Alice", 30, "😊"); // Security researcher Alice
let bob = ("Bob", 35, "😎"); // Security researcher Bob
let (name, age, mood) = alice; // Pattern matching on the tuple
println!("{}", name, age, mood);
println!("{}", bob.0, bob.1, bob.2);
```

Now, let us look at functions that work with tuples. We will look at functions that take tuples as input and return tuples as output.

A Tuple of Integers

This function accepts a space-time coordinate tuple and multiplies each component by a constant, returning the resulting tuple.

```
pub fn multiply_space_time_tuple(
(x, y, z, t): (i32, i32, i32, i32) // Pattern matching on the input tuple
) -> (i32, i32, i32, i32){
    (x*2, y*3, z*4, t*5)
}
```

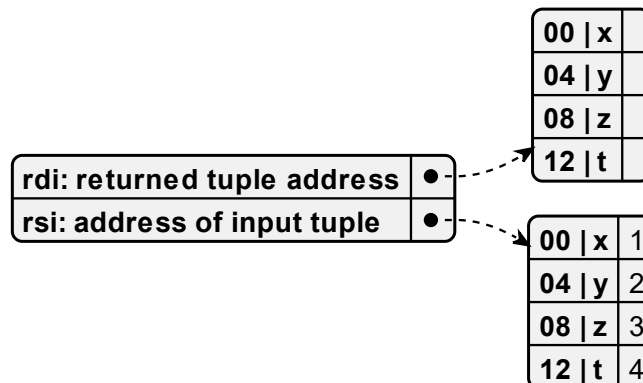
Before we proceed, let us look at how the compiler organizes the space-time tuple in memory. The compiler stores the tuple as a struct with four fields. The first field is at byte offset 0, the second is at offset 4, the third is at offset 8, and the fourth is at offset 12.

The following diagram displays the memory layout of the tuple, with the byte offsets appearing to the left of the field names.

00	x	42
04	y	94
08	z	32
12	t	43

Let us look at the sequence of steps that we will see in the generated assembly:

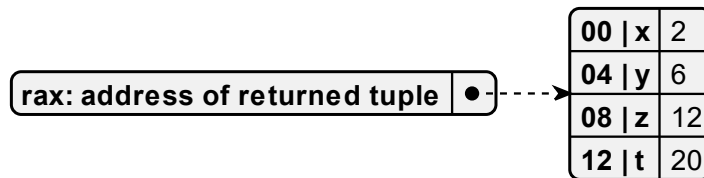
1. The caller supplies the address of the input tuple (`rsi` register) and the address where the function should return the results (`rdi` register). The following diagram shows the input registers and the memory locations used by the function.



2. The function then loads the tuple elements into registers.
3. The compiler uses different instructions for implementing the multiplications. The following table shows the instructions used for each multiplication:

Multiplication	Instruction	Notes
$x * 2$	<code>add ecx, ecx</code>	Adding a number to itself is equivalent to multiplying it by 2.
$y * 3$	<code>lea edx, [rdx + rdx * 2]</code>	Effective address calculation with a base value of <code>rdx</code> with an index of <code>rdx</code> and a scaling factor of 2
$z * 4$	<code>shl edi, 2</code>	Shift left by 2 is equivalent to multiplying by 4
$t * 5$	<code>lea esi, [rsi + rsi * 4]</code>	Effective address calculation with a base value of <code>rsi</code> with an index of <code>rsi</code> and a scaling factor of 4

4. The returned tuple receives the values after the multiplications. `rax` holds the address of the returned tuple. The following diagram illustrates the `rax` output address and the tuple it points to. It is worth noting that the caller passed the address of the returned tuple via the `rdi` register.



We will now look at the assembly generated for the function. The comments in the assembly code explain the steps we have just discussed.

- 12
34 Input:
 - rdi = address of the returned tuple
 - rsi = address of source tuple
- 12
34 Output:
 - rax = address of the returned tuple

example::multiply_i32_space_time_tuple:

```

; ← Copy the address of the tuple to be returned to rax
mov     rax, rdi

; ↓ Copy the x value from the source tuple to ecx
mov     ecx, dword ptr [rsi]

; ↓ Copy the y value from the source tuple to edx
mov     edx, dword ptr [rsi + 4]

; ↓ Copy the z value from the source tuple to edi
mov     edi, dword ptr [rsi + 8]

; ↓ Copy the t value from the source tuple to esi
mov     esi, dword ptr [rsi + 12]

; ✗ Multiply the x value by 2 using an addition and store the result in ecx
add     ecx, ecx

; ✗ Multiply the y value by 3 using the lea instruction
; Load effective address is used instead of mov because it is more efficient
lea     edx, [rdx + 2*rdx]

; ✗ Multiply the z value by 4 using the shift left (shl) instruction
shl     edi, 2

; ✗ Multiply the t value by 5 using the lea instruction
lea     esi, [rsi + 4*rsi]

; ↑ Store x in the returned tuple
mov     dword ptr [rax], ecx

; ↑ Store y in the returned tuple
mov     dword ptr [rax + 4], edx

; ↑ Store z in the returned tuple

```

```

mov     dword ptr [rax + 8], edi

;  Store t in the returned tuple
mov     dword ptr [rax + 12], esi

;  Return to the caller
ret

```

A Tuple of Floats

Now, let us look at a function that takes a tuple of floats as input and returns a tuple of floats as output.

```

pub fn multiply_f32_space_time_tuple(
    (x, y, z, t): (f32, f32, f32, f32)
) -> (f32, f32, f32, f32){
    (x*2.0, y*3.0, z*4.0, t*5.0)
}

```

The memory representation of the tuple follows. The compiler organizes the tuple as a struct with four fields. Each field is a 32-bit float.

00 x	11.200
04 y	21.999
08 z	13.011
12 t	42.412

Now, let us look at the assembly generated for the function. The tables have now turned, and the code for manipulating floating-point tuples is surprisingly more compact than that for the integer case.

The compiler has used SIMD (Single Instruction Multiple Data) instructions to perform four parallel operations per instruction. A single `movups` instruction copies all four values from the source tuple to the `xmm0` register. The compiler uses the `mulps` instruction to multiply the values in `xmm0` with the constants 2.0, 3.0, 4.0, and 5.0. The `movups` instruction copies the values from `xmm0` to the returned tuple. Finally, the function returns to the caller with the address of the returned tuple in `rax`.

-  Input:
 - `rdi` = address of the returned tuple
 - `rsi` = address of source tuple
-  Output:
 - `rax` = address of the returned tuple

```

.LCPI5_0:

.long    0x40000000 ; IEEE 754 single float 2.0
.long    0x40400000 ; IEEE 754 single float 3.0
.long    0x40800000 ; IEEE 754 single float 4.0
.long    0x40a00000 ; IEEE 754 single float 5.0

example::multiply_f32_space_time_tuple:

;  Copy the address of the tuple to be returned from rdi to rax
; This is the return value
mov     rax, rdi

```

Part A Introduction to Assembly and Basic Constructs

```

;  Perform a vector copy of x, y, z, and t from the tuple in rsi to
; xmm0. A single vector instruction copies all four values at once
movups  xmm0, xmmword ptr [rsi]

;  Multiply the values in xmm0 by the constants in .LCPI5_0.
; .LCPI5_0 contains the constants 2.0, 3.0, 4.0, and 5.0
; All four multiplications are performed in parallel
mulps   xmm0, xmmword ptr [rip + .LCPI5_0]

;  Copy the values in xmm0 to the tuple to be returned
; A single vector instruction copies all four values
movups  xmmword ptr [rdi], xmm0

;  Return to the caller, and return the result in rax.
ret

```

Learn More: Single Instruction Multiple Data (SIMD)

SIMD is a technique that allows a single instruction to perform the same operation on multiple data elements. A vector register stores the data elements. The vector register is a special-purpose register that can store multiple data elements. The size of the vector register is dependent on the architecture. For example, the `xmm0` register is 128-bit wide and can hold four 32-bit floats. The `ymm0` register is 256 bits wide and can store eight 32-bit floats. The `zmm0` register is 512 bits wide and can store sixteen 32-bit floats.

Experiment: Tuple Element Multiplications

Visit [Compiler Explorer \(https://godbolt.org/z/xahcT9oqc\)](https://godbolt.org/z/xahcT9oqc) to access the examples in this chapter.

We will experiment by adding functions to the Compiler Explorer editor and observing the generated assembly code. Compiler Explorer will compile the code and display the generated assembly code in the right pane as you add the following functions.

Use the following link to save some typing: <https://godbolt.org/z/MjsKzsGY3>

Multiply the i32 Tuple Elements by Larger Constants

Add the following function to Compiler Explorer editor. Compare the generated assembly with the `multiply_i32_space_time_tuple` function. You will find that the compiler has more tricks up its sleeve. The compiler uses `lea`, `add`, and `shl` instructions to perform the multiplications.

```

pub fn multiply_i32_space_time_tuple_larger_const_1(
    (x, y, z, t): (i32, i32, i32, i32),
) -> (i32, i32, i32, i32) {
    (x * 6, y * 7, z * 8, t * 9)
}

```

Now, add the following function to Compiler Explorer editor. Compare the generated assembly with the `multiply_i32_space_time_tuple` function. You will see that the compiler uses SIMD instructions to perform the multiplications.

```

pub fn multiply_i32_space_time_tuple_larger_const_2(
    (x, y, z, t): (i32, i32, i32, i32),
) -> (i32, i32, i32, i32) {

```



```
(x * 11, y * 12, z * 13, t * 14)
}
```

Square the Tuple Members

Add the following code to [Compiler Explorer](#) editor. Compare the generated assembly with the `multiply_i32_space_time_tuple` and `multiply_f32_space_time_tuple` functions.

```
pub fn square_i32_space_time_tuple(
    (x, y, z, t): (i32, i32, i32, i32)
) -> (i32, i32, i32, i32){
    (x*x, y*y, z*z, t*t)
}

pub fn square_f32_space_time_tuple(
    (x, y, z, t): (f32, f32, f32, f32)
) -> (f32, f32, f32, f32){
    (x*x, y*y, z*z, t*t)
}
```

AVX-512

Change the compiler options to use the AVX-512 instruction set using the following flag setting:

```
-C opt-level=3 -C target_cpu=skylake-avx512
```

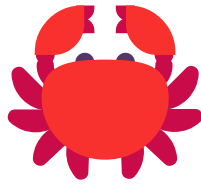
Apply the settings in the edit box on the top right of the Compiler Explorer window.

Now, the compiler uses AVX-512 instructions. The compiler uses SIMD instruction in many more places than the default `-opt-level=3` setting.



Key Takeaways

- The Rust compiler generates efficient assembly code. The compiler uses different approaches to optimize integer and floating-point operations.
- The compiler uses additions, shifts, and address arithmetic instructions (`lea`) to perform simple integer multiplications. If larger constants are involved, the compiler uses SIMD instructions to perform integer multiplications.
- The generated code will utilize SIMD instructions to enhance performance, as SIMD instructions enable the parallel execution of multiple operations.
- Rust represents tuples as structs with multiple fields stored in contiguous memory locations.
- When returning a large structure or tuple, the compiler passes the address of the returned structure to the function using a hidden parameter. This approach avoids creating temporary structures on the stack. Also, note that the called function copies the address of the returned structure to the `rax` register as the return value to comply with the x86-64 calling convention.



Chapter 2

Fundamentals of x86-64 Assembly

We got our first look at the x86-64 assembly in the previous chapter. We will now take a quick tour of the x86-64 assembly. We will look at the [registers](#), [calling convention](#), and [instruction set](#).

In the first reading, you can skip the details of the instructions and focus on the general concepts. You can come back to the details later when you need them.

x86-64 Registers

General-Purpose Registers

Programs use registers for general data manipulation, storing integers, pointers, and memory addresses. The following table shows the general-purpose registers available with their typical usage:

- `rax`: Accumulator
- `rbx`: Base index (for use with arrays)
- `rcx`: Counter for loops
- `rdx`: Extended precision accumulator (pairs with `rax` for 128-bit operations)
- `rsi`: Source index for string operations
- `rdi`: Destination index for string operations
- `r8` to `r15`: Additional general-purpose registers

Stack Registers

The compiler uses these registers for stack operations:

- `rsp`: Stack pointer
- `rbp`: Base pointer (often points to a fixed position within the stack)

Instruction Pointer

`rip` functions as the Instruction pointer, pointing to the next instruction slated for execution.

You will encounter `rip`-relative addressing in the assembly code. For instance, the instruction `lea rcx, [rip + .LJT157_0]` loads a `rip`-relative address of the label `.LJT157_0` into `rcx`. Using `rip` relative addressing aims to make the code position-independent, which is necessary for dynamic linking.

Flags Register

The `rflags` register contains flags that reflect the outcome of operations and control process execution in several ways.

Flag	Name	Description	Example
CF	Carry Flag	Set if an arithmetic operation generates a carry-out of the most significant bit or a borrow into it.	add al, 0xFF - If <code>al</code> contains <code>0x02</code> , there will be a carry, and the processor will set CF as the result exceeds 255.
ZF	Zero Flag	Set if the result of an operation is zero.	sub ax, ax - Subtracting <code>ax</code> from itself will result in zero, setting the ZF.
SF	Sign Flag	Set based on the most significant bit of the result (0 for positive or zero, 1 for negative signed numbers).	sub al, 1 - If <code>al</code> were 0, the result would be -1 (<code>0xFF</code> in two's complement), setting SF.
OF	Overflow Flag	Set if there is an overflow for signed arithmetic.	add al, 0x80 - If <code>al</code> also contains <code>0x80</code> , the result will overflow, setting the OF.
PF	Parity Flag	Set if the number of set bits (1s) in the least significant byte of the result is even.	mov al, 0x03; test al, al - <code>al</code> has an odd number of set bits, so the PF will be cleared.

These flags are crucial for understanding the results of arithmetic and logical operations, and they play an integral role in flow control through conditional jumps and conditional move instructions.

Floating-Point and SIMD Registers

- `xmm0` to `xmm15`: Used for floating point calculations and Single Instruction Multiple Data (SIMD) operations. These registers are 128 bits wide and store four single-precision floating-point values or two double-precision floating-point values.
- `ymm0` to `ymm15`: Used for floating point calculations and SIMD operations. These registers are 256 bits wide and can store eight single-precision floating-point values or four double-precision floating-point values. You can access the lower 128 bits of these registers as `xmm0` to `xmm15`.

Register Naming Convention for 8-bit, 16-bit, 32-bit, and 64-bit Access

One thing to note about the x86-64 architecture is that programs can access the same register as 8-bit, 16-bit, 32-bit, or 64-bit values. The following table shows the naming convention for the different access sizes.

64-bit name	32-bit name	16-bit name	8-bit name (higher byte)	8-bit name (lower byte)
<code>rax</code>	<code>eax</code>	<code>ax</code>	<code>ah</code>	<code>al</code>
<code>rbx</code>	<code>ebx</code>	<code>bx</code>	<code>bh</code>	<code>bl</code>
<code>rcx</code>	<code>ecx</code>	<code>cx</code>	<code>ch</code>	<code>cl</code>
<code>rdx</code>	<code>edx</code>	<code>dx</code>	<code>dh</code>	<code>dl</code>
<code>rsi</code>	<code>esi</code>	<code>si</code>	-	<code>sil</code>
<code>rdi</code>	<code>edi</code>	<code>di</code>	-	<code>dil</code>
<code>rbp</code>	<code>ebp</code>	<code>bp</code>	-	<code>bpl</code>
<code>rsp</code>	<code>esp</code>	<code>sp</code>	-	<code>spl</code>
<code>r8</code>	<code>r8d</code>	<code>r8w</code>	<code>r8b</code> (higher byte)	<code>r8b</code> (lower byte)
...	...	...	...	...
<code>r15</code>	<code>r15d</code>	<code>r15w</code>	<code>r15b</code> (higher byte)	<code>r15b</code> (lower byte)

x86-64 Calling Convention

The x86-64 calling convention dictates the methods for calling functions and passing arguments to them. Its importance lies in determining the generation of assembly code for function calls. It covers the following:

- Registers to use for passing arguments to functions.
- Registers to use for returning values from functions.
- The distinction between caller-saved and callee-saved registers.
- The usage of the stack in passing arguments and returning values.

Note that the calling convention is specific to the platform and the compiler. The assembly code presented in this book adheres to the calling convention used by the Rust compiler on x86-64 Linux (System V AMD64 ABI).

Registers for Passing Arguments

The calling convention specifies the order in which registers pass arguments to functions. The selection order is important because it determines which registers to use for which arguments.

The following table shows the register selection order for several types of arguments:

Argument Type	Register selection order
Integer types and pointers	<code>rdi</code> , <code>rsi</code> , <code>rdx</code> , <code>rcx</code> , <code>r8</code> , <code>r9</code>
Floating-point types	<code>xmm0</code> , <code>xmm1</code> , <code>xmm2</code> , <code>xmm3</code> , <code>xmm4</code> , <code>xmm5</code> , <code>xmm6</code> , <code>xmm7</code>

The generated code pushes arguments that do not fit into the available registers onto the stack in reverse order (right-to-left).

The `rdi` register also serves as a hidden pointer to return larger values from functions. In such cases, the caller is responsible for allocating space for the return value and passing the address of the allocated space in `rdi`. The callee then stores the return value at the address pointed to by `rdi`. In such cases, `rsi` receives the first argument instead of `rdi`.

Registers for Returning Values

The calling convention specifies the registers used for returning values from functions. The rules for selecting the registers are as follows:

- The called function returns integer types and pointers in `rax`, with `rdx` holding the high bits for 128-bit wide return values, such as a pair of 64-bit values.
- The called function returns floating-point values in `xmm0` and in `xmm1` if necessary.

For larger return values, the caller uses a hidden pointer as an extra argument to the function. It passes this pointer in `rdi` and stores the return value at the address the pointer indicates.

Caller vs. Callee Saved Registers

The calling convention dictates which registers the caller saves and which the callee saves. This distinction is crucial, as it specifies which registers need saving and restoring during function calls. The following table lists the caller-saved and callee-saved registers:

Register	Type	Description
<code>rax</code> , <code>rcx</code> , <code>rdx</code> , <code>rsi</code> , <code>rdi</code> , <code>r8</code> , <code>r9</code> , <code>r10</code> , <code>r11</code> , <code>xmm0</code> -15	Caller-saved	The calling function must save these registers to preserve their values across a function call.

Part A Introduction to Assembly and Basic Constructs

Register	Type	Description
rbx, rbp, rsp, r12, r13, r14, r15	Callee-saved	If a called function (callee) wants to use these registers, it must save and restore the original values before returning.

Function Prologue and Epilogue

In addition to the above, the calling convention outlines the function prologue and epilogue, which execute at the beginning and end of a function call. The prologue and epilogue set up and restore the stack and registers for the function call, ensuring their return to their original state before the function returns.

- The function's prologue prepares the stack and registers for the function's operation. Prologue preparation might involve setting up a base pointer (rbp), saving callee-saved registers, and allocating space for local variables.
- The epilogue reverts the state to its original form before the function call. It typically restores the saved registers and resets the stack pointer (rsp) to its initial value before concluding the function.

x86-64 Addressing Modes

Addressing modes determine how a processor locates the data or the operand of an instruction in assembly language. They specify a formula for calculating the effective memory address of an operand by using the information held in registers or constants contained within a machine instruction or elsewhere. The x86-64 instruction set offers a diverse set of addressing modes, providing programmers with the flexibility to reference memory in several ways. The following table delineates the frequently used addressing modes in x86-64 assembly:

Addressing Mode	Syntax	Description
Immediate	imm	This mode specifies a constant value.
Register	r64	This mode uses the value of a 64-bit general-purpose register as the address.
Direct Memory	moffset64	This mode utilizes a 64-bit immediate value as a direct memory address.
Register Indirect	[r64]	This mode employs the 64-bit register's value as a memory address.
Register with Offset	[r64 + imm32]	This mode adds a 32-bit immediate value to the register's value to determine the memory address.
Base with Index and Scale	[r64 + r64*scale]	This mode combines a base register's value with the product of an index register and a scale (1, 2, 4, or 8).
Base with Index, Scale, and Offset	[r64 + r64*scale + imm32]	This mode combines a base register, index register, scale factor, and a 32-bit immediate offset to compute the memory address.
RIP-relative with Offset	[rip + imm32]	This mode calculates the memory address using the RIP register's value plus a 32-bit offset, which aids in creating position-independent code.

The preceding table uses the following conventions:

- imm refers to an immediate value encoded directly within the instruction.
- r64 refers to a 64-bit general-purpose register.

- `moffset64` refers to a memory operand that holds 64 data bits.
- `scale` refers to a scale factor of 1, 2, 4, or 8.
- `rip` refers to the instruction pointer register.

x86-64 Instruction Set

The x86-64 instruction set is quite large, and it is beyond the scope of this book to cover it in detail. However, we will cover some of the most common instructions in the assembly code.

mov Instruction

The `mov` instruction transfers (or copies) data between registers, memory locations, and constant values. Although called *mov*, it does not actually “move” data by removing it from the source; it replicates the source data to the destination. Here are some examples of `mov` instructions:

Instruction	Explanation
<code>mov rax, rbx</code>	Copies the value from <code>rbx</code> into <code>rax</code>
<code>mov rax, [rbx]</code>	Loads the value pointed to by <code>rbx</code> into <code>rax</code>
<code>mov [rcx], rdx</code>	Stores the value in <code>rdx</code> into the memory location pointed to by <code>rcx</code>
<code>mov rax, 5</code>	Loads the constant value 5 into <code>rax</code>

lea Instruction

The `lea` (Load Effective Address) instruction computes the address of a memory operand and loads that address into a register without referencing memory. In other words, it does not fetch the value stored at the address but the address itself. The following table shows some examples of `lea` instructions:

Instruction	Explanation
<code>lea rax, [rbx]</code>	Loads the address in <code>rbx</code> into <code>rax</code>
<code>lea rax, [rbx + 4]</code>	Loads the address <code>rbx + 4</code> into <code>rax</code>
<code>lea rax, [rbx + rcx*4]</code>	Loads the address <code>rbx + (4 * rcx)</code> into <code>rax</code>
<code>lea rax, [rbx + rcx*4 + 8]</code>	Loads the address <code>rbx + (4 * rcx) + 8</code> into <code>rax</code>

Note that the compiler often uses `lea` to perform arithmetic operations on integers. For example, the following instruction adds `rbx` and `rcx` and stores the result in `rax`:

```
lea rax, [rbx + rcx]
```

The compiler picks the `lea` instruction when it will be more efficient than dedicated addition and multiplication instructions. For example, the following instruction multiplies `rcx` by 4, adds the result to `rbx`, and stores the result in `rax`:

```
lea rax, [rbx + rcx*4]
```

Arithmetic Instructions

Arithmetic instructions perform arithmetic operations on data in registers and memory locations. The following table shows a sample of the arithmetic instructions available in x86-64 assembly:

Instruction	Explanation
<code>add rax, rbx</code>	Adds <code>rbx</code> to <code>rax</code> and stores the result in <code>rax</code>

Part A Introduction to Assembly and Basic Constructs

Instruction	Explanation
<code>sub rax, rbx</code>	Subtracts <code>rbx</code> from <code>rax</code> and stores the result in <code>rax</code>
<code>mul rbx</code>	Multiplies <code>rax</code> and <code>rbx</code> and stores the result in <code>rdx:rax</code> (with <code>rdx</code> holding the high-order bits)
<code>imul eax, ebx</code>	Multiplies contents of <code>ebx</code> with <code>eax</code> (signed) and stores the result in <code>eax</code> .
<code>imul rax, rbx</code>	Same as above but for 64-bit.
<code>div rbx</code>	The instruction divides <code>rdx:rax</code> by <code>rbx</code> . After the execution of the instruction, the quotient is in <code>rax</code> and the remainder in <code>rdx</code> .
<code>idiv ebx</code>	The instruction divides the combined value of <code>edx:eax</code> by <code>ebx</code> (signed). After the execution of the instruction, the quotient is in <code>eax</code> and the remainder is in <code>edx</code> .
<code>idiv rbx</code>	Same as above but for 64-bit.
<code>inc rax</code>	Increments the value in <code>rax</code> by 1
<code>dec rax</code>	Decrements the value in <code>rax</code> by 1
<code>neg rax</code>	Computes the two's complement (negation) of <code>rax</code> and stores the result in <code>rax</code>

Bitwise Instructions

Bitwise instructions perform bitwise operations on data in registers and memory locations. The following table shows a sample of bitwise instructions available in x86-64 assembly:

Instruction	Explanation
<code>and rax, rbx</code>	Performs a bitwise AND between <code>rax</code> and <code>rbx</code> , storing the result in <code>rax</code>
<code>or rax, rbx</code>	Performs a bitwise OR between <code>rax</code> and <code>rbx</code> , storing the result in <code>rax</code>
<code>xor rax, rbx</code>	Performs a bitwise XOR between <code>rax</code> and <code>rbx</code> , storing the result in <code>rax</code>
<code>shl rax, 1</code>	Shifts the bits in <code>rax</code> to the left by 1, placing 0 in the least significant bit
<code>shr rax, 1</code>	Shifts the bits in <code>rax</code> to the right by 1, placing 0 in the most significant bit

The compiler uses `shl` instructions for multiplication by powers of 2 and `shr` instructions for division by powers of 2.

Compare and Test Instructions

In the x86-64 assembly, the compiler uses the `cmp` and `test` instructions for conditional operations. While they do not store their results in a register, they modify the flags in the `rflags` register that conditional jumps, moves, and set operations use. Here are some examples of `cmp` and `test` instructions:

Instruction	Operation	rflags Flags Affected	Explanation
<code>cmp rax, rbx</code>	Subtraction (<code>rax - rbx</code>)	CF, OF, PF, SF, ZF	Compares <code>rax</code> to <code>rbx</code> by subtracting <code>rbx</code> from <code>rax</code> and sets the <code>rflags</code> accordingly but does not save the result. Conditional jump instructions, then use the <code>rflag</code> settings to decide upon the branch.
<code>test rax, rbx</code>	Bitwise AND (<code>rax & rbx</code>)	CF (cleared), OF (cleared), PF, SF, ZF	This instruction tests the bits in <code>rax</code> against <code>rbx</code> using a bitwise AND, setting the <code>rflags</code> according to the outcome without saving the result. Programs commonly use it to verify if specific bits in a register/memory location are set or cleared.

Impacted Flags

Both `cmp` and `test` can affect several flags in the `rflags` register.

- **ZF (Zero Flag):** Set if the result of the operation is zero.
- **SF (Sign Flag):** Set if the result of the operation is negative (based on the most significant bit).
- **OF (Overflow Flag):** Set if there is a signed overflow because of the operation.
- **CF (Carry Flag):** Set if there is an unsigned overflow because of the operation.
- **PF (Parity Flag):** Set if the number of set bits in the least significant byte of the result is even.

Conditional Jump Instructions

Conditional branches rely on the state of the bits in the `rflags` register. Previous arithmetic, `cmp`, or `test` instructions often set these bits. The following table shows the conditional jump instructions available in x86-64 assembly:

Instruction	Condition	rflags bits used	Explanation
<code>je label</code>	Equal	ZF	If the Zero Flag (ZF) is set, the program jumps to <code>label</code> .
<code>jne label</code>	Not Equal	ZF	If the Zero Flag (ZF) is not set, the program jumps to <code>label</code> .
<code>jg label</code>	Greater (signed)	ZF, SF, OF	If ZF is clear and (SF is equal to OF), the program jumps to <code>label</code> .
<code>jb label</code>	Below (unsigned less than)	CF	If the Carry Flag (CF) is set, the program jumps to <code>label</code> .
<code>ja label</code>	Above (unsigned greater than)	CF, ZF	If CF is clear and ZF is clear, the program jumps to <code>label</code> .
<code>jae label</code>	Above or Equal (unsigned greater or equal)	CF	If the Carry Flag (CF) is clear, the program jumps to <code>label</code> .
<code>js label</code>	Sign (negative)	SF	If the Sign Flag (SF) is set, the program jumps to <code>label</code> .
<code>jns label</code>	No Sign (non-negative)	SF	If the Sign Flag (SF) is clear, the program jumps to <code>label</code> .
<code>jp label</code>	Parity (even bit)	PF	If the Parity Flag (PF) is set, the program jumps to <code>label</code> .

These conditional branches allow for the creation of loops, condition checks, and more intricate control flows in the assembly code. They form the foundation for constructs like “if” statements and “loops” in high-level languages.

Conditional Move Instructions

Conditional move (`cmov`) instructions in the x86-64 architecture transfer data between registers based on the state of the `rflag` bits while leaving the flags unaffected. Their usefulness lies in eliminating the need for some branches, helping avoid branch prediction penalties.

Here are some examples of conditional move instructions:

Part A Introduction to Assembly and Basic Constructs

Instruction	Condition	rflags bits used	Explanation
<code>cmovb rax, rbx</code>	Below (Carry)	CF	The instruction moves <code>rbx</code> to <code>rax</code> if the Carry Flag (CF) is set.
<code>cmovne rax, rbx</code>	Not Equal	ZF	The instruction moves <code>rbx</code> to <code>rax</code> if the Zero Flag (ZF) is not set.
<code>cmove rax, rbx</code>	Equal	ZF	The instruction moves <code>rbx</code> to <code>rax</code> if the Zero Flag (ZF) is set.

Unconditional Branches

Unconditional branches jump to a specific location in the code, regardless of the state of the `rflags`. The following table shows some of the unconditional jump instructions available in x86-64 assembly:

Instruction	Explanation
<code>jmp label</code>	Jump to the instruction labeled “label” unconditionally
<code>jmp rax</code>	Jump to the address stored in <code>rax</code>
<code>call label</code>	Pushes the address of the next instruction onto the stack and then jumps to the address specified by <code>label</code> . This mechanism allows the <code>ret</code> instruction to know where to return.
<code>call rax</code>	Like <code>call label</code> , but instead of jumping to a predefined address (label), the program jumps to the address in the <code>rax</code> register after pushing the address of the next instruction onto the stack.
<code>ret</code>	The instruction pops an address from the stack and jumps to it, typically returning to the instruction right after a <code>call</code> . This process usually involves an address previously pushed by a <code>call</code> instruction, enabling the program to resume execution post-call.

Stack Operations

In x86-64 assembly, the stack is a fundamental data structure for many tasks, including storing local variables and saving registers. The x86-64 architecture provides several instructions for stack operations.

The following table shows commonly used stack instructions:

Instruction	Explanation
<code>push rax</code>	The instruction pushes the value in <code>rax</code> onto the stack by decrementing the <code>rsp</code> (stack pointer) by 8 bytes and then storing the value of <code>rax</code> at the new <code>rsp</code> address.
<code>pop rax</code>	The instruction pops the top value from the stack into <code>rax</code> by loading the value pointed to by <code>rsp</code> into <code>rax</code> and then increments the <code>rsp</code> by 8 bytes.
<code>pushfq</code>	Pushes the 64-bit <code>rflags</code> register onto the stack.
<code>popfq</code>	Pops the top value from the stack into the <code>rflags</code> register.

SIMD Instructions

Here is a sample of SIMD (Single Instruction, Multiple Data) instructions available in x86-64 assembly. These instructions perform the same operation on multiple pieces of data simultaneously.

You can skim through the following table and return to it later when you need to understand the SIMD instructions in the assembly code.

Instruction	Operation	Explanation
<code>movdqa xmm1, xmm2</code>	Move aligned double quadword	Moves a 128-bit aligned value from <code>xmm2</code> to <code>xmm1</code> .
<code>movdqu xmm1, xmm2</code>	Move unaligned double quadword	Moves a 128-bit value from <code>xmm2</code> to <code>xmm1</code> . Data can be unaligned.
<code>vmovq xmm1, r64</code>	Move quadword	Moves a 64-bit value from a general-purpose register to the lower half of <code>xmm1</code> .
<code>vmovups ymm1, m256</code>	Move unaligned packed single precision	Moves 256-bits (8 floating-point values) from memory to <code>ymm1</code> . Data can be unaligned.
<code>vextracti128 xmm1, ymm2, imm8</code>	Extract 128-bit integer data from <code>ymm</code>	Extracts one of the two 128-bit values from <code>ymm2</code> based on <code>imm8</code> and stores it in <code>xmm1</code> .
<code>vzeroupper</code>	Zero out upper 128 bits of all <code>ymm</code> registers	Clears the upper 128 bits of all <code>ymm</code> registers to avoid AVX-SSE transition penalties.
<code>vpbroadcastq xmm1, r/m64</code>	Broadcast 64-bit integer to all elements of <code>xmm1</code>	Broadcasts a 64-bit integer value into both 64-bit positions of <code>xmm1</code> .
<code>psllw xmm1, xmm2/m128</code>	Packed shift left logical on word	Shifts eight 16-bit words in <code>xmm1</code> left by the number of bits specified in <code>xmm2/m128</code> .
<code>por xmm1, xmm2</code>	Packed bitwise OR	Performs a bitwise OR on packed 64-bit and 32-bit numbers in <code>xmm1</code> and <code>xmm2</code> .
<code>vpxor xmm1, xmm2, xmm3</code>	Packed bitwise XOR using AVX	Performs a bitwise XOR on packed 64-bit and 32-bit numbers in <code>xmm2</code> and <code>xmm3</code> , stores result in <code>xmm1</code> .
<code>vpshufd xmm1, xmm2/m128, imm8</code>	Shuffle packed doublewords	Shuffles four 32-bit doublewords in <code>xmm2/m128</code> based on <code>imm8</code> and stores the result in <code>xmm1</code> .
<code>vptestnmq xmm1, xmm2, xmm3</code>	Packed test negative and mask quadword	Performs a bitwise AND-NOT on 64-bit integers in <code>xmm2</code> and <code>xmm3</code> , setting bits in <code>xmm1</code> if results are zero.
<code>vpmullq xmm1, xmm2, xmm3</code>	Packed multiply logical quadword	Multiply two 64-bit quadwords in <code>xmm2</code> and <code>xmm3</code> , and store the result in <code>xmm1</code> .
<code>vpaddq xmm1, xmm2, xmm3</code>	Packed add quadword	Adds two 64-bit quadwords in <code>xmm2</code> and <code>xmm3</code> , stores result in <code>xmm1</code> .

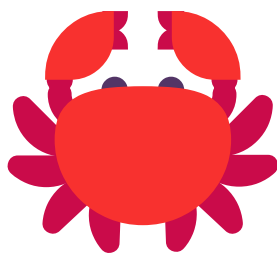
Note the following conventions used in the table:

- `imm8` refers to an 8-bit immediate value encoded directly within the instruction.
- `m64` refers to a memory operand that holds 64 data bits.
- `m128` refers to a memory operand that holds 128 data bits.
- `r64` refers to a 64-bit general-purpose register.
- `xmm` refers to a 128-bit XMM register.
- `ymm` refers to a 256-bit YMM register.

Part A Introduction to Assembly and Basic Constructs

Learn More: Aligned vs. Unaligned Instructions

The SIMD instructions in the x86-64 assembly come in two flavors: aligned and unaligned. Aligned instructions require that the memory addresses of the data they operate on are multiples of the data size. For instance, a 128-bit aligned instruction requires that the memory address is a multiple of 16 bytes. Unaligned instructions, however, do not have this restriction and can operate on data at any memory address.



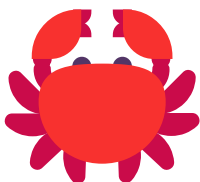
Part B

Control Structures and Enums in Assembly



Rust Under the Hood Companion (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews



Chapter 3

Deconstructing Match and If-Else Expressions

This chapter will cover the assembly code generated for match and if-else expressions. We will examine how the compiler optimizes the code to minimize branching and improve performance.

We will use the following examples to learn about match and if-else expressions.

- [Match Expression](#)
- [Match Expression with Ranges](#)
- [If-Else Expression](#)

Match Expression

Match expressions in Rust are a powerful control flow mechanism, like switch statements in other languages, but they are more expressive. They allow you to branch your code based on the pattern of a particular value. The keyword `match`, followed by an expression and then curly brackets `{}` containing one or more arms, defines the syntax of match expressions. Each arm is a pattern followed by `=>` and then the expression to execute if the match is successful. A key point to note is that Rust's match is exhaustive, meaning you must cover all possible values of the type. Here is an example:

```
pub fn get_emoji_0_to_10(score: u8) -> &'static str {  
    match score {  
        0 => "😱",  
        1 => "😨",  
        2 => "😧",  
        3 => "😞",  
        4 => "😓",  
        5 => "😬",  
        6 => "😏",  
        7 => "😄",  
        8 => "😁",  
        9 => "😃",  
        10 => "😄",  
    }
```

```

    _ => "❌",
}
}

```

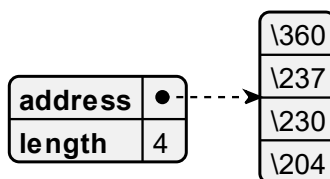
In this example, we have a function that takes a u8 and returns a string slice. The function uses a match expression to compare the value against a set of patterns and returns a string slice based on the pattern that matches. The `_` pattern is a catch-all that matches any value. If the value is not 0 to 10, it returns "❌".

Also, note that the `match` is an expression that returns a value. The `match` expression returns the value of the arm that matches. In our example, the `match` expression returns a string slice for the respective emoji.

This example requires an overview of static strings and string slices. Let us learn about them before we look at the generated assembly code.

Static Strings and String Slices

The compiler stores a static string in the read-only section of the executable. In our example, the compiler generates a static string for each emoji and returns a string slice. String slices are a reference to a string. They contain the address of the string and the length of the string. The compiler returns a string slice containing the emoji string's address and length. The following diagram shows a string slice pointing to the "😄".



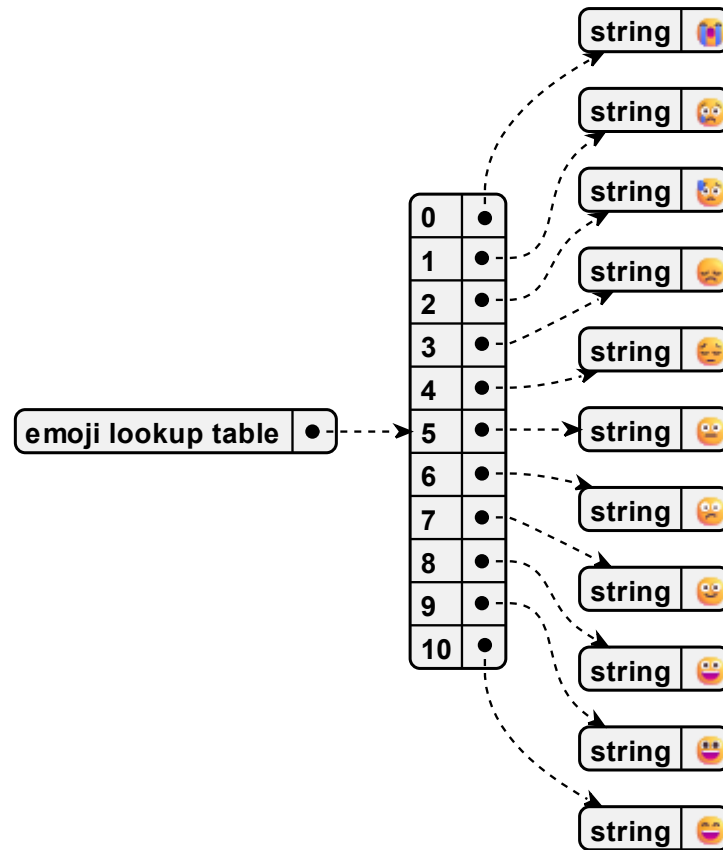
We will look at strings and string slices in more detail in the chapter *"Strings and Slices: Under the Surface"*.

Lookup Table for Match Expression

Let us lay the groundwork for understanding the assembly code generated for the match expression. We will examine how the compiler implements the match expression using a lookup table. The lookup table is a data structure that maps input values to output values. In our example, the input values are the scores from 0 to 10, and the output values are the emoji strings.

The compiler generates a lookup table-based implementation for this match expression. The lookup table stores the address offsets of the respective emoji strings. The compiler stores the emoji lookup table and the emoji strings in the read-only section of the executable.

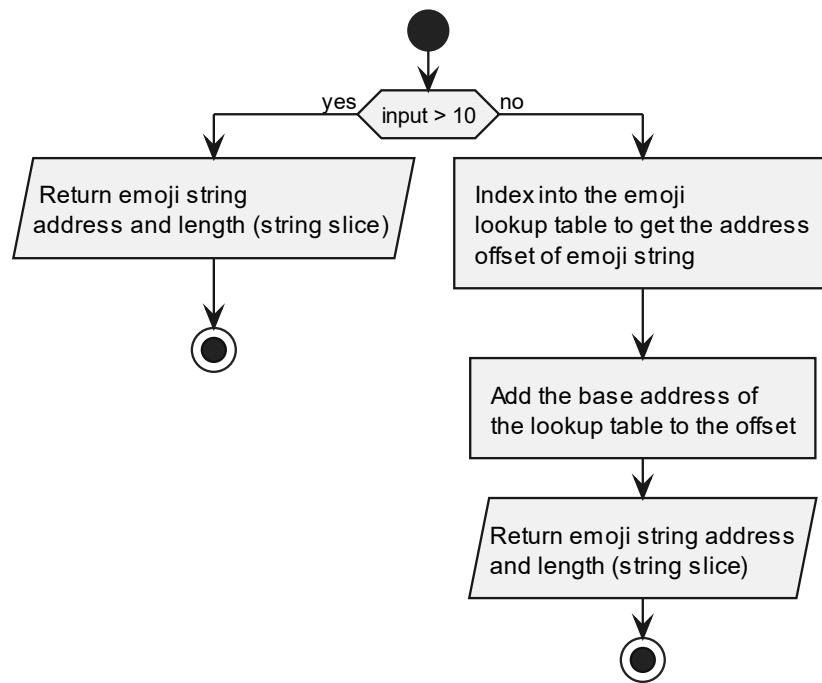
The following figure shows a high-level representation of the lookup table. It has 11 entries, one for each emoji. Each entry is 4 bytes long. The first entry is the address offset of the "😄" emoji string. The second entry is the address offset of the "😞" emoji string, and so on. The address offsets are relative to the base address of the lookup table.



Flowchart: Match Expression with a Lookup Table

The following flowchart presents the high-level implementation of the match expression. It compares the input value with 10. If the input exceeds 10, it returns the “✖” emoji. For inputs less than or equal to 10, the assembly code uses the input as an index into the emoji lookup table to find the address offset of the respective emoji string. It then adds the lookup table’s base address to the offset to locate the emoji string’s address. Finally, the function returns a string slice containing the address and length of the emoji string.

Part B Control Structures and Enums in Assembly



Assembly Code: Match Expression with a Lookup Table

This assembly code implements a switch-case statement. For input values in the range of 0-10, it jumps to the respective address in a lookup table, which points to the memory location of the associated emoji string. If the input value is out of range, it directly loads the address of the “✖” emoji string into the return register. The function also stores the string returned length in the edx register.

-  Input: dil = score
-  Output: rax = emoji string, edx = length of emoji string (string slice)

```

example::get_emoji_0_to_10:
; ⚖️ Compare the input value in dil (the lower 8 bits of rdi) with 10
cmp     dil, 10

; ➡️ If input > 10, jump to .LBB0_1, which will return the "✖" emoji
ja      .LBB0_1

; ⬅️ Move the input value into rax and sign-extend it
; (The sign extension is a nop for u8 but suitable for other types)
movsx   rax, dil

; ⬅️ Load the effective address of the relative emoji lookup table into rcx
lea     rcx, [rip + .Lreltable.example::get_emoji_0_to_10]

; 🔍 Index into the emoji lookup table with 4 * input (since each entry is
; 4 bytes) and load it into rax
movsxd  rax, dword ptr [rcx + 4*rax]

; ➕ Add the base address of the lookup table to the offset loaded into rax
add     rax, rcx

; ⬅️ Set edx to 4, the length of the emoji string in bytes

```

```

mov     edx, 4

;  Return to caller
ret

```

> : Branch when the input > 10.

```

.LBB0_1:

;  Load the effective address of the "X" emoji string into rax
lea     rax, [rip + .L__unnamed_1]

;  Set edx to 3, the length of the "X" emoji string in bytes
mov     edx, 3

;  Return to caller
ret

```

Look at [Compiler Explorer \(https://godbolt.org/z/MK7G86Kvs\)](https://godbolt.org/z/MK7G86Kvs) link for the emoji lookup table definition (.Lretable.example::get_emoji_0_to_10).

Match Expression with Ranges

Now, let us look at a match expression that uses ranges. We will change the previous example to return different emojis based on a score from 0 to 100. The match expression uses ranges to match the score to the relevant emoji. The ranges include the lower and upper bounds. The `_` is a catch-all pattern that matches any value.

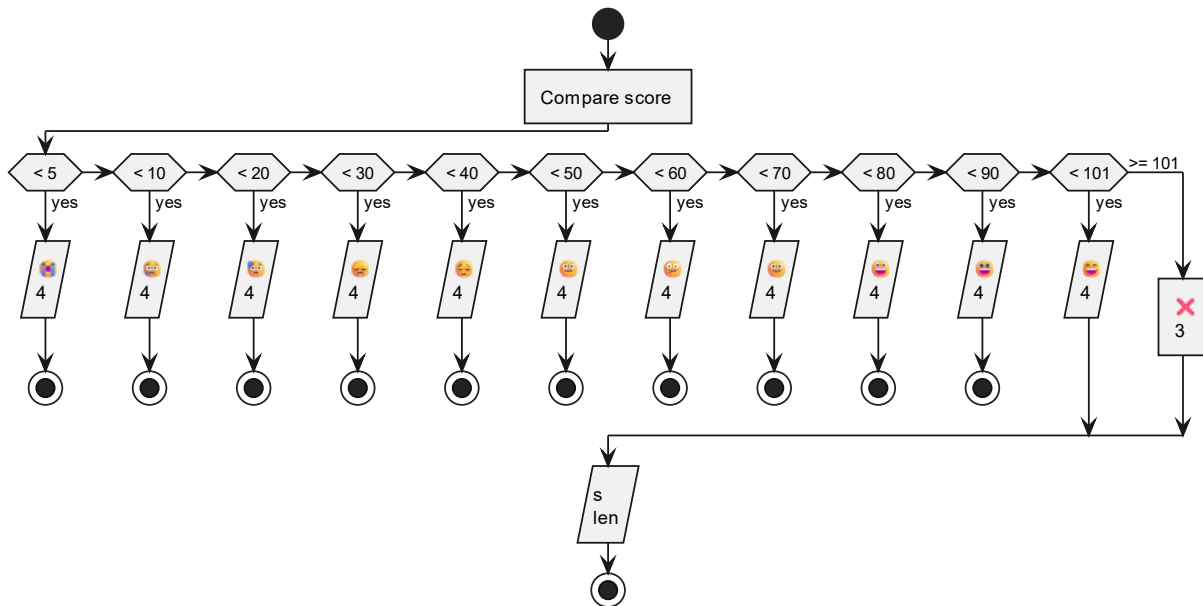
```

pub fn get_emoji_from_0_to_100(score: u8) -> &'static str {
    match score {
        0..=4 => "😱",
        5..=9 => "😬",
        10..=19 => "😓",
        20..=29 => "😞",
        30..=39 => "😔",
        40..=49 => "😟",
        50..=59 => "😟",
        60..=69 => "😬",
        70..=79 => "😬",
        80..=89 => "😬",
        90..=100 => "😬",
        _ => "X",
    }
}

```

The following flowchart summarizes the generated code. The code performs upper-bound checks for each range. If the input value exceeds the upper bound, it jumps to the following range. If the input value is less than or equal to the upper bound, it returns the respective emoji string. If the input value is out of range, it returns the "X" emoji string.

Part B Control Structures and Enums in Assembly



The following code highlights the compiler’s approach to handling the match comparisons. Instead of a lookup table, the compiler creates a series of comparisons and returns the corresponding emoji string. It stores the returned string length in the `edx` register.

Pay special attention to the last comparison: the default length 4 does not apply here because the “❌” emoji string is only 3 bytes long. The compiler uses the `adc` (add with carry) instruction to correct the length. Moreover, it employs a *conditional move below* instruction (`cmovb`) to set the return address to the memory address of the “❌” emoji string.

For each emoji, the function compares the input score with the upper limit of a range. If the input score is less than or equal to the upper limit, it jumps to a label where it sets the return address to the memory address of the emoji string.

- `12 34` Input: `dil` = score (u8 in Rust, represented as `dil` register here)
- `12 34` Output: `rax` = emoji string, `edx` = length of emoji string (string slice)

example::get_emoji_from_0_to_100:

```

mov     edx, 4      ; Set length of emoji string (4 bytes for each emoji)

cmp     dil, 5      ; ⚖️ Compare the input score with 5
jae     .LBB2_2     ; ➡️ If score >= to 5, jump to .LBB2_2
lea     rax, [rip + .L__unnamed_2] ; "🍌"
ret                                     ; ⬅️ BACK Return

.LBB2_2:

cmp     dil, 9      ; ⚖️ Compare the input score with 9
ja      .LBB2_5     ; ➡️ If score is > 9, jump to .LBB2_5
lea     rax, [rip + .L__unnamed_3] ; "😬"
ret                                     ; ⬅️ BACK Return

.LBB2_5:

cmp     dil, 20
jae     .LBB2_7

```

```

lea    rax, [rip + .L__unnamed_4] ; "😬"
ret

.LBB2_7:
cmp    dil, 30
jae    .LBB2_9
lea    rax, [rip + .L__unnamed_5] ; "😞"
ret

.LBB2_9:
cmp    dil, 40
jae    .LBB2_11
lea    rax, [rip + .L__unnamed_6] ; "😞"
ret

.LBB2_11:
cmp    dil, 50
jae    .LBB2_13
lea    rax, [rip + .L__unnamed_7] ; "😬"
ret

.LBB2_13:
cmp    dil, 60
jae    .LBB2_15
lea    rax, [rip + .L__unnamed_8] ; "😞"
ret

.LBB2_15:
cmp    dil, 70
jae    .LBB2_17
lea    rax, [rip + .L__unnamed_9] ; "😬"
ret

.LBB2_17:
cmp    dil, 80
jae    .LBB2_19
lea    rax, [rip + .L__unnamed_10] ; "😬"
ret

.LBB2_19:
cmp    dil, 90
jae    .LBB2_21
lea    rax, [rip + .L__unnamed_11] ; "😬"
ret

.LBB2_21:
xor    edx, edx ; 0 Clear the register edx

```

The following instructions set the correct length of the emoji string.

- Compare the input score with 101. Set the carry flag if `dil` is less than 101. Otherwise, clear the carry flag.
- If the carry flag is 1, the final length is 4 (length of "😬").

Part B Control Structures and Enums in Assembly

- If the carry flag is 0, the final length is 3 (length of “X”).

```
cmp    dil, 101      ; ⚖️ Compare the input score with 101
adc    rdx, 3        ; ✅ Add 3 and the carry flag to rdx
```

The following instructions prepare for the `cmovb` instruction.

- Set the comparison flags based on the comparison between the input score and 101.
- Set the `rcx` and `rax` registers to the addresses of the “😄” and “X” emoji strings, respectively.

```
cmp    dil, 101      ; ⚖️ Compare input score with 101 again
lea    rcx, [rip + .L__unnamed_12] ; Load the address of "😄" into rcx
lea    rax, [rip + .L__unnamed_1]  ; Load the address of "X" into rax
```

🔗 If the score is below 101 (i.e., within range), move the address in `rcx` (for “😄”) to `rax`. Otherwise, `rax` stays as “X”.

```
cmovb  rax, rcx
ret                                ; ⬅️ BACK Return the final emoji string
```

If-Else Expression

Rust features an `if-else` expression for branching based on a condition. The following example demonstrates using an `if-else` expression to determine a point’s quadrant in a Cartesian plane. Note that the `if` and `else` blocks act as expressions. The executed block’s output becomes the expression’s returned value.

```
pub fn get_quadrant(x: i64, y: i64) -> u8 {
    if x >= 0 && y >= 0 {
        1
    } else if x < 0 && y > 0 {
        2
    } else if x < 0 && y < 0 {
        3
    } else if x > 0 && y < 0 {
        4
    } else {
        0
    }
}
```

The function accepts two parameters, `x` and `y`, which it stores in the `rdi` and `rsi` registers, respectively. It stores the return value in the `rax` (`al`) register. Observe how the assembly code optimizes performance by employing bitwise operations to minimize branching.

- `12 34` Input: `rdi = x, rsi = y`
- `12 34` Output: `rax (al) = quadrant or 0`

example::get_quadrant:

```
; 1 If x is positive and y is positive, then the quadrant is 1
; 1 Set the quadrant to 1 by default
mov    al, 1
```

```
;  Move x into rcx
mov     rcx, rsi
```

 OR the x and y values to set the sign bit in rcx to 1 if x, y, or both are negative. The sign bit, the most significant bit in a signed integer, indicates the sign of the number.

Following this, the code checks the sign bit to determine the quadrant. If the sign bit is set, it indicates that either x, y, or both are negative. The code jumps to handle cases where x, y, or both are negative. If the sign bit is not set, the code returns the quadrant number in al.

```
or      rcx, rdi

;  If the sign bit is set, then either x, y, or both are negative.
; Jump to handle cases where x, y, or both are negative.
js      .LBB0_1

.LBB0_6:
;  Return the quadrant number in al
ret

.LBB0_1:

;  If x is negative and y is positive, then the quadrant is 2
;  Set the sign flag (SF) to 1 if x is negative
test    rdi, rdi

;  Jump if x is positive (not signed)
jns     .LBB0_3

;  Set the quadrant to 2
mov     al, 2

;  Set SF=0 if y is positive
test    rsi, rsi

;  Jump if y is greater than 0
jg      .LBB0_6

.LBB0_3:
```

 The test instruction sets the condition flags based on the result of the bitwise AND operation between the two operands. In this case, the operands are rsi and rdi. So, the signed bit in the condition flags will be set to 1 if both rsi (y) and rdi (x) are negative.

After the test instruction, the code checks the sign bit to determine the quadrant. If the sign bit is set, it indicates that both x and y are negative. The code jumps to handle the case where both x and y are negative. If the sign bit is not set, the code checks the signs of x and y to determine the quadrant.

```
test    rsi, rdi

;  Jump if x and y are both negative
js      .LBB0_4

;  Check for the sign of y
```

Part B Control Structures and Enums in Assembly

```
test    rsi, rsi

; 🚩 1 Set cl to 1 if y is negative
sets    cl

; 🔍 Check for the sign of x
test    rdi, rdi

; 🚩 1 Set al to 1 if x is greater than 0
setg    al

; ✅ al is set to 1 if x is positive and y is negative
; ❌ al is set to 0 if x is negative and y is positive
and     al, cl

; ✖ Multiply al by 4 to get the quadrant number
shl     al, 2

; ⬅️ Return the quadrant number in al
ret

.LBB0_4:

; 3 x and y are both negative, so the quadrant is 3
mov     al, 3

; ⬅️ Return the quadrant number in al
ret
```



Experiment: Reorder Match Ranges and If-Else vs Match

Experiment using [Compiler Explorer](https://godbolt.org/z/MK7G86Kvs) (<https://godbolt.org/z/MK7G86Kvs>) to observe changes in the generated assembly code when you change the `get_emoji_from_0_to_100` and `get_quadrant` functions.

Reorder Ranges

To observe how the generated assembly code changes based on the order of ranges in a match expression, you can add the provided code to the code pane in Compiler Explorer. The code includes reordered ranges, with the most common ranges checked first. By generating the code, you can see that the compiler follows the order of the ranges in the match expression.

```
pub fn get_emoji_from_0_to_100_rearranged(score: u8) -> &'static str {
    match score {
        50..=59 => "😞", // Moved
        80..=89 => "😄", // Moved
        90..=100 => "😁", // Moved
        0..=4 => "😱",
        5..=9 => "😞",
        10..=19 => "😞",
        20..=29 => "😞",
        30..=39 => "😞",
```



```

40..=49 => "😬",
60..=69 => "😬",
70..=79 => "😬",
_ => "❌",
}

```

A key takeaway from this experiment is that positioning the most common ranges at the top of the match expression improves performance by reducing the number of branches.

Discontinuous Ranges

You can add the provided function to the code pane in Compiler Explorer to demonstrate how the compiler handles missing ranges in a match expression. The function includes commented-out ranges to create a discontinuous match range. When you generate the code, you will see that the compiler maps the missing ranges to the default case.

```

pub fn get_emoji_from_0_to_100_discontinuous(score: u8) -> &'static str {
    match score {
        0..=4 => "😬",
        5..=9 => "😬",
        10..=19 => "😬",
        // 20..=29 => "😬",
        30..=39 => "😬",
        40..=49 => "😬",
        // 50..=59 => "😬",
        60..=69 => "😬",
        70..=79 => "😬",
        // 80..=89 => "😬",
        90..=100 => "😬",
        _ => "❌",
    }
}

```

Converting If-Else to Match

Let us examine the conversion of the if-else example into a match expression. We have rewritten the `get_quadrant` function using a match expression in the `get_quadrant_match` function, resulting in enhanced readability compared to the original `get_quadrant` function.

Add the following code to the code pane in Compiler Explorer to see the generated assembly code.

```

pub fn get_quadrant_match(x: i64, y: i64) -> u8 {
    match (x >= 0, y >= 0) {
        (true, true) => 1,
        (false, true) => 2,
        (false, false) => 3,
        (true, false) => 4,
    }
}

```

Part B Control Structures and Enums in Assembly

The assembly code produced for `get_quadrant_match` is significantly more optimized than `get_quadrant`. The code for `get_quadrant_match` is devoid of branches. Bitwise operations and conditional moves manage the comparisons entirely.

```
; Inputs: rdi = x, rsi = y
example::get_quadrant_match:

xor     eax, eax           ; Set eax to 0
test    rsi, rsi           ; Set SF=0 if y >= 0
setns   cl                 ; Set cl to 1 if y < 0, 0 otherwise
sets    al                 ; Set al to 1 if y >= 0, 0 otherwise
xor     cl, 3              ; Flip the value of cl (1 -> 2, 0 -> 3)
lea     edx, [rax + 2*rax] ; Multiply al by 3 and store in edx
inc     edx                ; Increment edx by 1
test    rdi, rdi           ; Set SF=0 if x >= 0
movzx   eax, cl            ; Move cl to eax and zero-extend
cmovns  eax, edx           ; If x < 0, move edx to eax
ret                                ; Return the value in eax
```

Learn More: Exhaustiveness Checking

The match expression in Rust enhances performance and readability and ensures safety through its exhaustiveness requirement. The compiler issues a compilation error if a match expression fails to cover all possible cases. Furthermore, the compiler identifies and warns about redundant match arms that will never execute.

When we converted `get_quadrant` to `get_quadrant_match`, the compiler issued a warning indicating the redundancy of the default leg returning 0. The compiler, employing tuple match arm patterns, determined the match expression's exhaustiveness and identified the default leg as redundant.

Optimizing the Code by Changing the Quadrant Numbering

When you look at the `get_quadrant_match` function, the quadrant number as a binary number, we see that it does not align with the tuple match arms.

Let us experiment with the quadrant numbering that aligns with the match arms. Refer to the following table for the quadrant numbering:

Quadrant	Quadrant in binary	Tuple match arm
3	11	(true, true)
1	01	(false, true)
0	00	(false, false)
2	10	(true, false)

Here is the modified function, with the quadrant numbering changed to align with the tuple match arms.

Our hunch is that the compiler will generate better code for this function. Let us see if that is indeed the case. Add the following code to the code pane in Compiler Explorer to see the generated assembly code.

```
pub fn get_quadrant_match_optimized(x: i64, y: i64) -> u8 {
    match (x >= 0, y >= 0) {
        (true, true) => 3,
        (false, true) => 1,
        (false, false) => 0,
        (true, false) => 2,
```

```

    }
}

```

The generated code outperforms the already efficient match version. The compiler has removed some instructions previously used to map the tuple arms to the quadrant numbers. Can you identify those instructions in the `get_quadrant_match` function?

```

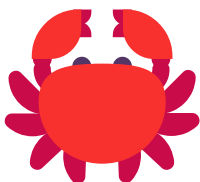
; Inputs: rdi = x, rsi = y
example::get_quadrant_match_optimized:

xor     ecx, ecx           ; Set ecx to 0
test    rsi, rsi           ; Set SF if y >= 0
setns   cl                ; Set cl to 1 if y < 0, 0 otherwise
lea     eax, [rcx + 2]      ; Add 2 to cl and store in eax
test    rdi, rdi           ; Test if x >= 0
movzx   eax, al            ; Move al to eax and zero-extend
cmovs   eax, ecx           ; If x < 0, move ecx to eax
ret                                           ; Return the value in eax

```

Key Takeaways

- If the number of matches is large and continuous, the compiler generates a lookup table to implement the match expression. Implementing a match expression this way is efficient, as it is a constant time operation.
- Implementing range-based match expressions involves a series of conditional jumps, where the number of jumps matches the number of ranges in the match expression. This implementation leads to a linear time operation.
- Arrange the match expression to check the most common ranges first for better performance. Doing this means the program checks the most used branches first, which cuts down on the total number of checks needed.
- Our experiments showed that formulating the `x` and `y` checks as a tuple match arm resulted in better code generation in the `get_quadrant_match`. The compiler removed branch instructions and used bitwise operations and conditional moves to implement the match expression.
- Recasting the problem with a match expression revealed an additional optimization opportunity. By synchronizing tuple matching with the quadrant numbering, we achieved enhanced code generation for the `get_quadrant_match_optimized` function.



Chapter 4

Deep Dive into Enums: Layout and Patterns

In this chapter, we will learn about enum representation in memory. We will go under the hood and see how the Rust compiler generates code for enums and pattern matching. We will also learn to use enums and pattern matching to write efficient code.

We will look at the following examples:

- [Pattern Matching an Enum](#)
- [Using Null Pointers as Discriminators](#)
- [Discriminator Size in a Small Enum](#)

Pattern Matching an Enum

Enums in Rust are discriminated unions that can hold one of several variants. The enum discriminator determines the current interpretation of the discriminated union.

The following code shows a simple enum in Rust that represents a generalized `Number` that can be an `Integer`, a `Float`, or a `Complex`. Here, `Number` is a container that can store a 64-bit integer (`i64`), a 64-bit floating-point number (`f64`), or a complex number (stored in a struct with two `f64` fields).

The code that follows the enumeration definition introduces a function named `double`. This function takes a parameter of type `Number` and outputs another `Number`, doubling each field of the input `Number` regardless of the specific `Number` variant from the enum. The `match` expression is employed within this function to discern the enum variant and generate the corresponding doubled variant.

```
pub enum Number {
    Integer(i64),
    Float(f64),
    Complex { real: f64, imaginary: f64 },
}

pub fn double(num: Number) -> Number {
    match num {
        Number::Integer(n) => Number::Integer(n + n),
        Number::Float(n) => Number::Float(n + n),
        Number::Complex { real, imaginary } => Number::Complex {
```

Part B Control Structures and Enums in Assembly

```
        real: real + real,
        imaginary: imaginary + imaginary,
    },
}
}
```

Memory Layout of a Rust Enum

Before proceeding further, let us look at the enum organization in memory. The size of the enum depends upon the largest variant. In this example, a `Number::Complex` requires two 64-bit floats. Hence, the total memory needed for the variant is 16 bytes. The enum occupies 24 bytes, with an extra 8 bytes dedicated to storing a 64-bit discriminator that identifies the variant currently saved in the enum.

The following table shows the memory layout of the `Number` enum.

Byte offset	Integer	Float	Complex
0	Discriminator (0)	Discriminator (1)	Discriminator (2)
8	i64	f64	f64
16			f64

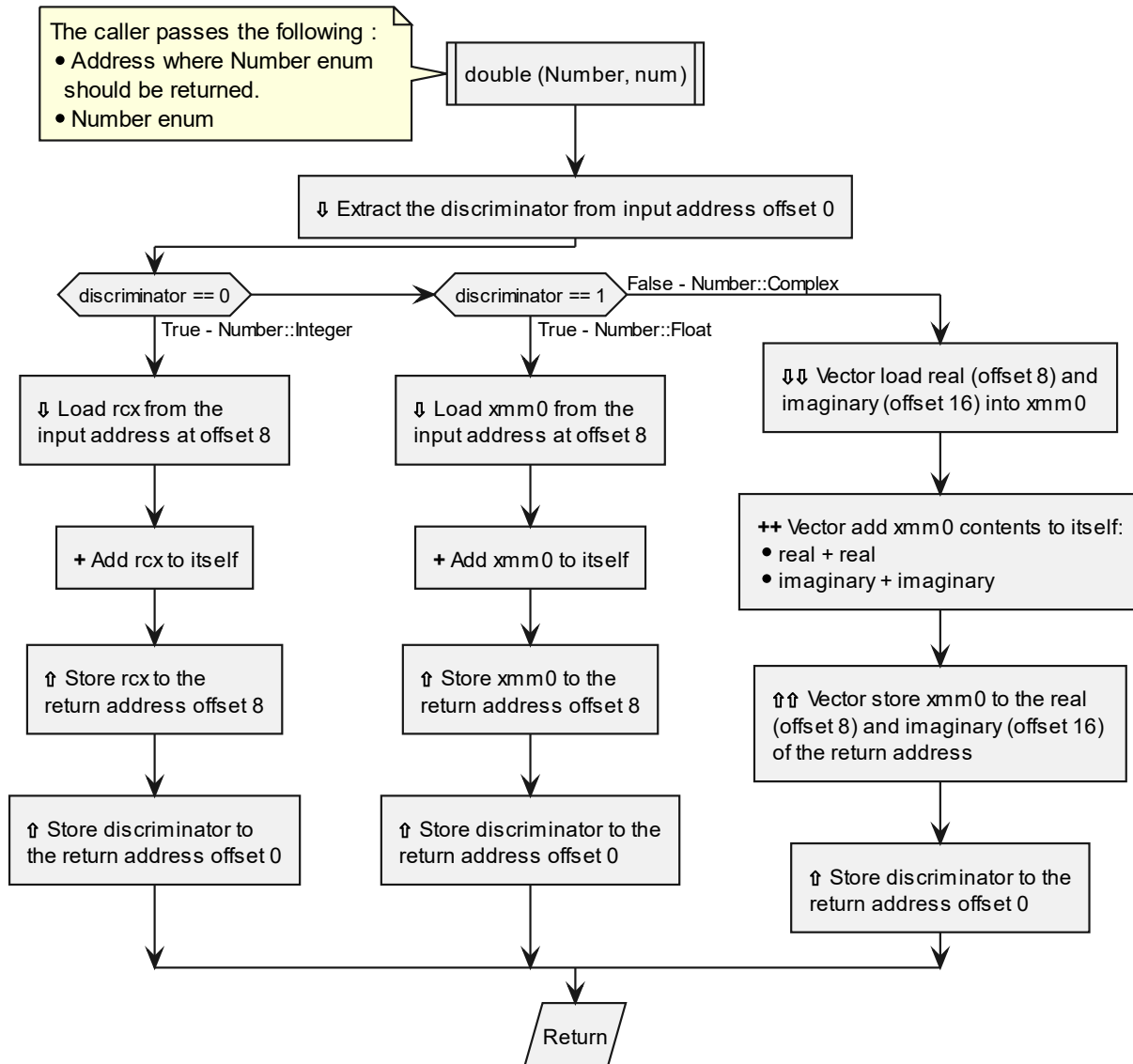
💡 Learn More: Enum Discriminator Size

A 64-bit discriminator seems wasteful here. Due to the padding rules, a smaller discriminator would not have saved any memory. Rust does switch to a smaller discriminator when doing so will save space.

Rust can also employ the Null pointer optimization to save space. For instance, in an `Option<Box<T>>`, the null pointer represents a `None`, while a pointer to the `T` value represents `Some`. No discriminator is necessary in this scenario. The [Option Wrapping a Box Type](#) section discusses this optimization.

Overview of the Generated Code

Before we dig deep into the assembly, let us get an overview of the generated code via the following flowchart. The code branches based on the enum discriminator and handles the processing of each enum discriminator separately. It then writes the results and discriminator values at the provided return address.



Deep Dive into the Generated Code

Now that we understand the basic flow, let us analyze the generated assembly code. The generated code examines the discriminator and then accesses the fields corresponding to selected variants. It doubles the individual fields associated with the variant and returns the enum with doubled values from the function. Additionally, the function copies the discriminator field to the returned enum.

In the generated assembly, you will notice that the processing of `Number::Complex` occurs using vector instructions. These instructions simultaneously operate on the real and imaginary fields.

- Input parameters:
 - `rdi`: The caller specified address where the called function should update the output enum
 - `rsi`: Address of the enum
- Output parameters:
 - `rax`: Output enum address (same as `rdi` passed by the caller)

example::double:

```
mov    rax, rdi ; rax now contains the address
```

Part B Control Structures and Enums in Assembly

```

; to the return value

mov    rcx, qword ptr [rsi] ; ⬇ Extract the union discriminator
test   rcx, rcx             ; 🔍 Check if the discriminator is 0
; (Number::Integer)

je     .LBB0_5              ; ➡ Jump if the discriminator is 0.

cmp    ecx, 1               ; ⚖ Check if the discriminator is 1
; (Number::Float).

jne    .LBB0_3              ; ➡ Jump if discriminator is 2
; (Number::Complex)

```

★ Number::Float match processing (discriminator is 1)

```

movsd   xmm0, qword ptr [rsi + 8] ; ⬇ Move the floating-point
; number in xmm0

addsd   xmm0, xmm0              ; + Double the number

movsd   qword ptr [rax + 8], xmm0 ; ⬆ Save the number to the return value

mov     qword ptr [rax], rcx     ; ⬆ Copy the discriminator
; into the return value

ret                                           ; ⬅ BACK Return the enum address in rax

```

★ Number::Integer match processing (discriminator is 0)

```

.LBB0_5:

mov     rdx, qword ptr [rsi + 8] ; ⬇ Move the integer

add     rdx, rdx                 ; + Double the number

mov     qword ptr [rax + 8], rdx ; ⬆ Write the number to the return value

mov     qword ptr [rax], rcx     ; ⬆ Write the discriminator

ret                                           ; ⬅ BACK Return the enum address in rax

```

★ Number::Complex match processing (discriminator is 2)

The code loads, doubles, and stores back the 64-bit real and imaginary parts using vector instructions that execute both operations in parallel.

```

.LBB0_3:

movupd  xmm0, xmmword ptr [rsi + 8] ; ⬇⬇ Read the real and
; imaginary parts into xmm0

```



```

addpd    xmm0, xmm0                ; ++ Double the real and
                                   ; imaginary parts

movupd   xmmword ptr [rax + 8], xmm0 ; ↑↑ Update the real and
                                   ; imaginary parts

mov      qword ptr [rax], rcx      ; ↑ Set the discriminator
                                   ; in the return value

ret                                             ; ← Return enum address in rax

```

Using Null Pointers as Discriminators

Let us focus on the `Option` enum from the standard library. The `None` variant does not have any fields. The `Some` variant contains type `T`. Here is the definition of the `Option` enum.

```

pub enum Option<T> {
    None,
    Some(T),
}

```

Rust eliminates the discriminator field when an `Option` contains a reference type. In such cases, the `Option` stores a pointer to the referenced type. If the pointer is `NULL`, the `Option` is `None`. If the pointer is not `NULL`, the `Option` is `Some`. We will look at two examples to understand how this works.

- [Option Wrapping a Reference Type](#)
- [Option Wrapping a Box Type](#)

Option Wrapping a Reference Type

Let us look at an example where `Option` wraps a `&u64`. `&u64` is a pointer to a 64-bit integer.

```

pub fn map_to_number(option: Option<&u64>) -> u64 {
    match option {
        Some(value) => *value,
        None => 0,
    }
}

```

The generated assembly code follows. The code checks if the address in `rdi` is `NULL`. If it is `NULL`, the code returns 0. If it is not `NULL`, the code loads the value from the address and returns it.

- Input:
 -  `rdi`: `Option<&u64>` address
- Output:
 -  `rax`: `u64` value

example::map_to_number:

```

; 🕒 Check if the address in rdi is NULL (This corresponds to None)
test     rdi, rdi

; ➡ Jump to .LBB0_1 if the address is NULL (None)
je       .LBB0_1

```

Part B Control Structures and Enums in Assembly

```

; [D] If the address is not NULL, load the value from the address in rax
mov     rax, qword ptr [rdi]

; [BACK] Return the value in rax
ret

.LBB0_1:

; [0] The address is NULL (None), return 0
xor     eax, eax

; [BACK] Return the value in rax
ret

```

Option Wrapping a Box Type

Consider the `Option<Box<i32>>` type, where `Box<i32>` represents a pointer to a 32-bit integer allocated on the heap. We will observe how the `Box` pointer functions as a discriminator to ascertain whether the `Option` is `None` or `Some`.

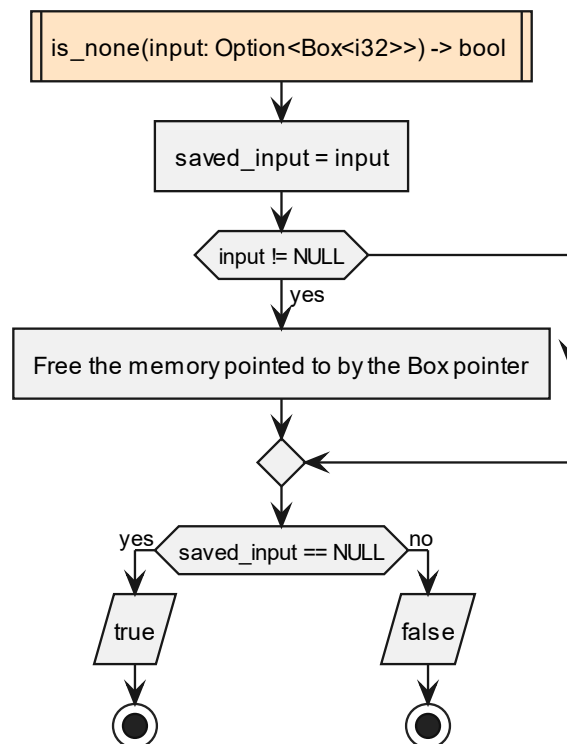
The `is_none` function accepts an `Option<Box<i32>>` as input and returns `true` if the input is `None` and `false` otherwise. In this scenario, the function invokes the `is_none` method for `Option`.

```

pub fn is_none(input: Option<Box<i32>>) -> bool {
    input.is_none()
}

```

The following flowchart illustrates the generated assembly for the `is_none` function. The function checks if the input is `NULL`. If it is `NULL`, the function returns `true`. The function deallocates the memory and returns `false` if it is not `NULL`.



The code generated for the function follows. The flow of the code is like the preceding flowchart. The point to note here is that the `sete` instruction sets the return value to `true` or `false`. The `sete` instruction sets the destination operand to 1 if the Zero flag is set. Otherwise, it sets the operand to 0.

- Input:
 -  `rdi: Option<Box<i32>>`
- Output:
 -  `rax: bool`

```
example::is_none:
push    rbx
mov     rbx, rdi

;  This operation tests if the pointer is NULL.
test    rdi, rdi
```

★ If the result of the test operation was zero (NULL, indicating None), jump to `.LBB1_2`, skipping the `dealloc` part as there is no memory to deallocate.

```
je      .LBB1_2
```

✂ Execute the following block if the pointer is not NULL (indicating Some). The code deallocates the memory pointed to by the `Box` pointer. We will learn more about `Box` memory management in the “*Arguments: Value, Reference, Box, Rc, and Arc*” chapter.

```
; Size to deallocate: 4 bytes (size of i32).
mov     esi, 4

; Alignment to use: 4 bytes (alignment of i32).
mov     edx, 4

; Pointer to deallocate: The Box<i32> pointer is stored in rbx.
mov     rdi, rbx

;  Call the dealloc function to free the memory.
call    qword ptr [rip + __rust_dealloc@GOTPCREL]
```

★ Common code for both `Some` and `None` cases.

```
.LBB1_2:

;  Perform a bitwise AND operation on rbx (which contains the initial
; pointer). This operation tests if the pointer is NULL (None).
test    rbx, rbx

;  Set the al register (lowest byte of the rax register, which stores
; function return values) to 1 if the result of the test operation was
; zero (NULL, indicating None), and 0 otherwise (Some).
sete    al

pop     rbx

;   Return from the function. The return value is in the rax register,
```

Part B Control Structures and Enums in Assembly

; or more specifically, its lowest byte al.

ret

Discriminator Size in a Small Enum

Consider the following example. The enum Point has two variants: Spherical and Cartesian. Note the type aliases we have defined for the fields. The enum implements the extract_cartesian and extract_spherical functions that return the fields of the enum variants. This section will look at the code generated for the extract_cartesian function.

We will compare the code for extract_cartesian and extract_spherical functions when we [experiment](#) with different type aliases that define the fields of Point.

```
type Coord = i8;
type Distance = i8;
type Angle = i8;

pub enum Point {
    Spherical {
        r: Distance,
        theta: Angle,
        phi: Angle,
    },
    Cartesian {
        x: Coord,
        y: Coord,
        z: Coord,
    },
}

impl Point {
    pub fn extract_cartesian(&self) -> Option<(Coord, Coord, Coord)> {
        if let Point::Cartesian { x, y, z } = self {
            Some((*x, *y, *z))
        } else {
            None
        }
    }

    pub fn extract_spherical(&self) -> Option<(Distance, Angle, Angle)> {
        if let Point::Spherical { r, theta, phi } = self {
            Some((*r, *theta, *phi))
        } else {
            None
        }
    }
}
```

There are only 3 bytes of data in the Spherical or Cartesian variants. The enum discriminator is 1 byte. The total size of the enum is 4 bytes. The layout of the enum follows.

Byte offset	Spherical	Cartesian
0	Discriminator (0)	Discriminator (1)
1	r	x

Byte offset	Spherical	Cartesian
2	theta	y
3	phi	z

The code generated for the `extract_cartesian` function follows. From the code, we can infer that the first byte of the enum stores the discriminator.

- Input:
 - `rdi: &self`
- Output:
 - `rax: Option<(Coord, Coord, Coord)>`

`example::Point::extract_cartesian:`

```
; ⚖️ If enum discriminator in rdi is 0 => Spherical variant.
; If it is 1 => Cartesian variant
cmp     byte ptr [rdi], 0
je      .LBB0_1          ; Jump if equal, i.e., Spherical variant.

; If here, the variant is Cartesian, and we are extracting x, y, and z.

movzx   eax, byte ptr [rdi + 1] ; ⬇ Load x into eax.
movzx   ecx, byte ptr [rdi + 2] ; ⬇ Load y into ecx.
movzx   edx, byte ptr [rdi + 3] ; ⬇ Load z into edx.
```

🟡 Combine the enum discriminator, x, y, and z into one value in `eax`. Since the Intel architecture follows the little-endian format, it reverses the contents of registers to align with the Rust enum layout. The generated code stores the discriminator in the lowest byte of the 32-bit register and places the x, y, and z values in the next three bytes of the register in reverse order.

```
shl     edx, 24          ; ⬅⬅⬅ Shift z 24 places left.
shl     ecx, 16          ; ⬅⬅ Shift y 16 places left.
shl     eax, 8           ; ⬅ Shift x 8 places left.

or      eax, ecx          ; + Combine shifted y into eax.
add     eax, edx          ; + Add shifted z into eax.

; 1 Increment eax by 1 to set the discriminator to 1 (Some).
inc     eax

ret ; ⬅ BACK eax holds the return value for Some((x, y, z)).
```

☐ Handling the `None` variant is straightforward. The code sets `eax` to 0 and returns.

`.LBB0_1:`

```
xor     eax, eax          ; ☐ Zero out eax, representing None in Rust.

ret     ; ⬅ BACK Return, eax holds the return value for None.
```



Experiment: Impact of Enum Size and Type on Discriminator Size

We will conduct the following experiments:

- [Change the Enum to Use 32-bit Types](#)
- [Compact Ways to Map Option<&T> to T](#)
- [Change Type Aliases Used in the Point Enum](#)

Change the Number Enum to Use 32-bit Types

Let us experiment with the Number enum example in [Compiler Explorer](https://godbolt.org/z/eqTKs79fd) (<https://godbolt.org/z/eqTKs79fd>).

Change the Number enum to use 32-bit types as shown here. Confirm that the compiler now uses a 32-bit discriminator.

```
pub enum Number {
    Integer(i32),
    Float(f32),
    Complex { real: f32, imaginary: f32 },
}
```

Compact Ways to Map Option<&T> to T

The [Compiler Explorer](https://godbolt.org/z/za71ebE5Y) (<https://godbolt.org/z/za71ebE5Y>) shows the code generated for the is_none and map_to_number functions we saw earlier.

Let us copy over the following functions to Compiler Explorer code window. Compare the generated code for the map_to_number function with the generated code for the map_to_number_with_unwrap and map_to_number_with_map functions.

```
pub fn map_to_number_with_unwrap(option: Option<&u64>) -> u64 {
    *option.unwrap_or(&0)
}

pub fn map_to_number_with_map(option: Option<&u64>) -> u64 {
    option.map(|value| *value).unwrap_or(0)
}
```

Change Type Aliases Used in the Point Enum

Let us use the experiment with the Point enum using [Compiler Explorer](https://godbolt.org/z/Yb1qoTGr7) (<https://godbolt.org/z/Yb1qoTGr7>). Fill in the following table with the results of your experiments.

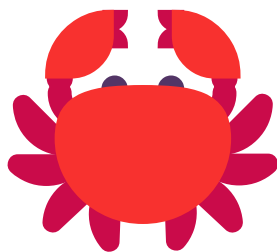
Coord	Distance	Angle	Size of enum	Size of discriminator	Offset of discriminator
i8	i8	i8	4 bytes	1 byte	0
i8	i16	i8			
i8	i8	i16			
f32	f64	f32			

Also, look at the generated code for the extract_cartesian and extract_spherical functions. See how the generated code changes with the change in the type aliases.



Key Takeaways

- Enums in Rust resemble unions in C, but they are safer because Rust enums maintain knowledge of the currently stored variant. Rust achieves this by adding a discriminator field to the enum.
- The size of the enum depends upon the largest variant. Our [experiments](#) with the `Point` enum demonstrate this.
- The size of the discriminator depends upon the range of variants in the enum. However, in most scenarios, the alignment requirements dictate the size of the discriminator.
- The generated assembly code for enum matching branches based on the discriminator and then processes the fields of the variant.
- The `Option` enum that wraps a reference type can take advantage of the fact that in Rust references are non-nullable. If the reference is `NULL`, then the enum is `None`. Otherwise, the enum is `Some` and the reference points to the value stored in the enum. No discriminator is needed in such cases. We saw this in the [Using Null Pointers as Discriminators](#) section.



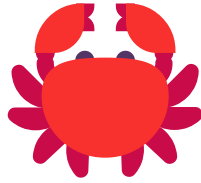
Part C

Data Structures and Memory Management



Rust Under the Hood Companion (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews



Chapter 5

Arguments: Value, Reference, Box, Rc, and Arc

This chapter will explore the performance implications of passing `self` by value, reference, and smart pointers (`Box`, `Rc`, and `Arc`). The generated assembly code will help us understand what happens under the hood.

We will be working with the `Complex` struct defined here. The code shows a struct and its associated method declarations. Note that, like Python, the `self` refers to the associated object as the first parameter in the method declaration.

```
use std::rc::Rc;
use std::sync::Arc;

type Num = f64;
const POWER: Num = 2.0;

#[derive(Copy, Clone)]
pub struct Complex {
    real: Num,
    imaginary: Num,
}

impl Complex {
    pub fn magnitude_self_copy(self) -> Num {
        (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
    }

    pub fn magnitude_self_reference(&self) -> Num {
        (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
    }

    // Passing smart pointers
    pub fn magnitude_self_box(self: Box<Self>) -> Num {
        (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
    }
}
```

Part C Data Structures and Memory Management

```

    }

    pub fn magnitude_self_rc(self: Rc<Self>) -> Num {
        (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
    }

    pub fn magnitude_self_arc(self: Arc<Self>) -> Num {
        (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
    }
}

```

The `self`-parameter in a method can specify the expected ownership model for the object. The following sections show `self` with different ownership models used in the methods associated with the `Complex` struct. The flowcharts and assembly code generated for each method will help us understand the performance implications of each ownership model.

We will cover the following ownership models for `self`:

- [Method Receives Self by Value](#)
- [Method Receives Self Reference &T](#)
- [Self Refers to Object on Heap via Box](#)
- [Method Receives Reference-Counted Smart Pointer Rc to Self](#)
- [Method Receives Atomic Reference-Counted Shared Reference Arc to Self](#)

Method Receives Self by Value

By default, passing `self` transfers the ownership of the parameter to the called function. In this example, however, the `Complex` type implements `Copy` and `Clone` traits. The compiler copies the content of the caller `Complex` struct.

```

pub fn magnitude_self_copy(self) -> f64 {
    (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
}

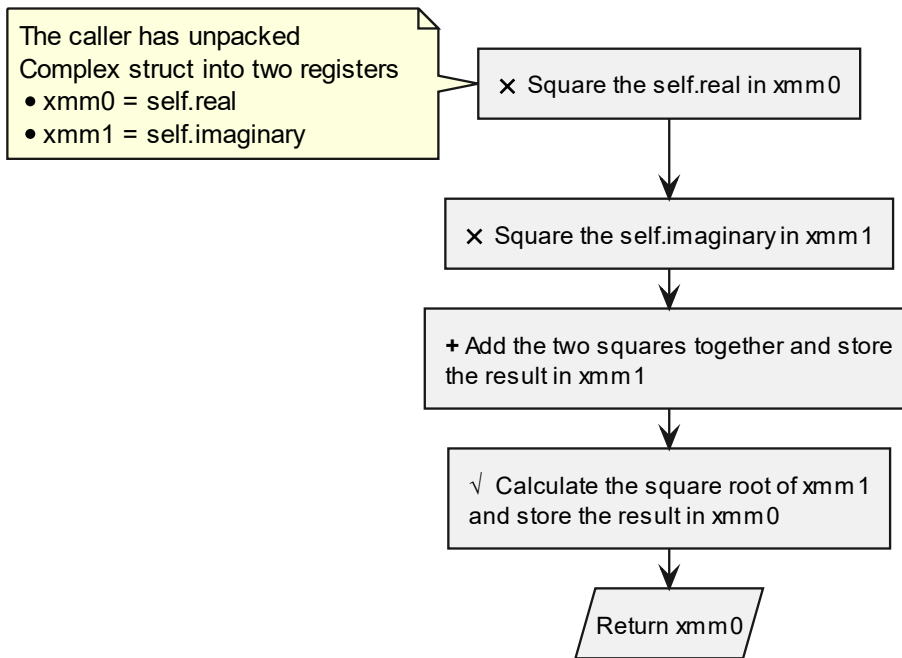
```

The following figure presents an example of a struct passed by value to the called function.

00 complex.real	1.0
08 complex.imaginary	2.0

Flowchart: Passing Self by Value

The flowchart of the generated assembly code follows.



Assembly Code: Passing Self by Value

The assembly code generated for the preceding function follows. One interesting thing to note here is that the compiler has optimized the passing of the Complex object by storing the `real` and `imaginary` fields in `xmm0` and `xmm1` registers, respectively. The method computes the result and returns it through the `xmm0` register, requiring no memory access since it stores both input and output in registers.

- 🏰 Input: `xmm0 = self.real`, `xmm1 = self.imaginary`
- 📦 Output: `xmm0 = result`

example::Complex::magnitude_self_copy:

```

mulsd  xmm0, xmm0 ; ✖ Square the real part
mulsd  xmm1, xmm1 ; ✖ Square the imaginary part
addsd  xmm1, xmm0 ; + Add the two squared numbers and store the result
                ; in xmm1
xorps  xmm0, xmm0 ; 0 Clear xmm0.
sqrtsd xmm0, xmm1 ; √ Perform the square root on the squared sum
                ; and store in xmm0.
ret    ; ← Return to the caller with the result in xmm0
  
```

Method Receives Self Reference &T

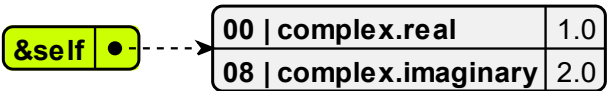
In the following method, the compiler passes the address of an immutably borrowed Complex object.

```

pub fn magnitude_self_reference(&self) -> f64 {
    (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
}
  
```

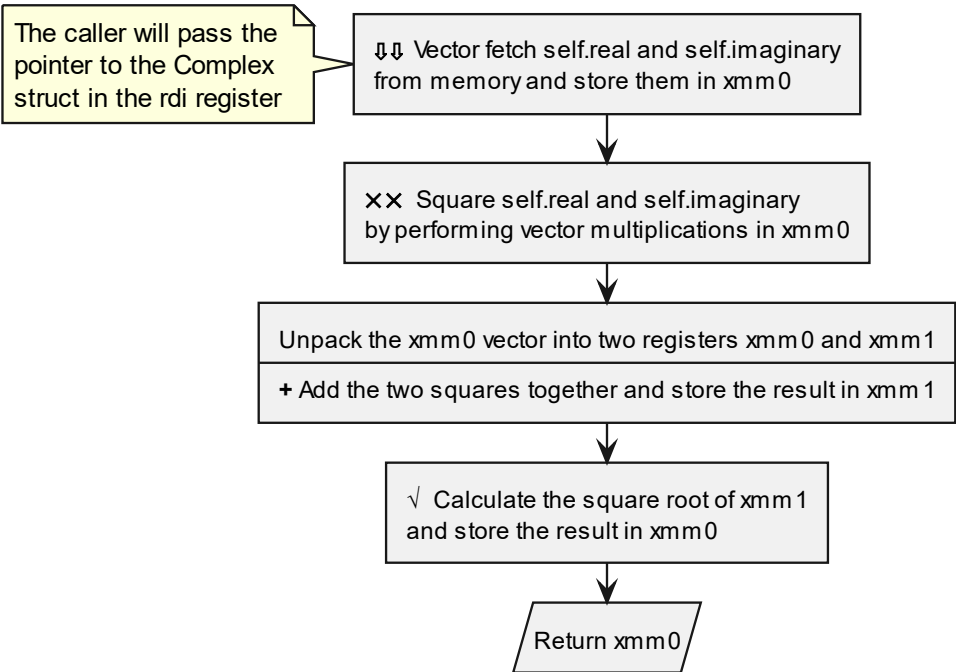
Part C Data Structures and Memory Management

Let us look at the representation of the `&self` and the `Complex` object in memory. The `&self` is a pointer to the `Complex` object. The method receives the `&self` pointer via the `rdi` register in the generated assembly.



Flowchart: Passing a Self Reference

The compiler has generated quite different code for the `&self` method than the `self` method. In the `&self` method, the compiler must copy the real and imaginary parts from the memory. The compiler, however, makes up for some of this performance loss by using vector operations to square and multiply the



numbers.

Assembly Code: Passing a Self Reference

- 👉 Input: `rdi = &self` (The caller will pass the pointer to the `Complex` struct in the `rdi` register.)
- 📦 Output: `xmm0 = result`

example::Complex::magnitude_self_reference:

```

movupd    xmm0, xmmword ptr [rdi] ; 📦📦 Vector load of real & imaginary
mulpd     xmm0, xmm0              ; ×× Square the real &
                                ; imaginary parts (vectorized)
movapd    xmm1, xmm0              ; Vector copy of squared real and
                                ; imaginary parts to xmm1
unpckhpd  xmm1, xmm0              ; 📦 Unpack real and imaginary parts
                                ; into xmm1 using a vector copy
                                ; and interleave operation
    
```

<code>addsd</code>	<code>xmm1, xmm0</code>	; + Add the real and imaginary parts
<code>xorps</code>	<code>xmm0, xmm0</code>	; Clear the complete xmm0 to 0
<code>sqrtsd</code>	<code>xmm0, xmm1</code>	; √ Square root of lower part of xmm1 -> xmm0
<code>ret</code>		; ← BACK Return result via xmm0

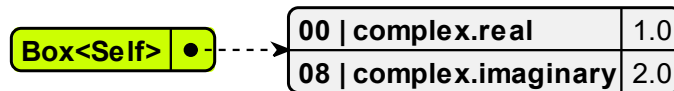
Self Refers to Object on Heap via Box

The Box smart pointer tracks objects on the heap. It functions similarly to the `unique_ptr` in C++ and points to an object on the heap. When you destroy the Box smart pointer, it destroys the heap object.

Let us look at the `magnitude_self_box` method. Here, the method passes a Box smart pointer to self. The Box contains a pointer to the Complex object stored on the heap.

```
pub fn magnitude_self_box(self: Box<Self>) -> f64 {
    (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
}
```

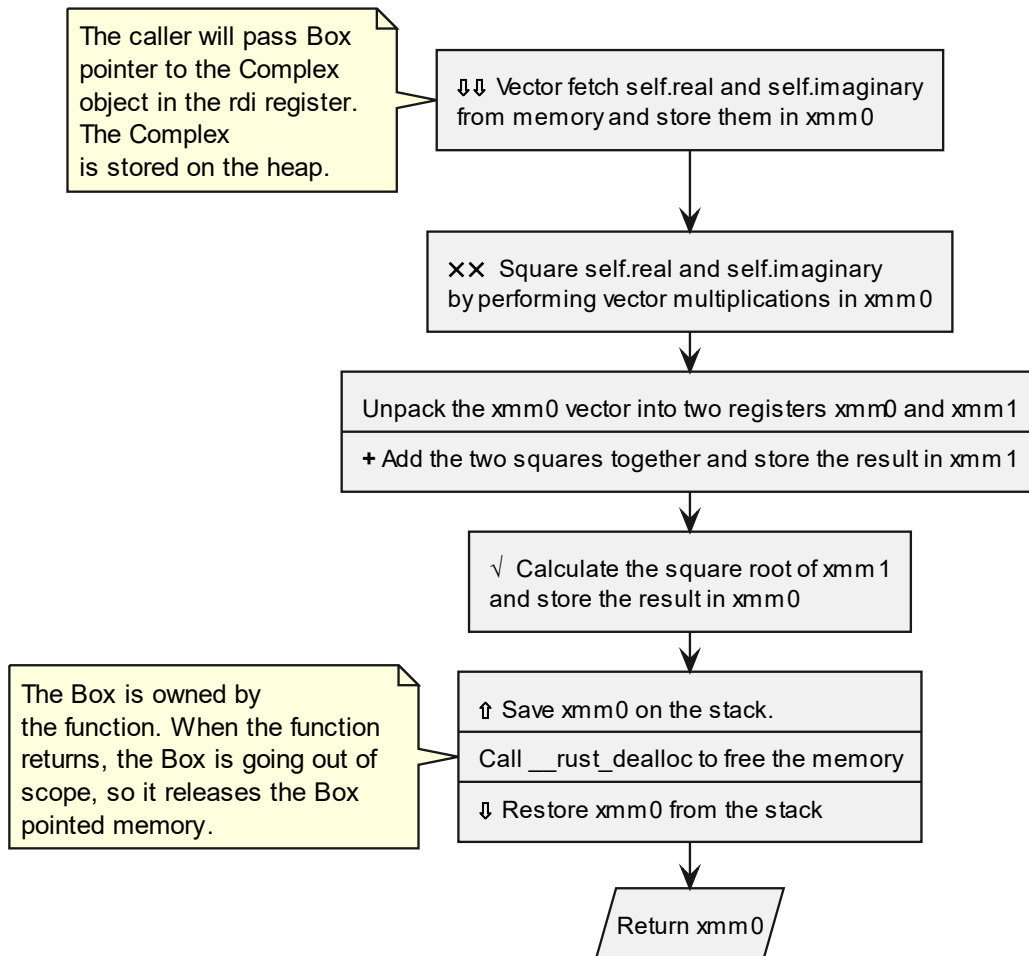
Before proceeding, we need to understand the memory layout of the Box pointer. It is 8 bytes long and points to the Complex object on the heap.



Flowchart: Passing a Box to Self

The flowchart resembles the `&self` case, with the main difference is due to Box handling. The function owns the Box smart pointer, and when it returns, the Box will go out of scope and free the memory allocated for the Complex object.

Part C Data Structures and Memory Management



Assembly Code: Passing a Box to Self

The generated assembly code looks like the &self case with the heap memory deallocation before the function returns.

- 📦 Input: rdi = Address of Box<Self>
- 📦 Output: xmm0 = result

example::Complex::magnitude_self_box:

```

push    rax                ; Save rax on the stack to align the stack
                        ; to 16 bytes - https://bit.ly/raxpush

movupd  xmm0, xmmword ptr [rdi] ; 📦 Vector load of the real & imaginary

mulpd   xmm0, xmm0         ; ✖✖ Square real & imaginary parts (SIMD)

movapd  xmm1, xmm0         ; Vector copy of the squared real &
                        ; imaginary parts to xmm1

unpckhpd xmm1, xmm0        ; 📦 Unpack the squared real & imaginary
                        ; parts into xmm1

addsd   xmm1, xmm0         ; + Add the squared real & imaginary parts
  
```



```

xorps    xmm0, xmm0           ; 0 Clear the complete xmm0 to 0
sqrtss   xmm0, xmm1           ; √ square root of lower part of xmm1 -> xmm0

```

📦 This method owns the Box, and as the function prepares to return, the Box is going out of scope and will soon be dropped. “Dropping” means freeing the heap memory allocated for the Complex object.

Note that the rdi register already points to the memory that needs freeing.

```

movsd    qword ptr [rsp], xmm0 ; ↑ Save xmm0 on the stack
mov      esi, 16                ; The size of the memory to free
                                      ; (Complex - 16 bytes)
mov      edx, 8                 ; The data is 8-byte aligned.

```

🔪 The parameters passed to the deallocation function are:

- rdi: Address of memory to free.
- esi: Size of memory to free.
- edx: Alignment of memory to free.

```

call     qword ptr [rip + __rust_dealloc@GOTPCREL] ; FREE 📦 Deallocate
movsd    xmm0, qword ptr [rsp] ; ↓ Restore the xmm0 from the stack.
                                      ; This is the return value.
pop      rax                      ; Dummy value restored from stack
ret                                             ; ← BACK Return the result in xmm0

```

Method Receives Reference-Counted Smart Pointer Rc to Self

Rust’s Rc (Reference Counted) smart pointer is a powerful tool for managing shared data ownership in a memory-safe and efficient manner. It allows multiple ownership of a value by keeping track of the number of references to it. Each time a new reference is created, the reference count increments, and it decrements when a reference is dropped. The code deallocates the value when the count reaches zero.

Now, consider the `magnitude_self_rc` method. Here, the method passes an Rc smart pointer to `self`, which contains a pointer to the Complex object stored on the heap.

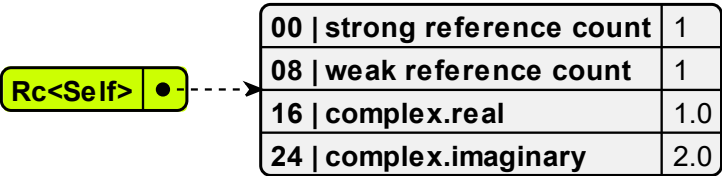
```

pub fn magnitude_self_rc(self: Rc<Self>) -> f64 {
    (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
}

```

The `magnitude_self_rc` method will share ownership to `self`. Multiple pointers to this object may be active in the same thread. When the method exits, it will decrement shared reference counts stored along with the Complex object. If this is the object's only reference, the object will be destroyed and released its memory to the heap; however, if the reference counts do not drop to zero, the object will continue to exist even after the method returns.

The following diagram illustrates the memory layout of the Complex object when passed as an Rc smart pointer to `self`. Creating an Rc allocates a memory block on the heap to store the reference count and the data. The Rc places the reference counts at the beginning of the memory block and the data after the reference counts. The Rc smart pointer then points to this memory block on the heap.



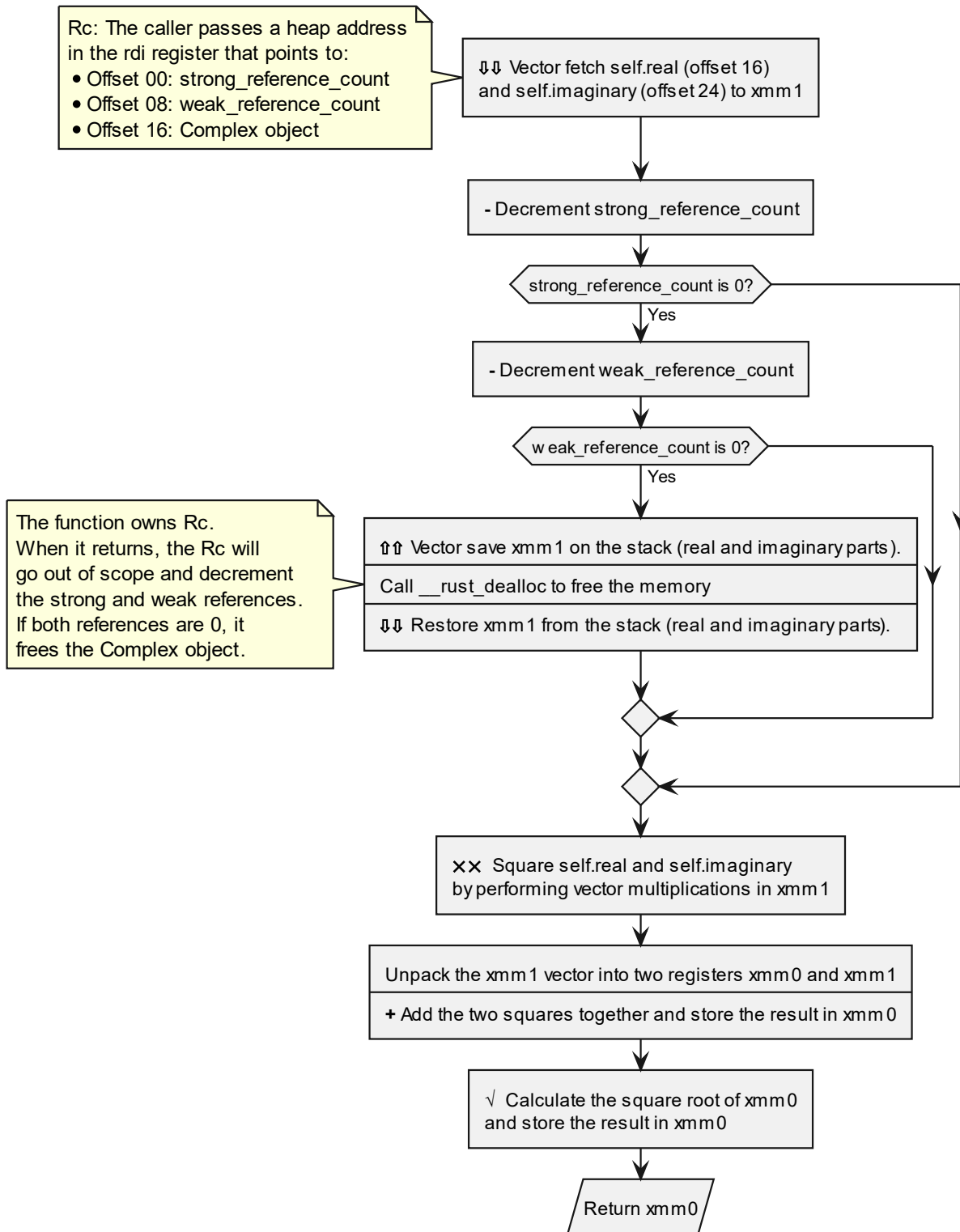
💡 Learn More: Strong and Weak References

The `Rc` (Reference Counted) smart pointer in Rust maintains two types of reference counts: strong and weak. The strong reference count tracks how many `Rc` pointers reference the same memory block. The weak reference count tracks the number of `Weak` pointers referencing that memory block.

You use a `Weak` pointer to prevent reference cycles, which occur when two objects reference each other, creating a cycle that prevents their reference counts from reaching zero. This situation can lead to memory leaks because the objects will never be deallocated. By using a `Weak` pointer, you can break this cycle. The `Weak` pointer does not increase the strong reference count, allowing the memory block to be deallocated when no `Rc` pointers remain, even if `Weak` pointers still exist.

Flowchart: Method Received Rc to Self

Let us look at the flowchart for `magnitude_self_rc` and how the function updates `Rc` reference counts and frees memory if the reference count drops to 0.



Assembly Code: Method Received Rc to Self

The generated assembly code resembles the `&self` case, with significant differences arising from handling Rc reference count decrements. Note the offsets for accessing the real and imaginary parts are 16 and 24, respectively, because two 64-bit reference counts precede the Complex object.

Once the method saves the real and imaginary parts in `xmm1`, it decrements the reference counts in preparation for going out of scope. If the reference count hits zero, the method deletes the object pointed to

Part C Data Structures and Memory Management

by the Rc. If not, the memory block containing the reference counts and Complex object will remain after the function returns.

Note: We have ignored the weak reference in this discussion.

-  Input: The caller passes a heap address in the `rdi` register that points to Rc with the following fields:
 - Offset 00: Strong reference count
 - Offset 08: Weak reference count
 - Offset 16: Complex object
-  Output: `xmm0` = result

```
example::Complex::magnitude_self_rc:
sub    rsp, 24                ; Reserve 24 bytes for local variables
movupd xmm1, xmmword ptr [rdi + 16] ;   Vector fetch real & imaginary
                                           ; parts of the struct from memory
dec    qword ptr [rdi]        ;  Decrement the strong reference
jne    .LBB4_3                ;  If not zero, skip deallocation
dec    qword ptr [rdi + 8]    ;  Decrement the weak reference
jne    .LBB4_3                ;  If not zero, skip deallocation
mov    esi, 32                ;  Size of the memory to free
                                           ; (Complex is 16 bytes) plus two 8-byte
                                           ; reference counters.
mov    edx, 8                 ; The data is 8-byte aligned.
movapd xmmword ptr [rsp], xmm1 ;   Vector save xmm1 on the stack
                                           ; (real & imaginary parts)
call   qword ptr [rip + __rust_dealloc@GOTPCREL] ;   Free the Rc memory
movapd xmm1, xmmword ptr [rsp] ;   Vector restore xmm1 from the
                                           ; stack (real & imaginary parts)
```

★ The method's core is the same as the `&self` case. Calculate the magnitude of the complex number and return the result in `xmm0`.

```
.LBB4_3:
mulpd   xmm1, xmm1 ;   Vector square the real and imaginary parts
movapd  xmm0, xmm1 ; Vector copy the squared real and imaginary
                                           ; parts to xmm0
unpckhpd xmm0, xmm1 ;  Unpack real & imaginary parts into xmm0
```

```

addsd    xmm0, xmm1 ; + Add the squared real and imaginary parts

sqrtsd   xmm0, xmm0 ; √ Square root of lower part of xmm0 -> xmm0

add      rsp, 24    ; Free the space saved for local storage

ret      ; ← BACK Return the result in xmm0

```

Method Receives Atomic Reference-Counted Shared Reference Arc to Self

Arc is a thread-safe smart pointer, like Rc, but it uniquely allows sharing among threads. Its implementation uses atomic reference counting, decrementing reference counts through atomic read-modify-write operations. This method ensures thread safety by decrementing the reference counts in a way that prevents concurrency issues.

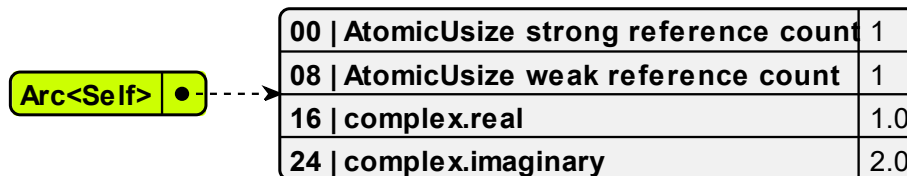
When passing an Arc to the method, let us examine the implementation of `Complex::magnitude_self_arc`. The method will now own the Arc smart pointer. As the method goes out of scope, it atomically decrements the shared reference counts stored with `Complex`. If these counts reach 0, the method will delete the object on the heap.

```

pub fn magnitude_self_arc(self: Arc<Self>) -> f64 {
    (self.real.powf(POWER) + self.imaginary.powf(POWER)).sqrt()
}

```

The Arc smart pointer points to a heap allocation containing `AtomicUsize` strong and weak references, with the `Complex` object stored after these two references (refer to the following figure for the memory representation).



💡 Learn More: 🧩 Atomic Operations

Atomic operations complete in a single step relative to other threads. No other thread can see the operation in a partly completed state, which makes them particularly useful for building multithreaded systems.

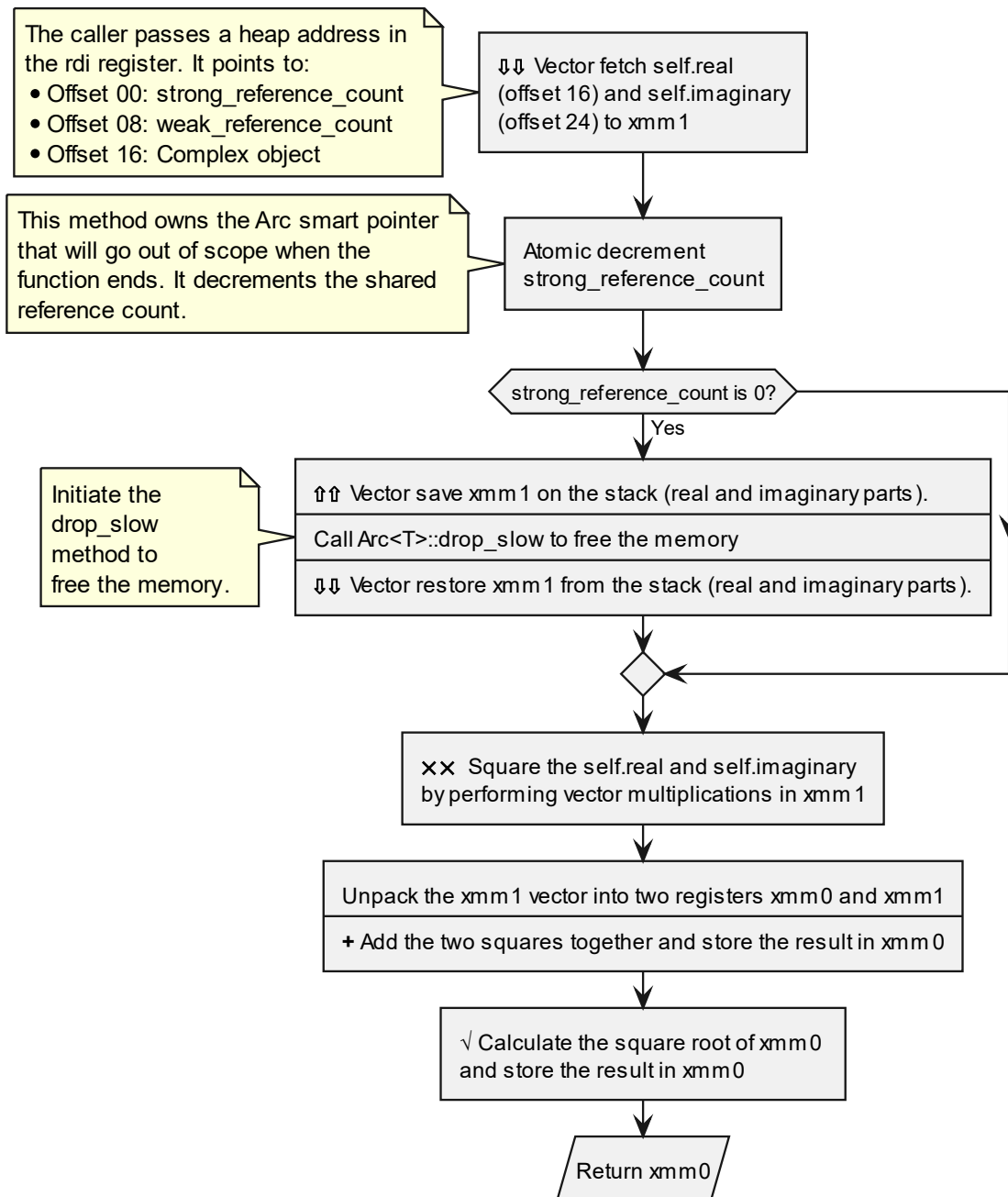
`AtomicUsize` in Rust is an atomic unsigned integer type that provides atomic operations for use in a lock-free programming style. It includes several methods to manipulate its value atomically, such as `fetch_add`, `fetch_sub`, `fetch_and`, `fetch_or`, `fetch_xor`, `load`, and `store`.

Special CPU instructions that guarantee atomicity typically implement these atomic operations at a low level.

On the x86-64 architecture, the “lock” prefix with various instructions implements these operations. This prefix guarantees the atomic execution of the instruction, ensuring that the processor maintains exclusive access to the memory location until the instruction completes.

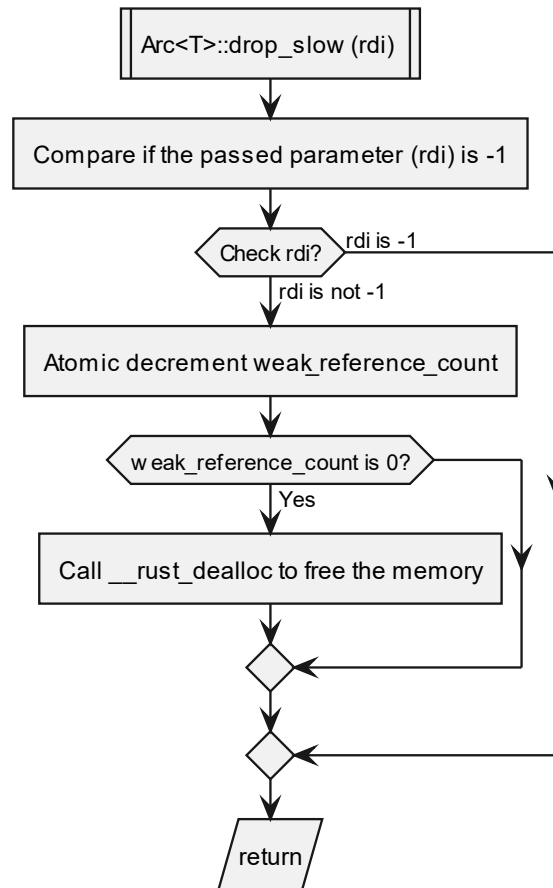
Flowchart: Method Receives Arc to Self

The following flowchart displays the steps when a method receives an Arc. This flowchart resembles the one we saw for Rc but uses atomic read-modify-write operations to decrement the reference counts. Additionally, the `drop_slow` function handles decrementing the weak reference count.



Drop Slow Utility Function

The `drop_slow` function activates when the strong reference count hits 0. It then decrements the weak reference count and verifies if it has reached 0. Should the weak reference count hit 0, the function frees the memory. The subsequent flowchart outlines the generated assembly.



Assembly Code: Method Receives Arc to Self

The code generated for Arc is like that for Rc. The significant differences from the Rc assembly code are:

- `lock sub qword ptr [rdi], 1` is generated to handle the atomic decrement of the reference count.
- The drop check and weak reference count decrement are handled in `alloc::sync::Arc<T>::drop_slow` function.

Let us look at the assembly code generated for the `magnitude_self_arc` method.

-  Input: The caller passes a heap address in the `rdi` register that points to Arc with the following fields:
 - Offset 00: Strong reference
 - Offset 08: Weak reference
 - Offset 16: Complex object
-  Output: `xmm0` = result

```
example::Complex::magnitude_self_arc:
```

```
sub    rsp, 24                ; Reserve for local variables
movupd xmm1, xmmword ptr [rdi + 16] ;   Fetch the real & imaginary
                                           ; parts of the struct from memory
```

Part C Data Structures and Memory Management

```

lock    dec    qword ptr [rdi]      ; 🔒 — Lock and perform an atomic
                                   ; decrement of the strong reference

jne     .LBB5_2                    ; ⚖️ If not 0, skip ahead & compute

movapd  xmmword ptr [rsp], xmm1    ; ⬆️ ⬆️ Save real & imaginary parts
                                   ; on stack

call    alloc::sync::Arc<T>::drop_slow ; Call the drop_slow function for
                                   ; delete processing.

movapd  xmm1, xmmword ptr [rsp]    ; ⬇️ ⬇️ Restore the real & imaginary
                                   ; parts from the stack.

```

★ The method's core is the same as the `&self` case. Calculate the magnitude of the complex number and return the result in `xmm0`.

```

.LBB5_2:

mulpd   xmm1, xmm1 ; ✖️ ✖️ Square real and imaginary parts

movapd  xmm0, xmm1 ; Vector copy the squared real and imaginary
                 ; parts to xmm0

unpckhpd xmm0, xmm1 ; 📦 Unpack real & imaginary parts into xmm0

addsd   xmm0, xmm1 ; ➕ Add the squared numbers and save them to xmm0

sqrtsd  xmm0, xmm0 ; √ Square root of lower part of xmm0 -> xmm0

add     rsp, 24    ; Free the space saved for local storage

ret     ; ⬅️ BACK Return the result in xmm0

```

When invoked, the `drop_slow` function, with `rdi` pointing to `Arc`, frees memory if the weak atomic reference count reaches 0.

Interestingly, the `__rust_dealloc` function undergoes tail call optimization. The `drop_slow` function jumps to the `__rust_dealloc` function. Upon returning from `__rust_dealloc`, the function returns directly to the caller of the `drop_slow` function.

We will dig deeper into tail call optimization in the “Traits, Vtables, and Tail Calls” chapter.

```

alloc::sync::Arc<T>::drop_slow:

cmp     rdi, -1      ; Check if the Arc block address is -1

je      .LBB0_2      ; If it is skip ahead and return.

lock    dec    qword ptr [rdi + 8] ; 🔒 🔒 — Perform an atomic decrement
                                   ; of the weak reference.

jne     .LBB0_2      ; If the weak reference is 0, proceed to free the Arc block

```



```

mov     esi, 32 ; Arc block size is 32 bytes
mov     edx, 8  ; Alignment is 8 bytes

; 🐮  Tail call __rust_dealloc function to free the memory

jmp     qword ptr [rip + __rust_dealloc@GOTPCREL] ; Free memory

.LBB0_2:

ret

```



Experiment: Cow Parameter, Change Power, and Quaternions

We have covered a lot of ground in this chapter. Let us experiment with the ideas we have learned so far. We will use Compiler Explorer to see the assembly code generated for the code we write.

We will conduct the following experiments:

- [What Happens When We Pass a Cow?](#)
- [Change the Type Alias and the Power Constant](#)
- [Look at Similar Code Generated for Quaternions](#)

What Happens When We Pass a Cow?

We have looked at Box, Rc, and Arc smart pointers. Let us look at the Cow type. The Cow type is a smart pointer that can wrap a borrowed or owned value. Let us see what happens when we pass a Cow to the magnitude function. The code must handle the borrowed and owned variants of the Cow type. The borrowed variant is a reference to the value, and the owned variant is the value itself. The code must handle both cases. Before we proceed, let us learn more about the Cow type.



Learn More: Cow - Clone on Write

The Cow type, a smart pointer, wraps either a borrowed or an owned value and helps avoid unnecessary allocations by providing immutable access. It will only lazily clone the value when mutation or ownership is required. It is an enum with two variants: Borrowed, which holds a reference to the borrowed value, and Owned, which stores the owned value. The Cow type also implements the Deref trait, enabling the treatment of a Cow value as a reference to the value it contains. Additionally, it implements the Borrow trait for both Borrowed and Owned variants, allowing the treatment of a Cow value as a reference to its held value.

The following code shows the definition of the Cow type.

```

pub enum Cow<'a, T: ToOwned + ?Sized> {
    Borrowed(&'a T),
    Owned(<T as ToOwned>::Owned),
}

```

In this definition:

- T represents the data type that can be borrowed or owned.
- ToOwned is a trait that specifies how to create an owned version from a borrowed reference.
- ?Sized is a trait bound that allows for types that do not have a constant size known at compile time.
- 'a is a lifetime parameter representing the borrowed reference's lifetime.

Cow Example

Now that we understand the Cow type let us look at the `magnitude_cow` function. This function takes a `Cow<Complex>` as input and returns the magnitude.

```
pub fn magnitude_cow(num: Cow<Complex>) -> Num {
    (num.real.powf(POWER) + num.imaginary.powf(POWER)).sqrt()
}
```

Note: We did not use Cow as self in this example. At the time of writing, Rust's stable release does not permit using Cow as the type for self.

The generated code must handle the borrowed and owned variants of the Cow type. The borrowed variant is a reference to the value, and the owned variant is the value itself. The following table shows the Cow type layout in memory for the borrowed and owned variants.

Byte offset	 Borrowed	 Owned
0	Discriminator (0)	Discriminator (1)
8	Address of Complex	real: f64
16		imaginary: f64

The following code shows the generated assembly for the `magnitude_cow` function. The first 8 bytes of the Cow type form the enum discriminator. The generated code performs the magnitude calculation for both the borrowed and owned variants of the Cow type using common code that references the `rdi` register. The code that handles the borrowed and owned variants prepares the `rdi` register with the address of the Complex struct before jumping to the common code.

-  Input: `rdi` = Address of `Cow<Complex>`
-  Output: `xmm0` = magnitude

```
example::magnitude_cow:
```

 Check the Cow's enum discriminator to determine if it is:

-  Borrowed (0) or
-  Owned (1)

```
cmp     qword ptr [rdi], 0
; If it is Owned, jump to .LBB0_1
jne     .LBB0_1
```

  Borrowed: We reach this point if the Cow is Borrowed. The `rdi` register holds the address of the Complex struct.

```
mov     rdi, qword ptr [rdi + 8]
; Skip the Owned branch-specific code
jmp     .LBB0_3
```

  Owned: We reach this point if the Cow is Owned. Incrementing `rdi` by 8 skips the discriminator and points to the Complex struct in memory.

```
.LBB0_1:
add     rdi, 8
```



Common handling for both Owned and Borrowed. At this point, `rdi` points to the `Complex` object in memory in the Borrowed and Owned case.

```
.LBB0_3:
movupd  xmm0, xmmword ptr [rdi]
mulpd   xmm0, xmm0
movapd  xmm1, xmm0
unpckhpd      xmm1, xmm0
addsd   xmm1, xmm0
xorps   xmm0, xmm0
sqrtsd  xmm0, xmm1
ret
```

You can further experiment with the code generated for the `magnitude_cow` function using the following link:

Compiler Explorer (<https://godbolt.org/z/19x4MYqvd>)

Change the Type Alias and the Power Constant

Let us look at the `Complex` struct that we have discussed in this chapter:

Compiler Explorer (<https://godbolt.org/z/nYa3rr4E3>).

We will experiment by changing the `Num` type alias and the `POWER` constant shown in the following code:

```
type Num = f64;
const POWER: Num = 2.0;
```

- Changing `type Num = f64` to `type Num = f32` does not change the structure of the code. You see a separate set of instructions for working with the `f32` type.
- Changing `const POWER: Num = 2.0` to `const POWER: Num = 3.0` results in the compiler generating a call to the `pow` function instead of using the `multiply` instructions.

Note: Changing the `POWER` to `3.0` is not a mathematically valid operation. We are doing this to see the impact of the change on the generated assembly code.

Look at Similar Code Generated for Quaternions

We have looked at the `Complex` struct. Let us experiment with a struct with more fields than `Complex` and see if the structure of the generated code changes. We will use the `Quaternion` struct for this experiment. The `Quaternion` struct has four fields: `w`, `x`, `y`, and `z`. We will calculate the magnitude of a quaternion.



Learn More: Quaternions

A quaternion extends the concept of complex numbers into a four-dimensional number system. It comprises one real and three imaginary parts, typically represented as $q = w + xi + yj + zk$. Here, w , x , y , and z are real numbers, while i , j , and k are quaternion units.

A quaternion's magnitude (or norm or absolute value) is a real number that is the same as the Euclidean norm for a vector in four-dimensional space. You compute it using the formula:

$$|q| = \sqrt{w^2 + x^2 + y^2 + z^2}$$

Quaternion Example

Look at the following Quaternion code in [Compiler Explorer](https://godbolt.org/z/7YWo5c3oe) (<https://godbolt.org/z/7YWo5c3oe>)

```
use std::rc::Rc;
use std::sync::Arc;

type Num = f64;
const POWER: Num = 2.0;

#[derive(Copy, Clone)]
pub struct Quaternion {
    real: Num,
    i: Num,
    j: Num,
    k: Num,
}

impl Quaternion {
    pub fn magnitude_self_copy(self) -> Num {
        (self.real.powf(POWER) + self.i.powf(POWER) +
         self.j.powf(POWER) + self.k.powf(POWER)).sqrt()
    }

    pub fn magnitude_self_reference(&self) -> Num {
        (self.real.powf(POWER) + self.i.powf(POWER) +
         self.j.powf(POWER) + self.k.powf(POWER)).sqrt()
    }

    // Passing smart pointers
    pub fn magnitude_self_box(self: Box<Self>) -> Num {
        (self.real.powf(POWER) + self.i.powf(POWER) +
         self.j.powf(POWER) + self.k.powf(POWER)).sqrt()
    }

    pub fn magnitude_self_rc(self: Rc<Self>) -> Num {
        (self.real.powf(POWER) + self.i.powf(POWER) +
         self.j.powf(POWER) + self.k.powf(POWER)).sqrt()
    }

    pub fn magnitude_self_arc(self: Arc<Self>) -> Num {
        (self.real.powf(POWER) + self.i.powf(POWER) +
         self.j.powf(POWER) + self.k.powf(POWER)).sqrt()
    }
}
```

The generated code for the `Quaternion::magnitude_self_copy` function follows:

-  Input: `rdi` points to the Quaternion object in memory
-  Output: `xmm0` contains the magnitude of the Quaternion object

example::Quaternion::magnitude_self_copy:

```
movupd  xmm0, xmmword ptr [rdi] ;   Load real and i parts
```

```

movupd  xmm1, xmmword ptr [rdi + 16] ; ⬇️⬇️ Load j and k parts

mulpd   xmm0, xmm0                    ; ✖️✖️ Square real and i parts

mulpd   xmm1, xmm1                    ; ✖️✖️ Square j and k parts
movapd  xmm2, xmm0

```

+++ Add the squared numbers and save them to xmm0.

```

unpckhpd xmm2, xmm0
addsd    xmm2, xmm0
addsd    xmm2, xmm1
unpckhpd xmm1, xmm1
addsd    xmm1, xmm2

xorps    xmm0, xmm0 ; 0 Set xmm0 to 0
sqrtsd   xmm0, xmm1 ; √ Square root of lower part of xmm1 -> xmm0
ret

```

Comparing the preceding code with the generated code for the `Complex::magnitude_self_copy` function, we see the following differences:

- In the case of `Complex`, the function receives the real and imaginary parts in the `xmm0` and `xmm1` registers. In the case of `Quaternion`, it receives a pointer to the `Quaternion` object in the `rdi` register. The function loads the real and i parts in the `xmm0` register. It then loads the j and k parts in `xmm1`.
 - Note from Compiler Explorer that the code generated for the `Quaternion::magnitude_self_copy` function is identical to the code generated for the `Quaternion::magnitude_self_reference` function. The `Quaternion` struct is too large to pass the fields in registers; thus, the compiler passes a pointer to the struct in the `rdi` register.
- We see that the code generated for the `Quaternion::magnitude_self_copy` function uses more vector operations than the code generated for the `Complex::magnitude_self_copy` function.

Look at the generated code for the following `Quaternion` functions and compare them with the equivalent code for the `Complex` functions:

- `Quaternion::magnitude_self_reference`
- `Quaternion::magnitude_self_box`
- `Quaternion::magnitude_self_rc`
- `Quaternion::magnitude_self_arc`

Key Takeaways

Small Types: Copying vs Reference

The compiler generated the most efficient code with the least memory overhead for the `Complex::magnitude_self_copy` method. Passing by value might be more efficient for small types than passing a reference.

The compiler will pass the address to the object in a register for larger types. We saw this in the generated code for the `Quaternion::magnitude_self_copy` function.

Passing a &T Is Efficient

In most scenarios, passing a reference will be more efficient than passing by value, as the compiler will not need to copy all the contents to the called function.

Passing a Box<T> Will Result in Object Deallocation

Passing a Box<T> will result in the deallocation of the object. If this is not the desired behavior, pass a &T or &mut T instead.

Prefer Passing a &T over Rc<T> and Arc<T>

The generated code demonstrates that passing an owned Rc<T> and Arc<T> introduces significant overhead. In scenarios where you do not expect sharing changes, we recommend passing a reference &T.

The [Reddit discussion](https://bit.ly/454rUPY) (<https://bit.ly/454rUPY>) on the subject defines the following rules for Arc<T>:

- If a function always needs to own its copy of the Arc, pass Arc<T> directly by value. The caller can decide whether to clone or move an existing Arc into it.
- If a function seldom requires copying the Arc, using &Arc<T> can be sensible. This approach avoids the need for atomic operations in typical cases at the expense of the inability to move the Arc in rare instances.
- If the function just wants to read from the T, just pass &T.

Consider the Memory Overhead of Rc and Arc

On a 64-bit machine, Rc and Arc add a 16 byte overhead on the heap.

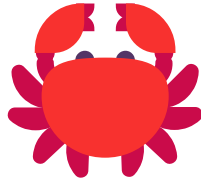
Consider Cow<T> When You Need to Work with Borrowed or Owned Data Interchangeably

We recommend using the Cow smart pointer when:

- You have a data structure that can be borrowed (&T) or owned (T).
- The data is potentially large or expensive to clone.
- You need the flexibility to work with borrowed or owned data interchangeably.

By using Cow, you can avoid unnecessary cloning by leveraging the Borrow trait, which allows you to work with borrowed references when possible and automatically clones the data when necessary.

Keep in mind that using Cow comes with some trade-offs. The choice between borrowing or owning can introduce runtime overhead. We explored these overheads in the [Cow experiment](#). We saw that the compiler did check if the Cow object was owned or borrowed at runtime. It cleverly optimized the code to keep most of the logic independent of the ownership of the Cow object.



Chapter 6

Structs: Alignment and Patterns

Let us look at how structs are [represented in memory](#) and how the compiler generates code to work with them. We will also look at code generated for [pattern-matching fields in a struct](#).

Field Ordering and Alignment

Let us begin by exploring a basic structure containing various fields. We will then examine the code generated for the `update_my_struct` function.

```
pub struct MyStruct {  
    a: u8,  
    b: u64,  
    c: i8,  
    d: i64,  
    e: i32,  
}  
  
pub fn update_my_struct(my_struct: MyStruct) -> MyStruct {  
    MyStruct {  
        a: my_struct.a + 0xA,  
        b: my_struct.b + 0xB,  
        c: my_struct.c - 0xC,  
        d: my_struct.d - 0xD,  
        e: my_struct.e - 0xE,  
    }  
}
```

Rust Reorders Struct Fields

Before we examine the generated code, let us explore the struct's memory layout. In Rust, unlike C, the fields of a struct might not appear in memory in the same order they are declared. The generated code shows that Rust aligns these fields based on size, with larger fields facing stricter alignment requirements.

The alignment of a field means the field's address must be a multiple of a certain number of bytes. For example, a `u64` requires an alignment of 8, meaning its address must be a multiple of 8. Conversely, a `u8`

Part C Data Structures and Memory Management

requires an alignment of 1 and thus can be stored at any address. Moreover, the entire struct requires an alignment of 8, reflecting the alignment requirement of the largest field.

`MyStruct` memory layout follows. Note that the compiler has reordered the fields and added a 2-byte padding at the end of the struct to align to an 8-byte boundary.

00	b	u64
08	d	i64
16	e	i32
20	a	u8
21	c	i8
22		2-byte padding

Ownership Transfer and Move Semantics

Note that the `update_my_struct` function takes ownership of the `MyStruct` argument and returns ownership of the `MyStruct` result. Because Rust uses move semantics by default and transfers ownership, the compiler can pass the input and output `MyStruct` as addresses in registers without generating code to copy the struct.

Reordering of Fields Facilitates Vectorization

The following is the generated code for `update_my_struct`. As expected, the compiler has produced code that copies the fields from the source struct to the destination struct and adds and subtracts constants from the source struct’s fields. An interesting aspect is that reordering the fields enables the code’s vectorization. The compiler employs vector operations to update `b` and `d`. The table below displays the operations performed on each field, executed in the order presented in the table.

The following table shows the operations performed on each field. The code executes these operations in the order presented in the table.

	Operation	Field	Offset in struct
↓	Read Input	a	20
+	Add 0xA	a	
↓	Read Input	c	21
−	Subtract 0xC	c	
↓	Read Input	e	16
−	Subtract 0xE	e	
↑	Write Output	a	20
↓ ↓	Vector Read Input	b and d	00 and 08
+ −	Vector Add 0xB and Subtract 0xD	b and d	
↑	Write Output	c	21
↑ ↑	Vector Write Output	b and d	00 and 08
↑	Write Output	e	16

Here is the generated code for the `update_my_struct` function.

-  Input:

- rdi = address of returned MyStruct
- rsi = address of the input MyStruct
-  Output:
 - rax = address of MyStruct returned struct

.LCPI0_0:

.quad 11 ; 0xb

.quad -13 ; -0xd

example::update_my_struct:

;  Copy the address of the MyStruct to be returned to rax

mov rax, rdi

;  Read a from the source MyStruct to ecx

movzx ecx, byte ptr [rsi + 20]

;  Add 10 (0xA) to a

add cl, 10

;  Read c from the source MyStruct to edx

movzx edx, byte ptr [rsi + 21]

;  Subtract 12 (0xC) from c

add dl, -12

;  Read e from the source MyStruct to edi

mov edi, dword ptr [rsi + 16]

;  Subtract 14 (0xE) from e

add edi, -14

;  Write a to the destination MyStruct

mov byte ptr [rax + 20], cl

;   Vector read b and d to xmm0

movdqu xmm0, xmmword ptr [rsi]

;   Vector add 11 (0xB) to b and -13 (0xD) to d

paddq xmm0, xmmword ptr [rip + .LCPI0_0]

;  Write c to the destination MyStruct

mov byte ptr [rax + 21], dl

;   Vector write b and d to the destination MyStruct

movdqu xmmword ptr [rax], xmm0

;  Write e to the destination MyStruct

mov dword ptr [rax + 16], edi

;  Return the address of the destination MyStruct in rax

ret

 Learn More: C Compatibility

You can turn off field reordering by adding the `#[repr(C)]` attribute to the struct, which is useful when interoperating with the C code. You can also specify the struct's alignment using the `#[repr(C)]` attribute. For instance, `#[repr(C, align(16))]` dictates that the struct should align to a 16-byte boundary.

Pattern Matching a Struct

Rust is a highly expressive language. It allows us to pattern-match fields of a struct. Let us look at an example of pattern-matching with the `OurStruct` struct defined here. The `are_we_compatible` function returns `true` if the pattern-matching conditions are satisfied. Otherwise, it returns `false`.

```
pub struct OurStruct {
    my_struct: MyStruct,
    your_struct: MyStruct,
}

pub fn are_we_compatible(our_struct: &OurStruct) -> bool {
    match *our_struct {
        OurStruct {
            my_struct: MyStruct { a: 10, .. },
            your_struct: MyStruct { a: 101, .. },
        } => true,
        OurStruct {
            my_struct: MyStruct { b: 11, .. },
            your_struct: MyStruct { c: 121, d: 42, .. },
        } => true,
        OurStruct { .. } => false,
    }
}
```

Here is the memory layout of `OurStruct`. The compiler arranges the two `MyStruct` fields within `OurStruct` adjacently in memory. Note that the display to the left of the field names indicates the byte offsets for each field in `OurStruct`.

The 2-byte padding at the end of each `MyStruct` ensures alignment to an 8-byte boundary, aiding in maintaining the alignment of the `MyStruct` fields within `OurStruct`.

00 my_struct.b	u64
08 my_struct.d	i64
16 my_struct.e	i32
20 my_struct.a	u8
21 my_struct.c	i8
22	2-byte padding
24 your_struct.b	u64
32 your_struct.d	i64
40 your_struct.e	i32
44 your_struct.a	u8
45 your_struct.c	i8
46	2-byte padding

The generated code for `are_we_compatible` follows. Note that the compiler has generated code to compare the fields of the struct with the constants specified in the pattern-matching conditions. The compiler has also generated code to compare the fields of the struct with the constants defined in the pattern-matching conditions. The generated code performs a logical AND of the results of the comparisons for each match leg.

The pattern-matching code is equivalent to the code provided below. Understanding this code will aid in comprehending the generated code.

```
pub fn alt_are_we_compatible(our_struct: &OurStruct) -> bool {
    if our_struct.my_struct.a == 10 &&
        our_struct.your_struct.a == 101 {
        true
    } else if our_struct.my_struct.b == 11 &&
        our_struct.your_struct.c == 121 &&
        our_struct.your_struct.d == 42 {
        true
    } else {
        false
    }
}
```

-  Input: `rdi` = address of `OurStruct`
-  Output: `al` = true if the pattern-matching conditions are satisfied; otherwise, false

example::are_we_compatible:

```
; ⚖️ Compare my_struct.a to 10
cmp     byte ptr [rdi + 20], 10

; ➡️ If my_struct.a is not equal to 10, jump to .LBB1_2
jne     .LBB1_2

; ✅ Storing true in al
mov     al, 1

; ⚖️ Compare your_struct.a to 101
cmp     byte ptr [rdi + 44], 101

; ➡️ If your_struct.a is not equal to 101, jump to .LBB1_1
jne     .LBB1_2

; ⬅️ BACK Return ❤️ true in al
ret

.LBB1_2:

; ⚖️ Compare my_struct.b to 11
cmp     qword ptr [rdi], 11

; ✅ Set al to 1 if my_struct.b is equal to 11
; ❌ Set al to 0 if my_struct.b is not equal to 11
sete    al
```

Part C Data Structures and Memory Management

```

; ⚖️ Compare your_struct.c to 121
cmp     byte ptr [rdi + 45], 121

; ✅ Set c1 to 1 if your_struct.c is equal to 121
; ❌ Set c1 to 0 if your_struct.c is not equal to 121
sete    c1

; Set c1 to 1 if my_struct.b is equal to 11 and
; your_struct.c is equal to 121
and     c1, al

; ⚖️ Compare your_struct.d to 42
cmp     qword ptr [rdi + 32], 42

; ✅ Set a1 to 1 if your_struct.d is equal to 42
sete    a1

; 🧮 Set a1 to 1 if my_struct.b is equal to 11 and
; your_struct.c is equal to 121 and
; your_struct.d is equal to 42
and     a1, c1

; ⬅️ BACK Return ❤️ true or ❌ false in al
ret

```



Experiment: Structs, Alignment, and Pattern-Matching

Let us run the following experiments with [Compiler Explorer](https://godbolt.org/z/M7b8jWfGK) (<https://godbolt.org/z/M7b8jWfGK>):

- [In-Place Update of a Struct](#)
- [Make a Rust Struct Compatible with a C Struct](#)
- [Adding a u128 Field to the Struct](#)
- [Compare the Code Generated When If-Else Is Used Instead of Pattern-Matching](#)

In-Place Update of a Struct

Add the `update_my_struct_mut` function to [Compiler Explorer](https://godbolt.org/z/M7b8jWfGK) (<https://godbolt.org/z/M7b8jWfGK>) code window. This function will update the fields of the `MyStruct` in-place. Compare the generated code with the assembly for `update_my_struct`.

You will find that the code generated for `update_my_struct_mut` is more efficient than that generated for `update_my_struct`. Updating a struct is more efficient because the compiler does not deal with an input and an output struct.

```

pub fn update_my_struct_mut(my_struct: &mut MyStruct) {
    my_struct.a += 0xA;
    my_struct.b += 0xB;
    my_struct.c -= 0xC;
    my_struct.d -= 0xD;
    my_struct.e -= 0xE;
}

```

Make a Rust Struct Compatible with a C Struct

Add the `#[repr(C)]` attribute to the Rust struct to make it compatible with C. The Rust compiler will not reorder the fields of the struct. It will follow the same memory layout as the C struct. Look at the generated assembly to see the difference in the code generated by adding the `#[repr(C)]` attribute.

```
#[repr(C)]
pub struct MyStruct {
    a: u8,
    b: u64,
    c: i8,
    d: i64,
    e: i32,
}
```

The layout of the C-compatible struct, which does not reorder fields but requires substantial padding to maintain alignment, follows. This results in an increase in the struct's size from 24 bytes to 36 bytes.

00 a	u8
01	7-byte padding
08 b	u64
16 c	i8
17	7-byte padding
24 d	i64
32 e	i32

Adding a u128 Field to the Struct

Add a `u128` field to the Rust struct. The lines with  show the changes needed to add the `u128` field.

```
pub struct MyStruct {
    a: u8,
    b: u64,
    c: i8,
    d: i64,
    e: i32,
     f: u128,
}

pub fn update_my_struct(my_struct: MyStruct) -> MyStruct {
    MyStruct {
        a: my_struct.a + 0xA,
        b: my_struct.b + 0xB,
        c: my_struct.c - 0xC,
        d: my_struct.d - 0xD,
        e: my_struct.e - 0xE,
         f: my_struct.f + 0xF,
    }
}
```

The new struct layout, which places the `u128` field at the top of the struct, follows.

Part C Data Structures and Memory Management

00 f	u128
16 b	u64
24 d	i64
32 e	i32
36 a	u8
37 c	i8
38	2-byte padding

Look for the following lines in [Compiler Explorer](https://godbolt.org/z/T8sEvsW3W) (<https://godbolt.org/z/T8sEvsW3W>) assembly code to see the code generated for the u128 update. Note that the compiler has generated code to update the u128 field using add and adc (add with carry) instructions to perform a 128-bit addition.

```
; <img alt="scissors icon" data-bbox="108 298 125 312"/>--- snip ---
    mov     rcx, qword ptr [rsi]
    mov     rdi, qword ptr [rsi + 8]
    add     rcx, 15
    adc     rdi, 0
; <img alt="scissors icon" data-bbox="108 382 125 396"/>--- snip ---
    mov     qword ptr [rax + 8], rdi
    mov     qword ptr [rax], rcx
    ret
```

Compare the Code Generated When If-Else Is Used Instead of Pattern-Matching

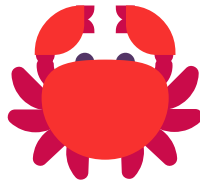
The `alt_are_we_compatible` function is equivalent to the `are_we_compatible` function. Paste this code in [Compiler Explorer](https://godbolt.org/z/M7b8jWfGK) (<https://godbolt.org/z/M7b8jWfGK>) code window and compare the assembly code generated for the two functions.

```
pub fn alt_are_we_compatible(our_struct: &OurStruct) -> bool {
    if our_struct.my_struct.a == 10 &&
        our_struct.your_struct.a == 101 {
        true
    } else if our_struct.my_struct.b == 11 &&
        our_struct.your_struct.c == 121 &&
        our_struct.your_struct.d == 42 {
        true
    } else {
        false
    }
}
```

Key Takeaways

- The Rust compiler will reorder struct fields to minimize padding.
- Reordering of fields reduces the size of the struct and increases the chances of vectorization.
- You can turn off the reordering of struct fields by using the `#[repr(C)]` attribute, which is particularly useful when interfacing with C code.
- The Rust compiler will add padding to the end of a struct to make the size of the struct a multiple of the alignment of the largest field.

- Implementing pattern matching on a struct involves executing a series of comparisons and jumps. The generated code achieves the same efficiency as a series of `if` statements that individually check each field.
- When you update a struct using a mutable reference, the Rust compiler will generate more efficient code. We saw this in the [In-place update of a struct](#) experiment.
- Rust, by default, employs move semantics, meaning a transfer of ownership does not necessitate copying the struct. The generated code for the `update_my_struct` function demonstrates this principle.



Chapter 7

Arrays, Tuples, Option, and Box Internals

We have already seen how Rust handles enums under the hood. We also looked at the code generation for the Box smart pointer. Here, we put these items together in an example that describes how arrays, tuples, Option enum, and Box smart pointer allocations operate at the assembly level.

Code Example: Arrays, Tuples, Option, and Box

We will be dissecting the assembly generated for the following Rust code. The code contains the following functions:

- `make_quad_coordinates` is a function that takes an optional named tuple and returns an optional Box smart pointer containing four coordinates.
- `cross_lines_from_quad_coordinates` is a function that takes an optional named tuple and returns an optional tuple by value.

```
type Num = f64;

// A named tuple with two fields.
pub struct Coordinate(Num, Num);

// A named tuple that contains two named tuples.
pub struct Line(Coordinate, Coordinate);

// This function takes an optional named tuple and returns an optional
// Box smart pointer containing four coordinates.
pub fn make_quad_coordinates(maybe_coordinate: Option<Coordinate>)
    -> Option<Box<[Coordinate; 4]>> {
    // Check if the Option uses the Some variant. If it does, then extract
    // the contents of the tuple into x and y variables. If a None variant
    // is used, the function returns. The question mark is syntactic
    // sugar for matching and extracting from a Some and returning on None.
    let Coordinate(x, y) = maybe_coordinate?;

    // Create a new Box smart pointer containing four coordinates.
    // Note that this involves memory allocation on the heap.
    Some(Box::new([
```

Part C Data Structures and Memory Management

```

        Coordinate(x, y),
        Coordinate(-x, -y),
        Coordinate(-x, y),
        Coordinate(x, -y),
    ]))
}

// This function takes an optional named tuple and returns an optional tuple
pub fn cross_lines_from_quad_coordinates(
    maybe_coordinate: Option<Coordinate>,
) -> Option<(Line, Line)> {
    // Pattern match and extract the contents of the array if the Option
    // uses the Some variant. Return None if the Option uses the None
    // variant.
    let [a, b, c, d] = *make_quad_coordinates(maybe_coordinate)?;

    // Form two lines from four coordinates and return them as a tuple.
    // The tuple is wrapped in a Some variant.
    Some((Line(a, b), Line(c, d)))
}

```

Let us look at the code and the generated assembly for `make_quad_coordinates` and `cross_lines_from_quad_coordinates`.

make_quad_coordinates: Memory Layout and Code Generation

```

pub fn make_quad_coordinates(maybe_coordinate: Option<Coordinate>)
    -> Option<Box<[Coordinate; 4]>> {
    let Coordinate(x, y) = maybe_coordinate?;

    Some(Box::new([
        Coordinate(x, y),
        Coordinate(-x, -y),
        Coordinate(-x, y),
        Coordinate(x, -y),
    ]))
}

```

When we analyze the generated assembly code, be on the lookout for the following:

- Since the function returns a `Box` smart pointer, the assembly code allocates memory on the heap using the `__rust_alloc` function.
- If the heap allocation fails, the function throws an exception using a special instruction (`ud2`).
- Typically, Rust enums generate a discriminator value to select the variant. In this case, the Rust code generator optimizes the `Option<Box>` implementation by using the `NULL` pointer as the discriminator value.

Understanding the Data Layout

We will encounter the following data types in this function:

- `Coordinate` - a named tuple with two fields.
- `Line` - a named tuple that contains two `Coordinate` tuples.

- `Box<[Coordinate; 4]>` - a Box smart pointer that points to a heap-allocated array of four coordinates.
- `Option<Coordinate>` - an Option enum that may contain a `Coordinate` tuple.
- `Option<Box<[Coordinate; 4]>>` - an Option enum that may contain a Box smart pointer that points to an array of four `Coordinates`.

Learning about the memory layout of various data types will aid in understanding the assembly code.

Representation of `Option<Coordinate>`

The `Option<Coordinate>` type has a memory layout where byte offset 0 is the discriminator, distinguishing between the `Some` and `None` variants. The `Option<Coordinate>` stores the `Coordinate` tuple in the following two entries.

Byte offset	None	Some
0	Discriminator (0)	Discriminator (1)
8		f64
16		f64

Representation of `Option<Box<[Coordinate; 4]>>`

There are two memory areas associated with the type `Option<Box<[Coordinate; 4]>>`:

- `Option<Box>` - a 64-bit pointer that points to the heap-allocated array of four `Coordinate` objects.
- `[Coordinate; 4]` - an array of four `Coordinate` objects. Each `Coordinate` object is a tuple of two f64 values.

Let us look at the memory layout of the `Option<Box>` and `[Coordinate; 4]` types and then visualize the memory layout of the `Option<Box<[Coordinate; 4]>>` type.

`Option<Box>` on the Stack

The Rust code generator optimizes the `Option<Box<>>` type to a single pointer on the stack. The pointer is `NULL` if the `Option` variant is `None`. A non-zero pointer indicates that the `Option` variant is `Some`.

Byte offset	None	Some
0	The Box pointer is NULL	The Box pointer contains a valid address that points to the heap.

`[Coordinate; 4]` Array on the Heap

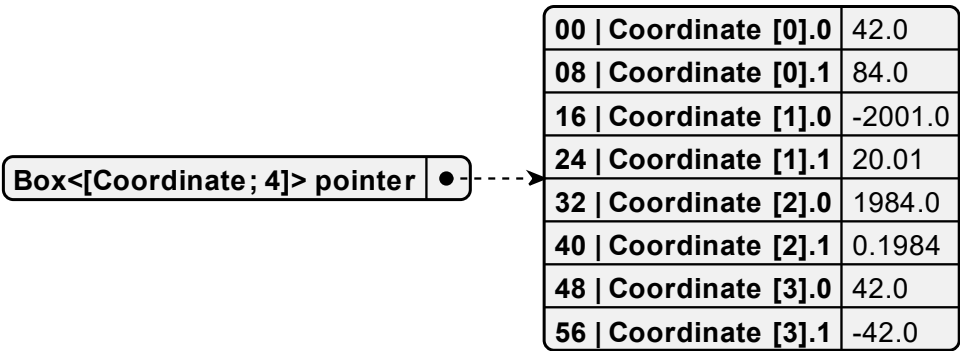
The `[Coordinate; 4]` array is allocated on the heap. The array contains four `Coordinate` objects. The memory layout of the array follows. The Box pointer points to the first element of the array.

Byte offset	Content
0	Coordinate
16	Coordinate
32	Coordinate
48	Coordinate

Visualizing the Memory Layout

Let us visually summarize what we have learned about the memory layout of the `Option<Box<[Coordinate; 4]>>` type.

`Some(Box::new([Coordinate; 4]))` representation:



None representation:

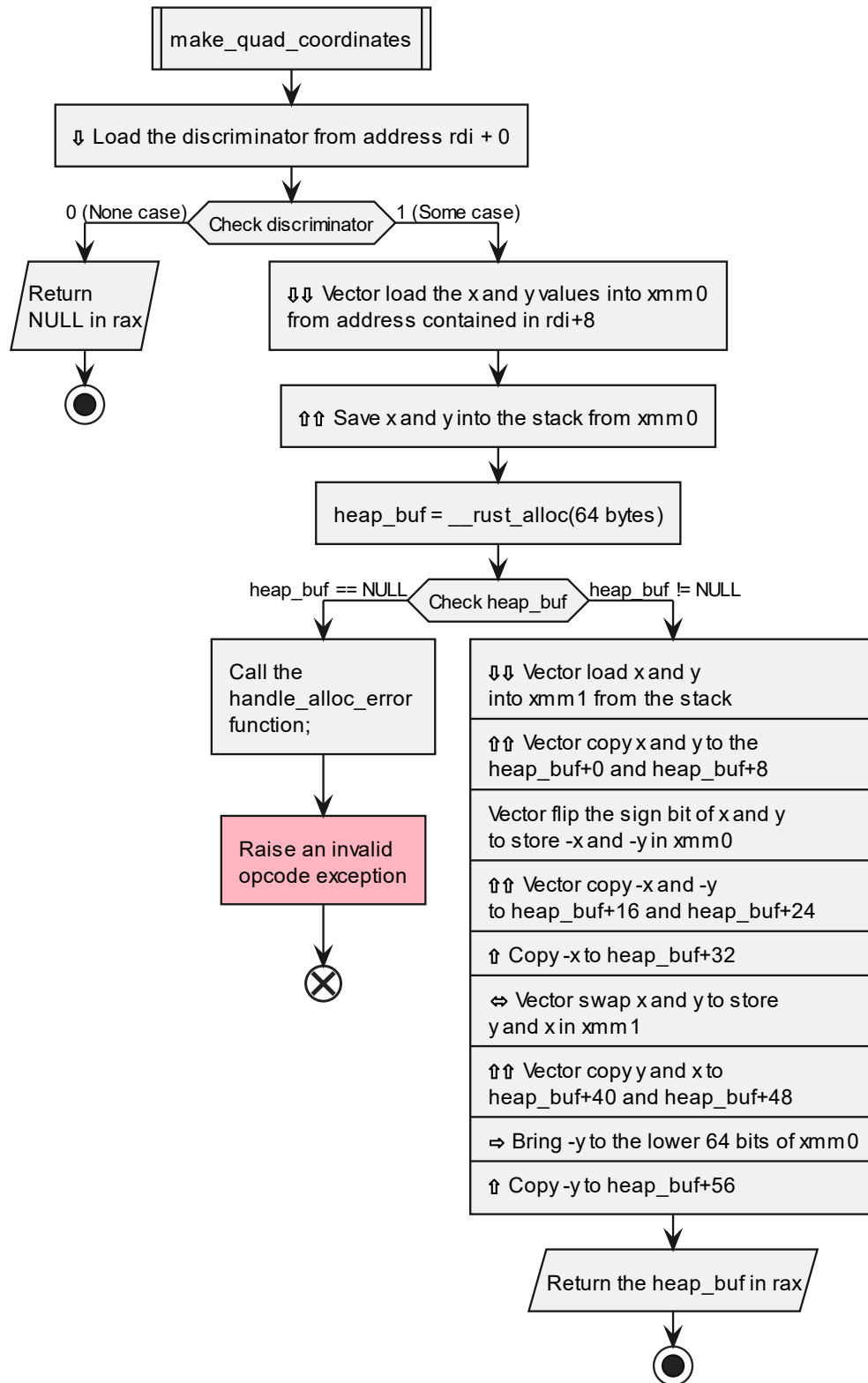


Flowchart for make_quad_coordinates

The flowchart for the `make_quad_coordinates` function illustrates how the function checks the input `Option` discriminator early on. The function returns immediately if the `Option` is `None`. If the `Option` is `Some`, the function extracts the tuple's contents and uses the `Box` smart pointer to allocate memory on the heap. The function then returns the `Box` pointer.

Key points to note:

- The compiler uses vector instructions to load and store two `f64` values simultaneously..
- The function will throw an exception if the `Box` allocation fails.
- The compiler uses the `NULL` pointer as the discriminator for the `Option<Box>` type, eliminating the need for a discriminator field.



Before we dive into the assembly code, let us review the IEEE 754 floating-point standard.

💡 Learn More: IEEE 754 and the Sign Bit

The `Coordinate` tuple comprises two `f64` fields, each a 64-bit floating-point number. The representation of these floating-point numbers adheres to the IEEE 754 floating-point standard, which defines the following:

Part C Data Structures and Memory Management

Sign	Exponent	Fraction (Mantissa)
bit 63	bit 62 to 52	Bit 51 to 0
1 bit	11 bits	52 bits

The critical point here is that floating-point numbers use a sign bit to define the sign of the number. The sign bit is the most significant bit. We will see in the assembly code that the generated code flips the sign bit of the f64 field to negate the number.

Assembly Code for make_quad_coordinates

The following assembly code for the make_quad_coordinates function includes annotations explaining the code.

-  Input: rdi = Address of Option<Coordinate>
-  Output: rax = Address of Option<Box<[Coordinate; 4]>>

```
.LCPI0_0:
.quad 0x8000000000000000 ; Constants used for flipping the sign bit..
.quad 0x8000000000000000 ; ..of a 64-bit floating-point number.

example::make_quad_coordinates:

sub    rsp, 24            ; Reserve space for local variables.

cmp     qword ptr [rdi], 0 ; ⚖️ Check if discriminator == 0 (None case)
je      .LBB0_1           ; 🔄 If discriminator == 0, jump to exit point.

; 👍 Processing the Some case of Option

movups  xmm0, xmmword ptr [rdi + 8] ; ⬇️ ⬇️ Get the 0th and 1st entry
                                           ; from the Coordinate tuple via a
                                           ; vector load (Note the 8-byte offset
                                           ; is needed to skip the
                                           ; discriminator)

movaps  xmmword ptr [rsp], xmm0      ; ⬆️ ⬆️ Save it into the x and y
                                           ; local variables on the stack.
```

  Allocating memory for Option<Box<[Coordinate; 4]>> involves reserving space for 4 array entries, with each entry requiring 16 bytes for the two f64 values, summing up to 64 bytes. It is important to note that there is no need for additional space for the Option discriminator; a NULL value (all 0s) indicates the None condition, while any non-zero heap address represents the Some condition.

```
mov     edi, 64            ; Request 64-byte memory
mov     esi, 8             ; Alignment is 8 bytes

call    qword ptr [rip + __rust_alloc@GOTPCREL] ; Request memory allocation.
                                           ; Allocation result in rax.

test    rax, rax           ; 🔍 Check if the memory allocation
                           ; returned an all-zero address (NULL)

je      .LBB0_5           ; ❌ If yes, jump to error handling code.
```

```

movaps  xmm1, xmmword ptr [rsp] ; ⬇️⬇️ Vector load x and y into xmm1

movups  xmmword ptr [rax], xmm1 ; ⬆️⬆️ Copy x and y to the heap

; ⬇️⬇️ Load constants with the most significant bit set
movaps  xmm0, xmmword ptr [rip + .LCPI0_0]

xorps   xmm0, xmm1                ; Flip the sign bits of x and y
                                       ; to store -x and -y in xmm0

movups  xmmword ptr [rax + 16], xmm0 ; ⬆️⬆️ Vector copy -x and -y to heap

movlps  qword ptr [rax + 32], xmm0  ; ⬇️ Copy -x to the heap

shufps  xmm1, xmm1, 78              ; 🔄 Swap the x and y values in xmm1

movups  xmmword ptr [rax + 40], xmm1 ; ⬆️⬆️ Copy y and x to the heap

pshufd  xmm0, xmm0, 238             ; ➡️ Bring -y to lower 64 bits of xmm0

movq    qword ptr [rax + 56], xmm0  ; ⬆️ Copy -y to the heap

add     rsp, 24                     ; Pop off the locals from the stack

ret                                          ; ⬅️👍📦 Return the result in rax

```

👉 Processing the None case of Option optimizes Option<Box> in Rust; it signals the None case by returning an all-zero (NULL) value to the caller, eliminating the need for a discriminator.

```

.LBB0_1:

xor     eax, eax                    ; 0 Setting the lower 32 bits of rax to 0
                                       ; Upper bits must be 0 to return an all-zero rax

add     rsp, 24                    ; Pop off the local variables

ret                                          ; ⬅️ Return NULL to signal None

```

❌ Exception processing (memory allocation failed).

```

.LBB0_5:

mov     edi, 64                    ; Size of the failed allocation (64-bit)
mov     esi, 8                     ; Alignment of the failed allocation (8-byte)
call    qword ptr [rip + alloc::alloc::handle_alloc_error@GOTPCREL]
ud2                                ; 💀 Raise invalid opcode exception

```

cross_lines_from_quad_coordinates: Inlining and deep optimization

Let us look at the cross_lines_from_quad_coordinates function and the generated assembly code.

```
pub fn cross_lines_from_quad_coordinates(
    maybe_coordinate: Option<Coordinate>,
) -> Option<(Line, Line)> {
    let [a, b, c, d] = *make_quad_coordinates(maybe_coordinate)?;
    Some((Line(a, b), Line(c, d)))
}
```

The assembly code for `cross_lines_from_quad_coordinates` gives us an unexpected result. Typically, we would think that when the `make_quad_coordinates` function returns a `Box`, it involves allocating memory on the heap. And since the function is supposed to use up the `Box`, we would also expect it to free up the heap memory before it finishes. But here is the surprising part: the generated code combines the `make_quad_coordinates` function directly into the main code and eliminates the need for the `Box`. Due to the inlining of the code, the compiler can skip both the memory allocation and deallocation steps.

Representation of Option<(Line, Line)>

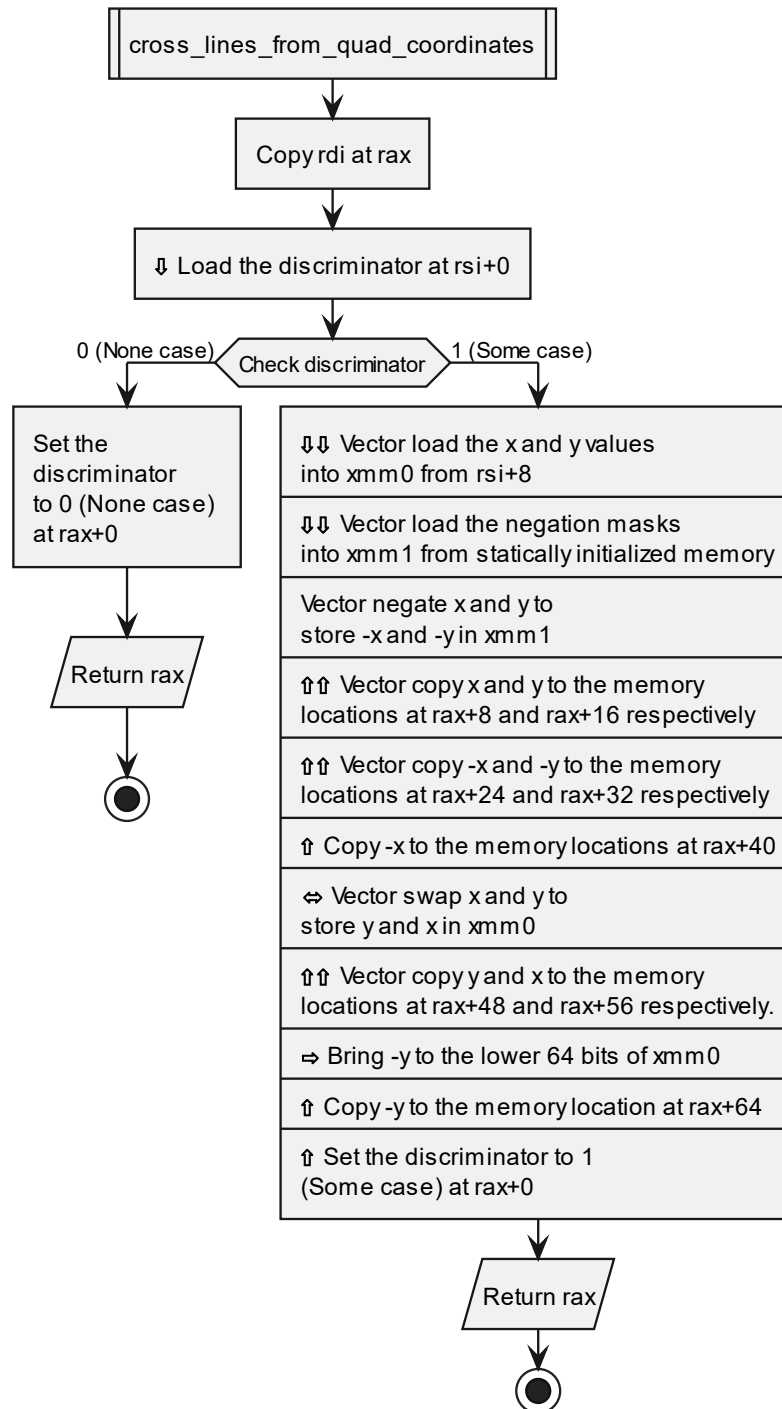
Understanding the representation of `Option<Line, Line>` will assist in keeping track of the flow of the assembly code. The memory layout starts with the `Option` discriminator. A value of 0 indicates the `None` case. A value of 1 represents the `Some` case. The two `Line` structs follow the `Some` case. The `Line` struct is a tuple of two `Coordinate` structs. The `Coordinate` struct is a tuple of two `f64` values.

Byte offset	None	Some
0	Discriminator (0)	Discriminator (1)
8		tuple.0 First Coordinate (x): f64
16		tuple.0 First Coordinate (y): f64
24		tuple.0 Second Coordinate (x): f64
32		tuple.0 Second Coordinate (y): f64
40		tuple.1 First Coordinate (x): f64
48		tuple.1 First Coordinate (y): f64
56		tuple.1 Second Coordinate (x): f64
64		tuple.1 Second Coordinate (y): f64

Flowchart of cross_lines_from_quad_coordinates

By looking at the flowchart of the `cross_lines_from_quad_coordinates` function, we glimpse the deep optimization of the generated code. This flowchart reveals the inlined code of the `make_quad_coordinates` function, eliminating the `Box` allocation and deallocation. With the elimination of the `Box`, a memory allocation failure exception becomes impossible, leading to the removal of the exception handling code.

The flowchart for the `cross_lines_from_quad_coordinates` function is simpler than the one for the `make_quad_coordinates` function it is supposed to call!



Assembly Code for cross_lines_from_quad_coordinates

Examine the assembly code using the flowchart blueprint.

Key points to note in the assembly code:

- The `make_quad_coordinates` function is inlined.
- The compiler eliminates the `Box` allocation and deallocation.
- The function cannot throw an exception because there is no heap allocation.
- The caller of `cross_lines_from_quad_coordinates` is responsible for the memory allocation of the return value.

Part C Data Structures and Memory Management

- The generated code uses vectorized instructions to operate simultaneously on two f64s.

12 **34** Input:

- rdi = Address of the return struct Option<(Line, Line)>
- rsi = Address of Option<Coordinate>

12 **34** Output: rax = Address of the return struct Option<(Line, Line)>

```
.LCPI1_0:
.quad  0x8000000000000000 ; Constants used for flipping the sign..
.quad  0x8000000000000000 ; .. bits in a 64-bit floating-point number.

example::cross_lines_from_quad_coordinates:

mov     rax, rdi           ; Copy return address to rax

cmp     qword ptr [rsi], 0 ; ⚖️ Check if discriminator == 0 (None case)
je      .LBB1_1           ; ➡️ If the discriminator is 0, jump to exit.
```

👍 Some variant in maybe_coordinate input. Note that the compiler has inlined the make_quad_coordinates function.

```
movups  xmm0, xmmword ptr [rsi + 8] ; ⬇️ ⬇️ Vector load x and y from the
                                     ; memory pointed by rsi

movaps  xmm1, xmmword ptr [rip + .LCPI1_0] ; ⬇️ ⬇️ Load the constant with
                                     ; the most significant bit set

xorps   xmm1, xmm0              ; Negate x and y by toggling
                                     ; sign bits
                                     ; xmm1 now contains -x and -y.

movups  xmmword ptr [rax + 8], xmm0 ; ⬆️ ⬆️ Vector store x and y to
                                     ; tuple.0 1st Coordinate(x, y)

movups  xmmword ptr [rax + 24], xmm1 ; ⬆️ ⬆️ Vector store -x and -y to
                                     ; tuple.0 2nd Coordinate(x, y)
```

Note that x86-64 byte ordering will write -x to the rax+24 address and -y to the rax+32 address.

```
movlps  qword ptr [rax + 40], xmm1 ; ⬆️ ⬆️ tuple.1 1st Coordinate (x):
                                     ; -x written to memory.

shufps  xmm0, xmm0, 78             ; ➡️ Swap the x and y in xmm0

movups  xmmword ptr [rax + 48], xmm0 ; ⬆️ ⬆️ Vector store y followed by x
                                     ; tuple.1 First Coordinate (y):
                                     ; y written to memory.
                                     ; tuple.1 Second Coordinate (x):
                                     ; x written to memory.

pshufd  xmm0, xmm1, 238            ; ➡️ Bring -y to xmm0 lower 64-bits
```

```

movq    qword ptr [rax + 64], xmm0    ; tuple.1 Second Coordinate (y):
                                       ; -y written to memory.

mov     ecx, 1                        ; 👍 Set the discriminator
                                       ; for Some (32-bit write)

mov     qword ptr [rax], rcx          ; ⬆ Write discriminator to memory
                                       ; (rax is the 64-bit register that
                                       ; contains ecx)

ret                                         ; ⬅ BACK Return the result in rax

```

👉 None variant in maybe_coordinate input.

```

.LBB1_1:
xor     ecx, ecx                      ; 🙅 None case (Discriminator = 0)

mov     qword ptr [rax], rcx          ; ⬆ Write the None discriminator

ret                                         ; ⬅ BACK Return the result in rax

```

🔬 Experiment: Changing the Type of Tuple Elements and Enabling AVX-512

Let us experiment with the code in this chapter in [Compiler Explorer](https://godbolt.org/z/cPefKdMr1) (<https://godbolt.org/z/cPefKdMr1>).

We will run a two-dimensional experiment. We will change the type of tuple elements and the compiler flags.

The first dimension of the experiment is the type of tuple elements. We will be changing the following type alias to f32, i64, i32, i16, and i8:

```

type Num = f64;

```

The second dimension of the experiment is choosing between the **Rust release** mode and **AVX-512**. We will be changing the following compiler flags in Compiler Explorer to switch between the two modes:

- **Rust release:** -C opt-level=3
- **AVX-512:** -C opt-level=3 -C target_cpu=skylake-avx512

Let us run through our two-dimensional experiment. In this chapter, we have already seen the assembly code for the **Rust release** mode for f64 tuple elements.

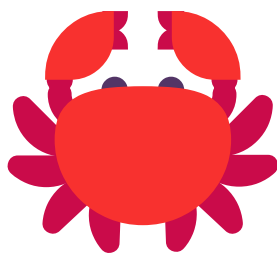
	f64	f32	i64	i32	i16	i8
Rust release	☒	☐	☐	☐	☐	☐
AVX-512	☐	☐	☐	☐	☐	☐

The AVX-512 experimentation is interesting as we see several new SIMD instructions with i8 and i16 tuple element sizes. Just hover over the assembly instructions in Compiler Explorer to learn more about them.



Key Takeaways

- The Rust compiler employs vector instructions to execute multiple operations in parallel, utilizing them for both floating-point and integer operations.
 - The compiler auto-vectorizes the code in the Rust release mode.
 - The compiler uses AVX-512 instructions when the target CPU is set to `skylake-avx512`. AVX-512 settings enable the use of more modern instructions and wider registers. The downside is that the code will not run on older CPUs.
- Rust's inlining of functions is deep and can eliminate expensive heap allocations. We saw this in the `cross_lines_from_quad_coordinates` function.
- The compiler represents an `Option` containing a `Box` as a 64-bit pointer. The `NULL` pointer indicates the `None` case. A non-`NULL` pointer indicates the `Some` case.
- Rust functions throw exceptions on memory allocation failure. One must be careful when using smart pointers and types that allocate memory from the heap. We saw the exception handling code in the `make_quad_coordinates` function.



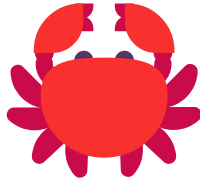
Part D

Iteration and Optimization



Rust Under the Hood Companion (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews



Chapter 8

Traditional vs. Functional Array Iteration

This chapter delves into the assembly code generated from functional style array iteration. We will contrast this with the assembly produced from a traditional for-loop style iteration. Additionally, we will explore the effects of varying the array size on the resulting assembly.

Iterator and Functional Style

The following code sums up the squares of even numbers in an array. The code uses a functional style to iterate over the array. The code is generic over the array size. The array size is a const generic parameter.

```
pub fn sum_of_even_squares<const N: usize>(arr: &[i64; N]) -> i64 {
    arr.iter()
        .filter(|&x| x % 2 == 0) // Filter out odd numbers
        .map(|&x| x.pow(2))      // Square remaining numbers
        .sum()                   // Sum them up
}

pub fn sum_of_even_squares_specific_size(arr: &[i64; 1024]) -> i64 {
    sum_of_even_squares(arr)
}
```

Generated Assembly: Loop Unrolling, Inlining, and Conditional Moves

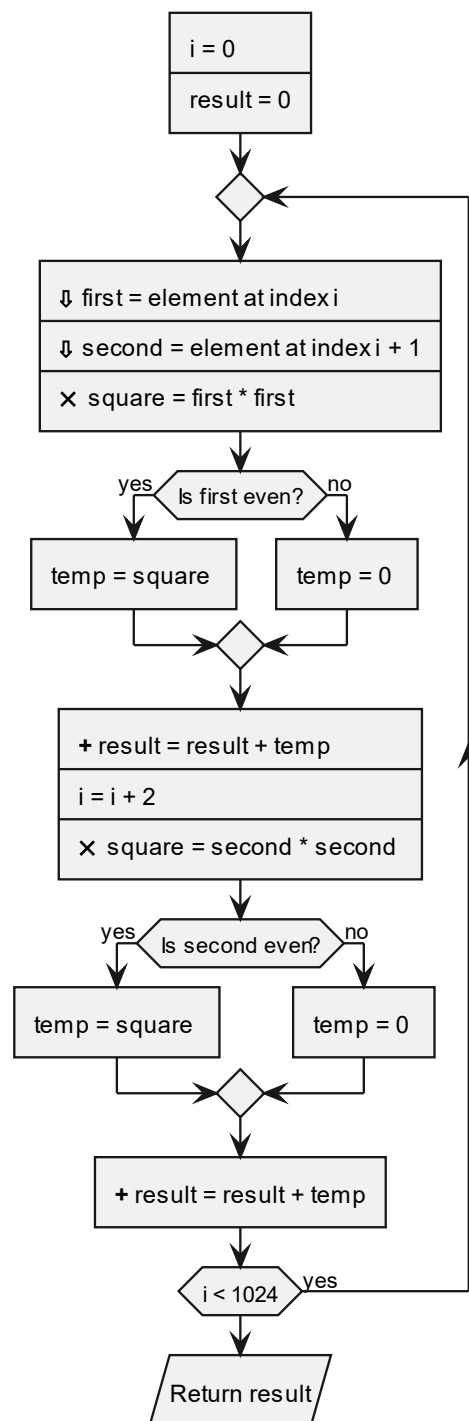
We will analyze the code generated for the `sum_of_even_squares_specific_size` function. The compiler generates no code for the `sum_of_even_squares` function as it is generic. Code generation occurs for specialized instances of generic functions rather than for the generic functions themselves.

We will see that in the `sum_of_even_squares_specific_size` function, the compiler inlines the `sum_of_even_squares` function. The compiler unrolls the loop by 2. So, the loop needs to go through 512 iterations to process 1024 elements. The compiler also uses conditional move instructions (`cmovne`) to avoid branching. The compiler calculates the squares of even as well as odd numbers. It uses `cmovne` to add the results of squaring even numbers while discarding the square of odd numbers. We will [experiment](#) with

Part D Iteration and Optimization

changing the power of x to find the point where the compiler adds code to avoid odd number multiplication.

Here is a flowchart of the assembly code generated by the Rust compiler for the `sum_of_even_squares_specific_size` function.



Let us dive into the assembly code generated by the Rust compiler for the `sum_of_even_squares_specific_size` function.

- 12
34 Input: `rdi` = address of the array
- 12
34 Output: `rax` = sum of even squares


```
example::sum_of_even_squares_specific_size:
```

;  The code loads a 0 constant into r8 for use in a conditional move.

```
xor    r8d, r8d
```

```
xor    eax, eax
```

;  Initialize the loop counter to 0. rdx serves as the loop counter.

```
xor    edx, edx
```

Loop start

The compiler unrolls this loop by 2, i.e., it processes two elements per iteration. So, the loop needs to go through 512 iterations to process 1024 elements.

```
.LBB0_1:
```

; Load the following two elements from the array.

;  The address of the first element in this iteration is rdi + 8*rdx.

```
mov    rcx, qword ptr [rdi + 8*rdx]
```

;  The address of the second element in this iteration is rdi + 8*rdx + 8.

```
mov    r9, qword ptr [rdi + 8*rdx + 8]
```

; Copy the first element to rsi.

```
mov    rsi, rcx
```

;  Square the first element.

```
imul   rsi, rcx
```

 Verify whether the initial element is even. The test instruction will set the zero flag to true when the result of performing a bitwise AND operation between the first element and 1 equals 0.

```
test   cl, 1
```

 If the zero flag is not set, it indicates the first element is odd, and the code loads 0 into rsi (since r8 is 0). Conversely, if the zero flag is set, the first element is even, and rsi already contains the square of the first element.

```
cmovne rsi, r8
```

 Add rax to rsi. Due to the conditional move instruction, rsi holds 0 for odd elements and the squared value for even ones. After this addition, rsi will have the cumulative sum of squares from even elements up to this point.

```
add    rsi, rax
```

;  Move the loop counter forward by 2.

```
add    rdx, 2
```

; Copy the second element to rax.

```
mov    rax, r9
```

;  Square the second element.

```
imul   rax, r9
```

Part D Iteration and Optimization

```
; 🧐 Check if the second element is odd or even.
test    r9b, 1
```

🔍 If the second element is odd, the code loads 0 into `rax` (as `r8` is 0). If the second element is even, then the square of the second element is already present in `rax`.

```
cmovne  rax, r8
```

✚ Add `rax` to `rsi`. Due to the conditional move instruction, `rax` contains 0 for odd elements and the square of the element for even elements. After this addition, `rax` will contain the cumulative sum of even squares up to this iteration.

```
add     rax, rsi
```

```
; ⚖️ Check if the loop counter is 1024.
cmp     rdx, 1024
```

```
; 🔄 If the loop counter is not 1024, jump to the loop start.
jne     .LBB0_1
```

🌀 ● Loop end

↩️ Return the sum in `rax`.

```
ret
```

🔬 Experiment: Change Array Size, Loop Code Generation, and Power of x

Experiment with [Compiler Explorer](https://godbolt.org/z/a3j746a53) (<https://godbolt.org/z/a3j746a53>) to understand how the generated code changes when you:

- [Change the Array Size](#)
- [Write the Code as a Loop](#)
- [Change the Power of x](#)

Change the Array Size

Change the array size as shown here and observe the generated code. The compiler unrolls the loop completely for small array sizes. The compiler restructures the code for odd array sizes to avoid unnecessary operations in the last iteration.

```
// Change the array size
pub fn sum_of_even_squares_specific_size(arr: &[i64; 1024]) -> i64 {
    sum_of_even_squares(arr)
}
```

Experiment with the following array sizes and confirm the observations:

Array size	Observation
8	The compiler completely unrolls the loop.

Array size	Observation
40	The compiler completely unrolls the loop.
64	The compiler unrolls the loop by 2 to execute 32 iterations.
65	The compiler unrolls the loop by 2 to execute 33 iterations. Since the loop count is odd, it restructures the code to permit just one operation in the last iteration.

Write the Code as a Loop

Check in [Compiler Explorer \(https://godbolt.org/z/oj1oa4chn\)](https://godbolt.org/z/oj1oa4chn) that the compiler generates the same assembly code for a handwritten loop as the functional style. With Rust, writing the code in a functional style does not incur any performance penalty. It is a zero-cost abstraction.

```
pub fn sum_of_even_squares_with_for<const N: usize>(arr: &[i64; N]) -> i64 {
    let mut sum = 0;
    for i in 0..N {
        let x = arr[i];
        if x % 2 == 0 {
            sum += x.pow(2);
        }
    }
    sum
}

pub fn sum_of_even_squares_specific_size_with_for(arr: &[i64; 1024]) -> i64 {
    sum_of_even_squares_with_for(arr)
}
```

Change the Power of x

Alter the power of x from 2 and observe the effect on the generated code. As the parameter of `x.pow` increases, the compiler shifts its priority from avoiding branches to avoiding multiplications. This shift occurs when the parameter reaches 9. At `x.pow(9)`, the compiler starts generating code with branching to minimize multiplications for odd numbers.

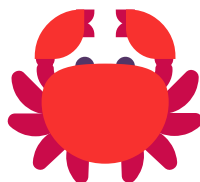
```
pub fn sum_of_even_squares<const N: usize>(arr: &[i64; N]) -> i64 {
    arr.iter()
        .filter(&x| x % 2 == 0) // filter out odd numbers
        .map(&x| x.pow(2)) // ← Change the power of x
        .sum() // sum them up
}
```

Key Takeaways

- Writing code in a functional style does not incur any performance penalty. It is a zero-cost abstraction. We saw this in our [for-loop experiment](#).
- The array size has a significant impact on the generated code. We saw this in our [array size experiment](#).
 - For small array sizes, the Rust compiler completely unrolls the loop.
 - The compiler optimizes iteration for odd array sizes by restructuring the code to avoid unnecessary operations in the last iteration.

Part D Iteration and Optimization

- The compiler goes out of its way to avoid branches. It prefers to calculate the power of x by a cascade of multiplications rather than using a branch to avoid the multiplications. We saw this in our [power of \$x\$ experiment](#).
- Code generation occurs for specialized instances of generic functions rather than for the generic functions themselves.
- The compiler may inline the code for a specialized instance of a generic function. For example, in the case of `sum_of_even_squares`, the compiler inlined it within `sum_of_even_squares_specific_size`. It generated no code for `sum_of_even_squares`.



Chapter 9

Optimizing Array Iteration with SIMD

We have already gone under the hood to look at the code generated by the compiler for array iteration in the “*Traditional vs. Functional Array Iteration*” chapter. This chapter will examine how the Rust compiler generates AVX-512 SIMD (Single Instruction Multiple Data) instructions to speed up array iteration.

Functional Style Loop

We will work with the same example as in the “*Traditional vs. Functional Array Iteration*” chapter. The code sums up the squares of even numbers in an array.

```
pub fn sum_of_even_squares<const N: usize>(arr: &[i64; N]) -> i64 {
    arr.iter()
        .filter(|&x| x % 2 == 0) // filter out odd numbers
        .map(|&x| x.pow(2)) // square remaining numbers
        .sum() // sum them up
}

pub fn sum_of_even_squares_specific_size(arr: &[i64; 1024]) -> i64 {
    sum_of_even_squares(arr)
}
```

AVX-512 SIMD Instructions

In this chapter, we will be using the following compiler flags to enable AVX-512 SIMD:

```
-C opt-level=3 -C target_cpu=skylake-avx512
```



Learn More: AVX-512 SIMD

AVX-512 SIMD instructions work with 512-bit registers. The processor supports dividing them into 16 32-bit elements, eight 64-bit elements, four 128-bit elements, or two 256-bit elements.

Compilers must balance utilizing 512-bit `zmm` registers and 256-bit `ymm` registers. As of this writing, LLVM employs 256-bit `ymm` registers for AVX-512 SIMD instructions to circumvent AVX-512 downclocking on Intel Skylake processors.

AVX-512: Vectorized Assembly Overview

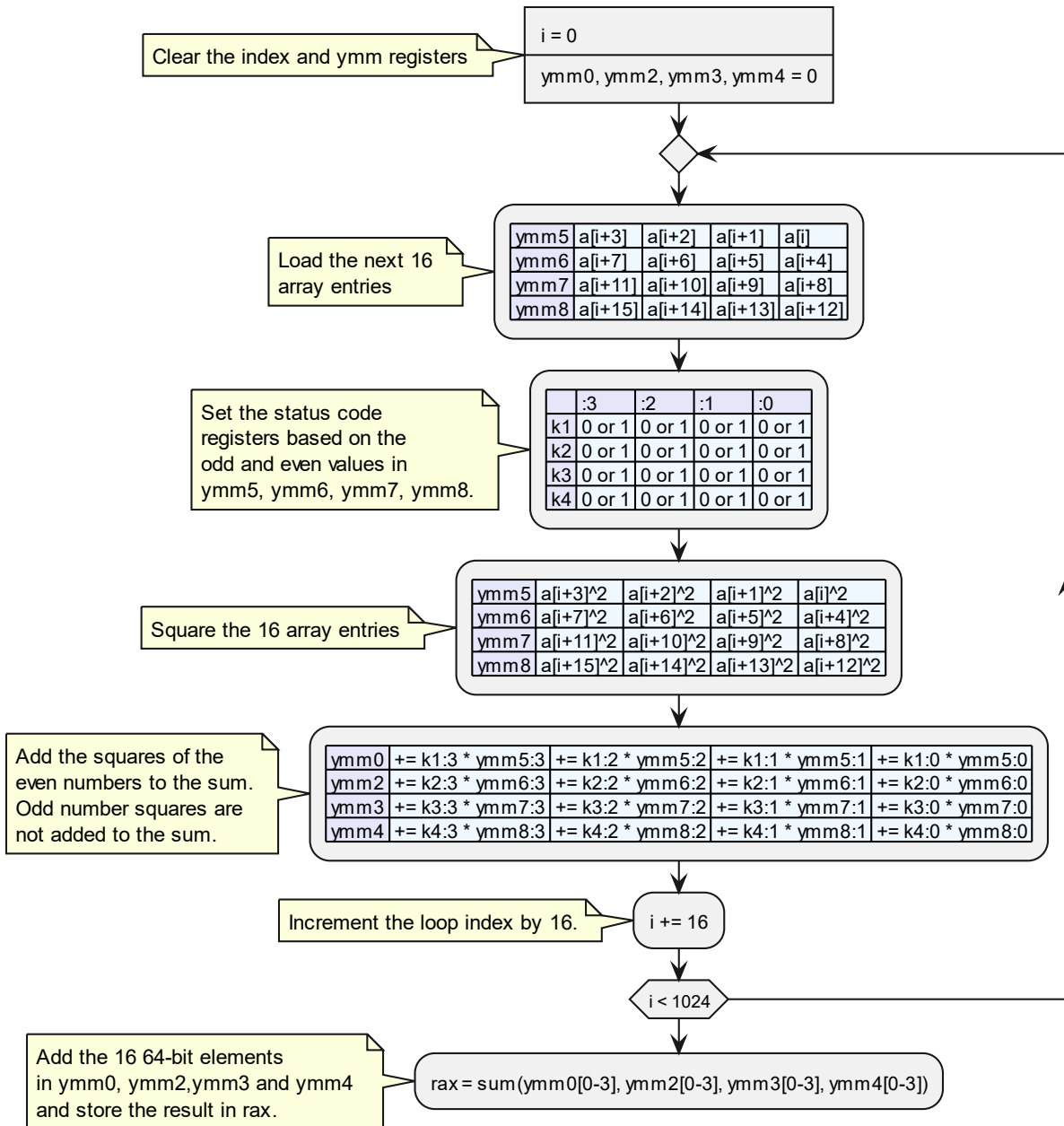
Before we dive into the assembly code, let us understand the high-level flow of the SIMD instructions in the generated assembly code.

When AVX-512 is enabled, the Rust compiler uses 256-bit `ymm` registers. It unrolls the loop by 16. The AVX-512 SIMD instructions perform four operations per instruction. The compiler generates four sets of SIMD instructions to process 16 elements per iteration.

The generated assembly has a lot to unpack (pun intended). We will start by looking at a flowchart of how the SIMD instructions perform the computation:

1. The code loads 16 array entries into four 256-bit `ymm` registers. It loads 16 array entries into 16 32-bit elements with four instructions.
2. The generated assembly then sets the status code registers based on the odd and even values in the 16 array entries.
3. Four SIMD instructions to square the 16 array entries. Each SIMD instruction squares four array entries in parallel.
4. The code then conditionally accumulates the results in 4 SIMD registers. The processor employs the status code register to ascertain whether the entry is even and should be added to the sum.
5. The loop index increments by 16 because the loop processes 16 array entries.
6. The code compares the loop index to the array size to determine if the loop should continue.
7. When the loop is complete, the code adds the 16 accumulated SIMD registers.
8. The function returns the result.

The following flowchart illustrates the highly vectorized loop.



Annotated Assembly: SIMD in Array Iteration

Now that we understand the SIMD flow and vectorized loop let us dive into the generated assembly code.

-  Input: `rdi` = address of the array
-  Output: `rax` = sum of even squares

 Label for a constant memory location for the broadcast instruction.

```
.LCPI0_0:
    .quad    1
```

 The function starts here.

```
example::sum_of_even_squares_specific_size:
```

Part D Iteration and Optimization

Zero `ymm0` by XORing it with itself. Although the instruction targets the lower 128 bits of the 256-bit register `ymm0`, it also zeroes the upper 128 bits (Learn more: <https://bit.ly/vpxor>).

```
vp xor    ymm0, ymm0, ymm0 ; 0 0 0 0
```

0 Initialize the loop index to 0. `rax` serves as the loop counter.

```
xor    eax, eax
```

1 1 1 1 Broadcasts the 64-bit value at the memory location `.LCPI0_0` to all the elements. `ymm1` is a 256-bit register that now stores four copies of a 64-bit value 1.

```
vp broadcastq    ymm1, qword ptr [rip + .LCPI0_0]
```

Zero the 256-bit registers `ymm2`, `ymm3`, and `ymm4`, which, along with `ymm0`, will store the sum of the squares of the even elements. The instruction zeroes the lower and upper 128 bits of these 256-bit registers.

```
vp xor    ymm2, ymm2, ymm2 ; 0 0 0 0
vp xor    ymm3, ymm3, ymm3 ; 0 0 0 0
vp xor    ymm4, ymm4, ymm4 ; 0 0 0 0
```

Loop start

```
.LBB0_1:
```

Load 16 elements from the array into the 256-bit registers `ymm5`, `ymm6`, `ymm7` and `ymm8`. The following four instructions load the loop index to loop index + 16. Each instruction loads four elements into the 256-bit register.

```
vmovdqu ymm5, ymmword ptr [rdi + 8*rax] ; ↓ ↓ ↓ ↓
vmovdqu ymm6, ymmword ptr [rdi + 8*rax + 32] ; ↓ ↓ ↓ ↓
vmovdqu ymm7, ymmword ptr [rdi + 8*rax + 64] ; ↓ ↓ ↓ ↓
vmovdqu ymm8, ymmword ptr [rdi + 8*rax + 96] ; ↓ ↓ ↓ ↓
```

Bitwise AND the 4 elements stored in `ymm5`, `ymm6`, `ymm7` and `ymm8` with the 4 copies of a 64-bit 1 stored in `ymm1`. The processor stores the status bits of the result in the mask registers `k1`, `k2`, `k3`, and `k4`. So, the following four instructions set $4 \times 4 = 16$ bits in the mask registers. The status bit is set to 1 if the AND operation results in a 0. It is set to 0 if the AND operation results in a 1. Thus, the `k1`, `k2`, `k3`, and `k4` mask registers will have 1s in the status bits if the corresponding elements are even and 0s in the status bits if the corresponding elements are odd.

```
vp testnmq    k1, ymm5, ymm1 ; 🔍 🔍 🔍 🔍
vp testnmq    k2, ymm6, ymm1 ; 🔍 🔍 🔍 🔍
vp testnmq    k3, ymm7, ymm1 ; 🔍 🔍 🔍 🔍
vp testnmq    k4, ymm8, ymm1 ; 🔍 🔍 🔍 🔍
```

Multiply the four elements stored in `ymm5`, `ymm6`, `ymm7`, and `ymm8` with themselves. The following four instructions perform $4 \times 4 = 16$ multiplications.

Note that the code performs multiplications for both even and odd values. Later, it will use the mask registers `k1`, `k2`, `k3`, and `k4` to ignore the squares of odd values.

```
vp mullq    ymm5, ymm5, ymm5 ; ✖ ✖ ✖ ✖
vp mullq    ymm6, ymm6, ymm6 ; ✖ ✖ ✖ ✖
```


<code>vpmullq ymm7, ymm7, ymm7</code>	; $\times \times \times \times$
<code>vpmullq ymm8, ymm8, ymm8</code>	; $\times \times \times \times$

Each of the four following instructions accumulates the result of its 4 multiplications into a single 256-bit register (ymm0, ymm2, ymm3 or ymm4). The mask registers k1, k2, k3, and k4 mask out the results of odd value multiplications.

The code accumulates the 16 intermediate results into four registers in the loop body. It then adds these registers to obtain the result in the loop epilogue.

<code>vpaddq ymm0 {k1}, ymm0, ymm5</code>	; $+$ $+$ $+$ $+$
<code>vpaddq ymm2 {k2}, ymm2, ymm6</code>	; $+$ $+$ $+$ $+$
<code>vpaddq ymm3 {k3}, ymm3, ymm7</code>	; $+$ $+$ $+$ $+$
<code>vpaddq ymm4 {k4}, ymm4, ymm8</code>	; $+$ $+$ $+$ $+$

Increment the loop index by 16. We are processing 16 elements in each iteration of the loop.

<code>add rax, 16</code>

Check if the loop index has not reached the end of the array.

<code>cmp rax, 1024</code>

↩ If the loop index has not reached the end of the array, jump to the beginning of the loop.

<code>jne .LBB0_1</code>

Loop end

Add the 16 intermediate results stored in ymm0, ymm2, ymm3, and ymm4 into four intermediate results stored in ymm0.

<code>vpaddq ymm0, ymm2, ymm0</code>	; $+$ $+$ $+$ $+$
<code>vpaddq ymm0, ymm3, ymm0</code>	; $+$ $+$ $+$ $+$
<code>vpaddq ymm0, ymm4, ymm0</code>	; $+$ $+$ $+$ $+$

Extract the upper 128 bits of ymm0 into xmm1, storing the four intermediate results in the xmm0 and xmm1 registers, with two intermediate results in each register.

<code>vextracti128 xmm1, ymm0, 1</code>

Before the next instruction:

- `xmm0` = [intermediate result 0, intermediate result 1]
- `xmm1` = [intermediate result 2, intermediate result 3]

After the next instruction:

- `xmm0` = [intermediate result 0 + intermediate result 2, intermediate result 1 + intermediate result 3]

<code>vpaddq xmm0, xmm0, xmm1</code>	; $+$ $+$
--------------------------------------	-----------

Swap the 64-bit halves of xmm0, placing the two intermediate results in the lower 64 bits of xmm0 and xmm1.

Part D Iteration and Optimization

```
vpshufd xmm1, xmm0, 238
```

++ Perform 64-bit additions on the upper and lower 64 bits of `xmm0` and `xmm1`. We only care about the lower 64 bits of the result.

```
vpaddq xmm0, xmm0, xmm1 ; ++
```

 Move the lower 64 bits of `xmm0` into `rax`.

```
vmovq rax, xmm0 ; ☐ ☒ 12 34
```

 Ensure all `ymm` registers have their upper 128 bits cleared to prevent invalid results from `ymm` access.

```
vzeroupper ; ☒ ☒ ☐ ☐
ret ;  Return result in rax
```

Learn More: Conditional Addition with Masks - `vpaddq`

We saw the following instruction in the assembly:

```
vpaddq ymm0 {k1}, ymm0, ymm5
```

This instruction performs a conditional addition, executing the addition only if the corresponding bit in the mask register `k1` is set. The setting of the mask register `k1` depends on the result of the `vptestnmq` instruction.

Let us look at the instruction in detail:

- `vpaddq`: This stands for Vector Packed ADD Quadword (64-bit). It adds packed quadword integers from the second operand (source operand) and the third operand and stores the packed integer results in the destination operand (first operand).
- `ymm0 {k1}, ymm0, ymm5`: This specifies the operands for the instruction. `ymm0` and `ymm5` are 256-bit `ymm` registers that can hold integers or floating-point numbers. The mask register `{k1}` determines which data lanes in the destination register are updated. The instruction adds `ymm5` to `ymm0` for the data lanes where `k1` is set. In cases where `k1` is not set, the data in `ymm0` will remain unaltered.

Experiment: Impact of Array Size and Power of `x` on Generated Code

Experiment with [Compiler Explorer](https://godbolt.org/z/az9Wj3Y1P) (<https://godbolt.org/z/az9Wj3Y1P>) to understand how the generated code changes when you:

- [Change the Array Size](#)
- [Change the Power of `x`](#)

Change the Array Size

Change the array sizes in the following code and observe the changes in the generated code.

```
// Change the array size 
pub fn sum_of_even_squares_specific_size(arr: &[i64; 1024]) -> i64 {
```

```
    sum_of_even_squares(arr)
}
```

Experiment with the following array sizes and confirm the observations:

Array size	Observation
64	The compiler completely unrolls the loop.
128	The compiler completely unrolls the loop.
1023	The compiler follows the same structure as 1024 iterations but limits the number of iterations to 1008 (16*63). It unrolls the remaining instructions and executes them after the loop.
1025	The code follows the same structure as 1024 (16*64) iterations. It handles the 1025th iteration outside the loop.

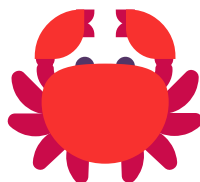
Change the Power of x

Change the power of x from 2 and see the impact on the generated code. The compiler prioritizes branch avoidance over calculating the power with a cascade of multiplications. Even at `x.pow(500)`, branch avoidance is prioritized. Just the number of `vpmullq` instructions increases.

```
pub fn sum_of_even_squares<const N: usize>(arr: &[i64; N]) -> i64 {
    arr.iter()
        .filter(|&x| x % 2 == 0) // filter out odd numbers
        .map(|&x| x.pow(2)) // ← Change power of x
        .sum() // sum them up
}
```

Key Takeaways

- AVX-512 processes 16 elements in each loop iteration by utilizing `ymm` registers, which handle four 64-bit elements per instruction.
- The `k` registers mask the results of multiplications on odd numbers, serving to avoid branching.
- When the array size is not a multiple of 16, the compiler executes the loop until the last multiple of 16 and runs the remaining iterations outside the loop.
- AVX-512 SIMD instructions can deliver a 4x speedup over the scalar code. However, memory bandwidth and thermal throttling can limit the speedup. AVX-512 instructions can increase power usage and may cause other cores on the CPU to downclock, potentially reducing overall system performance.
 - You should perform benchmarks to confirm the performance benefits of using AVX-512 instructions.



Chapter 10

In-Depth: Array Slices and Pattern Matching

This chapter will cover the assembly code generated for array slices and pattern matching. We will examine how the compiler optimizes the code to minimize branching and improve performance.

Array Slices

In Rust, array slices provide a view into a contiguous sequence of elements, such as arrays, vectors, or strings, without taking ownership. This feature is handy for referencing a portion of a data structure without copying the data.

Slices are dynamically sized, meaning their size is determined at runtime. Each slice is represented by a two-word object in memory: the first word is a pointer to the data, and the second is the slice's length. This design makes slices lightweight and efficient to pass around, as they simply point to existing data rather than own it.

By leveraging borrowed references, slices adhere to Rust's strict borrowing rules, which helps maintain memory safety and concurrency guarantees.

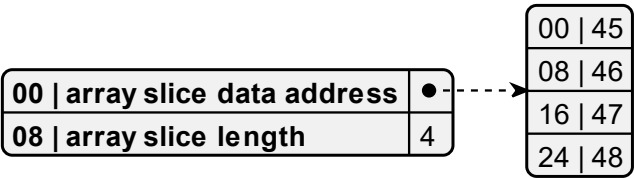
Array Slice Pattern Matching Example

The following code shows a function that takes an array slice as input and pattern matches on the slice to return the last element of the slice if the slice has a length of 2, 3, or 4 and matches the specified pattern. In all other cases, the function returns a 0.

```
pub fn process_array_slice(input: &[i32]) -> i32 {
    match input {
        [42, a] => *a,
        [43, 44, a] => *a,
        [45, 46, 47, a] => *a,
        [..] => 0,
    }
}
```

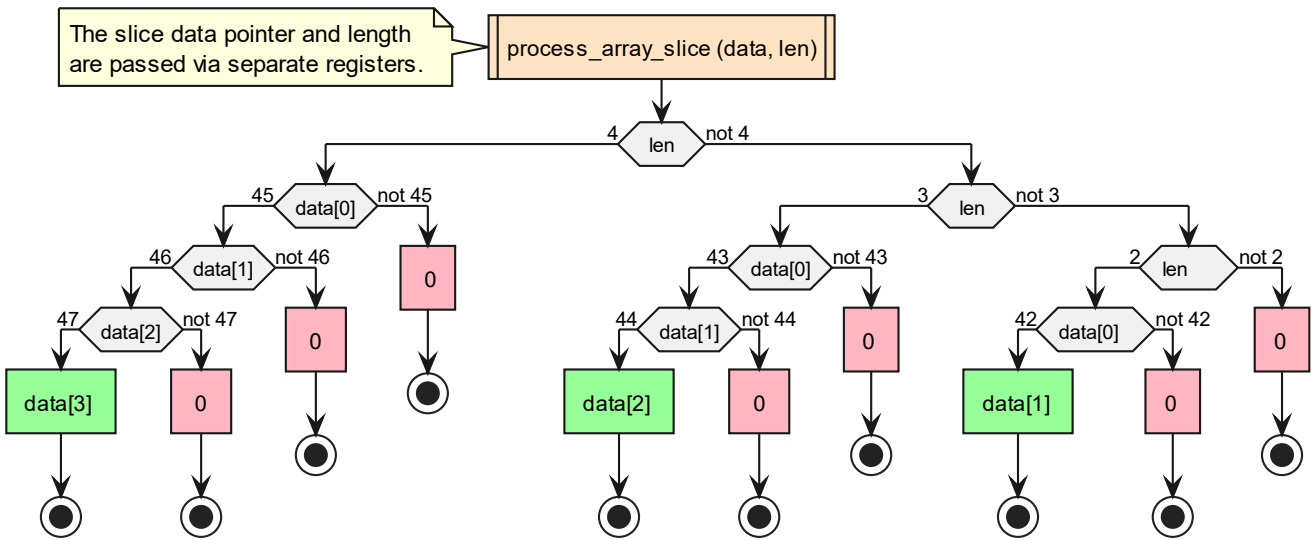
Part D Iteration and Optimization

The input parameter passed to `process_array_slice` is an array slice. The following diagram shows an example of an array slice. The first word of the slice is a pointer to the data, and the second word is the length of the slice.



Assembly Flowchart: Array Slice Processing with Pattern Matching

The following flowchart shows the assembly code generated by the Rust compiler for the `process_array_slice` function. Note that the generated code first checks the length of the slice and then checks the elements of the slice to match the patterns in the Rust code.



Array Slice Pattern-Matching Assembly Code

In this code, the `rsi` register holds the slice length, `rdi` points to the slice's data, and `eax` is the return register. The comparisons assess the slice's length and elements to match the patterns in the Rust code. The matching legs set an offset in the `ecx` register to identify the slice entry to return. This offset in `ecx` ensures the return of the correct slice element in the matched pattern.

- Input:**
 - `rdi` = array slice data address
 - `rsi` = array slice length
- Output:**
 - `rax` = return value

example::process_array_slice:

```
xor    eax, eax           ; 0 Initialize return value to 0
```

Check array slice lengths to determine which pattern to match.

```

cmp    rsi, 4          ; ⚖️ Compare input length to 4
je     .LBB0_8          ; ➡️ If length is 4, jump

cmp    rsi, 3          ; ⚖️ Compare input length to 3
je     .LBB0_6          ; ➡️ If length is 3, jump

cmp    rsi, 2          ; ⚖️ Compare input length to 2
jne    .LBB0_5          ; ➡️ If length is not 2, jump

```

2 Pattern match for the two-element case. Match the first element.

```

mov     ecx, 1          ; 1 Set offset to 1 (for two-element case)
cmp     dword ptr [rdi], 42 ; ⚖️ Compare the 1st element of the slice to 42
je     .LBB0_4          ; ➡️ If the 1st element is 42, jump

```

❌ Nothing matched, return 0

```

.LBB0_5:
ret     ; ⬅️ BACK Return 0 (default case)

```

3 Pattern match for the three-element case. Match the first two elements.

```

.LBB0_6:
cmp     dword ptr [rdi], 43 ; ⚖️ Compare the 1st element
                                ; of the slice to 43

jne     .LBB0_5          ; ➡️ If the 1st element is not 43,
                                ; jump to the default case

mov     ecx, 2          ; 2 Set offset to 2 (three-element case)
cmp     dword ptr [rdi + 4], 44 ; ⚖️ Compare the 2nd element to 44
jne     .LBB0_5          ; ➡️ If the 2nd element is not 44,
                                ; jump to the default case

jmp     .LBB0_4          ; ➡️ Jump to label .LBB0_4 to return
                                ; last element

```

4 Pattern match for four-element case. Match the first three elements.

```

.LBB0_8:
cmp     dword ptr [rdi], 45 ; ⚖️ Compare the 1st element
                                ; of the slice to 45
jne     .LBB0_5          ; ➡️ If the 1st element is not 45,
                                ; jump to the default case

cmp     dword ptr [rdi + 4], 46 ; ⚖️ Compare the 2nd element to 46
jne     .LBB0_5          ; ➡️ If the 2nd element is not 46,
                                ; jump to the default case

mov     ecx, 3          ; 3 Set the offset to 3
                                ; (four-element case)
cmp     dword ptr [rdi + 8], 47 ; Compare the 3rd element to 47

```

Part D Iteration and Optimization

```
jne      .LBB0_5          ;  If the 3rd element is not 47,
                        ; jump to the default case
```

 Return the last element of the slice. The offset in ecx is employed to compute the address of the last element of the slice.

```
.LBB0_4:
mov      eax, dword ptr [rdi + 4*rcx] ;  Load last element of slice
                        ; into eax for return
ret      ;  Return eax
```



Experiment: Array Match Patterns and If-Else Comparisons

Experiment with [Compiler Explorer](https://godbolt.org/z/449fr6Tqs) (<https://godbolt.org/z/449fr6Tqs>) to see how the generated assembly code changes when you change the patterns in the Rust code. A few things to try are:

- [Two Match Patterns of the Same Length](#)
- [Matching the Tail of a Slice](#)

We will also [compare the code generated using pattern matching with conventional if-else checks](#).

Two Match Patterns of the Same Length

Change the Rust code to match two patterns of the same length. The `[43, 44, 48, a]` and `[43, 44, 47, _]` patterns match the same length of array slice with the same first two elements. You will see that the compiler generates efficient code to check for the first two elements and then checks the third element to determine which pattern to match.

```
pub fn process_array_slice(input: &[i32]) -> i32 {
    match input {
        [42, a] => *a,
        [43, 44, 48, a] => *a,
        [43, 44, 47, _] => 42,
        [..] => 0,
    }
}
```

Matching the Tail of a Slice

Change the Rust code to match the tail of a slice. We will match the following patterns:

- `[4, .., 2]` pattern matches on a slice that starts with a 4 and ends with a 2.
- `[.., 9, 7]` pattern matches on a slice that ends with a 9 and a 7.
- `[..]` pattern matches on any other slice.

Here is the Rust code for the experiment.

```
pub fn process_array_slice(input: &[i32]) -> i32 {
    match input {
        [4, .., 2] => 42,
        [.., 9, 7] => 97,
        [..] => 0,
    }
}
```


Look at the generated code in [Compiler Explorer](https://godbolt.org/z/3MP8nWqG) (<https://godbolt.org/z/3MP8nWqG>) to see how the compiler generates efficient code to check the first and last elements of the slice to determine which pattern to match.

The assembly code generated in this experiment differs from the assembly code we saw earlier. The key differences are:

- Pattern matching on the tail of a slice requires the compiler to generate code to calculate the address of the last element of the slice.
- The compiler reorders the code to parallelize memory moves and branch execution.

The input and output registers for the function are:

-  Input:
 - `rdi` =  array slice data address
 - `rsi` =  array slice length
-  Output:
 - `rax` = return value

`example::process_array_slice:`

```
;  Initialize return value to 0
xor     eax, eax

;  Compare the slice length with 2
cmp     rsi, 2

;  If the slice length is less than 2, take the default case and return 0
jb      .LBB0_5

;  Compare the first element of the slice with 4
cmp     dword ptr [rdi], 4

;  Get the last element of the slice.
; Last_element_address = slice_data_address + (slice_length-1) * element_size
mov     ecx, dword ptr [rdi + 4*rsi - 4]

;  If the first element is not 4, move on to check the next pattern
jne     .LBB0_3

; Move 42 into the return value register in case the next pattern match
; succeeds.
mov     eax, 42

;  Compare the last element of the slice with 2
cmp     ecx, 2

;  If the last element is not 2, move on to check the next pattern
jne     .LBB0_3

.LBB0_5:

;  Return the return value in rax
ret
```

Part D Iteration and Optimization

```

.LBB0_3:
; 0 Set the return value to 0
mov     eax, 0

; ⚖ Compare the last element of the slice with 7
cmp     ecx, 7

; 🔄 If the last element is not 7, return 0 (default case)
jne     .LBB0_5

; 0 Set ecx to 0
xor     ecx, ecx

; ⚖ Compare the second to last element of the slice with 9
cmp     dword ptr [rdi + 4*rsi - 8], 9

; Load the value 97 into the return value register in anticipation
; of finding a match for the penultimate element.
mov     eax, 97

; 🔄 Conditionally return 97 if the second to the last element
; is 9 else move ecx into eax and return 0.
cmovne  eax, ecx

; ⬅️ Return the return value in rax
ret

```

Comparing with Code Written Without Pattern Matching

The following code is equivalent to the code we saw earlier. It is interesting to see how much more verbose the code is when we do not use pattern matching. Copy the following code into [Compiler Explorer](https://godbolt.org/z/449fr6Tqs) (<https://godbolt.org/z/449fr6Tqs>) to compare the code with the `process_array_slice` function that uses pattern matching.

```

pub fn process_array_slice_no_pattern_matching(input: &[i32]) -> i32 {
    let length = input.len();

    if length == 4 {
        if input[0] == 45 && input[1] == 46 && input[2] == 47 {
            return input[3];
        }
    } else if length == 3 {
        if input[0] == 43 && input[1] == 44 {
            return input[2];
        }
    } else if length == 2 {
        if input[0] == 42 {
            return input[1];
        }
    }
}

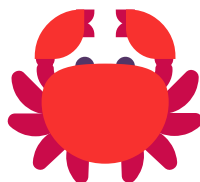
```

```
    return 0;  
}
```

Spoiler alert: The code generated by the compiler is the same as the code generated for the `process_array_slice` function that uses pattern matching.

Key Takeaways

- The array slice contains a pointer to the slice data and the length of the slice. In our examples, the compiler passes the pointer to the slice data and slice length via two registers.
- Rust offers a robust array slice pattern matching mechanism. With it, you can match various aspects of a slice: its length, first and last elements, and even its tail. This powerful feature ensures developers can efficiently handle different slice scenarios in their pattern-matching constructs.
- The compiler generates efficient code for array slice pattern matching. It minimizes the comparisons required to match a pattern. Array slice pattern matching is a zero-cost abstraction. We saw this when we [compared the assembly code generated with and without pattern matching](#).
- Pattern matching can target the tail of a slice, necessitating the compiler to generate code that calculates the address of the slice's last element using its length. We saw this in the [experiment where we matched on the tail of a slice](#).



Chapter 11

Vector Iteration in Assembly

Vectors are the most used collection type in Rust. This chapter will look under the hood to understand the assembly code generated for a vector iteration. We will see that the length of a vector plays a vital role in the runtime selection of the loop constructs used in the iteration. Finally, we will Experiment with Compiler Explorer to see how we can influence loop unrolling and vectorization of the loop.

Vector Memory Layout

Before we go any further, let us look at the memory layout of a vector. The definition of a vector follows. Vectors in Rust contain:

- A pointer to a heap vector buffer (`buf.ptr`).
- The capacity of the buffer (`buf.cap`).
- The length (`len`) of the vector.

The vector's buffer is a pointer to an array consisting of elements of type `T`. The capacity of this array defines its size, indicating the maximum number of elements it can accommodate without needing to reallocate the buffer. Meanwhile, the vector's length specifies the current count of elements contained within the vector.

The vector struct (`Vec`) is defined as follows. It contains a pointer to the heap-allocated buffer (`buf`) and the vector length (`len`).

```
pub struct Vec<T, A: Allocator = Global> {  
    // Points to the start of the vector data and keeps the capacity information  
    buf: RawVec<T, A>,  
    // Vector length  
    len: usize,  
}
```

The vector buffer is allocated using the `RawVec` struct. The `RawVec` struct contains a pointer to the heap-allocated buffer (`ptr`) and the buffer's capacity (`cap`). The `RawVec` struct is defined as follows.

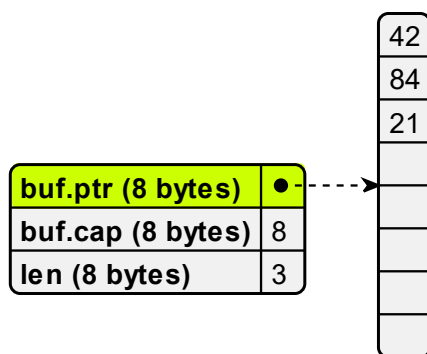
```
struct RawVec<T> {  
    // Points to the heap address that stores the vector data  
    ptr: NonNull<T>,  
}
```

Part D Iteration and Optimization

```
// Capacity of the vector
cap: usize,
_marker: PhantomData<T>,
}
```

Note: The `_marker` is 0-sized as it is of the `PhantomData<T>` type.

The following figure shows the memory layout of a vector. The heap pointer (`buf.ptr`) points to the start of the vector buffer. In this example, the vector buffer contains three `i64` integers. It has a length (`len`) of 3 and a capacity (`buf.cap`) of 8.



Sample Vector Handling Code

Let us explore the Rust to assembly mapping of the `Vec` struct using a simple function, `increment_by`, that increments the elements of a vector by a specified value (`num`). The sample also includes `increment_example` that calls the `increment_by` function.

```
// Increment each element of a i64 vector by num. A mutable
// reference to the vector is passed in.
pub fn increment_by(num: i64, list: &mut Vec<i64>) {
    // Iterate over the vector and increment each element.
    for item in list {
        // Note that the item needs to be dereferenced
        *item += num;
    }
}

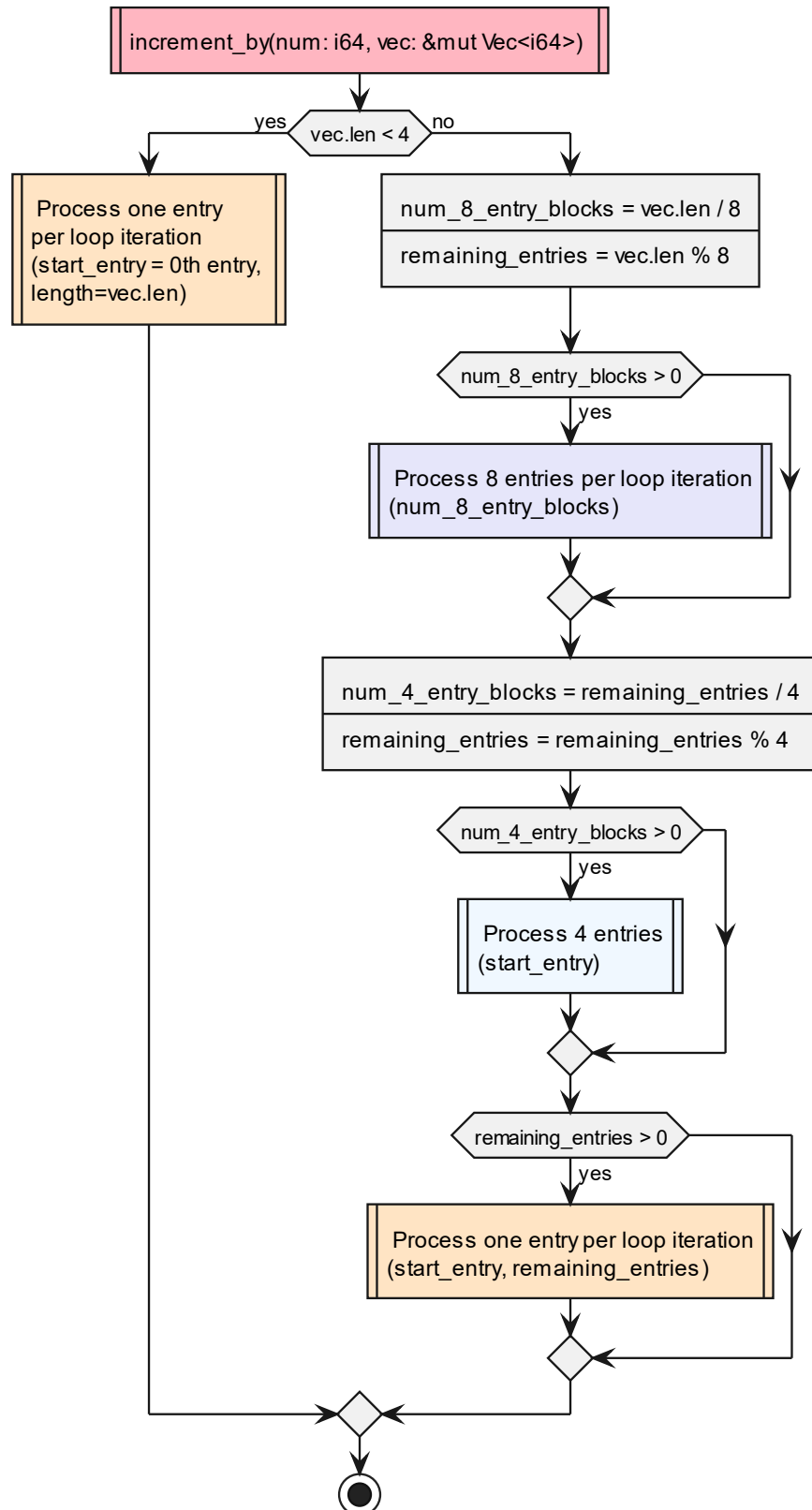
pub fn increment_example(inc_by: i64, array: [i64; 4]) -> Vec<i64> {
    let mut list = array.to_vec();
    increment_by(inc_by, &mut list);
    list
}
```

Assembly Insights: Optimizations for Varying Vector Lengths

The compiler optimizes the generated assembly code for the `increment_by` function to operate on vectors of various lengths. This section will examine the code's overall flow and then delve into the details of code blocks optimized for different vector lengths. With this knowledge, we can comprehend the assembly code generated for the `increment_by` function.

Arbitrary Length Handling Flowchart

The following flowchart of the `increment_by` function provides a high-level view of handling vectors of arbitrary length.



The preceding flowchart uses the following code blocks to handle vectors of different lengths.

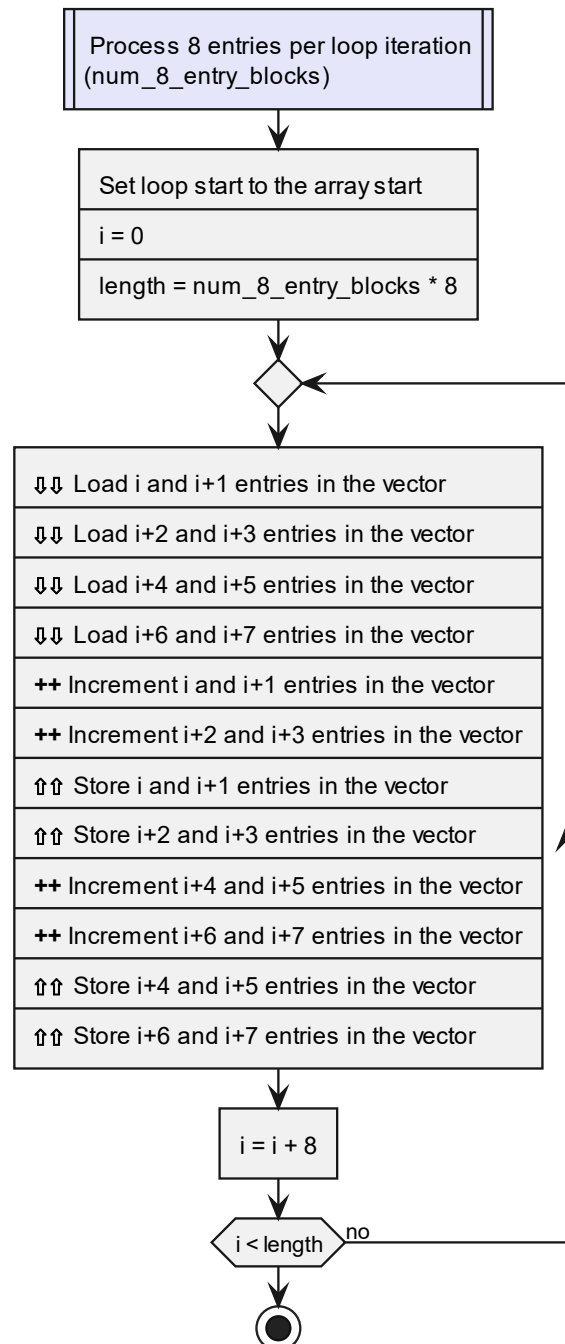
Part D Iteration and Optimization

- Vector Length Is a Multiple of 8
- Vector Length Is Equal to 4
- Vector Length Is Less than 4

Vector Length Is a Multiple of 8

The following code block handles vectors with a length that is a multiple of 8. The compiler unrolls the loop eight times and loads the vector elements into the SIMD registers. The code then increments the SIMD registers. Finally, the code stores back the SIMD registers in the vector buffer. Each SIMD instruction operates on two elements of the vector, so each operation in the loop results in four SIMD operations.

Flowchart for Processing Eight Entries per Loop Iteration



Now that we have looked at the flow for this code block, let us look at the generated assembly.

For vectors with lengths that are a multiple of 8, the code handles eight 64-bit operations per iteration. The code copies eight 64-bit numbers to the `xmm` registers using four `movdqu` instructions. Four `paddq` instructions perform the eight additions needed in the loop. Finally, four `movdqu` instructions copy the results into the vector buffer.

```
.LBB0_5:

movdqu  xmm1, xmmword ptr [rcx + rsi]      ; Load the 1st & 2nd number
movdqu  xmm2, xmmword ptr [rcx + rsi + 16] ; Load the 3rd & 4th number
movdqu  xmm3, xmmword ptr [rcx + rsi + 32] ; Load the 5th & 6th number
movdqu  xmm4, xmmword ptr [rcx + rsi + 48] ; Load the 7th & 8th number

paddq   xmm1, xmm0                        ; Increment 1st & 2nd number
paddq   xmm2, xmm0                        ; Increment 3rd & 4th number

movdqu  xmmword ptr [rcx + rsi], xmm1     ; Store the 1st & 2nd number
movdqu  xmmword ptr [rcx + rsi + 16], xmm2 ; Store the 3rd & 4th number

paddq   xmm3, xmm0                        ; Increment 5th & 6th number
paddq   xmm4, xmm0                        ; Increment 7th & 8th number

movdqu  xmmword ptr [rcx + rsi + 32], xmm3 ; Store the 5th & 6th number
movdqu  xmmword ptr [rcx + rsi + 48], xmm4 ; Store the 7th & 8th number

add     rsi, 64                          ; Increment the loop index
add     rax, -2                          ; Decrement the count as rax
                                           ; keeps track of the number of
                                           ; quad 64-bit numbers to process
jne     .LBB0_5                          ; Jump if there are still quad
                                           ; 64-bit numbers to process

test    r10b, 1                          ; Check if there are still
                                           ; entries to process
je      .LBB0_8                          ; Jump if less than 4 entries
                                           ; need to be processed.
```

💡 Learn More: Cache Line and Eight Times Unrolling

The 8-times unrolling technique aligns with the cache line size of x86-64 processors. A cache line, the data transfer unit between the main memory and the processor cache, determines how the processor reads from and writes to the main memory. On x86-64 systems, the cache line size is 64 bytes. Consequently, each iteration of the “multiple of 8” loop reads and writes an entire cache line.

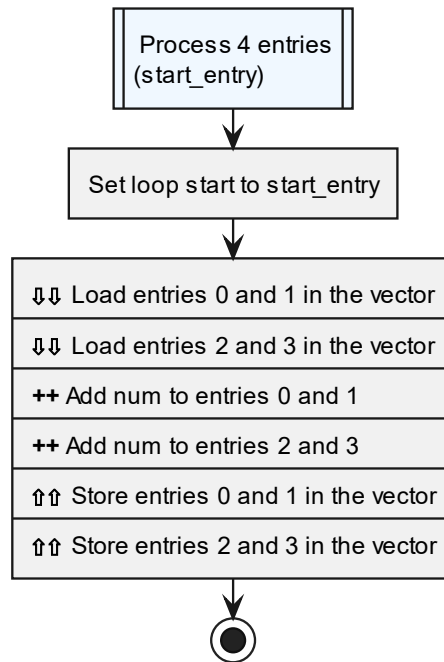
Dig deeper into cache organization and cache lines in the Medium post:

- Why software developers should care about CPU caches (<https://medium.com/p/8da04355bb8a>)

Vector Length Is Equal to 4

The code block handling vectors of length 4 consists of two vector loads, two vector additions, and two vector stores, with each vector operation acting on two 64-bit numbers.

Part D Iteration and Optimization



Now that we have looked at the flowchart, let us look at the generated assembly code.

Vector instructions perform two 64-bit operations in parallel. The `movdqu` instruction moves the vector data from the vector buffer to an `xmm` register. The `paddq` instruction adds the specified value to the vector data. The `movdqu` instruction moves the vector data back to the vector buffer.

```

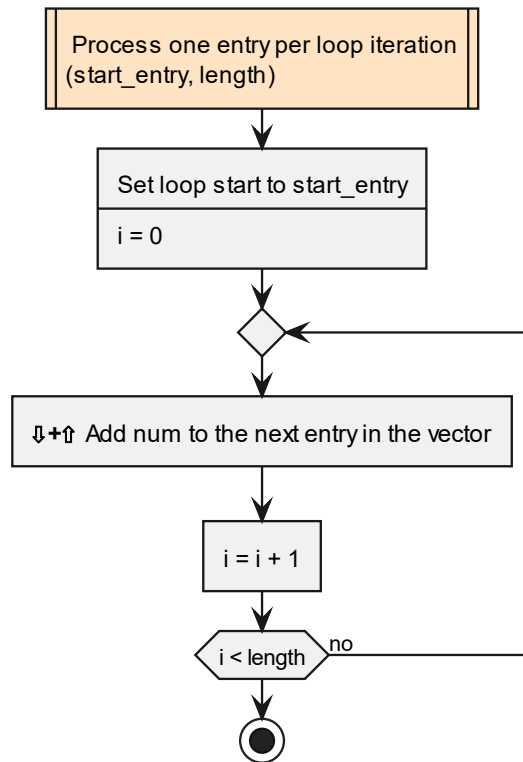
.LBB0_7:
movdqu  xmm1, xmmword ptr [rcx + rsi]      ; ⇓⇓ Load two 64-bit numbers
                                           ; from the vector
movdqu  xmm2, xmmword ptr [rcx + rsi + 16] ; ⇓⇓ Load two more 64-bit
                                           ; numbers from the vector

paddq   xmm1, xmm0                        ; ++ Add the increment value to
                                           ; the first two 64-bit number
paddq   xmm2, xmm0                        ; ++ Add the increment value to
                                           ; the 3rd and 4th of 64-bit number

movdqu  xmmword ptr [rcx + rsi], xmm1      ; ⇑⇑ Store the result back in
                                           ; the vector (1st and 2nd number)
movdqu  xmmword ptr [rcx + rsi + 16], xmm2 ; ⇑⇑ Store the result back in
                                           ; the vector (3rd and 4th number)
  
```

Vector Length Is Less than 4

When the vector length is less than 4, the compiler generates a loop that processes one entry per loop iteration. The flowchart for this code block follows.



For vectors shorter than 4, the generated assembly code loops over each entry in the vector and increments it by the specified value.

```

.LBB0_11:

add    qword ptr [rax], rdi ; ↓ + ↑ Add num to the next
                           ; entry in the vector

add    rax, 8               ; ↗ Move to the next entry in the vector

cmp    rax, rcx             ; ⚖ Check if at the end of the vector
jne    .LBB0_11             ; ↺ Jump if the vector's end has
                           ; not been reached.
  
```

Assembly Code for Handling Vectors of Arbitrary Length

We have analyzed the specialized assembly blocks for >8, 4, and <4 cases. Let us look at the following code that handles vectors of arbitrary length. The complete code of the function combines the three code blocks to handle vectors of arbitrary size. For example, the code will handle a vector of length 39 as follows:

Index Range	Total Entries	Iterations	Operations per Loop	Assembly Label
0-31	32	4	8 (4 sets of vector operations)	.LBB0_5
32-35	4	1	4 (2 vector operations)	.LBB0_7
36-38	3	1	1 per loop	.LBB0_11

Let us now look at the assembly code for the `increment_by` function that handles vectors of arbitrary length.

- **12** **34** Input:
 - rsi = address of the vector

Part D Iteration and Optimization

- rdx = increment value

example::increment_by:

```

mov    r9, qword ptr [rsi + 16] ;  r9 now contains length of vector
test   r9, r9                   ;  Check if the vector is empty
je     .LBB0_12                 ;  Jump to return if the vector is empty

mov    rcx, qword ptr [rsi]     ;  rcx = address of the vector data
lea    rax, [r9 - 1]           ; rax = index of last entry in vector

movabs rdx, 2305843009213693951 ; Hex 1FFF_FFFF_FFFF_FFFF (3 MSB cleared)

and     rdx, rax                ;  rdx = index of the last entry in
                                ; vector with 3 MSB cleared.

mov     rax, rcx                ; rax = address of the vector data
cmp     rdx, 3                  ;  Check if the vector length < 4
jb      .LBB0_10                ;  Jump if the vector length < 4

```

★ Handle vectors with 4 or more entries.

```

inc     rdx                     ;  Add 1 to the last entry index
                                ; to get the length

mov     r8, rdx                 ; r8 = number of items to be processed

and     r8, -4                  ; Mask of the lower 2 bits to get the count
                                ; of entries that are divisible by 4.

movq    xmm0, rdi               ; xmm 0 = Parameter num, the increment value

pshufd  xmm0, xmm0, 68          ;  Make a copy of num to the upper 64 bits to
                                ; facilitate vector addition.
                                ; 68 = 0100 0100 results in copying the lower
                                ; 64 bits to the upper 64 bits.
                                ; A reference with a detailed description
                                ; follows the assembly code.

lea     rax, [r8 - 4]           ; rax = 4 less than the number of items

mov     r10, rax                ; r10 = 4 less than the number of items

shr     r10, 2                  ; Divide r10 by 4 to get the index of the
                                ; last quad 64-bit numbers.

inc     r10                     ;  Add 1 to get the count of
                                ; quad 64-bit numbers

test    rax, rax                ;  Any more sets of 4 entries?
je      .LBB0_3                 ;  Jump if no more sets of 4 entries

mov     rax, r10                ; rax = count of quad 64-bit numbers
and     rax, -2                  ; Mask of the lower 2 bits to get the count of

```

```

; quad 64-bit numbers that are divisible by 4.
xor     esi, esi      ; 0 Set the quad 64-bit numbers index to 0

```

★ Process eight 64-bit additions per iteration.

```

.LBB0_5:
movdqu  xmm1, xmmword ptr [rcx + rsi]      ; ↓ ↓ Load 1st & 2nd numbers
movdqu  xmm2, xmmword ptr [rcx + rsi + 16]  ; ↓ ↓ Load 3rd & 4th numbers
movdqu  xmm3, xmmword ptr [rcx + rsi + 32]  ; ↓ ↓ Load 5th & 6th numbers
movdqu  xmm4, xmmword ptr [rcx + rsi + 48]  ; ↓ ↓ Load 7th & 8th numbers

paddq   xmm1, xmm0      ; ++ Increment 1st & 2nd
paddq   xmm2, xmm0      ; ++ Increment 3rd & 4th

movdqu  xmmword ptr [rcx + rsi], xmm1      ; ↑ ↑ Store 1st & 2nd numbers
movdqu  xmmword ptr [rcx + rsi + 16], xmm2 ; ↑ ↑ Store 3rd & 4th numbers

paddq   xmm3, xmm0      ; ++ Increment 5th & 6th
paddq   xmm4, xmm0      ; ++ Increment 7th & 8th

movdqu  xmmword ptr [rcx + rsi + 32], xmm3 ; ↑ ↑ Store 5th & 6th numbers
movdqu  xmmword ptr [rcx + rsi + 48], xmm4 ; ↑ ↑ Store 7th & 8th numbers

add     rsi, 64          ; ↻ Move the index to the
                        ; next set of 8 numbers

add     rax, -2          ; ↻ Decrement the count by 2
                        ; as two quad 64-bit numbers
                        ; have been processed.

jne     .LBB0_5          ; ↺ Jump if there are still
                        ; quad 64-bit numbers to handle

test    r10b, 1         ; ↻ Check if there are still quad entries to process
je      .LBB0_8          ; ↺ Jump if less than 4 entries need to be processed.

```

★ Block processing four 64-bit additions.

```

.LBB0_7:
movdqu  xmm1, xmmword ptr [rcx + rsi]      ; ↓ ↓ Load next two entries
movdqu  xmm2, xmmword ptr [rcx + rsi + 16]  ; ↓ ↓ Load next two entries

paddq   xmm1, xmm0      ; ++ Increment first two
paddq   xmm2, xmm0      ; ++ Increment next two

movdqu  xmmword ptr [rcx + rsi], xmm1      ; ↑ ↑ Store first two entries
movdqu  xmmword ptr [rcx + rsi + 16], xmm2 ; ↑ ↑ Store next two entries

.LBB0_8:

cmp     rdx, r8          ; ⚖ Check if all entries have been handled

```

Part D Iteration and Optimization

```

je      .LBB0_12      ; 🔄 Jump if all entries have been handled
lea     rax, [rcx + 8*r8] ; 🏠 rax = address of the last entry

.LBB0_10:

lea     rcx, [rcx + 8*r9] ; 🏠 rcx = address of last entry in vector
; rax points to the first entry in the vector

```

★ Process one 64-bit addition per iteration.

```

.LBB0_11:
add     qword ptr [rax], rdi ; ⬇ + ⬆ Add num to the next entry

add     rax, 8              ; 🏠 Move to the next entry in the vector
cmp     rax, rcx            ; ⚖ Check if at the end of the vector
jne     .LBB0_11           ; 🔄 Jump if vector end has not been reached

.LBB0_12:

ret                                           ; ⬅ BACK Return from the function

.LBB0_3:
xor     esi, esi          ; 0 esi is now 0

test    r10b, 1           ; 🔍 Check if a set of quad 64-bit numbers is available

jne     .LBB0_7           ; 🔄 Jump if quad 64-bit numbers are available

jmp     .LBB0_8           ; 🔄 Jump if less than four 64-bit numbers
; are available

```

💡 Learn More: pshufd Instruction Shuffling Options

The preceding code block uses `pshufd`, an instruction that shuffles the four 32-bit numbers in the source operand and stores the result in the destination operand. For an explanation of the `pshufd` shuffling options, refer to a [stack exchange article \(https://bit.ly/pshufd\)](https://bit.ly/pshufd).

Inlining, Loop Unrolling, and Memory Optimizations

We have looked at the code generated for `increment_by`. Now, let us examine a call to the function from the `increment_example` function that calls `increment_by` with a vector of length 4.

```

pub fn increment_example(inc_by: i64, array: [i64; 4]) -> Vec<i64> {
    let mut list = array.to_vec();
    increment_by(inc_by, &mut list);
    list
}

```

The generated assembly code for `increment_example` is much simpler than for the `increment_by` function. This simplification is evident from the absence of any call to `increment_by`. The compiler has inlined the relevant code from the function into `increment_example`.

- The compiler figures out that inlining the `increment_by` function would reduce a function call.

- Once the compiler decides to inline, it sees that the vector will only be of length 4. The compiler removes the logic for handling any arbitrary length vector. It optimizes the inlined code for a vector of length 4.
- The compiler also unrolls the loop to process four 64-bit additions.

Also, note that the assembly code for `increment_example` calls `__rust_alloc` for allocating the vector data on the heap. If the memory allocation fails, the function throws an exception using the `ud2` instruction.

Another point to note is that the compiler must copy the data from the array into the newly created vector.

-  Input:
 - `rdi` = `inc_by`
 - `rsi` = address of array
-  Output:
 - `rax` = address of vector

```
.LCPI1_0:
.quad 4          ; Vector length
.quad 4          ; Vector capacity

example::increment_example:

push    r15      ; Save r15 on the stack
push    r14      ; Save r14 on the stack
push    rbx      ; Save rbx on the stack

mov     r15, rdx  ; r15 = address of the array
mov     r14, rsi  ; r14 = inc_by
mov     rbx, rdi  ; rbx = address of vector on the stack
```

 Allocate memory for the vector buffer

```
mov     edi, 32   ; edi = number of 64-bit numbers to process
mov     esi, 8    ; esi = byte alignment is at 8 bytes

call    qword ptr [rip + __rust_alloc@GOTPCREL] ; rax = allocated address

test    rax, rax  ;  Check if the allocation failed
je      .LBB1_1   ;  Jump if the allocation failed
```

★ Copy the array to the vector

```
mov     qword ptr [rbx], rax          ;  Store address of data buffer

movups  xmm0, xmmword ptr [r15]      ;   Load first two numbers
movups  xmm1, xmmword ptr [r15 + 16] ;   Load 3rd & 4th num

movups  xmmword ptr [rax + 16], xmm1  ;   Store 3rd & 4th num
movups  xmmword ptr [rax], xmm0       ;   Store the first two num

movaps  xmm0, xmmword ptr [rip + .LCPI1_0] ;   Load length & capacity
movups  xmmword ptr [rbx + 8], xmm0     ;   Store length & capacity
```

Part D Iteration and Optimization

★ Increment the vector

```

movq    xmm0, r14                ;  Load the increment value

pshufd  xmm0, xmm0, 68           ;   The increment value is now
                                ; copied to the lower and upper
                                ; 64-bits of xmm0

movdqu  xmm1, xmmword ptr [rax]  ;   Load the first two numbers
                                ; from the vector

movdqu  xmm2, xmmword ptr [rax + 16] ;   Load the 3rd and 4th
                                ; numbers from the vector

paddq   xmm1, xmm0               ; ++ Add increment value to
                                ; the first two 64-bit numbers

movdqu  xmmword ptr [rax], xmm1   ;   Store the first two numbers
                                ; in the vector

paddq   xmm2, xmm0               ; ++ Add the increment value to
                                ; the third and fourth 64-bit numbers

movdqu  xmmword ptr [rax + 16], xmm2 ;   Store the 3rd and 4th numbers
                                ; in the vector

```

★ Return the vector

```

mov     rax, rbx                ; Return the vector in rax
pop     rbx                     ; Restore rbx
pop     r14                     ; Restore r14
pop     r15                     ; Restore r15
ret     ;  Return the vector in rax

```

.LBB1_1:

💀 Allocation failure handler

```

mov     edi, 32                 ; edi = Size of the attempted allocation
mov     esi, 8                  ; esi = Alignment is at 8 bytes
call    qword ptr [rip + alloc::alloc::handle_alloc_error@GOTPCREL]
ud2     ; Use the ud2 instruction to throw an exception

```



Experiment: Array Length Impact; AVX-512 Optimizations

Let us experiment with the vector iteration example in [Compiler Explorer](https://compiler-explorer.com/)
<https://godbolt.org/z/q7G3nerPd>.

Array Length

In the `increment_example` function, change array: `[i64; 4]` (line 11) and see how the compiler handles different array lengths:

- array: [i64; 128] - The compiler unrolls the loop even for 128 entries!
- array: [i64; 256] - The compiler generates an efficient loop that performs eight 64-bit additions in a single loop iteration.
- array: [i64; 257] - The loop performs 256 operations, and the code executes the remaining one outside the loop.
- array: [i64; 2048] - The generated code now uses `memcpy` to copy data from the input array to the vector's data buffer.

AVX-512

Change the compiler options to use the AVX-512 instruction set using the flag setting (change in the edit box on the top right):

```
-C opt-level=3 -C target_cpu=skylake-avx512
```

The compiler generates code that uses the `vaddq` instruction to perform four 64-bit additions with a single instruction. Try out some array lengths:

- array: [i64; 256] - The compiler unrolls the loop even for 256 entries!
- array: [i64; 512] - The compiler generates an efficient loop that performs 32 64-bit additions in a single loop iteration. The code performs four operations in parallel using the `vaddq` instruction.

Change Addition to Multiplication

In the `increment_by` function, change `*item += num` to `*item *= num` (line 7) and see the dramatic impact on the generated code.

- The compiler does not optimize for different vector lengths. It generates a single loop that performs the multiplication.

Impact of Passing an Array Slice

Add the following function to Compiler Explorer code pane:

```
pub fn increment_example_from_slice(inc_by: i64, array: &[i64]) -> Vec<i64> {
    let mut list = Vec::from(array);
    increment_by(inc_by, &mut list);
    list
}
```

You will see quite a few differences in the generated code.

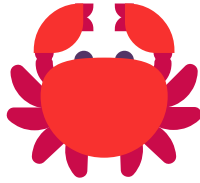
- Because the compiler does not know the array slice's length at compile time, it must generate code to handle vectors of arbitrary lengths.
- To support arbitrary lengths, the vector operations must handle the address overflow scenario, requiring the generation of code to manage capacity overflow.
- `memcpy` copies data from the input array to the vector's data buffer.

Key Takeaways

- When the compiler cannot determine the length of a vector at compile time, it generates code that conducts runtime length checks to select the most efficient code blocks for loop execution. We observed the compiler using the following code blocks to manage vectors of arbitrary lengths:

Part D Iteration and Optimization

- A loop that processes eight elements per iteration.
 - A code block dedicated to handling four elements.
 - A loop for managing fewer than four elements.
- The “8 elements per iteration” loop is the most efficient code block, aligning with the x86-64 architecture’s 64-byte cache line size. The compiler creates code that executes eight 64-bit additions in each loop iteration, effectively reading and writing a cache line per iteration.
- When the compiler knows the vector length at compile time, it can generate more efficient code by eliminating the need for runtime length checks to select the appropriate looping option. It can unroll the loop for a large number of iterations.
- When the vector comprises a multiple of 8 or 16 elements, the compiler can produce more efficient code, eliminating the need to generate code for handling residual elements (i.e., less than 8 or 16).
- Target a more recent CPU architecture to get more efficient code. The compiler can generate code that uses the AVX-512 instruction set to perform four 64-bit operations with a single instruction.
- Functions creating vectors can panic when the allocation fails. The compiler generates code that panics in such cases. We saw this in the section [Inlining, Loop Unrolling, and Memory Optimizations](#).



Chapter 12

Advanced Iterations: Mapping and Folding

In previous chapters, we explored array and vector iteration concepts. As we progress, this chapter aims to explore more complex forms of iteration, specifically those involving a two-tier structure with both an inner and an outer loop.

Exploring Inner and Outer Loops in Function Application

The following code defines a versatile `apply_functions` function. The role of this function is to accept an array of functions and apply each to every element of a given input array. As a result, the function produces a vector containing the outcomes of these operations.

```
type GenFunction<T> = fn(T) -> T;

fn apply_functions<T: Copy, const N: usize>(
    input: &[T],
    functions: &[GenFunction<T>; N],
) -> Vec<T> {
    input
        .iter()
        .map(|&item| functions.iter().fold(item, |acc, &f| f(acc)))
        .collect()
}
```

To better understand the code, we should examine the function signature first.

The function takes a generic type argument, `T`, and a generic constant, `N`. It accepts two parameters: a slice of `T` and an array of functions, where the functions are of `GenFunction<T>` type. Finally, the `apply_functions` function returns a vector of type `T` that contains the results of applying each function to every element in the input array.

There are two levels of iteration at work here:

- Inner Loop
- Outer Loop

Inner Loop

The inner loop iterates over the array of functions. It applies each function to the input item and returns a single result. The following code excerpt shows the inner loop:

```
|&item| functions.iter().fold(item, |acc, &f| f(acc))
```

Iter

The inner loop uses the `iter` function to iterate over the array of functions. The `fold` operation will apply to each function in the array that the `iter` function returns.

Fold

The inner loop uses the `fold` function to apply each function to the input item. The `fold` function takes an initial value and a closure.

The `item` is the initial value representing the item from the input array.

`|acc, &f| f(acc)` is the closure passed to the `fold`. The closure takes two arguments: `acc` and `&f`. `acc` is the accumulator representing the result of the previous function application. The closure applies `&f` to the input item and returns the result of this application to the accumulator. The generated code applies the closure to each function in the array of functions.

When the inner loop is complete, the `fold` function returns the result of applying all the functions to the input item.

Outer Loop

The outer loop goes through each item in the input array, using the `map` function to apply the inner loop to each item. This `map` function accepts a closure, representing the inner loop and acting on each item in the input array.

The `map` function returns the mapped iterator. The outer loop returns a vector of results by calling `collect` on the output iterator returned from the `map`.

Note that the `map` function is lazy and does not apply the inner loop to each item in the input array until the call to the `collect` function.

The following code excerpt demonstrates the application of the outer loop to the input array via the `map` function:

```
input
  .iter()
  .map(|&item| functions.iter().fold(item, |acc, &f| f(acc)))
  .collect()
```

Example: Applying Functions to an Array of Integers

Let us look at a complete example built around the `apply_functions` function. Key points to note in the example are:

- We define `Num` as a type alias for `i16`, which we can change to any unsigned or signed integer type. We will use this type alias to experiment with Compiler Explorer.
- We define `ARRAY_SIZE` as a constant to specify the size of the input array, and we can change it to any value. We will use this constant for experiments in Compiler Explorer.

- `FUNCTIONS_3` is an array of three functions that take a `Num` as input parameter and return a `Num`. The array contains two instances of the `multiply_by_two` function and one instance of the `add_one` function.
- `apply_array` is a function that takes an array of `Num` and applies the `FUNCTIONS_3` to each item in the array. The `apply_array` function returns a vector of `Num`.

```

type Num = i16;
const ARRAY_SIZE: usize = 1024;

type GenFunction<T> = fn(T) -> T;

fn apply_functions<T: Copy, const N: usize>(
    input: &[T],
    functions: &[GenFunction<T>; N],
) -> Vec<T> {
    input
        .iter()
        .map(|&item| functions.iter().fold(item, |acc, &f| f(acc)))
        .collect()
}

fn add_one(n: Num) -> Num {
    n + 1
}

fn multiply_by_two(n: Num) -> Num {
    n * 2
}

const FUNCTIONS_3: [GenFunction<Num>; 3] = [
    multiply_by_two,
    add_one,
    multiply_by_two,
];

pub fn apply_array(input: &[Num; ARRAY_SIZE]) -> Vec<Num> {
    let results = apply_functions(input, &FUNCTIONS_3);
    results
}

```

Optimizing an Array of Functions into a Single Polynomial

To understand the generated assembly code, let us discuss the optimizations we will see in the `apply_array` function.

We will see that the compiler has performed extensive optimizations on the `apply_array` function. It has inlined the `apply_functions` function along with the `map` and `fold` functions. It also folds the functions in the `FUNCTIONS_3` array into a single polynomial. The code applies the folded polynomial to each item in the input array. The following table shows the compiler's steps to arrive at the folded polynomial.

Function	Polynomial	Folded polynomial
<code>multiply_by_two</code>	$2x$	$2x$
<code>add_one</code>	$x + 1$	$2x + 1$
<code>multiply_by_two</code>	$2x$	$4x + 2$

Part E Strings, Dispatch, and Recursion

The compiler has folded the polynomial to $4x + 2$. It has replaced the three functions with a single function and then inlined it! The generated code loosely resembles the following Rust code:

```
pub fn apply_array_for_with_preallocation(input: &[Num; ARRAY_SIZE]) -> Vec<Num> {
    let mut results: Vec<Num> = vec![Num::default(); input.len()];

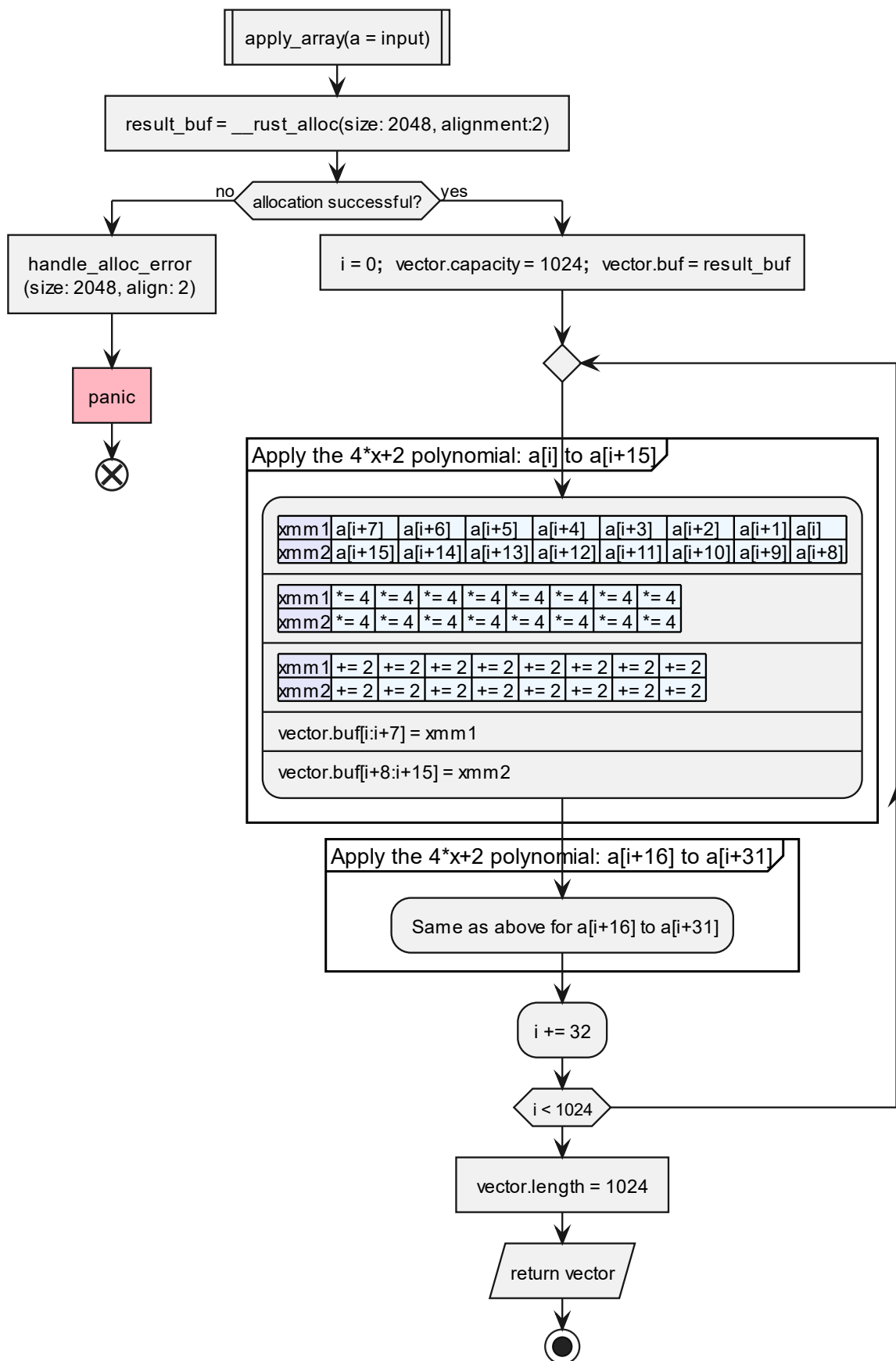
    for (i, &n) in input.iter().enumerate() {
        results[i] = 4 * n + 2;
    }

    results
}
```

Visualizing the Inner Loop Unrolling and Outer Loop SIMD Optimizations

Having discussed the optimizations, let us now examine the flowchart representing the structure of the generated assembly. We will see the following optimizations in the flowchart:

- The compiler completely unrolls the inner loop. It has inlined the `fold` function and replaced it with a single polynomial.
- The compiler has optimized the outer loop using SIMD instructions to process multiple elements in one iteration.



Polynomial Folding and SIMD Instructions

After going through the flowchart, let us examine the annotated assembly code.

-  Input: `rsi` = input array
-  Output: `rax` = result vector

```
; This is a constant value of 2s, will be used for the operations
.LCPI0_0:
.short 2
.short 2
.short 2
.short 2
.short 2
.short 2
.short 2
.short 2

example::apply_array:

; Save the registers that this function will use.
push    r14
push    rbx
push    rax

; rsi holds the second argument to the function, which is the input array
; rdi holds the first argument to the function, which is the result vector
mov     rbx, rsi
mov     r14, rdi
```

 Allocate memory for the result vector.

```
; Allocate space for 1024 i16 elements (2048 bytes)
mov     edi, 2048

; Alignment for i16 is 2 bytes
mov     esi, 2

;  Call Rust's allocator
call    qword ptr [rip + __rust_alloc@GOTPCREL]

;  Check if allocation was successful
test    rax, rax

;  If not, jump to error handling
je      .LBB0_4
```

 Allocation was successful.

```
;  Set the capacity of the result vector to 1024
mov     qword ptr [r14], 1024

;  Set the data pointer of the result vector to the allocated memory
mov     qword ptr [r14 + 8], rax
```



```
; 🌀 Zero out rcx to use it as an index.
xor     ecx, ecx

; 2 2 2 2 2 2 2 2 Load eight 2s into xmm0 for SIMD operations
; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
movdqa  xmm0, xmmword ptr [rip + .LCPI0_0]
```

★ Process 16 elements at a time using SIMD.

```
.LBB0_2:

; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ Load 8 i16 elements from the input array into xmm1
movdqu  xmm1, xmmword ptr [rbx + 2*rcx]

; ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ Load 8 i16 elements from the input array into xmm2
movdqu  xmm2, xmmword ptr [rbx + 2*rcx + 16]

; × × × × × × × × Multiply each of the 8 elements by 4 (shift left by 2)
psllw   xmm1, 2

; × × × × × × × × Multiply each of the 8 elements by 4 (shift left by 2)
psllw   xmm2, 2

; + + + + + + + + Add 2 to each of the 8 elements in xmm1
por     xmm1, xmm0

; + + + + + + + + Add 2 to each of the 8 elements in xmm2
por     xmm2, xmm0

; ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ Store the resulting 8 elements to the result vector
movdqu  xmmword ptr [rax + 2*rcx], xmm1

; ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ Store the resulting 8 elements to the result vector
movdqu  xmmword ptr [rax + 2*rcx + 16], xmm2
```

★ Process the next 16 elements in the same way.

```
movdqu  xmm1, xmmword ptr [rbx + 2*rcx + 32]
movdqu  xmm2, xmmword ptr [rbx + 2*rcx + 48]
psllw   xmm1, 2
psllw   xmm2, 2
por     xmm1, xmm0
por     xmm2, xmm0
movdqu  xmmword ptr [rax + 2*rcx + 32], xmm1
movdqu  xmmword ptr [rax + 2*rcx + 48], xmm2

; 🌀 Increment the index by 32 (Process 32 elements per iteration)
add     rcx, 32

; ⚖️ Check if all 1024 elements have been processed
cmp     rcx, 1024
```

Part E Strings, Dispatch, and Recursion

```
;  If not, continue processing by jumping to the beginning of the loop
jne      .LBB0_2
```

★ The code has processed all elements.

```
;   Set the length of the result vector to 1024
mov      qword ptr [r14 + 16], 1024

; Return the result vector via rax
mov      rax, r14

; Restore the saved registers
add      rsp, 8
pop      rbx
pop      r14

;  Return the result vector in rax
ret
```

😬 Panic handling.

```
.LBB0_4:

;  Allocation was not successful. Handle the error
; Report the error with the size of the allocation and alignment
mov      edi, 2048      ; Array size in bytes
mov      esi, 2         ; 2 byte alignment

;  Handle memory allocation failure
call     qword ptr [rip + alloc::alloc::handle_alloc_error@GOTPCREL]

;  Terminate the program with an undefined instruction (ud2)
ud2
```



Experiment: Array Size, Type, and Inner Loop Optimizations

We can run quite a few experiments with [Compiler Explorer](https://godbolt.org/z/YYr33rqro) (<https://godbolt.org/z/YYr33rqro>):

- [Change Array Size](#)
- [Change the Type of Array Elements](#)
- [Change the Number of Functions Applied](#)
- [AVX-512 Code Generation](#)
- [Iterating over a Vector](#)
- [Comparing Functional and Imperative Code](#)

Change Array Size

Change the array size to 256. You will see that the compiler completely unrolls the loop.

```
const ARRAY_SIZE: usize = 256;
```

Change the Type of Array Elements

Change the type of array elements to `u128`. The generated code processes two numbers per iteration. We also see that each `u128` results in two instructions to load or store the value.

```
type Num = u128;
```

At the other end of the spectrum, if you change the type of array elements to `u8`, you will see that the compiler uses SIMD instructions to process 64 elements in one iteration.

```
type Num = u8;
```

Change the Number of Functions Applied

Change the number of functions applied to the array elements to 8. See if you can figure out how the compiler generates the code for this case. Did it fold the eight operations into a single polynomial? Did it unroll the loop? Did it use SIMD instructions?

```
pub fn apply_array(input: &[Num; ARRAY_SIZE]) -> Vec<Num> {
    // Change to the FUNCTIONS_8 array
    let results = apply_functions(input, &FUNCTIONS_8);
    results
}
```

The [Compiler Explorer](https://godbolt.org/z/YYr33rqro) (<https://godbolt.org/z/YYr33rqro>) version of the code defines `FUNCTIONS_8` as follows:

```
const FUNCTIONS_8: [GenFunction<Num>; 8] = [
    add_one,
    multiply_by_two,
    subtract_one,
    cube,
    subtract_one,
    square,
    add_one,
    divide_by_two,
];
```

AVX-512 Code Generation

Change the compiler options to use the AVX-512 instruction set. Add the following to the compiler options in a box on the top right corner of Compiler Explorer page:

```
-C opt-level=3 -C target_cpu=skylake-avx512
```

We observed that with `type Num = i16`, the compiler generates AVX-512 instructions to process 32 elements per instruction. It has fully inlined the loop for 1024-sized arrays!

Iterating over a Vector

Uncomment the `apply_vec` function and comment out the `apply_array` function. The code generated for the `apply_vec` function is far more complex than the `apply_array` function. The main causes of complexity are:

Part E Strings, Dispatch, and Recursion

- The compiler does not know the size of the vector at compile time, so it needs to generate code that checks the length and determines the optimal looping strategy.
- Exception handling code is complicated as the compiler needs to generate cleanup code for two vectors.

```
pub fn apply_vec(input: Vec<Num>) -> Vec<Num> {
    let results = apply_functions(&input, &FUNCTIONS_3);
    results
}
```

Comparing Functional and Imperative Code

The following function in the imperative style is equivalent to `apply_array` in the functional style. Add this function to Compiler Explorer code window and compare the generated code with the `apply_array` function.

Spoiler alert: The code generated for this function is the same as for the `apply_array` function. This similarity is a prime example of what is known as a zero-cost abstraction, underscoring the fact that adopting a functional programming style does not necessarily come with a performance trade-off.

```
pub fn apply_array_imperative_with_preallocation(input: &[Num; ARRAY_SIZE])
    -> Vec<Num> {
    let mut results: Vec<Num> = vec![Num::default(); input.len()];

    for (i, &n) in input.iter().enumerate() {
        results[i] = 4 * n + 2;
    }

    results
}
```

In many cases, the handwritten code might employ sub-optimal strategies. For example, the following code uses `push_back` to add elements to the result vector. Add the following function to Compiler Explorer code window and compare the generated code with the `apply_array` function.

```
pub fn apply_array_imperative_with_push_back(input: &[Num; ARRAY_SIZE])
    -> Vec<Num> {
    let mut results: Vec<Num> = Vec::with_capacity(input.len());

    for &n in input {
        results.push(4 * n + 2);
    }

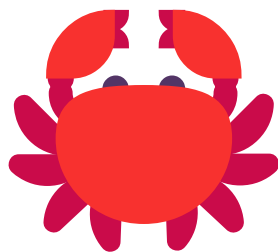
    results
}
```

The generated code for `apply_array_imperative_with_push_back` is inefficient on two accounts:

- `reserve_for_push` in the generated assembly reserves space for adding elements to the vector, although this operation is unnecessary because the compiler knows the vector's size at compile time.
- The generated code does not use SIMD operations to process the array elements.

Key Takeaways

- Rust enables the use of higher-order functions, bolstering its capabilities for functional programming. This feature allows developers to craft concise and expressive code without compromising speed. We observed that the compiler eliminates abstraction, yielding code as efficient as manually written for-loops.
 - Our observation extended to the compiler's ability to inline functions within the `FUNCTIONS_3` array while optimizing the inlined code by eliminating redundant operations. These optimizations extended to the outer and the inner loop.
 - Our study revealed that code written in a functional style matches the most optimized code that one can manually write for the same problem. We demonstrated this in the [experiment comparing functional code with imperative code](#).
 - Additionally, we noted that developers may write sub-optimal for-loops that perform worse than the equivalent functional code. We illustrated this in the [experiment comparing functional code with imperative code](#).
- The Rust compiler generates efficient code for array processing. We saw that the compiler unrolled the loop and used SIMD instructions to process multiple elements in one iteration.
 - We even observed that the compiler inlined all functions in the functions array and folded the operations into a single polynomial!
- Enabling the AVX-512 instruction set generates even more efficient code. With `type Num = i16`, the compiler generated AVX-512 instructions to process 16 `i16` operations per instruction. The compiler completely unrolled the 1024 iteration loop!
- The compiler generates the most efficient code for processing arrays when it knows the array size at compile time. Compared to the `apply_array` function, the code generated for the `apply_vec` function demonstrated this. The `apply_vec` function is more complex as the compiler needs to select the efficient looping strategy at runtime. Also, adding a second vector to the function increases the complexity of the exception-handling code.



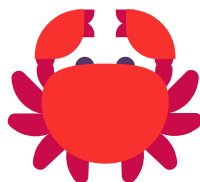
Part E

Strings, Dispatch, and Recursion



Rust Under the Hood Companion (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews



Chapter 13

Strings and Slices: Under the Surface

This chapter explores the representation of strings and string slices in memory and how Rust handles and manipulates them. We will dive into the generated assembly code for string and string slice operations.

We will cover the following topics:

- [String and String Slice Representation in Memory](#)
- [Combining Two Strings: String and &str in Action](#): Examine the assembly code generated by the Rust compiler for a function that combines two strings.
- [Builder Pattern: Optimize Away the Builder Struct](#): Understand the code generated by the Rust compiler for the builder pattern for a struct with multiple string fields.
- [Emoji Arithmetic: Overloading the + Operator](#): We will look at the code generated when we overload the + operator for a type that wraps a static string slice.

String and String Slice Representation in Memory

Let us examine the memory representation of strings. Understanding the memory layout of strings is essential for grasping how one passes around and manipulates them.

In Rust, the types `String` and `&str` represent strings. We will look at each of these types in detail.

String

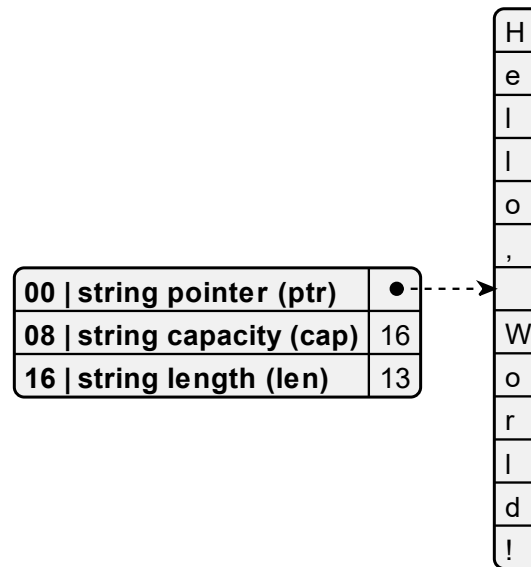
In Rust, a `String` is a growable, mutable, owned, UTF-8 encoded string type represented in memory as a three-part data structure. This structure includes a pointer to the raw string data (`ptr`), the string's byte length (`len`), and its total capacity (`cap`). The length indicates the actual size of the string. In contrast, the capacity represents the total buffer size, often larger than the string to accommodate growth without frequent memory reallocations. While the stack stores all this metadata, the actual string data resides on the heap. This memory representation strongly resembles the `Vec<T>` struct.

The `String` is a wrapper around `Vec<u8>`.

```
pub struct String {  
    vec: Vec<u8>,  
}
```

Part E Strings, Dispatch, and Recursion

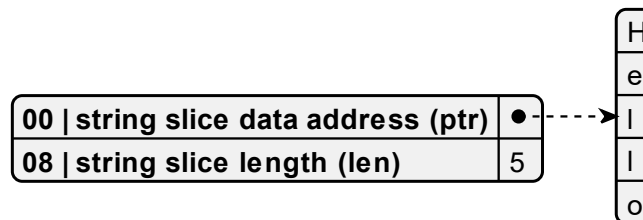
String in Memory:



&str (String Slice)

In Rust, `&str`, commonly known as a “string slice”, is a reference to a string and often appears in function signatures to accept any string. Comprising two parts, a string slice consists of a pointer to the string data’s start and the byte length of the slice. Notably, a string slice does not own the memory it references and lacks capacity information, rendering it unmodifiable and non-resizable. However, its smaller size makes it more efficient to pass around than a `String`.

&str in Memory:



Combining Two Strings: String and &str in Action

Let us explore how Rust combines two strings. This code snippet shows a function that takes two strings and returns their concatenation as a new string. The first string is an owned `String`, and the second is a string slice `&str`.

```
pub fn combine_strings(s1: String, s2: &str) -> String {
    let mut result = String::new();
    result.push_str(&s1);
    result.push_str(s2);
    result
}
```

Flowchart of `combine_strings`

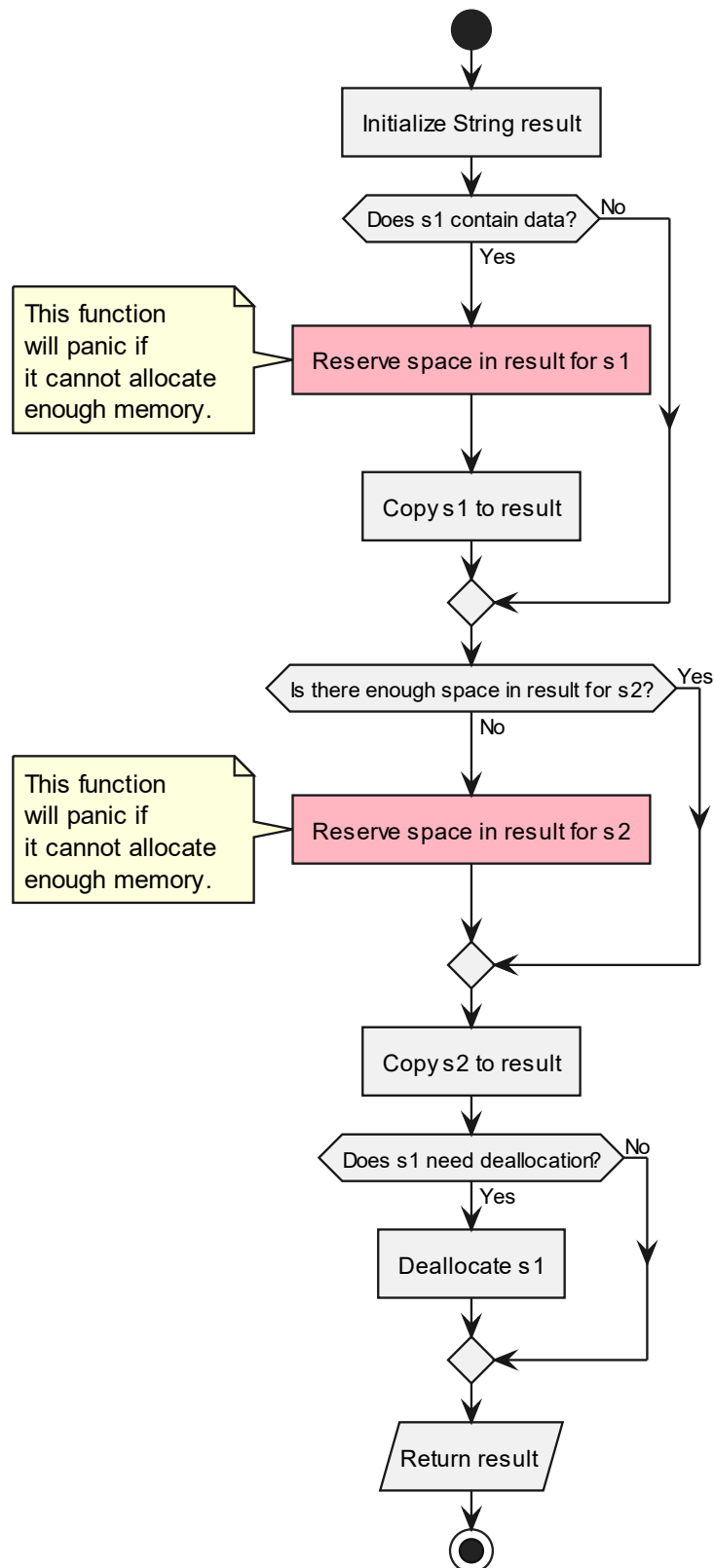
The following diagram outlines the main steps performed by the assembly code. It begins with function setup, initializes the result `String`, and then checks whether the input strings contain data. Depending on these checks, it may reserve additional space in the `String` or set it as empty. Then, it copies the input strings into the result, deallocating the original `String` and returning the result.

The `String` reserves space by calling the `reserve` method in `RawVec`. The function signature is as follows:

```
pub fn reserve(&mut self, additional: usize)
```

This method takes a mutable reference to the `Vec` (because it may need to modify the `Vec`'s capacity), and a `usize` argument that specifies the number of additional elements for which to reserve space. It does not return a value because any errors that occur would typically be handled by panicking.

The following diagram shows the main steps performed by the `combine_strings` function:



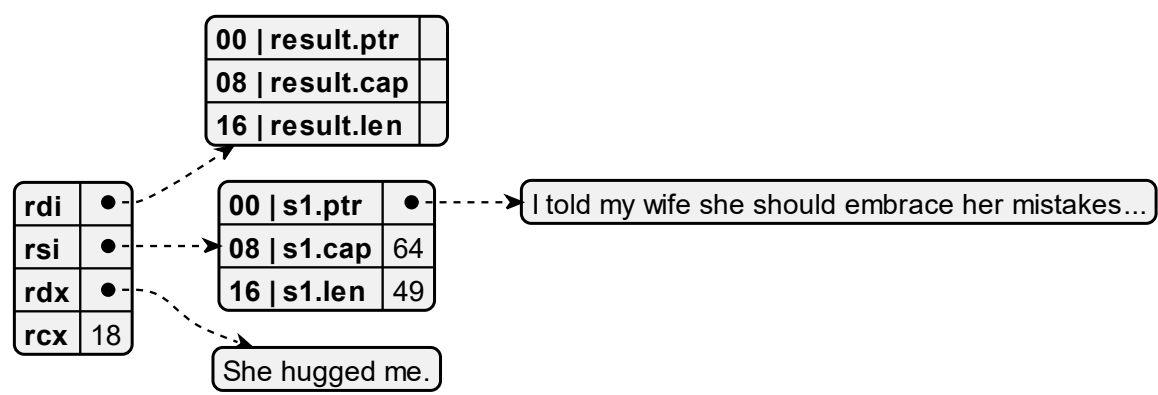
Input and Output Registers

To understand the assembly code, we need to understand the input and output registers the function uses.

Input Registers

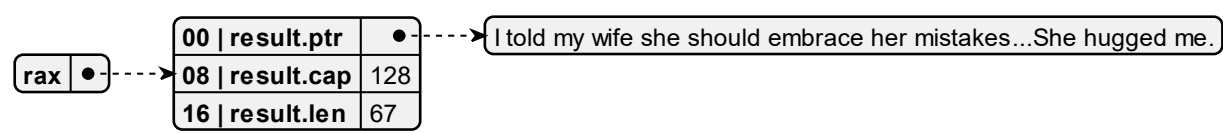
- `rdi`: The caller passes the destination string’s address. In Rust, functions that return a struct, such as `String`, write the return value directly into the caller-provided memory. The function receives the address of this memory as a “hidden” first argument.
- `rsi`: This is a pointer to the `String` struct for `s1`.
- `rdx`: Pointer to string slice `s2`.
- `rcx`: Length of `s2` slice.

The following diagram shows the input parameters passed to the function in registers.



Output Registers

The function returns the output in `rax`, which holds the address of the destination string, serving as the function’s return value. The following diagram illustrates the `String` struct in memory after the function’s return.



Assembly Code for `combine_strings`

Input:

- `rdi = &result`
- `rsi = &s1`
- `rdx = s2.ptr`
- `rcx = s2.len`

Output:

- `rax = &result`

 Function prologue: saving registers and making room for local variables.

```
example::combine_strings:
push    rbp
push    r15
push    r14
push    r13
```

Part E Strings, Dispatch, and Recursion

```

push    r12
push    rbx
sub     rsp, 24

```

🔧 Setting up arguments: Saving registers for future use.

```

mov     r12, rcx           ; 🧰 r12 = s2.len
mov     qword ptr [rsp + 16], rdx ; 📁📌 Save s2.ptr for later use
mov     rbx, rdi           ; 📁📌 rbx = &result

```

🔧 Initialization of result String.

```

mov     qword ptr [rdi], 0      ; 📁📌 result.ptr = 0
mov     qword ptr [rdi + 8], 1  ; 📁📌 result.capacity = 1
mov     qword ptr [rdi + 16], 0 ; 🧰📌 result.len = 0

```

🔧 Obtain len and ptr for s1 and check if s1.len == 0.

```

mov     rax, qword ptr [rsi + 8] ; 📁📌 rax = s1.cap
mov     qword ptr [rsp + 8], rax ; 📁📌 Save s1.cap for later use
mov     r15, qword ptr [rsi + 16] ; 🧰📌 r15 = s1.len

test    r15, r15             ; 🔍🧰 Check if s1.len == 0
mov     qword ptr [rsp], rsi   ; 📌 Save s1 address for later use
je      .LBB3_1               ; 🔄 Jump to .LBB3_1 if s1.len == 0

```

📁📌 Reserve capacity in result for the first String.

```

mov     r14, rsi              ; r14 = &s1
mov     rdi, rbx              ; rdi = &result
xor     esi, esi              ; rsi = 0
mov     rdx, r15              ; 🧰📌 rdx = s1.len

call    alloc::raw_vec::RawVec<T,A>::reserve::do_reserve_and_handle

mov     rbp, qword ptr [rbx + 16] ; 🧰📌 rbp = result.len
mov     r14, qword ptr [rbx]      ; 📁📌 r14 = result.ptr
mov     r13, qword ptr [rbx + 8] ; 📁📌 r13 = result.cap
jmp     .LBB3_5                 ; 🔄

```

🔧 Handle the case when s1.len == 0.

```

.LBB3_1:
mov     r13d, 1                ; 📁📌 r13 = 1 (result.cap)
xor     r14d, r14d              ; 📁📌 r14 = 0 (result.ptr)
xor     ebp, ebp               ; 🧰📌 rbp = 0 (result.len)

```

📄 Copying the contents of s1 to the result string.

```

.LBB3_5:
; Setting up memcpy destination, source, and length arguments
mov     rdi, r13               ; 📁📌 rdi = result.ptr

```

```

add    rdi, rbp                ;  + rdi = result.ptr + result.len
mov     rsi, qword ptr [rsp + 8] ;   rsi = s1.ptr
mov     rdx, r15               ;  rdx = s1.len

call    qword ptr [rip + memcpy@GOTPCREL] ; Copy s1 to result

```

Updating the length of the result after copying and adjusting the capacity.

```

add     rbp, r15               ;  + rbp = result.len + s1.len
mov     qword ptr [rbx + 16], rbp ;   result.len += s1.len
sub     r14, rbp               ;  - r14 = result.capacity - result.len

```

Copying the contents of s2 to the result string.

```

cmp     r14, r12              ;   Compare result.capacity - result.len with s2.len
jae     .LBB3_6               ;  Jump if result.capacity - result.len >= s2.len

```

Handle the case when `result.capacity - result.len < s2.len`.

Reserve capacity in result for the second String.

```

mov     rdi, rbx              ; rdi = &result
mov     rsi, rbp              ;  rsi = result.len
mov     rdx, r12              ;  rdx = s2.len
mov     r14, qword ptr [rsp] ; 

; Calling reserve function
call    alloc::raw_vec::RawVec<T,A>::reserve::do_reserve_and_handle

mov     r13, qword ptr [rbx + 8] ;   r13 = result.capacity
mov     rbp, qword ptr [rbx + 16] ;   rbp = result.len
jmp     .LBB3_9               ;  Jump to .LBB3_9

.LBB3_6:

mov     r14, qword ptr [rsp]   ;   r14 = s1.ptr

```

Copying the contents of s1 to the result string.

```

.LBB3_9:
; Setting up memcpy destination, source, and length arguments
add     r13, rbp              ;  r13 = result.capacity + result.len
mov     rdi, r13              ;  rdi = result.ptr

mov     rsi, qword ptr [rsp + 16] ;   rsi = s2.ptr
mov     rdx, r12              ;  rdx = s2.len

call    qword ptr [rip + memcpy@GOTPCREL] ; Copy s2 to result

add     rbp, r12              ;  rbp = result.len + s2.len
mov     qword ptr [rbx + 16], rbp ;   result.len = result.len + s2.len

```

Part E Strings, Dispatch, and Recursion

🔪 Check if `s1` is allocated on the heap. If so, deallocate it.

```
mov    rsi, qword ptr [r14]      ; 📄 rsi = s1.ptr
test   rsi, rsi                 ; 🕒📄 Check if s1.ptr == 0
je     .LBB3_11                 ; 🔄 Skip cleanup if s1.ptr == 0
```

🔪 Cleaning up the memory used by the first String (`s1`).

```
mov     edx, 1                  ; Set alignment to 1
mov     rdi, qword ptr [rsp + 8] ; 📄 rdi = s1.cap
call    qword ptr [rip + __rust_dealloc@GOTPCREL] ; Deallocate s1
```

🏠 Return the result.

```
.LBB3_11:
mov     rax, rbx                ; rax = &result
add     rsp, 24                 ; Pop the stack
pop     rbx
pop     r12
pop     r13
pop     r14
pop     r15
pop     rbp
ret                                ; 🏠 Return result in rax
```

💀 Exception handling code. It is not invoked directly from the function.

```
mov     rbp, rax
mov     rdi, rbx
call    core::ptr::drop_in_place<alloc::string::String>
mov     rdi, r14
call    core::ptr::drop_in_place<alloc::string::String>
mov     rdi, rbp
call    _Unwind_Resume@PLT
ud2
```

Builder Pattern: Optimize Away the Builder Struct

The builder pattern, a design pattern suitable for the step-by-step construction of complex objects, is effective when the same construction process can yield several types or representations of the object. It proves especially useful for objects with numerous optional or configurable properties. In Rust, implementing the builder pattern involves using a struct and a corresponding builder struct. This builder struct includes methods for setting the values of the struct's fields. It also features a `build` method that consumes the builder, returning a new struct instance.

Builder for Address

Let us look at an example of a builder for an `Address` struct:

```
pub struct Address {
    pub street: String,
    pub city: String,
    pub postal_code: u64,
```



```
}

#[derive(Default)]
pub struct AddressBuilder {
    street: String,
    city: String,
    postal_code: u64,
}

impl AddressBuilder {
    fn new() -> AddressBuilder {
        AddressBuilder::default()
    }

    fn street(mut self, value: String) -> Self {
        self.street = value;
        self
    }

    fn city(mut self, value: String) -> Self {
        self.city = value;
        self
    }

    fn postal_code(mut self, value: u64) -> Self {
        self.postal_code = value;
        self
    }

    fn build(self) -> Address {
        Address {
            street: self.street,
            city: self.city,
            postal_code: self.postal_code,
        }
    }
}

pub fn make_address(street: String, city: String, postal_code: u64)
    -> Address {
    AddressBuilder::new()
        .street(street)
        .city(city)
        .postal_code(postal_code)
        .build()
}
```

Let us break down the preceding code:

- `Address` is a public struct that represents an address. It has three public fields: `street`, `city` and `postal_code`.
- `AddressBuilder` is a public struct that is a builder for the `Address` struct, containing the same fields as `Address`. The `#[derive(Default)]` annotation above the struct definition enables the creation of a default `AddressBuilder` instance, where all fields are set to their default values.

Part E Strings, Dispatch, and Recursion

- The `impl AddressBuilder` block provides methods for the `AddressBuilder` struct:
 - `new` is a function that creates a new `AddressBuilder` instance with default values.
 - `street`, `city`, and `postal_code` are functions that take a mutable reference to `self` (the current `AddressBuilder` instance) and a value to set for the respective field. They each return `self`, allowing the methods to be chained together.
 - `build` takes `self` and constructs an `Address` instance with the current state of the `AddressBuilder`.
- `make_address` is a public function that takes `street`, `city`, and `postal_code` as parameters, creates an `AddressBuilder`, sets the fields with the provided values, builds the `Address`, and returns it.

The use of the builder pattern makes the code more readable and flexible. For example, if developers add additional fields to the `Address` struct in the future, they can also add them to the `AddressBuilder` in a way that does not break existing code.

We wish to answer the question: Does the builder pattern have a runtime cost? In the next section, let us look at the assembly generated for the `make_address` function.

Builder Pattern Assembly

The generated assembly shows that the builder pattern is a zero-cost abstraction. The `main` function inlines the `make_address` function, leading to the optimization of the `AddressBuilder` struct. Additionally, the compiler optimizes away the `new`, `street`, `city`, and `build` functions, resulting in no generated code for these functions.

After the optimizations, the effective code for `make_address` is as follows:

```
pub fn make_address(street: String, city: String, postal_code: u64)
    -> Address {
    Address {
        street,
        city,
        postal_code,
    }
}
```

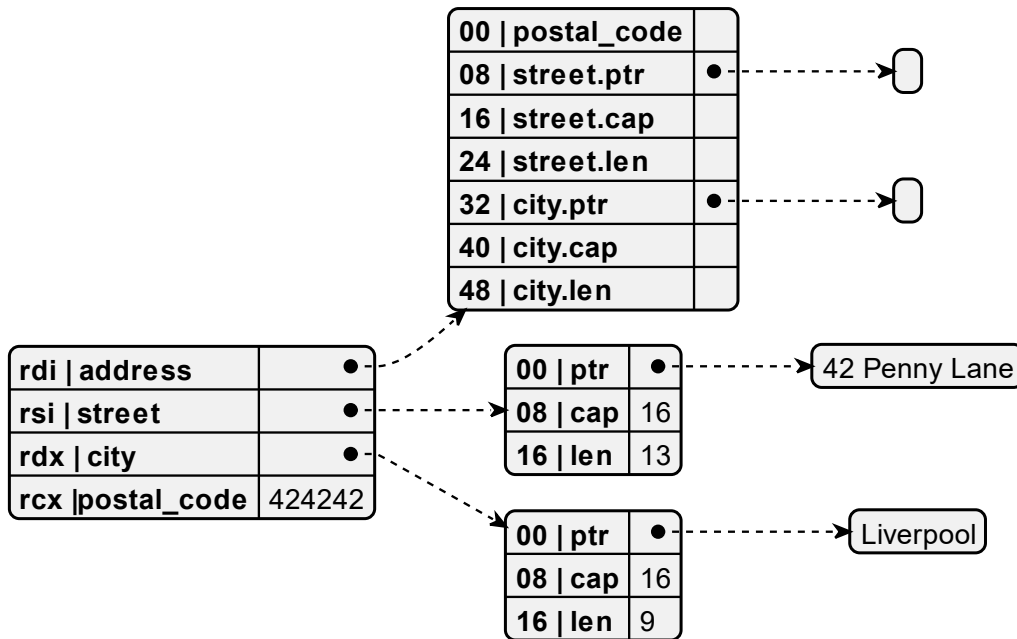
Now let us look at the input and output registers for the `make_address` function:

Input Registers for `make_address`

The `make_address` function uses the following registers for input:

Register	Value	Description
<code>rdi</code>	Pointer to <code>Address</code>	The caller passes a hidden parameter to specify the address for copying the returned <code>Address</code> .
<code>rsi</code>	Pointer to the <code>street</code>	The first parameter to <code>make_address</code> .
<code>rdx</code>	Pointer to <code>city</code>	The second parameter to <code>make_address</code> .
<code>rcx</code>	<code>postal_code</code> value	The third parameter to <code>make_address</code> .

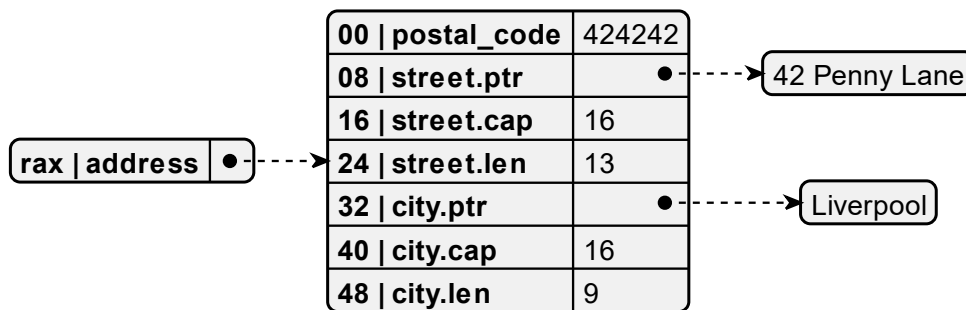
The following figure shows an example of the input registers passed to the `make_address` function:



Output Registers for make_address

The function stores the return address of the `Address` struct in the `rax` register. Note that this address is the same as the one passed in the `rdi` register from the caller. The compiler optimizes the `AddressBuilder` struct away, allowing the `Address` struct to be constructed directly in the memory location specified by the `rdi` register.

The following figure shows an example of the output registers returned from the `make_address` function:



Assembly for make_address

The generated code loads the values into the registers and returns the `Address` struct. The compiler also opportunistically uses SIMD instructions to load and store two 64-bit values simultaneously.

Input:

- `rdi` = pointer to 🏠 `Address` struct
- `rsi` = pointer to 📮 `street` string
- `rdx` = pointer to 🏙️ `city` string
- `rcx` = 📮 `postal_code`

Output:

- `rax` = 🏠 pointer to `Address` struct

example::make_address:

```
; 🏠 Move the result a=Address to rax
mov    rax, rdi

; 🗺️ abc ⬇ Copy the street string pointer to rdi
mov    rdi, qword ptr [rsi]

; 🗺️ 📏 ⬇ 📏 SIMD copy the street string capacity & length to xmm0
movups xmm0, xmmword ptr [rsi + 8]

; 🗺️ abc ⬇ 📏 ⬇ SIMD copy the city string pointer & capacity to xmm1
movups xmm1, xmmword ptr [rdx]

; 🗺️ 📏 ⬇ Copy the city string length to rdx
mov    rdx, qword ptr [rdx + 16]

; 🗺️ abc ⬆ Copy the street address pointer to the Address struct
mov    qword ptr [rax + 8], rdi

; 🗺️ 📏 ⬆ 📏 ⬆ SIMD copy the street string capacity & length to the output
movups xmmword ptr [rax + 16], xmm0

; 🗺️ abc ⬆ 📏 ⬆ SIMD copy the city string pointer & capacity to the output
movups xmmword ptr [rax + 32], xmm1

; 🗺️ 📏 ⬆ Copy the city string length to the output Address struct
mov    qword ptr [rax + 48], rdx

; 📞 ⬆ Copy the postal code to the output Address struct
mov    qword ptr [rax], rcx

; ⬅ 🏠 Return the output Address struct in rax
ret
```

Emoji Arithmetic: Overloading the + Operator

Now, let us look at a more complex example. The following code defines an `Emoji` struct representing a single emoji. The `Emoji` struct implements the `Add` trait to add two emojis together. The `add_two_emojis` function adds two emojis, and the `add_three_emojis` function adds three.

Note that the `add` function returns an `Option<Emoji>` because the code may not support the addition of two emojis. For example, the code defines the addition of the 🤗 and 🤗 emojis, but it does not support the addition of the 🤗 and 🤔 emojis.

💡 Learn More: `Option<T>`

The `Option<T>` type represents an optional value. It can be `Some(T)` or `None`. Where `T` is the type of the optional value. The `Option<T>` type is an enum defined as follows:

```
pub enum Option<T> {
    None,
```

```
    Some(T),
}
```

The representation of enums in Rust is like that of unions in C. An additional discriminator field is added to the struct to indicate which enum variant is stored. In many cases, Rust can optimize away the discriminator field. We will see an example of this optimization in the assembly code for the `add` function.

Emoji Example

```
#[derive(PartialEq)]
pub struct Emoji(&'static str);

impl std::ops::Add for Emoji {
    type Output = Option<Self>;

    fn add(self, rhs: Self) -> Self::Output {
        let Emoji(self_emoji) = self;
        let Emoji(rhs_emoji) = rhs;

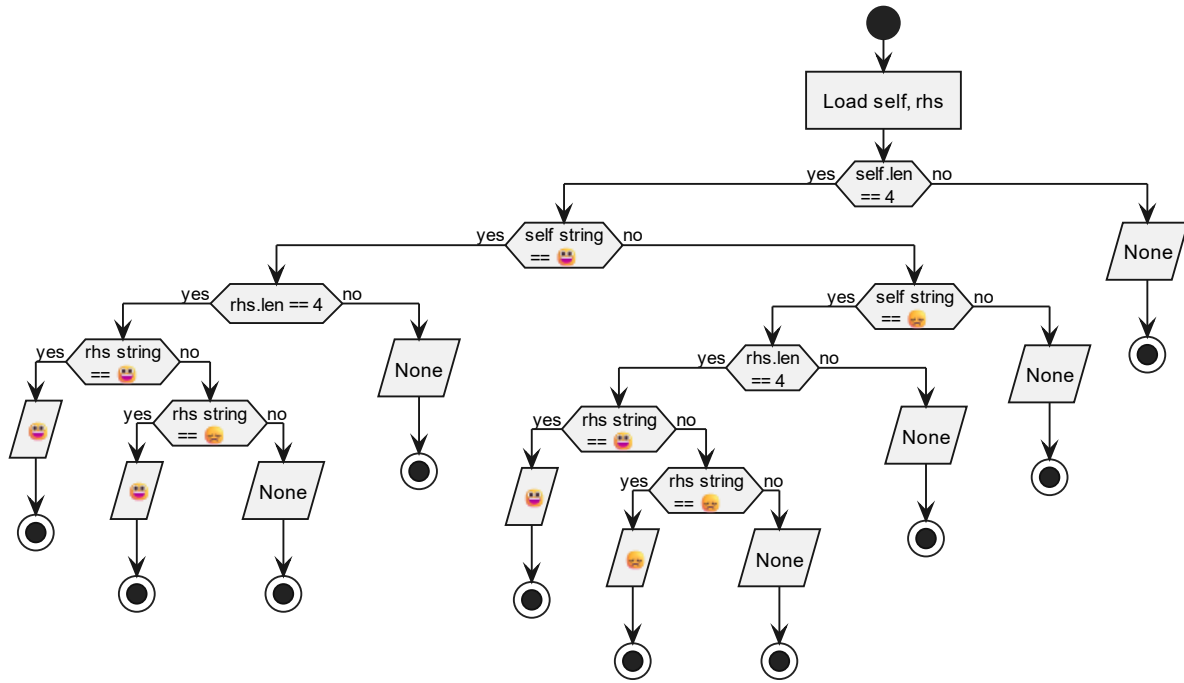
        match (self_emoji, rhs_emoji) {
            ("😄", "😄") => Some(Emoji("😄")),
            ("😄", "😞") => Some(Emoji("😄")),
            ("😞", "😄") => Some(Emoji("😄")),
            ("😞", "😞") => Some(Emoji("😞")),
            _ => None,
        }
    }
}

pub fn add_two_emojis(a: Emoji, b: Emoji) -> Option<Emoji> {
    a + b
}

pub fn add_three_emojis(a: Emoji, b: Emoji, c: Emoji) -> Option<Emoji> {
    (a + b)? + c
}
```

std::ops::Add Assembly Flowchart

The following flowchart shows a high-level view of the generated assembly code for the `add` function. The control flow presented here will help you understand the following assembly code.



std::ops::Add Input and Output Registers

Before we look at the assembly code, let us look at the input and output of the add function. The add function takes two `Emoji` string slices as input and returns an `Option<Emoji>` string slice as output. The compiler uses two registers for each of the input `Emoji` string slices and two for the output `Option<Emoji>` string slice. The input and output registers used by the add function are as follows:

- **Input:**
 - rdi: `abc` Pointer to the first `Emoji` bytes - `self.ptr`.
 - rsi: `Length` Length of the first `Emoji` slice - `self.len`
 - rdx: `abc` Pointer to the second `Emoji` bytes - `rhs.ptr`
 - rcx: `Length` Length of the second `Emoji` slice - `rhs.len`
- **Output:**
 - rax: This contains the return value of the function:
 - `abc` Pointer to the output `Emoji` bytes **or**
 - `None` (all-zero) if the operation is not supported.
 - edx: `Length` This contains the length of the return string slice. Valid if rax is not `None`.

std::ops::Add Assembly Code

Let us understand the assembly code of the Add trait implementation for `Emoji`. A few points to note here:

- The code checks the emoji strings to see if the contents of the `Emoji` string slices are valid by comparing the entire integer representation of the string slice with known valid values. This approach is a significant optimization because it allows the code to check the whole 4-byte emoji string at once rather than checking each byte individually.
- The function returns the `Option<Emoji>` via the rax and edx registers. The rax register contains the pointer to the output `Emoji` string slice or `None` (all-zero) if the operation is not supported. The edx register contains the length of the return string slice. This register is only valid if rax is not `None`.

```
<example::Emoji as core::ops::arith::Add>::add:
```

🚦 If the length of the self `Emoji` string slice is not equal to 4, jump to `.LBB0_7` and return `None`.

```
cmp    rsi, 4          ; 🔧 Compare 1st string length to 4.
jne    .LBB0_7         ; Jump if they are not equal
```

🌈 Jump to `.LBB0_8` if:

- The self emoji bytes are not equal to 🤪 emoji bytes.
- The length of the rhs emoji string slice is not equal to 4.

```
xor     eax, eax        ; Zero out eax

cmp     dword ptr [rdi], -2087149584 ; 📄 Compare four bytes of self
                                           ; emoji to the bytes of "🤪"

setne   sil             ; Set sil to 1 if they are not equal

cmp     rcx, 4          ; 🔧 Compare 2nd string length to 4

jne     .LBB0_8         ; Jump if they are not equal

mov     al, sil         ; Move the result of the
                       ; previous comparison to al

test    eax, eax        ; Test al (AND operation with itself)

; Jump to .LBB0_8 if the result of the test is not zero,
; i.e., emojis are not "🤪"

jne     .LBB0_8         ; Jump if the result of the test
                       ; is not zero, i.e., emojis are
                       ; not "🤪"
```

🌈 Jump to `.LBB0_12` if the four bytes of the rhs emoji string slice are equal to 🤪 or 😞.

```
lea     rax, [rip + .L__unnamed_1] ; 📄 Load address of "🤪" into rax

cmp     dword ptr [rdx], -2087149584 ; 📄 Compare four bytes of rhs
                                           ; emoji to bytes of "🤪"

je      .LBB0_12        ; Jump if they are equal

cmp     dword ptr [rdx], -1634164752 ; 📄 Compare four bytes of rhs
                                           ; emoji to bytes of "😞"

je      .LBB0_12        ; Jump if they are equal
```

🌈 Jump to `.LBB0_10` if the self emoji string slice equals 😞.

```
cmp     dword ptr [rdi], -1634164752 ; 📄 Compare four bytes of self
                                           ; emoji to bytes of "😞"
```

Part E Strings, Dispatch, and Recursion

```
je .LBB0_10 ; Jump if they are equal
```

☐ If none of the comparisons are successful, return None.

```
.LBB0_7:
xor     eax, eax           ; ☐ Returning None. Zero out eax.
mov     edx, 4             ; Move 4 into edx
ret      Returning None
```

 Jump to .LBB0_12 with rax set to None if:

- The four bytes of the rhs emoji string slice do not match 🙄 bytes
- The length of the rhs emoji string slice is not equal to 4

```
.LBB0_8:
xor     esi, esi           ; Zero out esi
cmp     dword ptr [rdi], -1634164752 ;  Compare four bytes of
                                         ; the self emoji to bytes of "🙄"
setne   dil               ; Set dil to 1 if they are not equal
xor     eax, eax           ; Zero out eax (None)
cmp     rcx, 4             ;  Compare length of second
                                         ; string with 4
jne     .LBB0_12           ; Jump if they are not equal
mov     sil, dil           ; Move the result of the previous
                                         ; comparison to sil
test    esi, esi          ; Test sil (AND with itself)
jne     .LBB0_12           ; Jump if the result of test is not
                                         ; zero, i.e., emojis are not "🙄"
```

 Jump to LBB0_13 if the rhs emoji bytes match 😊.

```
.LBB0_10:
cmp     dword ptr [rdx], -2087149584 ;  Compare the first four bytes
                                         ; of rhs emoji to bytes of "😊"
je      .LBB0_13           ; Jump if they are equal
```

 Check if the rhs emoji matches 🙄:

- If it does, return 🙄.
- If it does not, return None.

```
xor     ecx, ecx           ; Zero out ecx
cmp     dword ptr [rdx], -1634164752 ;  Compare the first four bytes
```



```

; of rhs emoji to bytes of "😞"
setne    cl                ; Set cl to 1 if they are not equal
xor      eax, eax          ; Zero out eax
test     ecx, ecx          ; Test cl (AND operation with itself)
lea      rcx, [rip + .L__unnamed_2] ; Load address of "😞" into rcx
cmov     rax, rcx          ; Conditionally move rcx into rax
                        ; (if ZF is not set)

```

 Return to the caller with the result in rax and edx. The result depends upon the rax value.

```

.LBB0_12:
mov      edx, 4            ;  Move 4 into edx (length of the
                        ; returned string)
ret       Return

```

 Return 😊.

```

.LBB0_13:
lea      rax, [rip + .L__unnamed_1] ;  Load address of "😊" into rax
mov      edx, 4            ;  Move 4 into edx (length of the
                        ; returned string)
ret       Return

```

“😊” and “😞” representations in assembly.

```

.L__unnamed_1:
.ascii   "\360\237\230\203"        ; UTF-8 representation of "😊" emoji
.L__unnamed_2:
.ascii   "\360\237\230\236"        ; UTF-8 representation of "😞" emoji

```

add_two_emojis Assembly

```

pub fn add_two_emojis(a: Emoji, b: Emoji) -> Option<Emoji> {
    a + b
}

```

The `add_two_emojis` function is a simple wrapper around the `add` function. The compiler uses tail call optimization to avoid allocating a new stack frame for the `add` function. Instead, it reuses the stack frame of the `add_two_emojis` function for the `add` function.

```

example::add_two_emojis:
;  Jump to the add method of Emoji. When the add method returns,
; it will directly return to the caller of add_two_emojis.
jmp      qword ptr [rip + <example::Emoji as Add>::add@GOTPCREL]

```



Learn More: Tail Call Optimization

Tail call optimization allows the compiler to reuse the current function's stack frame for the following function call. This optimization is possible under the following conditions:

- The last operation of the current function is a function call.
- The arguments of the following function call are already in the proper registers.
- The return value of the current function is the return value of the following function call.

This optimization proves beneficial for wrapper functions, allowing the compiler to bypass allocating a new stack frame for the wrapped function. Similarly, it is advantageous for recursive functions, enabling the compiler to reuse the stack frame for each recursive call. We will explore recursive functions in greater detail later in the book.

add_three_emojis Assembly

```
pub fn add_three_emojis(a: Emoji, b: Emoji, c: Emoji) -> Option<Emoji> {
    (a + b)? + c
}
```

This function starts by saving the pointers to the third emoji and its length in r14 and r8, respectively. It then delegates to the Add implementation to add the first two emojis.

If the result is None, it returns None. Otherwise, it calls the Add implementation again to add the third emoji to the result of adding the first two. Finally, it restores the saved registers and returns the result. This logic implements the expression $(a + b)? + c$. The expression is equivalent to the following code:

```
if let Some(a_plus_b) = a + b {
    a_plus_b + c
} else {
    None
}
```

Input:

- rdi =  a.ptr
- rsi =  a.len
- rdx =  b.ptr
- rcx =  b.len
- r8 =  c.ptr
- r9 =  c.len

Output:

- rax =  result.ptr or ☐ None
- rdx =  result.len

```
example::add_three_emojis:
push    r14                ; Save r14
push    rbx                ; Save rbx
push    rax                ; Save rax

mov     r14, r9             ;  Move c.len to r14
mov     rbx, r8             ;  Move c.ptr to rbx
```

- `a.ptr`, `a.len`, `b.ptr`, `b.len` are already in `rdi`, `rsi`, `rdx`, `rcx`
- The arguments are already in the right registers for the `add`. Call `add` method of `Emoji` for `a + b`

```
call    qword ptr [rip + <example::Emoji as Add>::add@GOTPCREL]

; Test the return value (the result of the add operation is not None)
test    rax, rax

je      .LBB2_1          ; Jump if it is None

mov     esi, 4           ; Move 4 (emoji string length of a+b) into esi

mov     rdi, rax         ; Move result of a+b value into rdi

mov     rdx, rbx         ; Move c.ptr into rdx
mov     rcx, r14         ; Move c.len into rcx
```

★ Call `add` method of `Emoji` for `(a + b) + c`

```
call    qword ptr [rip + <example::Emoji as Add>::add@GOTPCREL]

jmp     .LBB2_3          ; Skip setting eax to 0 as the result is not None

.LBB2_1:
xor     eax, eax         ; Returning None. Zero out eax.

.LBB2_3:
mov     edx, 4           ; Move 4 into edx (length
                        ; of the None return value)

add     rsp, 8           ; Add 8 to rsp to balance the stack

pop     rbx              ; Restore rbx
pop     r14              ; Restore r14

ret     ; Return result.ptr in rax and
        ; result.len in rdx
```

Experiment: String Manipulation and Optimization

Combine Strings Experiment

Let us use [Compiler Explorer](https://godbolt.org/z/nj9rc5Y1z) (<https://godbolt.org/z/nj9rc5Y1z>) to compare with the assembly generated for the following functions:

- `combine_strings_ref`: Here, we pass both strings by reference. The function allocates a new string for the result.

```
pub fn combine_strings_ref(s1: &str, s2: &str) -> String {
    let mut result = String::new();
    result.push_str(s1);
    result.push_str(s2);
}
```

Part E Strings, Dispatch, and Recursion

```
    result
}
```

- `combine_strings_mut`: Here, we pass the first string by mutable reference and the second as an immutable reference. The function directly modifies the first string. This way, we can avoid allocating a new string for the result.

```
pub fn combine_strings_mut(s1: &mut String, s2: &str) {
    s1.push_str(s2);
}
```

Builder Pattern Experiment

Edit the builder pattern example in [Compiler Explorer](https://godbolt.org/z/6rdeTcoxe) (<https://godbolt.org/z/6rdeTcoxe>) and make the following functions pub one at a time:

- `new`
- `street`
- `city`
- `postal_code`
- `build`

You will see that the compiler generates code for every pub function even though it has inlined the calls to these functions in the `make_address` function.

Emoji Arithmetic Example with Owned Strings

Use [Compiler Explorer](https://godbolt.org/z/G4cs3731K) (<https://godbolt.org/z/G4cs3731K>) to experiment with the assembly generated when the `Emoji` struct owns the strings. Replace the `Emoji` struct and the `add` function with the following code:

```
#[derive(PartialEq)]
pub struct Emoji(String);

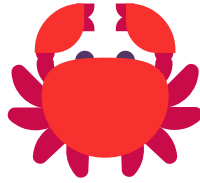
impl std::ops::Add for Emoji {
    type Output = Option<Self>;

    fn add(self, rhs: Self) -> Self::Output {
        let Emoji(self_emoji) = self;
        let Emoji(rhs_emoji) = rhs;

        match (self_emoji.as_str(), rhs_emoji.as_str()) {
            ("😄", "😄") => Some(Emoji("😄".to_string())),
            ("😄", "😞") => Some(Emoji("😄".to_string())),
            ("😞", "😄") => Some(Emoji("😄".to_string())),
            ("😞", "😞") => Some(Emoji("😞".to_string())),
            _ => None,
        }
    }
}
```

Key Takeaways

- `String` is a struct that contains the following fields:
 - Pointer to a heap-allocated buffer
 - Capacity of the heap-allocated buffer
 - Length of the string
- `&str` is a struct that contains a pointer to a string slice and length.
- If a `String` parameter transfers ownership, the caller passes a pointer to the `String` struct. The called function is responsible for deallocating the heap-allocated buffer in the `String` when the function is going out of scope. We saw this in the `combine_strings` function.
- If a function returns a `String`, the caller passes a pointer where the called function should store the result `String`. We saw this in the `combine_strings` function.
- The compiler uses two registers to pass a string slice `ptr` and length `len`. We observed this in the `add_three_emojis` function.
- For small strings, the compiler can treat a string's UTF-8 bytes as a single integer. This optimization allows the compiler to generate efficient code for string comparisons. We saw this in the `Emoji` struct's `add` method.
- Tail call optimization reduces the overhead of stack preparation. We saw this in the `add_two_emojis` function. This optimization reduces the overhead of calling wrapper functions. We will see in later chapters that this optimization is also helpful for recursive functions.
- The Rust compiler removes the discriminator for `None` and `Some` variants for non-nullable types. It represents the `None` variant as a null pointer and the `Some` variant as a valid pointer. We observed this in the `add_three_emojis` function.



Chapter 14

Static vs. Dynamic Dispatch

Traits in Rust are like interfaces in other languages. Traits permit the implementation of a standard interface for multiple types. Developers can compose their code using traits instead of concrete types.

When calling a method specified by a trait, the compiler generates code to dispatch the call to the concrete method that implements the trait specification. This chapter will examine how the Rust compiler implements traits and generates code for static and dynamic dispatch.

Rust supports two types of dispatch:

- **Static dispatch** serves as the default mode, employed when the compiler can determine the concrete type at compile time.
- **Dynamic dispatch** comes into play when the concrete type implementing the trait remains unknown at compile time.

Example: Static vs. Dynamic Dispatch

We will use the following example to illustrate the difference between static and dynamic dispatch.

```
// The Shape trait requires that the implementor have a method called area
// that returns the area as the associated type Shape::T.
pub trait Shape {
    type T;
    fn area(&self) -> Self::T;
}

// A generic Point for a specified type T.
pub struct Point<T> {
    x: T,
    y: T,
}

// A generic Rectangle for a specified type T.
pub struct Rectangle<T> {
    top_left: Point<T>,
    bottom_right: Point<T>,
}
```

```

}

// Implement the Shape trait for the Rectangle using a generic type T.
// The T is the return type of the area method. The where clause specifies
// that T must support subtraction, multiplication, and the ability to copy.
impl<T> Shape for Rectangle<T>
where
    T: std::ops::Sub<Output = T> + std::ops::Mul<Output = T> + Copy,
{
    type T = T;
    fn area(&self) -> T {
        let width = self.bottom_right.x - self.top_left.x;
        let height = self.top_left.y - self.bottom_right.y;

        width * height
    }
}

// A function that calculates the area of two shapes and returns a tuple
// containing the areas. This function requires that the two shapes implement
// the Shape trait. The function will call the area function via a static
// dispatch. Code will be generated for the function only if concrete
// types are specified for `a` and `b`.
pub fn area_pair_static(a: impl Shape<T = f64>,
                       b: impl Shape<T = f64>) -> (f64, f64) {
    (a.area(), b.area())
}

pub fn static_dispatch_pair(a: Rectangle<f64>,
                           b: Rectangle<f64>) -> (f64, f64) {
    // The following line will generate code for the function as concrete
    // types are specified for `a` and `b`.
    area_pair_static(a, b)
}

// This function performs the same function as `area_pair_static` but uses
// dynamic dispatch. The compiler will generate code for this function.
// The function called will be determined at runtime.
pub fn area_pair_dynamic(a: &dyn Shape<T = f64>,
                        b: &dyn Shape<T = f64>) -> (f64, f64) {
    (a.area(), b.area())
}

```

Static dispatch

The `static_dispatch_pair` function requires `a` and `b` parameters as values with concrete types (`impl Shape<T = f64>` serves as syntactic sugar for generic parameters implementing the specified trait). Consequently, the function understands the layout of `a` and `b` at compile time, allowing direct calls to the `area` method in the `Shape` trait. These calls, as we will explore, are also candidates for inlining.

The accompanying diagram illustrates the memory layout of a sample `Rectangle` object, suitable for passing as either `a` or `b`.

00 top_left.x	0.0
08 top_left.y	0.0
16 bottom_right.x	42.0
24 bottom_right.y	84.0

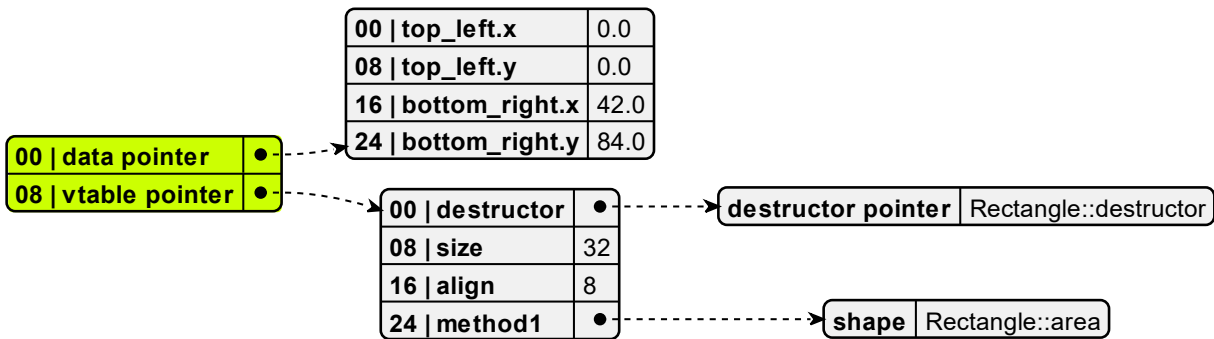
Dynamic dispatch

The `area_pair_dynamic` function takes two parameters, `a` and `b`; both are dynamic trait objects of type `&dyn Shape<T = f64>`. The function uses trait objects without knowing the specific types that implement the `Shape<T>` trait. As a result, the code calls the `area` method on these objects through a function pointer. How does the compiler generate code for this function? We will dissect the assembly code and the `vtable` to understand the machinery behind dynamic dispatch.

For each unique type that implements `Shape<T>`, the compiler generates a `vtable` (virtual table) containing pointers to the trait's methods as implemented by that type. Each type has its own `vtable`, stored in the program's static data section. The `vtable` facilitates the runtime dispatch of calls to the appropriate method.

Instances of the same type share the same `vtable`, but distinct types have separate `vtables`. The `vtable` includes pointers to the trait method implementations specific to each type. The Rust compiler uses a **fat pointer** for `&dyn Shape<T = f64>`, a tuple containing a pointer to the data and a pointer to the `vtable`.

The `area_pair_dynamic` receives `a` and `b` as fat pointers. The accompanying diagram shows the memory layout of a sample `Rectangle` object, which can serve as either `a` or `b`. The leftmost box represents the fat pointer, with the data pointer pointing to the `Rectangle` object and the `vtable` pointer pointing to the `vtable` of the `Rectangle` type.



Now that we understand the differences between static and dynamic dispatch let us look at the code generated for the two functions in the following two sections.

- [Static Dispatch: Assembly of Trait-Based Generics and Inlining](#)
- [Dynamic Dispatch with Trait Objects: Dissecting the Assembly and Vtables](#)

Static Dispatch: Assembly of Trait-Based Generics and Inlining

```
pub fn area_pair_static(a: impl Shape<T = f64>,
                       b: impl Shape<T = f64>) -> (f64, f64) {
    (a.area(), b.area())
}

pub fn static_dispatch_pair(a: Rectangle<f64>,
```

Part E Strings, Dispatch, and Recursion

```

    b: Rectangle<f64>) -> (f64, f64) {
    area_pair_static(a, b)
}

```

The static dispatch occurs in the call to the `area_pair_static` function. The function requires that `a` and `b` belong to a type that implements the `Shape<T = f64>`. As mentioned, the syntax for `impl` based parameter passing serves as syntactic sugar for a generic-based representation. The desugared generic representation of `area_pair_static` follows.

```

pub fn area_pair_static<S, T> (a: S, b: T) -> (f64, f64)
where
    S: Shape<T = f64>,
    T: Shape<T = f64>,
{
    (a.area(), b.area())
}

```

The generic representation shows that the `area_pair_static` function is generic over two types, `S` and `T`. The function requires that `S` and `T` should implement the `Shape<T = f64>` trait. The function body calls the `area` method on the `a` and `b` parameters. The compiler will only generate code for the function if it receives concrete types for `a` and `b`. The function `static_dispatch_pair` does exactly this, defining `S` and `T` as `Rectangle<f64>`.

`Rectangle<f64>` layout follows. The `top_left` and `bottom_right` fields are of type `Point<f64>`. The `x` and `y` fields are of type `f64`. The layout of `Rectangle<f64>` also shows the byte offset from the starting address on the left side of each field.

00 top_left.x	0.0
08 top_left.y	0.0
16 bottom_right.x	42.0
24 bottom_right.y	84.0

The compiler generates the following code for the `static_dispatch_pair` function and inlines the code for the `area_pair_static` and `area` functions. This generated code employs vector instructions to accelerate the computation of the two area calculations. It returns the two area tuples in the `xmm0` and `xmm1` registers.

12 34 Input parameters:

- `a`: `Rectangle<f64>` in `rdi`
- `b`: `Rectangle<f64>` in `rsi`

12 34 Output parameters:

- (`f64`, `f64`) in (lower `xmm0`, lower `xmm1`)

```
example::static_dispatch_pair:
```

■ This code calculates rectangle `a`'s area and stores the result in the lower part of `xmm0`.

```

movupd  xmm1, xmmword ptr [rdi + 8] ; Vector load a.top_left.y and
                                     ; a.bottom_right.x into xmm1
                                     ; xmm1 = (a.top_left.y, a.bottom_right.x)

```

```

movsd    xmm0, qword ptr [rdi + 24] ; Load a.bottom_right.y into lower xmm0

movhpd   xmm0, qword ptr [rdi]      ; Load a.top_left.x into upper xmm0
                                           ; xmm0 = (a.top_left.y, a.top_left.x)

subpd    xmm1, xmm0                 ; xmm1 = (a.top_left.y-a.bottom_right.y,
                                           ; a.bottom_right.x-a.top_left.x)
                                           ; xmm1 = (a:height, a:width)

movapd   xmm0, xmm1                 ; xmm0 = (a:height, a:width)

unpckhpd xmm0, xmm1                 ; xmm0 = (a:height, a:height)
                                           ; xmm1 = (a:height, a:width)

mulsd    xmm0, xmm1                 ; lower xmm0 = a:width*a:height = a:area

```

■ Calculate rectangle b's area and copy the results to lower xmm1.

```

movupd   xmm2, xmmword ptr [rsi + 8] ; Vector load b.top_left.y and
                                           ; b.bottom_right.x into xmm2
                                           ; xmm2 = (b.top_left.y, b.bottom_right.x)

movsd    xmm1, qword ptr [rsi + 24] ; Load b.bottom_right.y into lower xmm1

movhpd   xmm1, qword ptr [rsi]      ; Load b.top_left.x into upper xmm1
                                           ; xmm1 = (b.top_left.y, b.top_left.x)

subpd    xmm2, xmm1                 ; xmm2 = (b.top_left.y-b.bottom_right.y,
                                           ; b.bottom_right.x - b.top_left.x)
                                           ; xmm2 = (b:height, b:width)

movapd   xmm1, xmm2                 ; xmm1 = (b:height, b:width)

unpckhpd xmm1, xmm2                 ; xmm1 = (b:height, b:height)
                                           ; xmm2 = (b:height, b:width)

mulsd    xmm1, xmm2                 ; lower xmm1 = b:width*b:height = b:area

ret                                             ; Return (a:area, b:area) in (xmm0, xmm1)

```

Dynamic Dispatch with Trait Objects: Dissecting the Assembly and Vtables

Let us focus on the `area_pair_dynamic` function and understand the machinery behind dynamic dispatch.

```

pub fn area_pair_dynamic(a: &dyn Shape<T = f64>,
                        b: &dyn Shape<T = f64>) -> (f64, f64) {
    (a.area(), b.area())
}

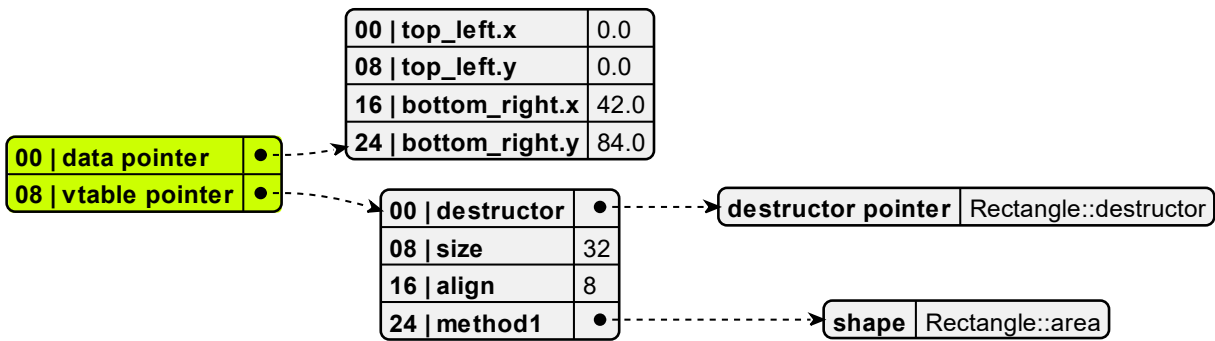
```

The `area_pair_dynamic` function takes two trait objects as parameters. The function body calls the `area` method on the `a` and `b` parameters. The compiler generates code for the function referencing the trait object. There is no reference to the concrete types.

Part E Strings, Dispatch, and Recursion

The layout of the trait object fat pointer follows. As we learned earlier, the trait object is a tuple of two pointers. The first pointer is the address of the object. The second pointer is the address of the vtable of the object. The vtable contains the addresses of the trait methods implemented by the object.

In addition to the function pointers, the vtable also includes a pointer to the drop function, the size, and the alignment of the concrete type. When the object goes out of scope, it calls the drop function. This drop typically happens when a smart pointer like Box contains a trait object. If the Box gets dropped, it calls the drop function on the object. The code uses the size and alignment of the concrete type to free memory for the object.



The compiler generates the following code for the `area_pair_dynamic` function.

The compiler passes each trait object pointer via two registers. One register holds the address of the object. The other register carries the address of the vtable of the object. The generated code obtains the address of the area function from the vtable of the object. It sets the `self`-parameter to the address of the object. Then, it calls the area function using the area function pointer. It then invokes the area function twice, once for each object.

12 34 Input parameters:

- rdi contains the address of the first object (a).
- rsi contains the address of the vtable of the first object (a)
- rdx contains the address of the second object (b).
- rcx contains the address of the vtable of the second object (b)

12 34 Output parameters:

- Tuple<f64, f64> in (xmm0, xmm1)

example::area_pair_dynamic:

```

push    r14                ; Save the previous r14
push    rbx                ; Save the previous rbx
push    rax                ; Save the previous rax

mov     r14, rcx            ; r14 = address of the vtable of b
mov     rbx, rdx            ; rbx = b (address of b)
    
```

Dynamic dispatch:

- rdi contains the address of a (self)
- rsi contains the address of the vtable for a's type

```

call    qword ptr [rsi + 24] ; Get address of the area function from
                                ; the vtable and call it.
    
```

```
movsd    qword ptr [rsp], xmm0    ; Save a.area() return value in a local
                                           ; variable

mov      rdi, rbx                  ; rdi = address of b
```

■ Dynamic dispatch:

- rdi contains the address of b (self)
- r14 contains the address of the vtable for b's type

```
call     qword ptr [r14 + 24]      ; Get address of the area function from
                                           ; the vtable and call it.
```

(■, ■) Save the return value of b.area() in xmm1 and restore the return value of a.area() from the local variable. Return the tuple in xmm0 and xmm1.

```
movaps   xmm1, xmm0              ; Save b.area() return value in xmm1

movsd    xmm0, qword ptr [rsp]    ; Get a.area() return value from stack

add      rsp, 8                   ; Remove local variable from the stack

pop      rbx                      ; Restore previous rbx
pop      r14                     ; Restore previous r14

ret                                             ; Return a.area() and b.area()
                                           ; as (xmm0, xmm1)
```



Experiment: Trait Objects in a Box and Optimize Away Dynamic Dispatch

Click on [Compiler Explorer](https://godbolt.org/z/95aPbMjsG) (<https://godbolt.org/z/95aPbMjsG>) link to experiment with what we have covered in this chapter.

Let us try out a few things in Compiler Explorer.

- [Passing Trait Objects in a Box](#)
 - [Box Owning a Trait Object](#)
 - [Box Owning a Trait Object Reference](#)
- [Auto Conversion of Dynamic Dispatch to Static Dispatch](#)

Passing Trait Objects in a Box

Let us experiment with passing trait objects wrapped in a Box. We will run two experiments to understand the difference between passing a Box owning a trait object and a Box owning a reference to a trait object.

Box<dyn Trait> vs Box<&dyn Trait>

Box<dyn Trait>: This Box owns a trait object directly. It is a “fat pointer” that points to the heap-allocated data and has a second pointer to the vtable that provides the metadata about the concrete type that implements the trait.

Box<&dyn Trait>: This Box owns a reference to a trait object, containing a pointer to a “fat pointer” stored on the heap.

Part E Strings, Dispatch, and Recursion

Let us add the following functions to Compiler Explorer code window. We will dissect the generated code for these functions in the following two sections.

```
pub fn area_pair_dynamic_box_dyn(a: Box<dyn Shape<T = f64>>,
                                b: Box<dyn Shape<T = f64>>) -> (f64, f64) {
    (a.area(), b.area())
}

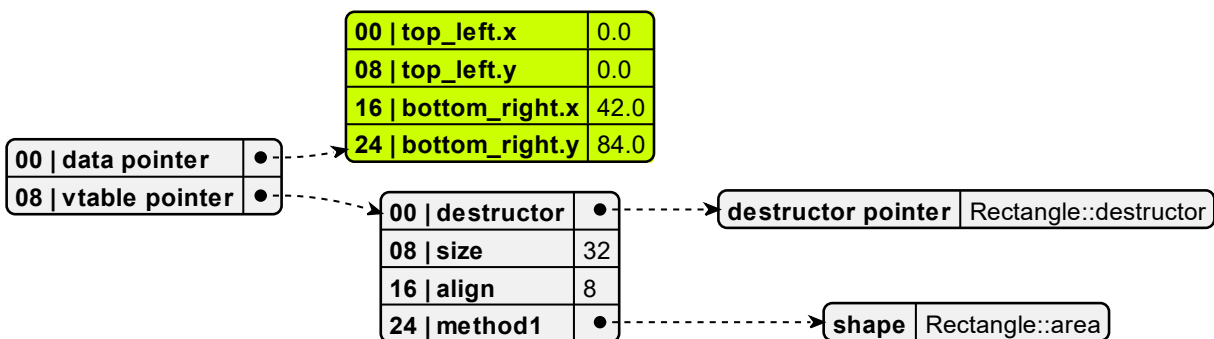
pub fn area_pair_dynamic_box_ref_dyn(a: Box<&dyn Shape<T = f64>>,
                                     b: Box<&dyn Shape<T = f64>>) -> (f64, f64) {
    (a.area(), b.area())
}
```

Box Owning a Trait Object

```
pub fn area_pair_dynamic_box_dyn(a: Box<dyn Shape<T = f64>>,
                                b: Box<dyn Shape<T = f64>>) -> (f64, f64) {
    (a.area(), b.area())
}
```

Here, the `area_pair_dynamic_box_dyn` function takes two `Box<dyn Shape<T = f64>>` parameters. When the function returns, it drops the `Box` and frees the heap memory.

How does the compiler generate code to release the heap memory when it does not know the size of the heap-allocated data? The following figure of a `Box<dyn Shape<T = f64>>` holds the key. The `vtable` stores information about the size and alignment of the heap-allocated data. The compiler uses this information to free the heap memory. It also finds the address of the destructor from the `vtable` and calls the destructor.



The annotated assembly code for the `area_pair_dynamic_box_dyn` function follows. The compiler generates code to  free the heap memory and call the  destructor of the trait object. You will see the destructor call and a call to `__rust_dealloc` with the size and alignment information from the `vtable`.

12 34 Input parameters:

-  `rdi` contains the address of the first object (a).
-  `rsi` contains the address of the `vtable` of the first object (a)
-  `rbx` contains the address of the second object (b).
-  `rcx` contains the address of the `vtable` of the second object (b)

12 34 Output parameters:

- `Tuple<f64, f64>` in `(xmm0, xmm1)`

```
example::area_pair_dynamic_box_dyn:
```

★ Saving the registers and setting up the stack.

```
push    r15
push    r14
push    r13
push    r12
push    rbx
sub     rsp, 48
```

★ Setting up the parameters.

```
mov     rbx, rcx ; Move the second box's vtable into rbx
mov     r13, rdx ; Move the second box's data into r13

mov     r12, rsi ; Move the first box's vtable into r12
mov     r14, rdi ; Move the first box's data into r14
```

★ Saving the parameters onto the stack.

```
mov     qword ptr [rsp + 16], rdi ; Save the first box's data
mov     qword ptr [rsp + 24], rsi ; Save the first box's vtable

mov     qword ptr [rsp + 32], rdx ; Save the second box's data
mov     qword ptr [rsp + 40], rcx ; Save the second box's vtable
```

■ Call the area function on the first shape.

```
call    qword ptr [rsi + 24]

movsd   qword ptr [rsp + 8], xmm0 ; Save the first area result
```

■ Call the area function on the second shape.

```
mov     rdi, r13

call    qword ptr [rbx + 24]

movsd   qword ptr [rsp], xmm0 ; Save the second area result
```

♻ Deallocating the second box.

```
mov     rdi, r13

call    qword ptr [rbx] ; 🗡 Call the destructor from the vtable

mov     rsi, qword ptr [rbx + 8] ; Fetch the size from the vtable

test    rsi, rsi ; if size == 0, skip dealloc

je      .LBB6_5

mov     rdx, qword ptr [rbx + 16] ; Fetch the alignment from vtable
```

Part E Strings, Dispatch, and Recursion

```
mov     rdi, r13 ; Pointer to the data

call    qword ptr [rip + __rust_dealloc@GOTPCREL]
```

♻️ Deallocating the first box.

```
.LBB6_5:

mov     rdi, r14

call    qword ptr [r12] ; 🗡️ Call the destructor from the vtable

mov     rsi, qword ptr [r12 + 8] ; Fetch the size from the vtable

test    rsi, rsi ; if size == 0, Skip dealloc

je      .LBB6_8

mov     rdx, qword ptr [r12 + 16] ; Fetch the alignment

mov     rdi, r14 ; Pointer to the data

call    qword ptr [rip + __rust_dealloc@GOTPCREL]

.LBB6_8:
```

★ Restore the results from the stack and set up the return values.

```
movsd   xmm0, qword ptr [rsp + 8]
movsd   xmm1, qword ptr [rsp]
```

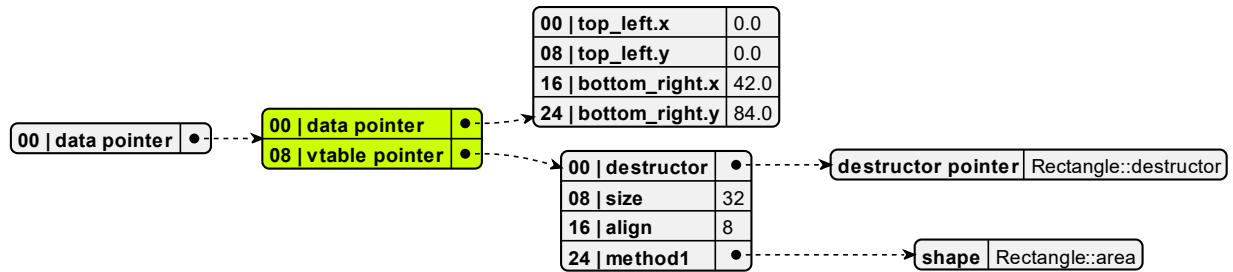
★ Restoring the stack and the registers.

```
add     rsp, 48
pop     rbx
pop     r12
pop     r13
pop     r14
pop     r15
ret     ; Return
```

Box Owning a Trait Object Reference

```
pub fn area_pair_dynamic_box_ref_dyn(a: Box<&dyn Shape<T = f64>>,
                                     b: Box<&dyn Shape<T = f64>>) -> (f64, f64) {
    (a.area(), b.area())
}
```

In this case, the Box only owns a reference to the trait object. It does not own the trait object. The following diagram shows the memory layout of Box<&dyn Shape<T = f64>>.



The annotated assembly code for the `area_pair_dynamic_box_ref_dyn` function follows. The function invokes `__rust_dealloc` with a size of 16 bytes and an alignment of 8 bytes to release the fat pointer. Note that the code does not free the trait object, nor does it call the destructor, as the `Box` does not own the trait object.

Input parameters:

- `rdi` contains the address of the fat pointer for the first object (a).
- `rsi` contains the address of the fat pointer for the second object (b).

Output parameters:

- `Tuple<f64, f64>` in `(xmm0, xmm1)`

```

example::area_pair_dynamic_box_ref_dyn:  ; function label
push    r15
push    r14
push    rbx
sub     rsp, 16 ; Reserve space for local variables on the stack

```

Compute the area for the first shape.

```

mov     rbx, rsi ; Copy pointer to the second shape to rbx
mov     r14, rdi ; Copy pointer to the first shape to r14
mov     rdi, qword ptr [rdi] ; Load first data ptr to rdi
mov     rax, qword ptr [r14 + 8] ; Load first vtable ptr to rax
call    qword ptr [rax + 24] ; Call area method via vtable
movsd   qword ptr [rsp + 8], xmm0 ; Store the first shape's
                                   ; area on the stack

```

Compute the area for the second shape.

```

mov     rdi, qword ptr [rbx] ; Load second data ptr to rdi
mov     rax, qword ptr [rbx + 8] ; Load second vtable ptr to rax
call    qword ptr [rax + 24] ; Call area method via vtable
movsd   qword ptr [rsp], xmm0 ; Store the second shape's area on the stack

```

Deallocate the fat pointers stored on the heap.

```

mov     r15, qword ptr [rip + __rust_dealloc@GOTPCREL] ; Load Rust dealloc
mov     esi, 16 ; Set the size of the fat pointer in bytes

```

Part E Strings, Dispatch, and Recursion

```

mov     edx, 8    ; Set alignment of the fat pointer in bytes
mov     rdi, rbx  ; Pass a pointer to the second box to be deallocated

call    r15       ; Call the deallocation function

mov     esi, 16   ; Set the size of the fat pointer in bytes
mov     edx, 8    ; Set alignment of the fat pointer in bytes
mov     rdi, r14  ; Pass a pointer to the first box to be deallocated

call    r15       ; Call the deallocation function

```

(■, ■) Return the tuple in xmm0 and xmm1.

```

movsd   xmm0, qword ptr [rsp + 8] ; Load first shape's area from stack
movsd   xmm1, qword ptr [rsp]     ; Load the second shape's area from stack

add     rsp, 16 ; Restore the stack pointer
pop     rbx     ; Restore the original value of rbx
pop     r14     ; Restore the original value of r14
pop     r15     ; Restore the original value of r15
ret     ; Return from the function

```

Auto Conversion of Dynamic Dispatch to Static Dispatch

The Rust compiler can optimize away dynamic dispatch and convert it to static dispatch if it can determine the concrete type at compile time.

Add the following function in the code and see the assembly generated. In this case, the compiler can optimize the code and generate the same assembly as the static dispatch version as it knows that the type of the arguments is `Rectangle<f64>`, and it can inline the code for the area function.

```

pub fn dynamic_dispatch_pair(a: Rectangle<f64>,
                             b: Rectangle<f64>) -> (f64, f64) {
    area_pair_dynamic(&a, &b)
}

```

Key Takeaways

Static Dispatch Vs Dynamic Dispatch

Static dispatch

The compiler can inline the static dispatch code for the concrete type.

The compiler produces code tailored to each concrete type that implements the applicable trait. However, using static dispatch for numerous types can lead to code bloat, as it generates specific code for each type.

Dynamic dispatch

The code generated for a dynamic dispatch is less efficient as the compiler does not know the concrete type, and it invokes the called trait method via a function pointer contained in the `vtable` associated with the concrete type.

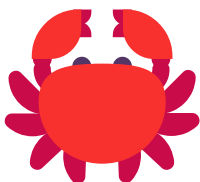
The compiler generates code that works for all types that implement the referenced trait.

Static dispatch	Dynamic dispatch
On a 64-bit system, the compiler generates code that passes the concrete type as a 64-bit pointer.	On a 64-bit system, the compiler generates code that references a trait object via a 128-bit fat pointer. The fat pointer is a tuple of the 64-bit pointer to the object and a 64-bit pointer to the <code>vtable</code> associated with the concrete type.

Rust Supports Deallocation of the Concrete Type from a Trait Object

Rust can free the concrete type implementing the trait when dropping the trait object because it records the size and alignment of the concrete type in the `vtable` associated with that type.

We observed this in the [Box owning a trait object](#) example. The compiler generated code to release the heap-allocated `Rectangle` when dropping the `Box<dyn Shape<T = f64>>`.



Chapter 15

Traits, Vtables, and Tail Calls

We have looked at static and dynamic dispatch in the “*Static vs. Dynamic Dispatch*” chapter. This chapter will examine the assembly code generated when one trait inherits from another. By analyzing this code, we will see how the compiler reduces some overhead associated with dynamic dispatch using tail call optimization. We will also examine how the compiler frees memory when a trait object wrapped in a `Box` goes out of scope. To illustrate these concepts, we will work with the following example code:

```
type Num = f64;

pub trait Shape {
    type T;
    fn area(&self) -> Self::T;
}

pub trait Draw: Shape {
    fn draw(&self);
}

// The function operates with a trait object reference
pub fn draw_dynamic(a: &dyn Draw<T = Num>) {
    a.draw();
}

// Invokes the draw method and then the area method on the trait object
pub fn draw_and_report_area_dynamic(a: &dyn Draw<T = Num>) -> Num {
    a.draw();
    a.area()
}

// The function operates with a trait object wrapped in a Box.
// The trait object is freed when the wrapping Box
// goes out of scope.
pub fn draw_and_report_area_dynamic_box(a: Box<dyn Draw<T = Num>>) -> Num {
    a.draw();
    a.area()
}
```

Part E Strings, Dispatch, and Recursion

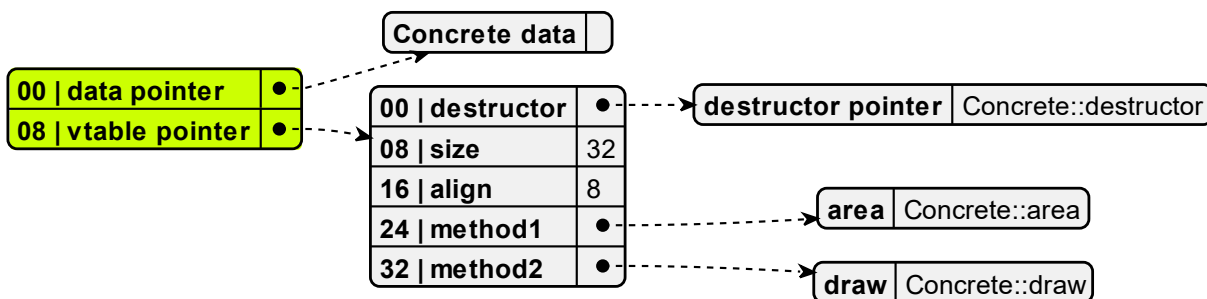
The preceding code incorporates two traits: `Shape` and `Draw`. The `Shape` trait pertains to geometric figures with an area and requires a method named `area`. The `Draw` trait, an extension of `Shape`, denotes items that support drawing. The `Draw` trait mandates the implementation of the `draw` method.

We will be looking at the assembly code generated for the following functions:

- `draw_dynamic`: This function takes a trait object as an argument and calls the `draw` method of the trait object.
- `draw_and_report_area_dynamic`: This function takes a trait object as an argument and calls the `draw` method of the trait object, followed by the `area` method. The function returns the result of the `area` method.
- `draw_and_report_area_dynamic_box`: This function takes a trait object wrapped in a `Box` as an argument and calls the `draw` method of the trait object, followed by the `area` method of the trait object. The function returns the result of the `area` method. We will also see that the function frees the memory allocated for the trait object.

Trait Object Fat Pointer and the Vtable

As the previous chapter covers, the Rust compiler uses fat pointers to realize trait objects. To revisit this concept, let us delve into the fat pointer associated with the `Draw` trait object:



The first pointer points to the data of the concrete type that implements the `Draw` trait, and the second pointer points to the `vtable` (virtual table) for the concrete type. The `vtable` data structure stores pointers for concrete type methods, destructor, and alignment and size information. The compiler needs this information to free the memory allocated for the trait object when it goes out of scope.

Dynamic Dispatch and Tail Call Optimization

The generated assembly code for the `draw_dynamic` function includes a jump to the `vtable` address of the `draw` function rather than a call. This optimization, known as *Tail Call Optimization (TCO)*, eliminates creating a new stack frame and return address for the call chain's last (tail) function.

In this case, the `draw` function returns directly to the caller of `draw_dynamic` using the return address that was already present on the stack. This optimization is possible because the `rdi` register already points to the concrete object that implements the `Draw` trait, satisfying the `self`-parameter requirement for the `draw` function.

Inputs:

- `rdi`: Address of `a`
- `rsi`: Address of the `vtable` associated with `a`

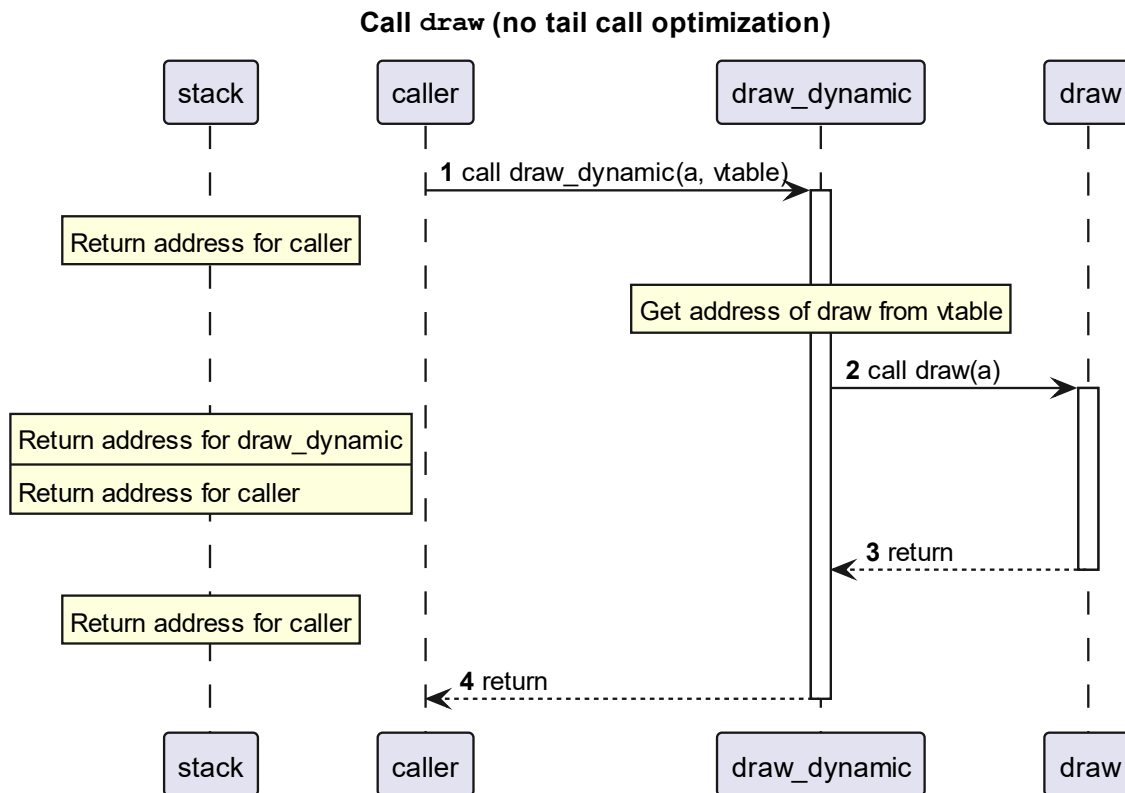
```
example::draw_dynamic:
  jmp     qword ptr [rsi + 32] ; ↪ Transfer control to a.draw()
                               ; via the `vtable`
```

Tail Call Optimization Illustrated

Let us dig deeper into the tail call optimization we have just examined.

The following diagram illustrates the call chain for the `draw_dynamic` function if no tail call optimization existed.

1. The caller function invokes the `draw_dynamic` function.
 - The stack now holds the return address in the caller function.
 - It passes `a` and `vtable` to the `draw_dynamic` function as a fat pointer.
2. The `draw_dynamic` function calls the `draw` function via the `vtable`.
 - The stack now holds the return addresses in the `draw_dynamic` and caller functions.
 - It passes `a` to the `draw` function.
3. The `draw` function returns control to the `draw_dynamic` function.
 - The `draw_dynamic` function pops its return address from the stack.
 - The stack now contains the return address for the caller function.
4. The `draw_dynamic` function returns control to the caller function.
 - The caller function pops its return address from the stack.

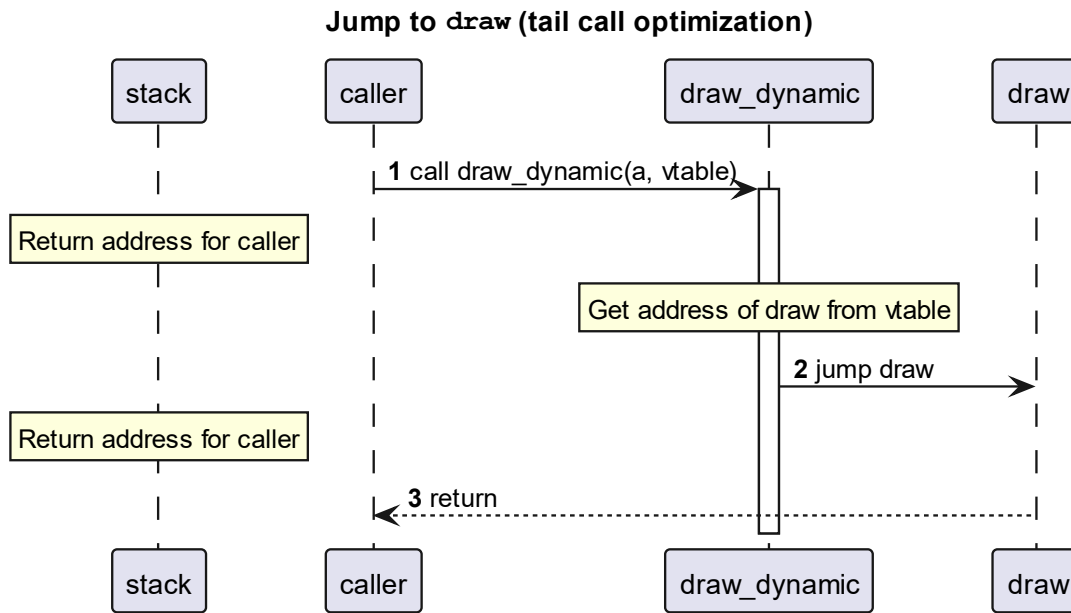


With tail call optimization, we save on stack frame preparation and return. The following diagram illustrates the call chain for the `draw_dynamic` function with tail call optimization.

1. The caller function calls the `draw_dynamic` function.
 - The stack now contains the caller function's return address.

Part E Strings, Dispatch, and Recursion

- The caller function passes `a` and `vtable` to the `draw_dynamic` function as a fat pointer.
- 2. The `draw_dynamic` function uses the `vtable` to jump to the `draw` function.
 - The stack still holds the caller function's return address.
 - `a` is where the `draw` function expects to find it.
- 3. The `draw` function returns directly to the caller function.
 - The caller function's return address gets popped from the stack, making the `draw` function think the caller function directly calls it.



Vtable Method Dispatch and Tail Calls

The generated assembly code for the `draw_and_report_area_dynamic` function is more complex than `draw_dynamic`. It first obtains the addresses of the `draw` and `area` methods from the `vtable` using offsets of 32 and 24, respectively. It then calls the `draw` method using a `call` instruction, which creates a new stack frame and return address.

The code transfers control to the `area` method using a `jump` instruction. The compiler can tail call optimize the `area` method as it is the last function call in the body of `draw_and_report_area_dynamic`. The compiler optimizes away the call by replacing it with a jump, eliminating the need for a new stack frame and return address. The return value of the `area` method is also the return value of `draw_and_report_area_dynamic`, so the optimization has no impact on the correctness of the code.

Inputs:

- `rdi`: Address of `a`
- `rsi`: Address of the `vtable` associated with `a`

Outputs:

- `xmm0`: Result of function

```
example::draw_and_report_area_dynamic:
push    r14 ; Save the value of r14
push    rbx ; Save the value of rbx
push    rax ; Save the value of rax
```


★ Call `a.draw()`

`draw` is not the last function call in the function body, so it is not eligible for tail call optimization.

```

mov     r14, rsi ; Save the address of the vtable
mov     rbx, rdi ; Save the address of a

call    qword ptr [rsi + 32] ; Call a.draw() via the vtable

mov     rdi, rbx ; Restore the address of a
mov     rax, r14 ; Restore the address of the vtable

```

★ Remove the three pushes from the stack in preparation for the tail call. This action leaves the stack clean, enabling the execution of a return instruction following the jump to `a.area()`, which will return from `draw_and_report_area_dynamic`.

```

add     rsp, 8 ; Adjust the stack pointer
pop     rbx ; Restore the value of rbx
pop     r14 ; Restore the value of r14

```

↪ Tail call `a.area()`

`area` is the last function call in the function body, and its return value is also the return value of `draw_and_report_area_dynamic`.

```

jmp     qword ptr [rax + 24] ; Jump to a.area() via the vtable.

```

Boxed Trait Object Memory Management

The function `draw_and_report_area_dynamic_box` takes a `Box` containing a trait object as an argument and returns the result of calling the `area` method of the trait object. It then calls the `draw` method of the trait object contained in the `Box`, followed by the `area` method. The function then calls the destructor of the trait object contained in the `Box` and frees the memory allocated for the trait object. The function then returns the result of the `area` method.

Inputs:

- `rdi`: Address of `a`
- `rsi`: Address of the vtable associated with `a`

Outputs:

- `xmm0`: Result of the function

```

example::draw_and_report_area_dynamic_box:
push    r15
push    r14
push    rbx
sub     rsp, 32 ; Adjust stack pointer to reserve space for temporaries

mov     rbx, rsi ; Save the address of the vtable into rbx
mov     r14, rdi ; Save the address of the Box into r14

mov     qword ptr [rsp + 16], rdi ; Save address of the Box on the stack
mov     qword ptr [rsp + 24], rsi ; Save address of the vtable on the stack

```

Part E Strings, Dispatch, and Recursion

★ Call the draw method of the trait object in the Box.

```
call    qword ptr [rsi + 32]    ; Call the draw method of the trait object
mov     rdi, r14                ; Restore the address of the Box into rdi
```

★ Call the area method of the trait object in the Box.

```
call    qword ptr [rbx + 24]    ; Call the area method of the trait object
movsd   qword ptr [rsp + 8], xmm0 ; Save the result of the area method
                                           ; on the stack
```

★ Call the destructor of the trait object in the Box.

```
mov     rdi, r14                ; Load the address of the Box into rdi
call    qword ptr [rbx]         ; Call the destructor of the value
                                           ; in the Box
```

🔄 Free the memory allocated for the trait object in the Box.

```
mov     rsi, qword ptr [rbx + 8] ; Load the size of the value in
                                           ; the Box into rsi

test    rsi, rsi                ; Check if the size of the value
                                           ; in the Box is 0

je      .LBB5_5                 ; If the size of the value in the
                                           ; Box is 0, jump to .LBB5_5

mov     rdx, qword ptr [rbx + 16] ; Load the alignment of the value
                                           ; in the Box into rdx

mov     rdi, r14                ; Load the address of the Box into rdi
call    qword ptr [rip + __rust_dealloc@GOTPCREL] ; Call __rust_dealloc
```

Load the result of the area method into xmm0 and return.

```
.LBB5_5:
movsd   xmm0, qword ptr [rsp + 8] ; Load the result of the area method into xmm0
add     rsp, 32                    ; Adjust the stack pointer to free space
                                           ; used by temporary variables
pop     rbx
pop     r14
pop     r15
ret
```

Rectangle - A Concrete Type Implementing the Draw Trait

Let us examine a specific type that implements the Draw trait. The relevant implementations, including the `vtable`, `destructor`, `area`, and `draw` methods, are also explored.

```
pub struct Point<T> {
    x: T,
    y: T,
}

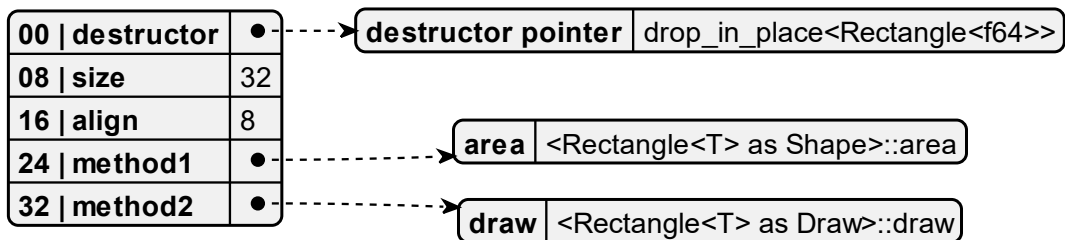
pub struct Rectangle<T> {
    top_left: Point<T>,
    bottom_right: Point<T>,
}

impl<T> Shape for Rectangle<T>
where
    T: std::ops::Sub<Output = T> + std::ops::Mul<Output = T> + Copy,
{
    type T = T;
    fn area(&self) -> T {
        let width = self.bottom_right.x - self.top_left.x;
        let height = self.top_left.y - self.bottom_right.y;
        width * height
    }
}

impl<T> Draw for Rectangle<T>
where
    T: std::ops::Sub<Output = T> + std::ops::Mul<Output = T> + Copy,
{
    fn draw(&self) {
        println!("Draw a rectangle");
    }
}
```

Vtable for Rectangle

This illustration shows the vtable for Rectangle. Compare this definition with the [trait object and vtable figure](#).



Here is the assembly declaration for the vtable for Rectangle:

```
.L__unnamed_3:
; Destructor
.quad    core::ptr::drop_in_place<example::Rectangle<f64>>

; Size: 32 (leading space); Alignment: 8 (\b character)
.asciz   " \000\000\000\000\000\000\000\000\b\000\000\000\000\000\000"
```

Part E Strings, Dispatch, and Recursion

```
; Address of the area method for the Rectangle
.quad <example::Rectangle<T> as example::Shape>::area

; Address of the draw method for the Rectangle
.quad <example::Rectangle<T> as example::Draw>::draw
```

Destructor for Rectangle

The destructor for Rectangle returns.

```
core::ptr::drop_in_place<example::Rectangle<f64>>:

ret
```

Area Method for Rectangle

The area method for Rectangle is `<example::Rectangle<T> as example::Shape>::area`. The annotated assembly code for the area method follows:

```
<example::Rectangle<T> as example::Shape>::area:

movupd  xmm1, xmmword ptr [rdi + 8] ; Load bottom_right.x and
                                   ; bottom_right.y fields into xmm1

movsd   xmm0, qword ptr [rdi + 24] ; Load top_left.x field into xmm0

movhpd  xmm0, qword ptr [rdi]      ; Load top_left.y field into higher xmm0

subpd   xmm1, xmm0                ; Perform vector subtractions to compute
                                   ; the width and height

movapd  xmm0, xmm1                ; Copy the width and height into xmm0

unpckhpd xmm0, xmm1               ; Unpack width & height into xmm0

mulsd   xmm0, xmm1                ; Perform vector multiplication to
                                   ; compute the area

ret                                         ; Return the result in xmm0
```

Draw Method for Rectangle

The draw method for Rectangle is `<example::Rectangle<T> as example::Draw>::draw`. Here is the annotated assembly code for the draw method:

```
<example::Rectangle<T> as example::Draw>::draw:

sub     rsp, 56 ; Allocate space for the arguments on the stack

lea     rax, [rip + .L__unnamed_1] ; Load address of format string into rax

mov     qword ptr [rsp + 8], rax    ; Store address of format string on stack

mov     qword ptr [rsp + 16], 1     ; Store the number of arguments on stack
```

```

mov    qword ptr [rsp + 24], 0    ; Store address of 1st argument on stack

lea    rax, [rip + .L__unnamed_2] ; rax = address of the argument

mov    qword ptr [rsp + 40], rax  ; Store address of the argument on stack

mov    qword ptr [rsp + 48], 0    ; Store the address of the 2nd argument
                                      ; on the stack

lea    rdi, [rsp + 8]             ; Load address of the arguments into rdi

call   qword ptr [rip + std::io::stdio::_print@GOTPCREL] ; Call print

add    rsp, 56                   ; Remove local variables from the stack
ret

```



Experiment: Impact of Call Order on TCO; Dynamic Dispatch Inlined

Edit the code in [Compiler Explorer](https://godbolt.org/z/Px6sE9faY) (<https://godbolt.org/z/Px6sE9faY>) to perform the following experiments.

Reverse the Order of the area and draw Calls

Add the `draw_and_report_area_dynamic_switched` function that reverses the order of the area and draw calls as shown. You will see that the compiler cannot apply tail call optimization to the draw call as it is no longer the last function call in the function body. Compare the assembly code with that for the `draw_and_report_area_dynamic` function. What differences do you see?

```

pub fn draw_and_report_area_dynamic_switched(a: &dyn Draw<T = f64>) -> f64 {
    let area = a.area();
    a.draw();
    area
}

```

Function Inlining and Dynamic Dispatch Elimination

Look at the assembly code generated for the `draw_rectangle` function (included in [Compiler Explorer](#) link). You will see that the compiler has inlined the functions called by `draw_rectangle` and removed dynamic dispatch. The compiler can do this because the type of the trait object is known at compile time.

```

type Num = f64;
pub fn draw_rectangle(rectangle: Rectangle<Num>) -> (Num, Num) {
    draw_dynamic(&rectangle);
    let a = draw_and_report_area_dynamic(&rectangle);
    let b = draw_and_report_area_dynamic_box(Box::new(rectangle));
    (a, b)
}

```

Look at the Code and Vtable as 'Num' Changes

Change the `Num` type to the listed types and analyze the generated assembly.

```
type Num = f64;
```

f64	f32	i64	i32	i16	i8
☒	☐	☐	☐	☐	☐

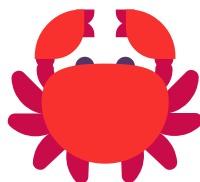
Keep an eye on the changes to the `vtable` as you change the type of `Num`. For example, here is the `vtable` for `Rectangle<i8>`.

```
.L__unnamed_3:
    .quad    core::ptr::drop_in_place<example::Rectangle<i8>>
    .asciz   "\004\000\000\000\000\000\000\000\000\001\000\000\000\000\000\000"
    .quad    <example::Rectangle<T> as example::Shape>::area
    .quad    <example::Rectangle<T> as example::Draw>::draw
```



Key Takeaways

- The Rust compiler uses Tail Call Optimization (TCO) to optimize away the last function call in a function body. This optimization eliminates creating a new stack frame and return address for the last (tail) function call in a call chain.
- Reordering functions to prepare the return value in the last called function might improve performance as tail call optimization will reduce the overhead of a function call.
- Trait objects are represented as a pair of pointers (aka fat pointer). The first pointer points to the data, and the second points to the `vtable`.
- The compiler typically uses two registers to pass a trait object's and its `vtable` addresses.
- The compiler stores the `Vtables` in the read-only data section of the binary. It saves the following information in the `vtable`:
 - Size and alignment of the concrete type.
 - The address of the destructor.
 - The addresses of the functions that implement the methods of the trait.
- When a trait object wrapped in a `Box` goes out of scope, the compiler frees the memory allocated for the trait object by calling the destructor and the `__rust_dealloc` function. It obtains the destructor's address and the concrete type's size from the `vtable`.
- The compiler can inline functions and remove dynamic dispatch when the type of the trait object is known at compile time.



Chapter 16

Optimizing Recursive Functions

This chapter will examine how the Rust compiler optimizes tail call recursive functions by mapping them to loops. Additionally, we will explore how the compiler eliminates the enum discriminator when it can infer it.

Tree Structure

We begin by defining a tree structure in Rust, characterized as an enum with two variants: `Leaf` and `Node`. The `Leaf` variant contains a value of type `T`, while the `Node` variant contains a value of type `T`, a left child of type `Box<Tree<T>>`, and a right child of type `Box<Tree<T>>`.

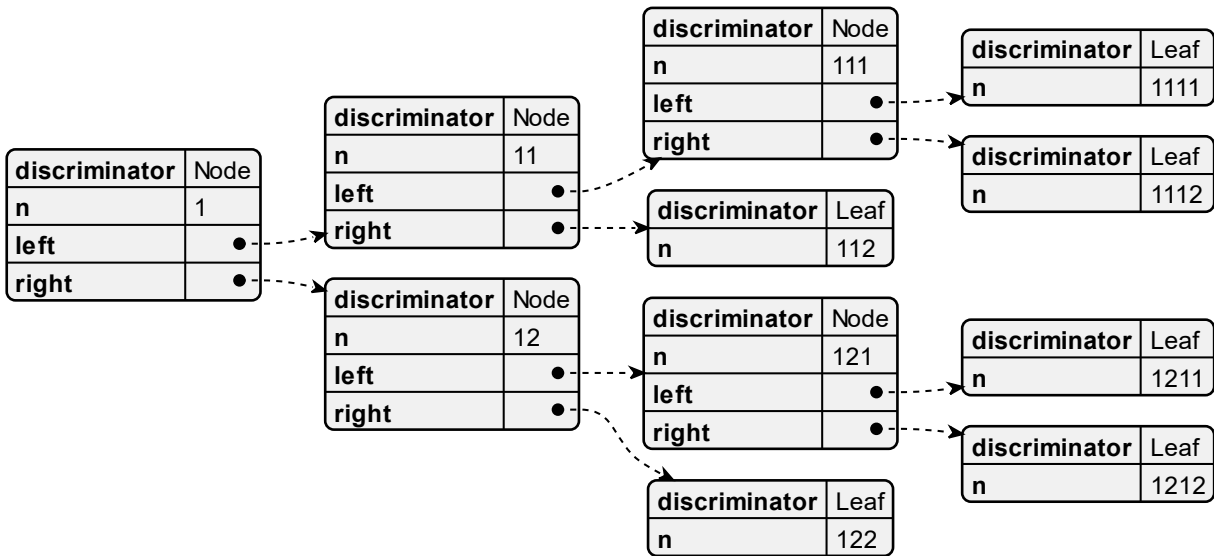
We also define a `sum` function that sums the values of all the nodes in the tree. The `sum` function is a recursive function that calls itself for the left and right subtrees of a `Node` variant. We will examine the code generated for the `sum` function and see how the Rust compiler optimizes some recursive calls into loops.

```
pub enum Tree<T> {
    Node(T, Box<Tree<T>>, Box<Tree<T>>),
    Leaf(T),
}
use Tree::{Leaf, Node};

pub fn sum(tree: &Tree<u64>) -> u64 {
    match tree {
        Leaf(n) => *n,
        Node(n, left, right) => *n + sum(left) + sum(right),
    }
}
```

Tree Representation in Memory

As discussed in the *“Deep Dive into Enums: Layout and Patterns”* chapter,” Rust usually includes a discriminator in the memory representation of an enum to keep track of the currently stored variant. Hence, the `Tree` enum should also contain a discriminator to track its stored variant. If a discriminator were present, the memory representation of the `Tree` object would look like the following diagram.



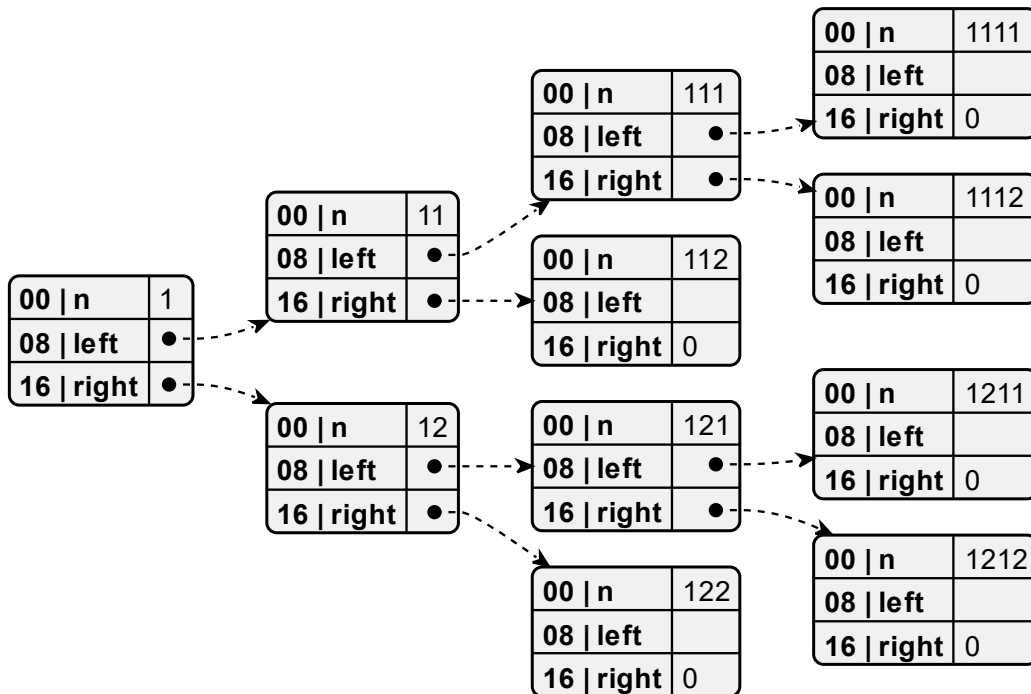
The generated tree structure is simpler than the preceding diagram. Rust optimizes the code by inferring the variant from the context and eliminating the need for a discriminator.

In the `Tree` enum, the `Node` variant's `right` field, a `Box<Tree<T>>`, acts as a pointer to another `Tree` object. The compiler uses this field's presence or absence to determine the `Tree` object's variant:

- A non-null `right` field indicates that the `Tree` object is a `Node`.
- A null `right` field signifies that the `Tree` object is a `Leaf`.

Thus, the `right` field functions as a discriminator and a pointer to the next `Tree` object.

The following diagram shows the simplified memory representation of the `Tree` object.



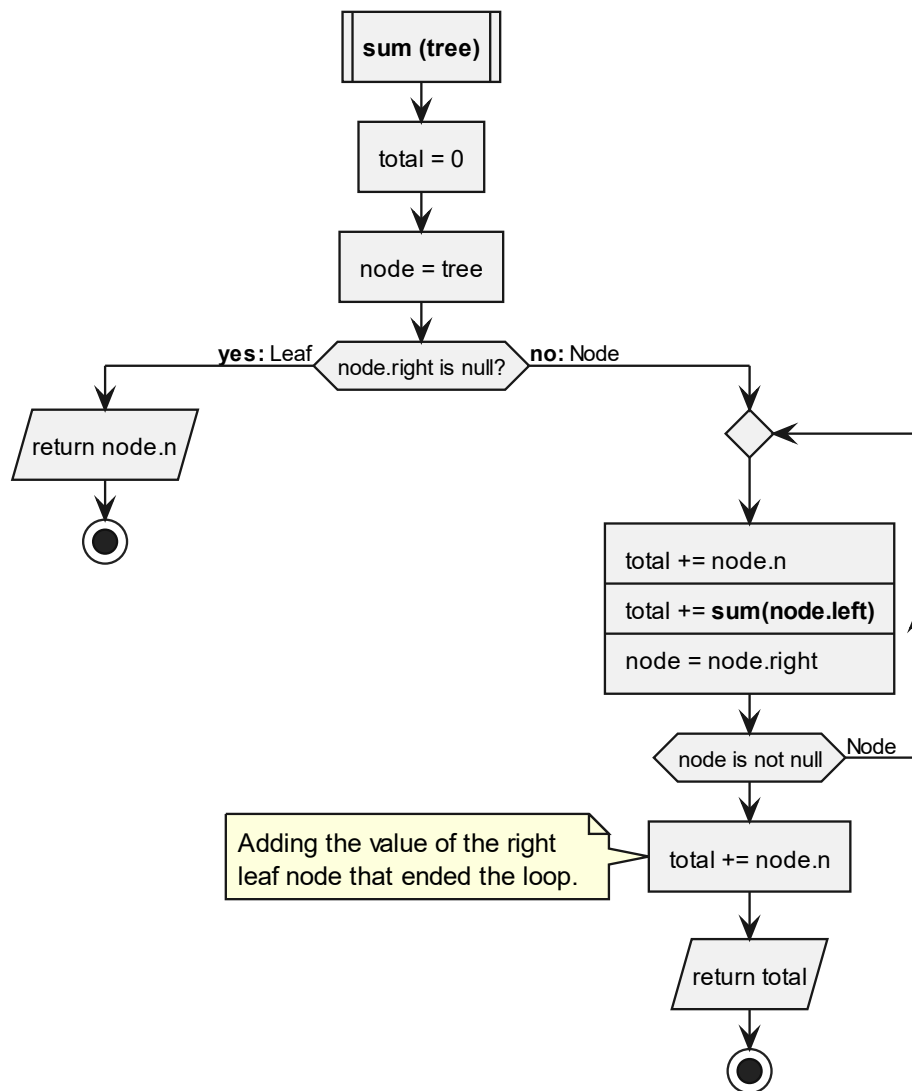
The preceding diagram also shows the byte offsets on the left side of each `Node` or `Leaf` object. These offsets will come in handy in understanding the generated assembly code.

Flowchart of Partially Tail Call Optimized sum Function

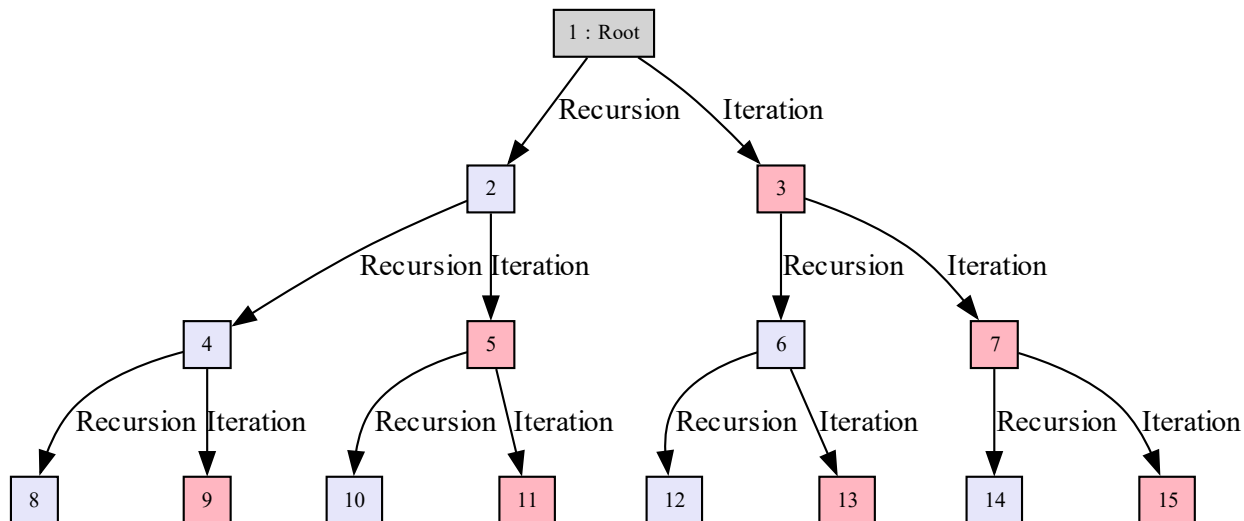
The following flowchart will help you understand the generated assembly code. The Rust compiler optimizes away the recursive call to the sum function for the `right` of the `Node` variant. The sum function is tail-recursive for the `sum (right)` call.

Converting the recursive call to a loop is a two-step process. Here is how the compiler thinks about this:

1. The function's last call, `sum(right)`, mirrors the structure of the calling function `sum(tree)`. The compiler can reduce overhead by eliminating the stack frame for this call.
2. The `sum(right)` call, targeted for elimination by the compiler, is a recursive call to the same function. The compiler can transform this tail call into a loop, enhancing performance and minimizing stack usage.



The following diagram illustrates the traversals from the preceding flowchart. Node visits on the left are recursive calls, while node visits on the right are tail call optimized into iteration.



Annotated Assembly Code for Transforming Recursive Tail Calls into Loops

Now, let us look at the generated assembly code. Review the memory representation of the Tree object and the flowchart of the sum function to understand the assembly code better.

- 🌳 Input: rdi: &Tree<u64>
- 📦 Output: rax: u64

example::sum:

```
push    r15                ; save callee-saved registers
push    r14
push    rbx
mov     r14, rdi            ; r14 = tree
```

🍀 Verify whether the present entry is a Leaf by examining the right field (offset 16) of the Node for a null value.

```
cmp     qword ptr [rdi + 16], 0 ; if tree is a leaf Node
je      .LBB0_1                ; then return the value
```

(🔄 🌳, ➡️ 🌳) Since the Tree entry's right field is not null, we infer that the entry represents a Node. Therefore, we must process this Node's left and right fields. To calculate the total sum, add the Node's value to the sums of its left and right subtrees and return this result.

```
xor     ebx, ebx            ; rbx = 0
mov     r15, qword ptr [rip + example::sum@GOTPCREL] ; r15 = sum
```

★ The compiler has converted the right-side recursive call to the sum function into a loop. The loop starts here and continues until the right field of the Node is a Leaf. The left-side Node visits are recursive.

🌀 🟢 ➡️ 🌳 **sum (right) Loop starts.**

.LBB0_3:

```

mov    rdi, qword ptr [r14 + 8] ; rdi = left
add    rbx, qword ptr [r14]      ; rbx += node.n

```

🏠 Recursive call for the left side of the Node.

```

call    r15                      ; call sum(left)

mov     r14, qword ptr [r14 + 16] ; r14 = right
add     rbx, rax                  ; rbx += sum(left)

cmp     qword ptr [r14 + 16], 0   ; check if right is a Leaf
jne     .LBB0_3                   ; if the right side is a Node, continue

```

🔄🔴🏠 **sum(right)** Loop back to the top.

```

jmp     .LBB0_4                   ; the right side is a Leaf - proceed to
                                   ; return after adding the value

```

🍀 Leaf Node: Return the value of the Leaf node.

```

.LBB0_1:

xor     ebx, ebx                  ; Set rbx to 0
                                   ; This way, the addition in the next
                                   ; instruction will add to 0.

```

🏠 Return the value from the function. Adding the final node .n is necessary to incorporate the n value corresponding to the right leaf Node that terminates the loop.

```

.LBB0_4:

add     rbx, qword ptr [r14]      ; rbx += node.n (right side Leaf)
mov     rax, rbx                  ; rax = rbx

pop     rbx                       ; restore callee-saved registers
pop     r14
pop     r15
ret

```



Experiment: TCO for factorial and fibonacci Functions

You can experiment with the code in [Compiler Explorer \(https://godbolt.org/z/zG46WvTjz\)](https://godbolt.org/z/zG46WvTjz). You can also add the following recursive functions to Compiler Explorer code editor to see how the compiler optimizes recursive calls in different scenarios.

- A version of the sum function that uses an accumulator to sum the values of all the nodes in the tree. The recursive call to `sum_acc` is a tail call. Compare this with the sum function we saw earlier.

```

pub fn sum_acc(tree: &Tree<u64>, acc: u64) -> u64 {
    match tree {
        Leaf(n) => *n + acc,
        Node(n, left, right) => sum_acc(right, sum_acc(left, *n)),
    }
}

```

Part E Strings, Dispatch, and Recursion

- The compiler optimizes the recursive `factorial` call into a loop.

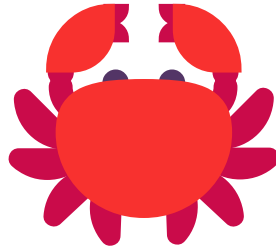
```
pub fn factorial(n: u64) -> u64 {
    if n == 0 {
        1
    } else {
        n * factorial(n - 1)
    }
}
```

- The compiler optimizes away one of the two recursive calls to the `fibonacci` function. The situation here is like the `sum` function we saw earlier. The compiler optimizes the recursive call to `fibonacci(n - 2)` to a loop but does not optimize the recursive call to `fibonacci(n - 1)` to a loop.

```
pub fn fibonacci(n: u32) -> u32 {
    if n == 0 {
        return 0;
    } else if n == 1 {
        return 1;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Key Takeaways

- The Rust compiler only converts recursive calls into loops when they are tail calls.
 - In the `sum` function, the compiler optimized the tail call `sum(right)` into a loop but did not optimize the `sum(left)` call.
 - For the `fibonacci` function, the compiler converted the `fibonacci(n - 2)` call into a loop, but it did not do the same for the `fibonacci(n - 1)` call.
 - The `factorial` function, being tail recursive, saw its `factorial(n - 1)` call eliminated, with the factorial calculation transformed into a loop.
- The Rust compiler eliminates enum discriminators when it can deduce the enum variant from the context. This elimination was evident in the `Tree` example; specifically, the `Node` variant's `right` field allowed the compiler to deduce the `Tree` object's variant.



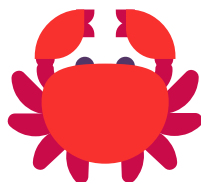
Part F

Closures and Async/Await



Rust Under the Hood Companion (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews



Chapter 17

Closures: From Environment to Execution

In this chapter, we will explore the inner workings of Rust closures by comparing the assembly code generated for closures returned as `impl Fn` and `Box<dyn Fn>`. We will investigate how Rust stores captured variables on the stack and the heap. Additionally, we will examine the implementation of dynamic dispatch for `Box<dyn Fn>` closures. To better understand `vtables`, we recommend reading the chapter *“Static vs. Dynamic Dispatch”*.

Rust Closures Traits - `Fn`, `FnMut`, and `FnOnce`

In Rust, closures serve as anonymous functions with the capacity to capture variables from their defining scope. You can utilize closures in multiple ways - passing them as arguments to functions, returning them from functions, or assigning them to variables.

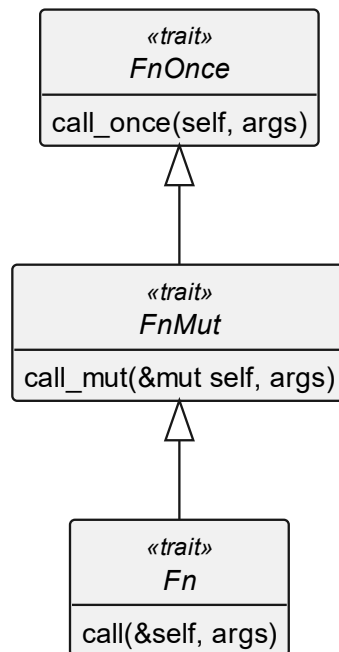
In terms of invocation, closures provide versatility—you can call them repeatedly or exclusively once. Additionally, closures demonstrate adaptable behavior in handling borrowed variables. They can borrow these immutably, ensuring no alteration to the variables or, mutably, allowing variable modifications. Furthermore, closures can assume ownership of captured variables.

Implementing the `Fn`, `FnMut`, and `FnOnce` traits achieves this broad range of capabilities. Here is a brief overview of these traits:

- The `Fn` trait applies to closures that take a borrowed version of the environment. It allows a closure to be called multiple times without altering the environment.
- The `FnMut` trait is for closures that mutably borrow the environment. These closures can be called multiple times, but each call must be sequenced.
- The `FnOnce` trait is for closures that take ownership of the environment. These closures can be called only once, as they consume the environment upon being called.

In this context, “environment” refers to the captured variables.

The following diagram shows `FnOnce` as a supertrait of `FnMut`, and `FnMut` as a supertrait of `Fn`. If a closure implements `Fn`, it also implements `FnMut` and `FnOnce`.



💡 Learn More: Closure Environment

In Rust, the closure environment refers to the scope in which the closure is defined and includes any variables from the scope that the closure captures or “closes over”.

When a closure captures a variable from its environment, it can do so in one of three ways:

1. **By reference (&T):** The closure takes an immutable borrow of the variable, allowing it to utilize its value without any modifications. The original variable remains accessible and usable.
2. **By mutable reference (&mut T):** The closure takes a mutable borrow of the variable, enabling it to alter its value. The original variable is no longer usable for modifications within the same scope as the closure.
3. **By ownership (T):** The closure takes ownership of the variable. For types implementing the Copy trait, the value remains available after being consumed. For complex types, ownership of the data itself is transferred. Using move might be necessary to ensure the closure can access the captured variable even after its original scope ends. This approach prevents dangling references and upholds memory safety.

These mechanisms determine how a closure interacts with the variables from its environment, shaping its behavior and usage.

💡 Learn More: Closures and Move Semantics in Rust

Rust’s closure traits, `Fn`, `FnMut`, and `FnOnce`, come with distinct behaviors and use cases, especially when considering concurrency and ownership. The `Fn` trait allows a closure to be called simultaneously from multiple threads, making it suitable for concurrent applications, as demonstrated by its use in Rayon’s `ParallelIterator` interface. In contrast, `FnMut` closures, used in the standard `Iterator` interface, are not inherently thread-safe and are designed for single-threaded contexts.

Ownership in closures is another critical concept. While all three closure types can capture and own environment variables upon creation, `FnOnce` is unique in its ability to consume those variables within the closure body. This property stems from its design, which permits only a single call.

The concept of move semantics in Rust plays a vital role with closures. This mechanism ensures the original variable becomes inaccessible after the move, preventing dangling references and upholding Rust’s

memory safety. Understanding this is critical for working with closures that capture variables by value, especially when maintaining Rust's strict safety guarantees.

Stack and Heap-Allocated Closures

Before we proceed, let us explore two common methods for working with closures in Rust:

- **impl Fn:** We use this notation when a function returns a closure without specifying its exact type. The compiler infers that the closure implements the Fn trait. While `impl Fn` does not inherently specify where to store the closure (stack or heap), we typically store closures typed this way on the stack unless explicitly boxed.
- **Box<dyn Fn>:** This denotes a heap-allocated closure that employs dynamic dispatch (indicated by `dyn`). This method proves useful when you cannot statically determine the exact type of a closure or when the type might vary. Boxing a closure ensures its storage on the heap and facilitates the return of closures from functions or their storage in data structures without precise type knowledge.

Here is a basic example illustrating the difference between `impl Fn` and `Box<dyn Fn>` in Rust:

```
// A function that returns a closure, typically stored on the stack
pub fn returns_impl_fn() -> impl Fn(i32) -> i32 {
    let capture_var = 10;
    move |x| x + capture_var
}

// A function that returns a boxed closure stored on the heap
pub fn returns_dyn_fn() -> Box<dyn Fn(i32) -> i32> {
    let capture_var = 20;
    Box::new(move |x| x + capture_var)
}
```

Now that we have a basic understanding of the two ways of working with closures, let us examine their implementation in assembly.

Return an impl Fn from a Function

The following example defines the function `make_quadratic` that takes three `f64` parameters: `a`, `b`, and `c`. It returns a closure that implements the `Fn(f64) -> f64` trait. The closure captures the variables `a`, `b`, and `c` from the surrounding scope. It implements the logic of a quadratic equation with input `x` by calculating `a*x*x + b*x + c` and returning the result.

```
pub fn make_quadratic(a: f64, b: f64, c: f64) -> impl Fn(f64) -> f64 {
    move |x| a*x*x + b*x + c
}
```



Learn More: move Keyword

Note that the `move` keyword before the returned closure signals the compiler to move `a`, `b`, and `c` into the closure environment. Without the `move` keyword, the closure will immutably borrow variables `a`, `b`, and `c`. However, it is not feasible to immutably borrow the variables `a`, `b`, and `c` because the closure is returned from the function, causing these variables to be dropped when the function returns.

Assembly Code for `make_quadratic`

The assembly generated for the `make_quadratic` function demonstrates how it prepares the closure environment, capturing the variables `a`, `b`, and `c`.

The diagram below illustrates the memory layout of this closure environment, with byte offsets on the left. When calling `make_quadratic`, the caller uses the `rdi` register to specify the closure's save location. The function stores the closure environment at this address and returns it in the `rax` register.

00 a	5.0
08 b	4.0
16 c	3.0

Input:

- `xmm0`: `a`
- `xmm1`: `b`
- `xmm2`: `c`
- `rdi`: Address to store the closure

Output:

- `rax`: Address of the closure

```
example::make_quadratic:
mov     rax, rdi                ; rax = Address of the closure
movsd   qword ptr [rdi], xmm0   ; closure.a = a
movsd   qword ptr [rdi + 8], xmm1 ; closure.b = b
movsd   qword ptr [rdi + 16], xmm2 ; closure.c = c
ret                                           ; Return address of closure in rax
```

Note that the assembly output only shows the return of the environment. When defining a closure, the compiler automatically implements the appropriate trait(s) for the closure's unique type. This generation includes the `call` method, executing the closure's function logic. The auto-generated `call` method accesses the closure's captured environment and uses this environment to execute the function logic.

Calling the Closure

The following example calls the closure returned by `make_quadratic`.

```
pub fn call_make_quadratic(x: f64) -> f64 {
    let quad_fn = make_quadratic(5.0, 4.0, 3.0);
    quad_fn(x)
}
```

Assembly Code for `call_make_quadratic`

The following assembly, generated for the `call_make_quadratic` function, does not include a call to the closure as the compiler has inlined it.

Input: `xmm0`: `x`

Output: `xmm0`: Result of the quadratic function.

```
example::call_make_quadratic:
```

```
movsd    xmm1, qword ptr [rip + .LCPI1_0] ; xmm1 = 5.0
mulsd    xmm1, xmm0                      ; xmm1 = 5.0 * x
mulsd    xmm1, xmm0                      ; xmm1 = 5.0 * x * x
mulsd    xmm0, qword ptr [rip + .LCPI1_1] ; xmm0 = 4.0 * x
addsd    xmm0, xmm1                      ; xmm0 = 5.0 * x * x + 4.0 * x
addsd    xmm0, qword ptr [rip + .LCPI1_2] ; xmm0 = 5.0 * x * x + 4.0 * x + 3.0
ret                                             ; Return the result in xmm0
```

The following assembly directives define the constants used in the function `call_make_quadratic`.

```
.LCPI1_0:
    .quad    0x4014000000000000 ; 5.0_f64
.LCPI1_1:
    .quad    0x4010000000000000 ; 4.0_f64
.LCPI1_2:
    .quad    0x4008000000000000 ; 3.0_f64
```

Returning a Boxed Closure `Box<dyn Fn>` from a Function

Let us look at an example where the closure returns as a `Box<dyn Fn>`. In this scenario, the code allocates the closure on the heap and returns a `Box`.

The function `make_quadratic_box` takes three `f64` parameters: `a`, `b`, and `c`. The function returns a `Box<dyn Fn(f64) -> f64>` that implements the logic of a quadratic equation with input `x` by calculating $a*x*x + b*x + c$ and returning the result.

```
pub fn make_quadratic_box(a: f64, b: f64, c: f64) -> Box<dyn Fn(f64) -> f64> {
    Box::new(make_quadratic(a, b, c))
}
```

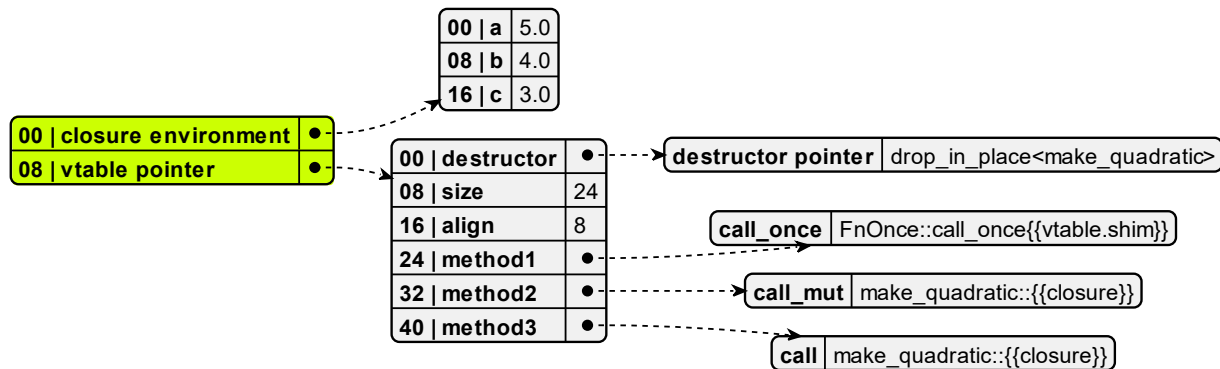
Before we look at the assembly code, let us look at the memory layout of the closure environment.

Memory Layout of a `Box<dyn Fn>` Closure

`Box<dyn Fn>` is a fat pointer that is a tuple of the closure environment and the `vtable` pointer for the type. The closure environment contains the captured variables `a`, `b`, and `c`. The `vtable` pointer is a pointer to the `vtable` for the trait `Fn(f64) -> f64`. The `vtable` contains the function pointers for the trait methods. In this case, the `vtable` contains the function pointer for the method `call` defined by the `Fn` trait. The `vtable` also contains the `call_mut` and `call_once` function pointers. The `call_mut` and `call_once` functions are the same as the `call` function.

The `vtable` also includes the destructor function pointer. When dropping the `Box`, the code calls this destructor function. Furthermore, the `vtable` stores the size and alignment of the closure environment necessary for dropping the `Box`.

The following diagram shows the memory layout of the closure environment and the `vtable` pointer.



Assembly Code for Returning a Box<dyn Fn> Closure

First, the generated code stores the captured fields on the stack. It allocates heap memory by calling the `__rust_alloc` function. If the memory allocation succeeds, it saves the captured fields in the closure environment. Finally, the system returns the address of the closure environment on the heap in the `rax` register. The `rdx` register holds the vtable pointer for the type. The system uses the tuple `(rax, rdx)` to return the `Box<dyn Fn>`.

Note that if the memory allocation for the closure fails, the function will throw an exception.

Annotated assembly code for the function `make_quadratic_box` follows.

- **Input:**
 - `xmm0`: `a`
 - `xmm1`: `b`
 - `xmm2`: `c`
- **Output:**
 - `rax`: Address of the closure environment
 - `rdx`: Vtable pointer for the trait `Fn(f64) -> f64`

```
example::make_quadratic_box:
sub     rsp, 24                ; Allocate stack space for the
                                ; captured variables
movsd   qword ptr [rsp + 16], xmm2 ; capture.c = c
movsd   qword ptr [rsp + 8], xmm1  ; capture.b = b
movsd   qword ptr [rsp], xmm0      ; capture.a = a
```

Allocate space for the closure environment and return the result in the `rax` register.

```
mov     edi, 24                ; Size of the closure
mov     esi, 8                 ; Alignment of the closure
call    qword ptr [rip + __rust_alloc@GOTPCREL]
```

Check if allocation was successful. Jump to the error handler if not.

```
test    rax, rax                ; Check if allocation was successful
je      .LBB4_1                 ; If not, call the error handler.
```

Closure environment heap allocation was successful. Save the captured fields in the closure environment.

```

movsd    xmm0, qword ptr [rsp]           ; xmm0 = capture.a
movsd    qword ptr [rax], xmm0          ; environment.a = a

movsd    xmm0, qword ptr [rsp + 8]       ; xmm0 = capture.b
movsd    qword ptr [rax + 8], xmm0       ; environment.b = b

movsd    xmm0, qword ptr [rsp + 16]      ; xmm0 = capture.c
movsd    qword ptr [rax + 16], xmm0      ; environment.c = c

```

← BACK Return the address of the closure environment in rax and the vtable pointer in rdx. Copy the address of the vtable. Note that rax already points to the start of the closure environment.

```

lea      rdx, [rip + .L__unnamed_1]      ; Address of the vtable
add      rsp, 24                         ; Deallocate space for the closure.
; Return the address of the closure in rax and the vtable in rdx.
ret

```

✗ Memory allocation has failed. Call the error handler and throw an exception.

```

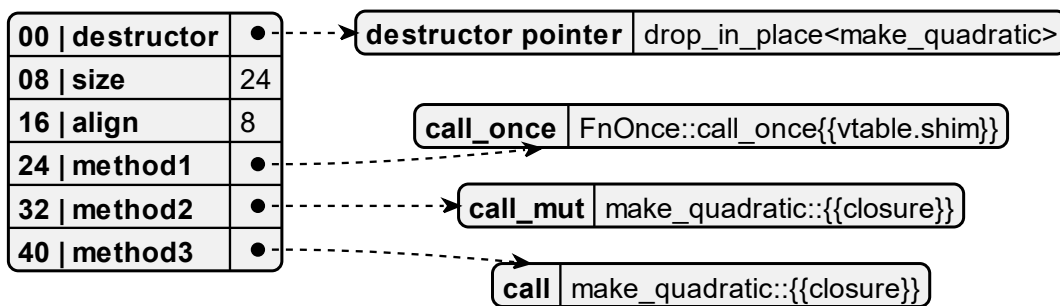
.LBB4_1:

mov      edi, 24                         ; Size of the closure.
mov      esi, 8                         ; Alignment of the closure.
; 🦴 Call the error handler.
call    qword ptr [rip + alloc::alloc::handle_alloc_error@GOTPCREL]
; 🦴 Throw an invalid instruction exception.
ud2

```

Vtable for Returned Box<dyn Fn> Closure

We have already seen the [memory layout of a vtable](#). The vtable is reproduced here for convenience.



The vtable for the returned Box<dyn Fn> follows. The assembly directives define the vtable in the .rodata (read-only data) section. Compare the assembly declaration of the vtable with the preceding figure.

```

.L__unnamed_1:

; Call when dropping a closure environment.
.quad    core::ptr::drop_in_place<example::make_quadratic::{{closure}}>

; The closure's size is 24 bytes, and the alignment is 8 bytes.
.asciz   "\030\000\000\000\000\000\000\000\000\b\000\000\000\000\000\000"

```

Part F Closures and Async/Await

```

; Call function for FnOnce (call_once).
.quad    core::ops::function::FnOnce::call_once{{vtable.shim}}

; Call function for FnMut (call_mut).
.quad    example::make_quadratic::{{closure}}

; Call function for Fn (call).
.quad    example::make_quadratic::{{closure}}

```

Destructor for the Closure

When the Box is dropped, the code calls the closure's destructor. In this case, the destructor takes no action.

```

core::ptr::drop_in_place<example::make_quadratic::{{closure}}>:
    ret

```

call Method for the Closure

The generated code invokes its call method when invoking the closure. The vtable for the closure maps the call and call_mut methods to the make_quadratic::{{closure}} function.

Input:

- rdi: Pointer to the closure
- xmm0: x

Output:

- xmm0: Result of the quadratic function

```

example::make_quadratic::{{closure}}:
movupd  xmm1, xmmword ptr [rdi]      ; xmm1 = closure.a, closure.b

movapd  xmm2, xmm0                    ; xmm2 = x
unpcklpd xmm2, xmm0                  ; xmm2 = x, x

mulpd   xmm2, xmm1                    ; xmm2 = closure.a * x, closure.b * x
mulsd   xmm0, xmm2                    ; xmm0 = closure.a * x * x

unpckhpd xmm2, xmm2                  ; xmm2 = closure.b * x, closure.b * x

addsd   xmm0, xmm2                    ; xmm0 = closure.a * x * x
                                           ;      + closure.b * x

addsd   xmm0, qword ptr [rdi + 16]    ; xmm0 = closure.a * x * x
                                           ;      + closure.b * x + closure.c

ret                                         ; Return the result in xmm0

```

call_once Shim for the Closure

The compiler also generates a shim for the call_once method, as the call_once method is part of the FnOnce trait. The generated code is identical to the [call method for the closure](#), but the compiler generates another copy.

Passing a Dynamic Closure dyn Fn to a Function

Let us look at an example where a function receives dyn Fn as a parameter. We will look at the following cases:

- Passing a &dyn Fn as an Argument
- Passing a &Box<dyn Fn> as an Argument
- Passing a Box<dyn Fn> as an Argument

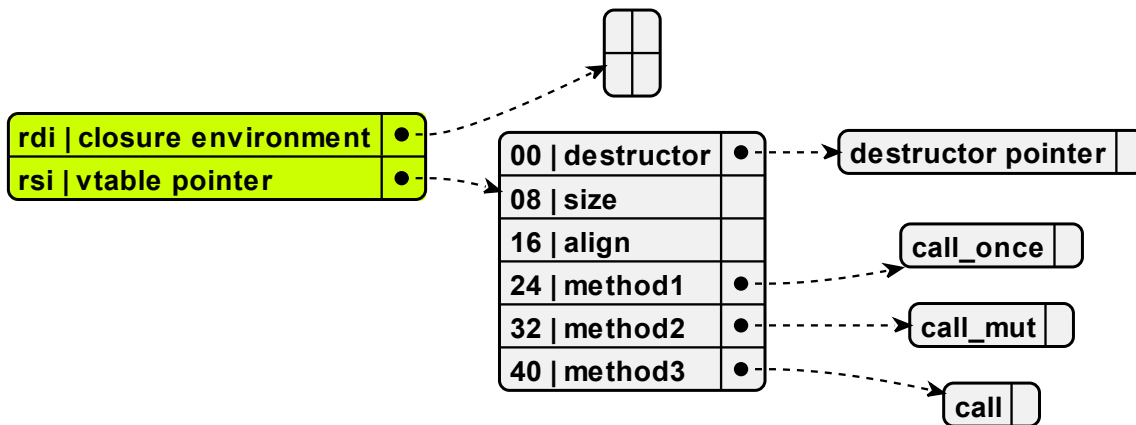
It is essential to highlight that the functions receiving the argument do not know the specific closure type in all these instances. They only understand that the argument adheres to the dyn Fn trait.

Passing a &dyn Fn as an Argument

```
pub fn call_dyn_ref_closure(closure: &dyn Fn(f64) -> f64, x: f64) -> f64 {
    closure(x)
}
```

In this scenario, the code passes the closure as a fat pointer. It stores a pointer to the closure environment in rdi and the vtable pointer in rsi. The x argument passes in xmm0.

The following diagram illustrates how the function receives the fat pointer using the rdi and rsi registers.



The generated code takes the following steps:

1. Load the closure's call function address from the vtable. The call function is at an offset of 40 from the start of the vtable. The address of the vtable is obtained from rsi.
2. Perform a tail call to the closure function—the jmp instruction tricks the closure into thinking the caller of call_dyn_ref_closure directly called it. When the closure function returns, it will return to the caller of call_dyn_ref_closure. A tail call works here as the parameters the closure function needs are already in the correct registers, rdi, and xmm0. The closure's return register xmm0 also serves as the return register for the function call_dyn_ref_closure function.

The annotated assembly code for the function call_dyn_ref_closure follows:

12 Input:

- rdi: Pointer to the closure
- rsi: Pointer to the vtable
- xmm0: x

Part F Closures and Async/Await

Output:

- `xmm0`: Result from the closure function

```
example::call_dyn_ref_closure:
mov     rax, qword ptr [rsi + 40] ; Load the address of the call function
                                           ; from the vtable for the closure

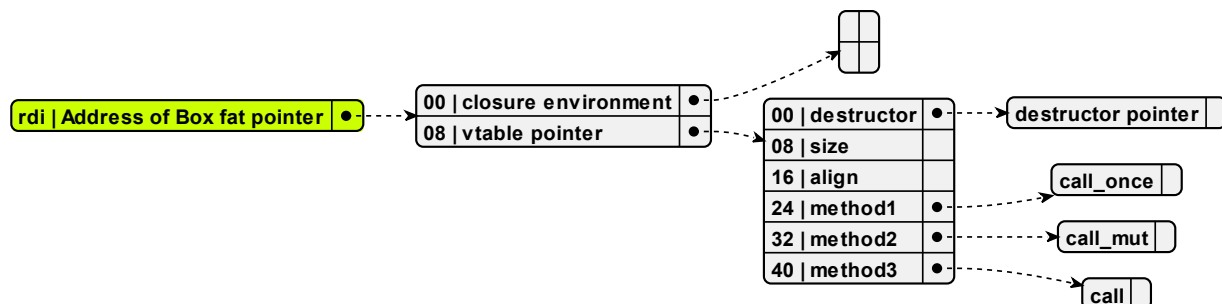
jmp     rax                        ; Jump to the closure function
                                           ; (i.e., call the closure function)
```

Passing a `&Box<dyn Fn>` as an Argument

The following function receives a reference to a `Box<dyn Fn>` as a parameter. Here, `rdi` contains the address of the `Box<dyn Fn>` fat pointer.

```
pub fn call_box_ref_closure(closure: &Box<dyn Fn(f64) -> f64>, x: f64)
    -> f64 {
    closure(x)
}
```

The following diagram illustrates how the function receives a pointer to the fat pointer via the `rdi` register. In addition, the function receives `x` in the `xmm0` register.



Let us examine the assembly code for the `call_box_ref_closure` function. As the code passes a reference to `Box` in `rdi`, it first loads the closure address and the vtable. Then, it prepares the `rdi` register for the closure function call. Finally, the function makes a tail call to the closure function.

Input:

- `rdi`: Address of the `Box` fat pointer
- `xmm0`: `x`

Output:

- `xmm0`: Result from the closure function

```
example::call_box_ref_closure:
mov     rax, qword ptr [rdi]           ; Load address of the closure environment
                                           ; from the address of the Box fat pointer.

mov     rcx, qword ptr [rdi + 8]       ; Load the address of the vtable

mov     rcx, qword ptr [rcx + 40]      ; Closure's call function is located at
                                           ; offset of 40 in the vtable
```



```

mov     rdi, rax           ; Move closure environment pointer to rdi
                               ; as the closure function expects it in rdi

jmp     rcx               ; Tail call to the closure function
    
```

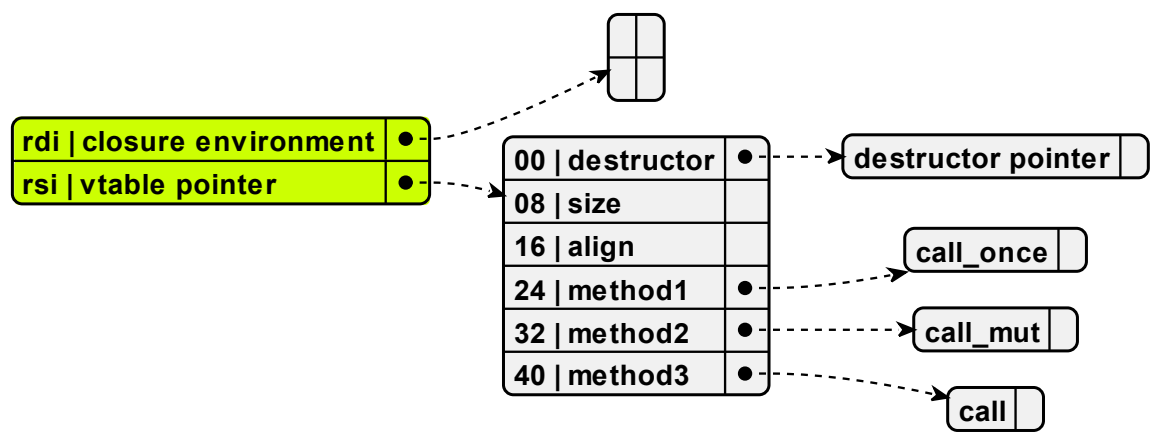
Passing a Box<dyn Fn> as an Argument

Now, we will look at the implications of passing a Box<dyn Fn> as an argument. The call_box_closure function presented here takes ownership of the Box<dyn Fn>. Consequently, it is also responsible for freeing the environment when the function returns.

```

pub fn call_box_closure(closure: Box<dyn Fn(f64) -> f64>, x: f64) -> f64 {
    closure(x)
}
    
```

The caller passes the Box as a fat pointer, storing the closure environment in rdi and the vtable in rsi. It also passes the x argument in xmm0.



The assembly code for the function call_box_closure follows. Key points to note are:

- Before the function returns, it must drop the Box. The vtable stores all the information necessary for this, including the destructor for the closure environment and the size and alignment of the closure environment.
- The function saves many registers to the stack due to the parameter passing requirements of __rust_dealloc.
- The function does not make a tail call to the closure function because it must drop the Box before returning from call_box_closure.

12 34 Input:

- rdi: Pointer to the closure environment
- rsi: Pointer to the vtable
- xmm0: x

12 34 Output:

- xmm0: Result from the closure function

```

example::call_box_closure:
push    r15               ; Save callee registers to the stack
push    r14
push    rbx
    
```

Part F Closures and Async/Await

```

sub     rsp, 32                ; Allocating 32 bytes on stack for locals

mov     rbx, rsi               ; Save the vtable pointer to rbx
mov     r14, rdi               ; Save closure environment pointer to r14

mov     qword ptr [rsp + 16], rdi ; Saving closure pointer to the stack
mov     qword ptr [rsp + 24], rsi ; Saving vtable pointer to the stack

```

- Load the address of the closure's call function from the vtable.
- Call the closure function.
- Save the results as a local variable on the stack in preparation for the Box drop call.

```

call     qword ptr [rsi + 40]    ; Obtain the address of the call function
                                   ; from the vtable and call the closure
                                   ; function

movsd    qword ptr [rsp + 8], xmm0 ; Storing the result of the function call
                                   ; in a local variable on the stack

```

🔪 This function owns the boxed closure and drops the closure before returning.

```

mov     rdi, r14                ; Move closure pointer to rdi for next call
call     qword ptr [rbx]         ; Obtain the address of the destructor from
                                   ; the vtable and call it.

mov     rsi, qword ptr [rbx + 8] ; Obtain the size of the closure
                                   ; environment from the vtable

test     rsi, rsi               ; Checking if the size is non-zero
je       .LBB8_4                ; Jumping to .LBB8_4 if it is zero
                                   ; (no need to deallocate)

mov     rdx, qword ptr [rbx + 16] ; Obtain the alignment from vtable
mov     rdi, r14                ; Moving closure pointer to rdi
                                   ; for the deallocation call

call     qword ptr [rip + __rust_dealloc@GOTPCREL] ; Free closure environment

```

← END Load the result in xmm0 and return from the function.

```

.LBB8_4:

movsd    xmm0, qword ptr [rsp + 8] ; Loading the result of the closure call
                                   ; back to xmm0 for return

add     rsp, 32                ; Restoring the stack pointer effectively
                                   ; deallocating local variables

pop     rbx                    ; Restoring saved registers
pop     r14
pop     r15
ret                                     ; Returning from the function

```

Experiment: Return `impl Fn` and `Box<dyn Fn>` with No Capture

Use [Compiler Explorer](https://godbolt.org/z/Grhao7qs3) (<https://godbolt.org/z/Grhao7qs3>) to experiment with the code in this chapter.

Let us also look at some examples where the closure does not capture anything from the enclosing scope. In these cases, the compiler does not need to allocate any memory for the closure environment.

- [Return `impl Fn` with No Capture](#)
- [Return `Box<dyn Fn>` with No Capture](#)

Return `impl Fn` with No Capture

Add the following code in the editor window and examine the generated assembly code.

```
pub fn make_quadratic_no_capture() -> impl Fn(f64) -> f64 {
    |x| 42.0*x*x + 84.0*x + 0.0
}
```

You will see that the compiler returns from `make_quadratic_no_capture`. There is no environment to capture and return. Here is the generated assembly code:

```
example::make_quadratic_no_capture:
ret                                ; Return from the function
```

Return `Box<dyn Fn>` with No Capture

Add the following code in the editor window and examine the generated assembly code. This example is like the previous example except that it returns the closure as a `Box<dyn Fn>`. There is no environment to capture and return.

```
pub fn make_quadratic_box_no_capture() -> Box<dyn Fn(f64) -> f64> {
    Box::new(make_quadratic_no_capture())
}
```

The `make_quadratic_box_no_capture` function returns a fat pointer in `eax` and `edx`, with `eax` holding the address of the closure environment and `edx` containing the `vtable` address. In this case, the function does not need to save any closure environment to `eax`, so the compiler stores a 1 for the closure environment pointer.

```
example::make_quadratic_box_no_capture:
    lea     rdx, [rip + .L__unnamed_2]    ; Address of the vtable
    mov     eax, 1                        ; No closure environment to store
    ret
```

The `vtable` for the closure follows. Note that the size of the closure environment is set to 0 as the closure does not capture anything from its enclosing scope.

```
.L__unnamed_2:
; Destructor for the closure environment.
.quad core::ptr::drop_in_place<example::make_quadratic::{{closure}}>

; Size of the closure environment is 0 bytes, and alignment is 1 byte.
.asciz "\000\000\000\000\000\000\000\000\000\001\000\000\000\000\000"
```

Part F Closures and Async/Await

```

; call_once: Call function for FnOnce.
.quad    core::ops::function::FnOnce::call_once{{vtable.shim}}

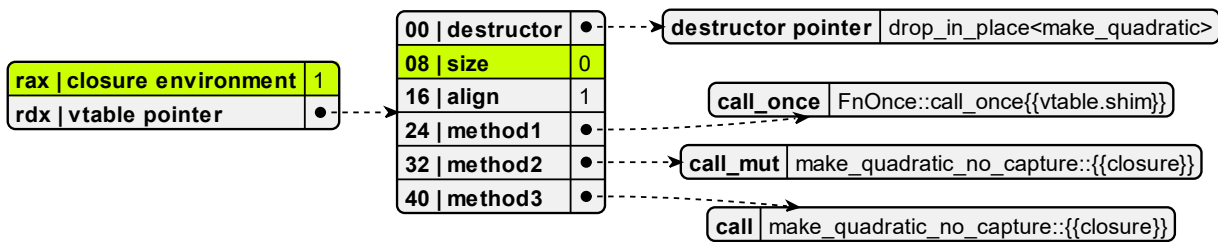
; call_mut: Call function for FnMut.
.quad    example::make_quadratic_no_capture::{{closure}}

; call: Call function for Fn.
.quad    example::make_quadratic_no_capture::{{closure}}

```

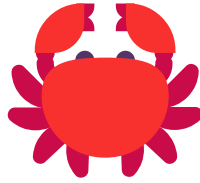
The returned result from the function follows:

- The `rax` register contains 1 as no closure environment is allocated in this case.
- The `rdx` register contains the address of the `vtable`. The size of the closure environment is set to 0 to reflect that there is no environment to capture.



Key Takeaways

- The Rust compiler captures and stores the environment in a closure environment struct.
- Closures returned as `impl Fn` typically store their environments on the stack.
- The compiler may completely inline a closure. In such cases, it does not store the closure environment.
- If the function returns a closure as `Box<dyn Fn>`, it stores the closure environment on the heap and returns a fat pointer to the caller. This fat pointer includes the address of the closure environment and the vtable address.
- For closures in the form of `Box<dyn Fn>`, passing a `&dyn Fn` as an argument to a function is more efficient. Passing `Box<dyn Fn>` is less efficient since the compiler must handle the dropping of the `Box`.
- The `vtable` includes the destructor for the closure environment, the size and alignment of the closure environment, and the closure's call method.
- Prefer returning `impl Fn` over `Box<dyn Fn>` as it stores the closure environment on the stack and eliminates the need for heap memory allocation. In many cases, `impl Fn` is more efficient since the compiler can inline the closure.



Chapter 18

Async/Await: Internal State Machine Dynamics

This chapter digs into the inner workings of Rust’s asynchronous functions, exploring the mechanics that enable writing non-blocking, concurrent code with simple and intuitive syntax. We will understand how async functions transform into state machines and how this code translates into assembly language.

Async Functions in Rust

Rust’s async functions rely heavily on closures, special functions capturing variables from their enclosing scope. These closures are crucial for async functions, allowing them to maintain state between invocations. For a deeper understanding, refer to the chapter *“Closures: From Environment to Execution”*, which explains the fundamental workings of closures in Rust.

Asynchronous programming in Rust primarily uses futures and the `async/await` syntax, simplifying the writing of non-blocking, concurrent code. Futures are abstractions for operations that may not have finished executing, representing potentially ongoing computations. The `Future` trait in Rust describes these asynchronous computations. A future holds a value that may not be ready yet.

The `async/await` construct lets code pause. It hands control back to the runtime. Later, the code resumes without blocking the thread. This approach allows writing code that appears synchronous while the compiler automatically transforms it into asynchronous code.

💡 Learn More: Single Threaded Async Executor

In Rust, async functions effectively act as syntactic sugar over futures, representing computations that might not yet be complete. However, futures and `async/await` functions do not perform any computation independently; they require an environment for execution provided by the async executor. An async executor in Rust takes charge of polling futures to completion, managing their state, and overseeing task scheduling and execution.

A single-threaded async executor handles the execution of tasks in an asynchronous context using only one thread. It multiplexes multiple tasks onto the same thread, switching between them as tasks voluntarily suspend their execution, often due to waiting for an I/O operation or a timer.

Part F Closures and Async/Await

Internally, a single-threaded executor operates with an event loop and a task queue. When scheduled for execution, futures enter the task queue. The executor then launches into the event loop, cycling through futures from the queue and polling them. If a future is not ready, typically awaiting the completion of an I/O operation, it hands control back to the executor, allowing the executor to proceed to the next task. Once a future is ready, the executor resolves it, enabling the caller of the async function to access its result.

💡 Learn More: Simple Async Local Executor

We will use the [simple async local executor](https://github.com/enlightware/simple-async-local-executor) in our study of the async internals. The following table contains links to the executor's source code and the example we will use in this and the following two chapters.

GitHub Link	Description
https://github.com/enlightware/simple-async-local-executor	enlightware's single-threaded polling executor.
https://github.com/eventhelix/simple-async-local-executor	eventhelix's fork of enlightenment's single-threaded polling executor.
https://github.com/eventhelix/simple-async-local-executor/blob/main/examples/game-units.rs	game-units.rs example from eventhelix's fork of the single-threaded polling executor.

The code in the preceding links is dual licensed under the MIT and Apache 2.0 licenses.

Async Function goto: Await Reaches the Target Position

We will use the following code to understand Rust's async machinery.

```
#[derive(Default)]
struct Unit {
    /// The 1-D position of the unit. In a real game, it would be 2D or 3D.
    pub pos: i32,
}
type UnitRef = Rc<RefCell<Unit>>;

/// A future that will move the unit towards `target_pos` at each step,
/// and complete when the unit has reached that position.
struct UnitGotoFuture {
    unit: UnitRef,
    target_pos: i32,
}

impl Future for UnitGotoFuture {
    type Output = ();
    fn poll(self: Pin<&mut Self>, _cx: &mut Context) -> Poll<Self::Output> {
        let unit_pos = self.unit.borrow().pos;
        if unit_pos == self.target_pos {
            Poll::Ready(())
        } else {
            self.unit.borrow_mut().pos
                += (self.target_pos - unit_pos).signum();
            Poll::Pending
        }
    }
}
```

```

/// Helper async function to write unit behavior nicely
async fn goto(unit: UnitRef, pos: i32) {
    UnitGotoFuture {
        unit,
        target_pos: pos,
    }
    .await;
}

```

As previously discussed, a future embodies a value that might still be under computation. In programming models, this concept handles tasks scheduled for future completion. It acts as a placeholder for a value that will eventually become available.

Async support in Rust allows you to write asynchronous code that can “wait” for these futures to complete without blocking the rest of your program.

Here is a breakdown of the code:

Item	Description
<code>struct Unit</code>	The struct represents a simple model for a game character or object that has a position in 1-D space. It saves the position in the <code>pos</code> field.
<code>type UnitRef</code>	A type alias for a reference-counted (Rc) smart pointer to a <code>Unit</code> that supports mutation using <code>RefCell</code> . This setup allows for multiple ownership (Rc) and the ability to mutate the value even when there are numerous owners (<code>RefCell</code>).
<code>struct UnitGotoFuture</code>	A struct responsible for moving a unit to a specific position. It moves the unit one step closer to the target position with each poll. The struct includes the unit and the target position.
<code>impl Future for UnitGotoFuture</code>	Defines how <code>UnitGotoFuture</code> behaves. The executor should repeatedly call the <code>poll</code> method to check if the future’s value is ready. If the unit’s position has reached the target position, it returns <code>Poll::Ready(())</code> . If the unit has not reached the target, it moves one step closer and returns <code>Poll::Pending</code> , indicating that the future is not yet ready.
<code>goto</code> function	An asynchronous function that moves a given unit to a given position. It creates a <code>UnitGotoFuture</code> with the given unit and position and then <code>.awaits</code> on it.

The `.await` keyword enables the code to “wait” for the future to complete. This does not block the whole program, only this async function. Other parts of your program can continue running while `UnitGotoFuture` is still not ready. The `goto` function pauses until `UnitGotoFuture` is ready.

When you call `goto(unit, pos).await`, the execution of the `goto` function pauses at the `.await` until `UnitGotoFuture` is complete. This pause does not affect the rest of the program because it runs in an async context.

The advantage of this design is that you can have many units all moving simultaneously (in the larger context of your program), even though each `goto` operation behaves like a simple, linear sequence of steps.

Desugaring the Async Example into a State Machine

Before exploring the assembly implementation of the async function `goto`, let us examine equivalent non-async Rust code that could achieve the same functionality. This code will help us understand the state machine the compiler generates for the async function `goto`.

Part F Closures and Async/Await

Awaiting on `UnitGotoFuture` breaks down the `goto` function into different states. An enum tracks these states, saving where the execution paused and resuming from that point when the executor calls the future's `poll` function.

The following code shows the equivalent non-async Rust code for the `goto` function. It uses an enum (`GotoFuture`) to model the state machine.

```
#[repr(u8)]
enum GotoFuture {
    // 🚀 Initial state
    Start(UnitRef, i32) = 0,
    // ⌚ Waiting for UnitGotoFuture to complete
    Waiting(UnitGotoFuture) = 3,
    // ✅ Final state
    Done = 1,
}

// Implementing Future for GotoFuture
impl Future for GotoFuture {
    type Output = ();

    // The async executor will call the Future's poll method to check
    // if the future is ready and if the execution can continue.
    fn poll(mut self: Pin<&mut Self>,
            cx: &mut Context<'_>) -> Poll<Self::Output> {
        // The loop transitions between states.
        loop {
            match &mut *self {
                // 🚀 Start (0): In the start state, create a UnitGotoFuture
                // and move to the waiting state
                GotoFuture::Start(unit, pos) => {
                    let fut = UnitGotoFuture {unit: unit.clone(),
                                              target_pos: *pos };
                    *self = GotoFuture::Waiting(fut);
                }
                // ⌚ Waiting (3): In the waiting state, poll the
                // UnitGotoFuture and move to the done state if it is ready
                GotoFuture::Waiting(ref mut fut) => {
                    match Pin::new(fut).poll(cx) {
                        Poll::Ready(()) => *self = GotoFuture::Done,
                        Poll::Pending => return Poll::Pending,
                    }
                }
                // ✅ Done (1): In the done state, return ready
                GotoFuture::Done => return Poll::Ready(()),
            }
        }
    }
}

// The original async function is equivalent to creating a new
// GotoFuture instance in the start state
fn goto(unit: UnitRef, pos: i32) -> impl Future<Output = ()> {
```



```
GotoFuture::Start(unit, pos)
}
```

In the preceding code, the `GotoFuture` enum defines three states that correspond to the three stages of the async function's execution:

1.  **Start:** This state fires when the function receives the unit and target position. In this state, the function retains a reference to the unit and the target position, preparing to shift to the subsequent state.
2.  **Waiting:** In this state, the function awaits the completion of `UnitGotoFuture`. It possesses an instance of `UnitGotoFuture`, which undergoes repeated polling until it yields `Poll::Ready()`. This result signals the completion of its task.
3.  **Done:** This is the concluding stage, marking the function's completion. In this state, the function, upon polling, instantly returns `Poll::Ready()`.

We will see that the compiler-generated code for the `goto` function is like the state machine we just described. It uses a closure to implement the state machine and a state variable to track the state machine's current state.

Relationship between Closures, Futures, and Async Functions

Before we dig deeper into the assembly code, let us understand the relationship between futures, closures, and async executors.

Futures and Closures

In Rust, `async/await` mechanisms build on the concept of closures and Futures. When you write an async function in Rust, the compiler transforms it into a state machine, where each state represents a particular "await point" in your function.

The compiler transforms each `async` function you write in Rust into a closure that captures its environment and returns a Future. Each time the executor polls this Future, it effectively resumes the closure from where it last left off (the last await point).

Here is a simplified view of how the compiler transforms `async` functions. This view is not the exact transformation, but it should give you an idea of what is happening.

Let us say you have an `async` function like this:

```
async fn my_func() {
    // some code
    foo().await;
    // more code
    bar().await;
    // even more code
}
```

The compiler transforms this function into something like this:

```
fn my_func() -> impl Future<Output = ()> {
    async move {
        // some code
        foo().await;
        // more code
    }
}
```

Part F Closures and Async/Await

```

    bar().await;
    // even more code
}

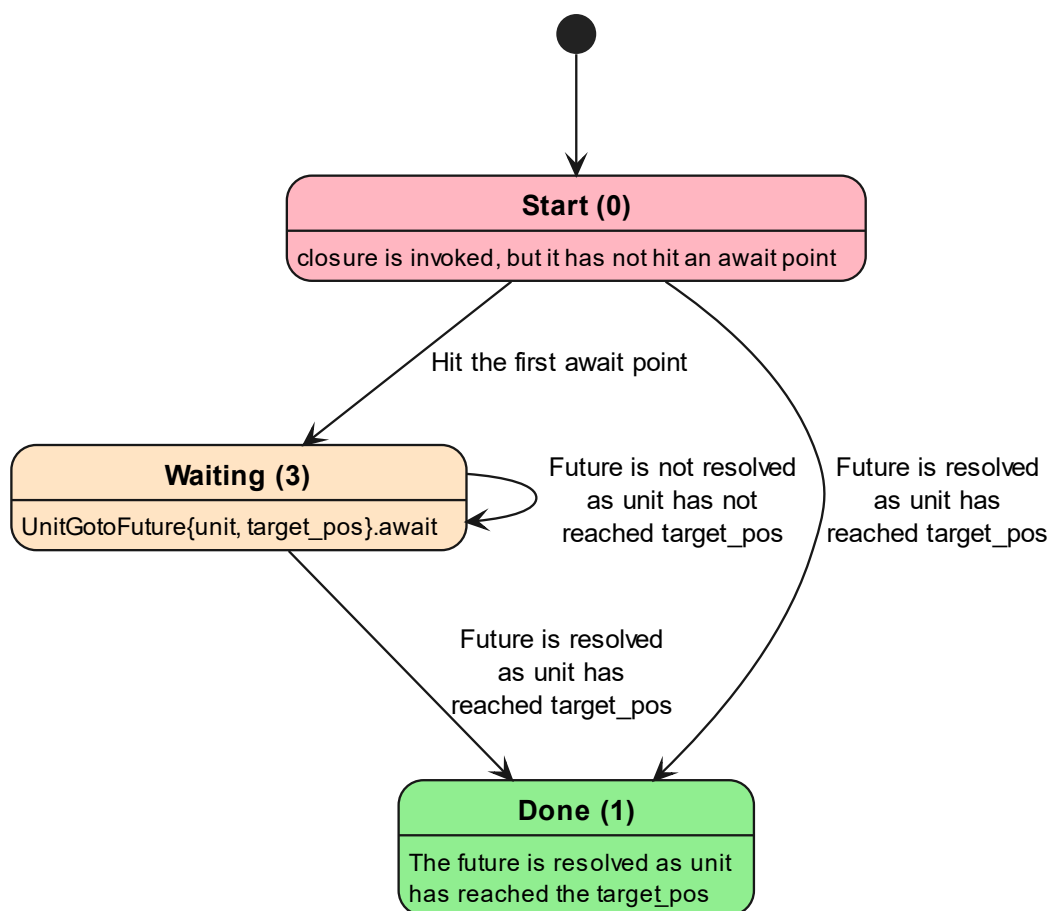
```

After transformation, `my_func` returns a Future created by the `async move { ... }` block, which acts like a closure. Inside this closure, the original body of `my_func` remains mostly unchanged, but the compiler alters its control flow to allow pausing and resuming at each `.await`. When polled, this Future runs the closure until it encounters a `.await` on a future that is not ready. Now, it saves its current state and returns `Poll::Pending`. Upon subsequent polling, it resumes from the saved state until it reaches the closure's end, eventually returning `Poll::Ready`.

Closure State Machine

We have just seen a Rust `async` function mapping into an `async` closure. Let us return to the `goto` function we are exploring in this chapter.

The `goto` closure includes a state variable to track the state machine's current state. Initially set to 0 in the `Start` state, the closure checks the state variable each time the executor calls the `poll` method. Based on the current state, closure takes appropriate action. The following diagram gives a high-level view of the closure state machine.



The following figure displays the closure environment implementing the state machine, containing the state variable and the captured variables. In the figure, the `goto_state` variable at offset 28 tracks the state machine's current state.

00 goto_unit_local	goto saves the local variable unit here
08 goto_target_pos_local	goto saves the local variable pos here
16 goto_unit_captured	
24 goto_target_pos_captured	Caller pos for goto
28 goto_state	state to identify the current execution state for goto

00 strong	strong reference count
08 weak	weak reference count
16 borrow	Borrow flag
24 pos	Position

The following assembly shows how the closure checks the state variable and uses a jump table to jump to the appropriate code block. The jump table switch happens at the start of the closure.

```

movzx eax, byte ptr [rdi + 28]      ; Load the state to determine which
                                   ; block to execute. The state is
                                   ; stored at the offset 28 in the
                                   ; closure environment.

lea rcx, [rip + .LJTI57_0]         ; rcx = address of the jump table
                                   ; The jump table is a list of offsets
                                   ; from the start of the jump-table to
                                   ; each block.

movsxd rax, dword ptr [rcx + 4*rax] ; Get the jump offset from the entry
                                   ; corresponding to the state. The
                                   ; index in rax is multiplied by 4
                                   ; because the jump table is an
                                   ; array of 32-bit jump offsets
                                   ; indexed by the state.

add rax, rcx                       ; Add the jump offset to the start
                                   ; of the jump table to get the
                                   ; address of the block to execute.

jmp rax                            ; Jump to the block to execute.

```

Here is the compiler-generated jump table. The jump table contains the offsets from the start of the jump table to each block.

```

.LJTI57_0:
.long .LBB57_4-.LJTI57_0 ; 🚀 Start (0) : Entry point to the goto closure.
.long .LBB57_3-.LJTI57_0 ; ✅ Done (1) : Throw a panic if polled after
                           ; completion of the future.
.long .LBB57_2-.LJTI57_0 ; 💀 UnitState2: Panic if async function
                           ; is resumed after panicking.
.long .LBB57_1-.LJTI57_0 ; ⌚ Waiting (3): Future is pending

```

The goto async function returns the Future struct that wraps the goto: :{{closure}}. The contents of the returned Future follow. They are the closure environment of goto: :{{closure}}. The returned closure environment contains the captured parameters at offsets 16 and 24. The state variable is stored at offset 28. The closure also has local variables that store the intermediate results.

00 goto_unit_local	goto saves the local variable unit here
08 goto_target_pos_local	goto saves the local variable pos here
16 goto_unit_captured	
24 goto_target_pos_captured	Caller pos for goto
28 goto_state	state to identify the current execution state for goto

00 strong	strong reference count
08 weak	weak reference count
16 borrow	Borrow flag
24 pos	Position

Part F Closures and Async/Await

The following function presents the assembly code of the `goto async` function. It stores the `Unit` and `target_pos` parameters at offsets 16 and 24, respectively, and initializes the state variable to 0 (Start) at offset 28. The assembly indicates that no code execution has taken place yet.

12 **34** Input:

- `rdi`: `goto::{{closure}}` environment
- `rsi`: `unit`
- `rdx`: `target_pos`

12 **34** Output:

- `rax`: `goto::{{closure}}` environment/Future

`playground::goto:`

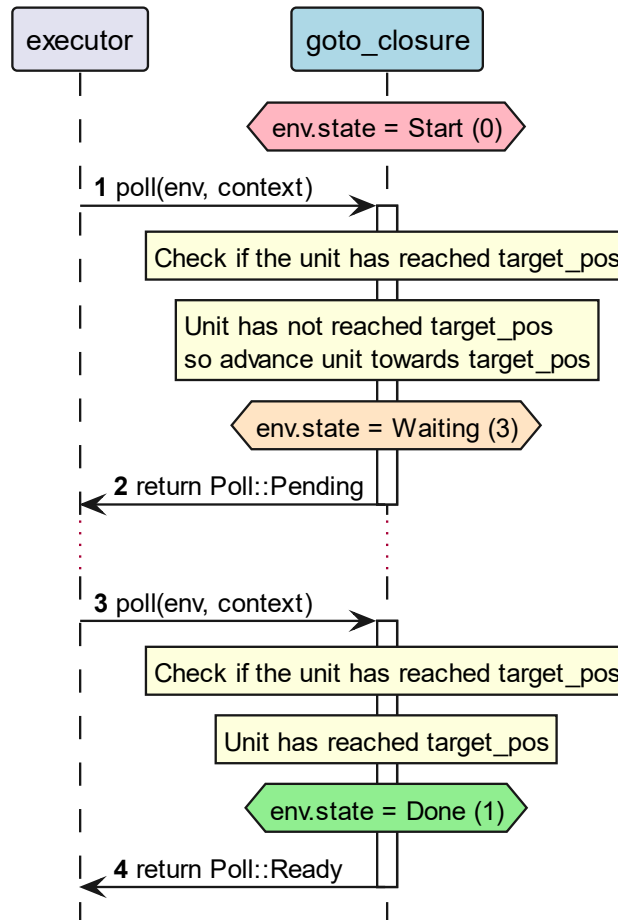
```

mov rax, rdi          ; rax = Set the return value to the future
mov qword ptr [rdi + 16], rsi ; Save the unit.
mov dword ptr [rdi + 24], edx ; Save the target_pos
mov byte ptr [rdi + 28], 0   ; 🚀 Start (0)
ret                        ; Return

```

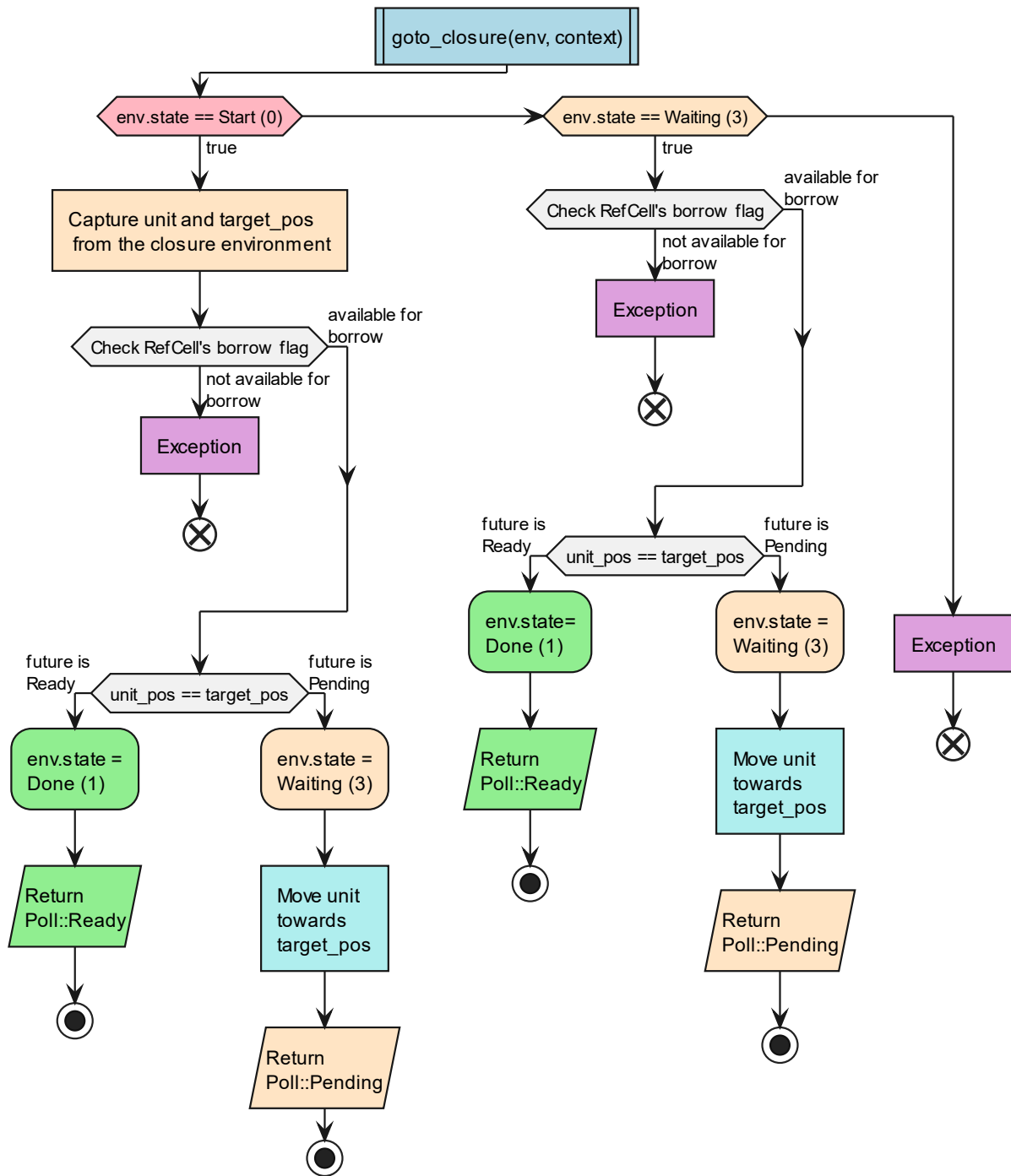
Role of the Async Executor

Rust requires an async executor to run the async functions. The executor is responsible for polling the future returned by the async function. The following sequence diagram shows how the executor polls the future returned by the `goto async` function. It calls the `poll` method of the `Future` trait. The `poll` method calls the `goto::{{closure}}`. The `goto::{{closure}}` checks the state variable and executes the appropriate code block based on the current state. The `goto::{{closure}}` then updates the state variable and returns the `Poll` enum. The `Poll` enum contains the result of the poll. The executor then checks the poll results and decides whether to continue polling.



Visualizing the Generated Assembly Code for the Async State Machine

Now that we better understand the async function's state machine and async executors, let us look at a high-level flowchart of the generated assembly code. We see that the compiler has inlined the poll for `UnitGotoFuture` into the `goto::{{closure}}`. Also, note that the goto closure can result in an exception if the `RefCell` borrowing fails.



Generated Assembly Code for the Async State Machine

Let us look at the generated assembly of the `goto :: {{closure}}` function. The function receives a mutable reference to a closure environment and a Context object. It returns `Poll::Pending` or `Poll::Ready`.

The assembly code implements a state machine. The following table describes the states of the state machine.

State	Description
 Start (0)	The entry point to the function. The function initializes the closure environment and loads the <code>Unit</code> and <code>target_pos</code> fields from the closure environment into

State	Description
	registers. It then saves these fields into the closure environment and jumps to the  Waiting (3) state.
 Waiting (3)	The state where the future is pending. The function loads the Unit field from the closure environment and calls the inlined UnitGotoFuture::poll logic. It calculates the difference between the unit_pos and the target_pos and updates the unit_pos accordingly. The function jumps to an error handler if the unit has already been borrowed. If the unit has reached the target_pos, it sets the state to  Done (1) . Otherwise, it returns Poll::Pending.
 Done (1)	The state where the future is completed. The function decrements the strong reference count of the unit. If the strong reference count reaches zero, it uses the __rust_dealloc function to free the memory associated with the unit. The function then returns Poll::Ready to indicate the completion of the future.

The assembly code begins by consulting a jump table, using the current state of the goto::{{closure}} to identify and jump to the correct code block. This closure retrieves the future's state from its environment, employing this information for the jump table lookup to find the corresponding code segment.

Let us now dive into the assembly code.

 Input:

- rdi: Closure environment
- rsi: &mut Context

 Output:

- rax: Poll<()>

```
playground::goto::{{closure}}:
push rbp
push r15
push r14
push rbx
push rax

mov r15, rdi ; r15 = closure environment
```

 Jump to code block based on closure state

Index into the jump table using the state in the closure environment and jump to the code block corresponding to the state.

```
movzx eax, byte ptr [rdi + 28] ; Load the state to select the block
                                ; to execute. The state is stored at
                                ; offset 28 in the closure environment.

lea rcx, [rip + .LJTI57_0] ; Load the address of the jump table rcx
                            ; The jump table is a list of offsets
                            ; from the start of the jump table to
                            ; each block.

movsxd rax, dword ptr [rcx + 4*rax] ; Get the jump offset from the entry
                                    ; corresponding to the state.
```

Part F Closures and Async/Await

	; The index in rax is multiplied by 4
	; because the jump table is an array of
	; 32-bit jump offsets indexed by state.
<code>add rax, rcx</code>	; Add the jump offset to the start of
	; the jump table to get the
	; address of the block to execute.
<code>jmp rax</code>	; Jump to the block to execute.

 **Start (0)** block entry point

 Note that the caller of the async function is responsible for setting the state to 0 and initializing the closure environment.

```
.LBB57_4:
mov rdi, qword ptr [r15 + 16] ; rdi = unit object from the closure environment
mov eax, dword ptr [r15 + 24] ; eax = target_pos from the closure env

mov qword ptr [r15], rdi ; Save the unit in the closure env
mov dword ptr [r15 + 8], eax ; Save the target_pos in the closure env
jmp .LBB57_5
```

 **Waiting (3)** block entry point

★ Resume the future after a poll that returned Pending.

```
.LBB57_1:
mov rdi, qword ptr [r15] ; rdi = unit from the closure environment

.LBB57_5:
; Inlined call to UnitGotoFuture::poll
mov rax, qword ptr [rdi + 16] ; Load the borrow flag from unit's
; RefCell into rax

movabs rcx, 9223372036854775807 ; rax = maximum signed 64-bit integer

cmp rax, rcx ; Check if the borrow flag is greater
; than max signed 64-bit value.

jae .LBB57_6 ; If the value in rax is greater than
; or equal to max signed 64-bit value,
; jump to .LBB57_6, as the unit has
; already been borrowed.
```

★ The unit has not been borrowed.

```
mov ebx, dword ptr [rdi + 24] ; Load the unit_pos from unit into ebx
mov ebp, dword ptr [r15 + 8] ; Load the target_pos from the closure
; environment into ebp

mov ecx, ebp ; Set ecx to target_pos

sub ecx, ebx ; Subtract unit_pos from target_pos
```



```

; and store the result in ecx

jne .LBB57_8                ; If the difference is not equal to 0,
                           ; jump as the unit has not reached the
                           ; target position

dec qword ptr [rdi]        ; Decrement the strong reference count
                           ; of the unit

mov r14b, 1                ; Set the state to  Done (1)

jne .LBB57_17              ; Check if the strong reference count is
                           ; not equal to 0. If it is not equal to
                           ; 0, jump to .LBB57_17

```

 Freeing Rc memory as the reference count is 0.

```

dec qword ptr [rdi + 8]    ; Decrement the weak reference count
                           ; of the unit

jne .LBB57_17              ; Jump if the weak reference count is
                           ; not equal to 0.

mov esi, 32                ; Deallocating 32 bytes.
mov edx, 8                  ; Memory alignment is 8 bytes

call qword ptr [rip + __rust_dealloc@GOTPCREL]
jmp .LBB57_17

```

 Check if the unit can be borrowed.

```

.LBB57_8:
test rax, rax              ; Check if the borrow flag is 0.

jne .LBB57_9              ; If the borrow flag is not 0, jump
                           ; to .LBB57_9, as the unit has
                           ; already been borrowed.

xor eax, eax               ; Set eax to 0.

```

 `signum` function inlined - begin

```

test ecx, ecx              ; Check if the unit_pos is greater
                           ; than the target_pos

setg al                    ; Set the value of al to 1 if unit_pos
                           ; is greater than target_pos
                           ; Set the value of al to 0 if unit_pos
                           ; is less than or equal to target_pos

lea eax, [rbx + 2*rax]     ; Add 2 to unit_pos if unit_pos is less
                           ; than target_pos

dec eax                    ; Subtract one from the result of

```

Part F Closures and Async/Await

```

; the previous addition
; (signum addition of 1 or -1)

```

🚧 `signum` function inlined - end

🕒 Transition to Waiting (3) state

```

mov dword ptr [rdi + 24], eax    ; Save the new unit_pos in the unit
mov qword ptr [rdi + 16], 0     ; Set the borrow flag to 0.

mov r14b, 3                    ; Set the state to 🕒 Waiting (3)

```

🔙 Save the state in the closure environment and return the future status.

- `Poll::Pending` to indicate that the future is not ready.
- `Poll::Ready` to indicate that the future is ready.

```

.LBB57_17:
cmp ebp, ebx                    ; Compare unit_pos with target_pos

setne al                        ; Set the value of al to 1 (future not
                                ; ready) if unit_pos is not equal to
                                ; target_pos
                                ; al = 0 (future ready)
                                ; if unit_pos is equal to target_pos

mov byte ptr [r15 + 28], r14b   ; Save the state in closure environment

add rsp, 8                      ; Free local variables.
pop rbx
pop r14
pop r15
pop rbp
ret                             ; Return the future status.

```

💀 Prepare the panic message.

```

.LBB57_6:
lea r8, [rip + .L__unnamed_21]
lea rcx, [rip + .L__unnamed_20] ; drop_in_place
mov esi, 24
lea rdi, [rip + .L__unnamed_19] ; Load "already borrowed".
jmp .LBB57_10

```

💀 `UnitState2`: Panic if async function is resumed after panicking.

```

.LBB57_2:
lea rdi, [rip + str.1]          ; Load address of string
                                ; "`async fn` resumed after panicking"

lea rdx, [rip + .L__unnamed_23]
mov esi, 34
call qword ptr [rip + core::panicking::panic@GOTPCREL]
ud2

```

✅ Done (1) block entry point

This block should be unreachable as the future has been completed.

```
.LBB57_3:

lea rdi, [rip + str.2]           ; Load address of string "`async fn`
                                ; resumed after completion"
lea rdx, [rip + .L__unnamed_23]
mov esi, 35
call qword ptr [rip + core::panicking::panic@GOTPCREL]
ud2

.LBB57_9:

lea r8, [rip + .L__unnamed_22]
lea rcx, [rip + .L__unnamed_8]
mov esi, 16
lea rdi, [rip + .L__unnamed_7]

.LBB57_10:

mov rdx, rsp
call qword ptr [rip + core::result::unwrap_failed@GOTPCREL]
ud2
mov r14, rax
mov rdi, qword ptr [r15]
call core::ptr::drop_in_place<playground::UnitGotoFuture>
mov byte ptr [r15 + 28], 2
mov rdi, r14
call _Unwind_Resume@PLT
ud2
```

🌈 Jump table for the goto closure. The async closure uses the jump table to find the starting address of code blocks handling different async states.

```
.LJTI57_0:

.long .LBB57_4-.LJTI57_0        ; 🚀 Start (0): Entry point to the
                                ; goto closure.
.long .LBB57_3-.LJTI57_0        ; ✅ Done (1): Throw a panic if polled
                                ; after completion of the future.
.long .LBB57_2-.LJTI57_0        ; 💀 UnitState2: Panic if async function
                                ; is resumed after panicking.
.long .LBB57_1-.LJTI57_0        ; ⌚ Waiting (3): Future is pending.
```

📁 vtable for the goto closure. Note that the vtable contains the address of the goto::{{closure}} function. The vtable also stores the size and alignment of the closure.

```
.L__unnamed_26:
; Destructor for the FnOnce trait object
.quad core::ptr::drop_in_place<playground::goto::{{closure}}>
```

Part F Closures and Async/Await

```
; Size of object: 32 bytes (Leading space)
; Alignment of the Future trait object: 8 bytes (\b)
.asciz " \000\000\000\000\000\000\000\b\000\000\000\000\000\000"

; call_once method of the FnOnce trait
.quad playground::goto::{closure}}
```



Experiment: Equivalent Async and Non-Async Functions

Let us experiment with the following code in the [Rust playground](https://bit.ly/44D8Gl3) (<https://bit.ly/44D8Gl3>)

The essential functions in the code are:

- `simulated_async_operation` is an async function that simulates an asynchronous operation by sleeping for 2 seconds and returning the sum of the two parameters.
- `example_function` is a regular function that returns a `Future` that will return the sum of the two parameters.
- `example_function_async` is an async function that returns the sum of the two parameters.

```
pub async fn simulated_async_operation(a: i32, b: i32) -> i32 {
    tokio::time::sleep(std::time::Duration::from_secs(2)).await;
    a + b
}

pub fn example_function(a: i32, b: i32) ->
    impl std::future::Future<Output = i32> {
    async move { simulated_async_operation(a, b).await }
}

pub async fn example_function_async(a: i32, b: i32) -> i32 {
    simulated_async_operation(a, b).await
}

#[tokio::main]
async fn main() {
    let result = example_function(5, 10).await;
    println!("The example_function result is {}", result);
    let result = example_function_async(42, 84).await;
    println!("The example_function_async result is {}", result);
}
```

Now that we have looked at the code let us run some experiments in the Rust playground.

Run

Use the **RUN** button to run the code (top-left button in the Rust playground). You will see the following output with each line printed after a 2-second delay. The two functions, `example_function` and `example_function_async`, are equivalent.

Here is the output from the Rust playground:

```
The example_function result is 15
The example_function_async result is 126
```

Explore the Assembly Code

1. Now set the **Optimization level** to **RELEASE** using the second drop-down menu from the left.
2. Switch the leftmost button to **ASM** to see the assembly code.

Some interesting searches to navigate the assembly code:

Search for	Description
<code>playground::main:</code>	The <code>main</code> function invokes the <code>main::{{closure}}</code> . The generated code results from expanding the <code>#[tokio::main]</code> . Look for a call to <code>playground::main::{{closure}}</code>
<code>playground::main::{{closure}}:</code>	The <code>main</code> function invokes this closure. The compiler has inlined the <code>example_function</code> and <code>example_function_async</code> functions. Look for calls to <code>playground::simulated_async_operation::{{closure}}</code> . You will see two calls that will return the future that is awaited in the <code>main::{{closure}}</code> .
<code>playground::simulated_async_operation::{{closure}}:</code>	The function begins with <code>.LJTI106_0</code> based lookup table access to determine the execution block. Also, look for calls to the <code>tokio::time::sleep</code> function. Note that <code>tokio::time::sleep</code> is an <code>async</code> function that does not block the thread. Following the call to <code>sleep</code> , we see a lot of stack and memory operations required to preserve the environment across multiple invocations of the <code>simulated_async_operation</code> function.

Run Clippy

Use the Rust playground's **Tools** menu to run **Clippy**. You will see that Clippy recommends we change:

```
pub fn example_function(a: i32, b: i32)
    -> impl std::future::Future<Output = i32> {
    async move { simulated_async_operation(a, b).await }
}
```

To:

```
pub async fn example_function(a: i32, b: i32) -> i32 {
    simulated_async_operation(a, b).await
}
```

The Clippy message matches our understanding that an `async` function in Rust returning a type `T` is equivalent to a regular function that returns an anonymous type implementing `Future<Output = T>`. We covered this in the [Futures and Closures](#) section.

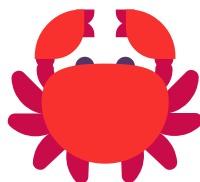
To summarize:

Part F Closures and Async/Await

- **pub async fn example_function_async(a: i32, b: i32) -> i32:** This asynchronous function takes two parameters of type `i32` and returns a value of type `i32`. However, because it is marked `async`, it returns a `Future` that, when awaited, produces an `i32`.
- **pub fn example_function(a: i32, b: i32) -> impl std::future::Future<Output = i32>:** This is a function that takes two parameters of type `i32` and returns a type that implements the `Future` trait with an `Output` of `i32`. The Rust compiler desugars an `async` function into this function. It provides the same functionality as the preceding `async` function but requires manual handling of the `Future`.

Key Takeaways

- An `await` in an `async` function typically results in a `poll` call on the future. If the future is not ready, the `poll` method returns `Poll::Pending`. The `poll` method returns `Poll::Ready` if the future is ready.
- The `async` function itself returns a future that wraps a closure. The `poll` method of the future calls the closure.
- The closure functions as a state machine, with the compiler representing the `await` points as state transitions.
 - A compiler-generated closure stores the state.
 - The compiler generates a jump table that maps from the current state to the code handling the state.
- An `async` function in Rust returning a type `T` is equivalent to a regular function that returns an anonymous type implementing `Future<Output = T>`.
- In an `async` function, the closure environment stores the local variables that must persist across invocations. Too many such local variables can cause the closure environment to be large.



Chapter 19

Nested Async/Await: Desugaring to Loops

This chapter builds on the previous one, which covered the desugaring and assembly code generation for `async/await`. This chapter will explore the [desugaring](#) and [assembly code generated for nested `async/await` constructs](#) in Rust. Additionally, we will examine how `async` functions deal with [infinite loops](#).

`patrol` Function Awaits `goto` Function in an Infinite Loop

Our primary focus will be on the `patrol` function. The `patrol` function is an `async` function that moves a unit between two positions. The core of the `patrol` function involves the `goto` function, which directs the unit to move to a particular position. By utilizing the `goto` function, the `patrol` function continuously toggles the unit's movement between the two positions, `poses[0]` and `poses[1]`, creating an infinite loop of patrolling. The `.await` waits for the `goto` function to reach the target position.

Here is the code representation of the `patrol` function:

```
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
    loop {
        goto(unit.clone(), poses[0]).await;
        goto(unit.clone(), poses[1]).await;
    }
}
```

`patrol` and `goto` Functions: Nested Async/Await Constructs

Here is the complete code for the `patrol`, `goto` functions and the `Unit` struct. We looked at the `goto` function in the “*Async/Await: Internal State Machine Dynamics*” chapter. Here, we will explore the `patrol` function in detail.

```
#[derive(Default)]
struct Unit {
    /// The 1-D position of the unit. The unit can only move along this axis.
    pub pos: i32,
}
type UnitRef = Rc<RefCell<Unit>>;
```

Part F Closures and Async/Await

```

/// A future that will move the unit towards `target_pos` at each step,
/// and complete when the unit has reached that position.
struct UnitGotoFuture {
    unit: UnitRef,
    target_pos: i32,
}

impl Future for UnitGotoFuture {
    type Output = ();
    fn poll(self: Pin<&mut Self>, _cx: &mut Context) -> Poll<Self::Output> {
        let unit_pos = self.unit.borrow().pos;
        if unit_pos == self.target_pos {
            Poll::Ready(())
        } else {
            self.unit.borrow_mut().pos
                += (self.target_pos - unit_pos).signum();
            Poll::Pending
        }
    }
}

/// Helper async function to write unit behavior nicely
async fn goto(unit: UnitRef, pos: i32) {
    UnitGotoFuture {
        unit,
        target_pos: pos,
    }
    .await;
}

/// Let a unit go back and forth between two positions
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
    loop {
        goto(unit.clone(), poses[0]).await;
        goto(unit.clone(), poses[1]).await;
    }
}

```

A bit of a refresher on the `goto` function: it takes a `UnitRef` and a `pos` as input and returns a `UnitGotoFuture` instance. The `UnitGotoFuture` instance implements the `Future` trait and has a `poll` method that moves the unit towards the `pos` at each step and completes when the unit has reached that position.

After this brief recap, let us dig deeper into the `patrol` function's async machinery.

Desugaring the Patrol Function into a State Machine

Before exploring the generated assembly, let us see how we might desugar the async `patrol` function into a state machine. The transformation of the async `patrol` function into a state machine results in a structure resembling the following code:

```

// The state machine enum
#[repr(u8)]
enum PatrolFuture {
    // Initial state

```



```

Start(UnitRef, [i32; 2]),
// Waiting for goto to complete
WaitingToReachPosition0(UnitRef, [i32; 2],
    Pin<Box<dyn Future<Output = ()>>>),
WaitingToReachPosition1(UnitRef, [i32; 2],
    Pin<Box<dyn Future<Output = ()>>>),
// Final state (which is unreachable)
Done,
}

// Implementing Future for PatrolFuture
impl Future for PatrolFuture {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
        -> Poll<Self::Output> {
        // Convert the pinned reference to &mut PatrolFuture
        let this = self.get_mut();

        loop {
            match this {
                // In the start state, create a goto future
                // and move to the waiting state
                PatrolFuture::Start(unit, poses) => {
                    let fut = Box::pin(goto(unit.clone(), poses[0]));
                    *this = PatrolFuture::WaitingToReachPosition0(
                        unit.clone(), *poses, fut);
                }
                // In the waiting state for goto1, poll the goto future and
                // move to the next waiting state if it is ready
                PatrolFuture::WaitingToReachPosition0(
                    unit, poses, ref mut fut) => {
                    match Pin::as_mut(fut).poll(cx) {
                        Poll::Ready(()) => {
                            let fut = Box::pin(goto(unit.clone(), poses[1]));
                            *this =
                                PatrolFuture::WaitingToReachPosition1(
                                    unit.clone(), *poses, fut);
                        }
                        Poll::Pending => return Poll::Pending,
                    }
                }
                // In the waiting state for goto2, poll the goto future
                // and move back to the start state if it is ready
                PatrolFuture::WaitingToReachPosition1(unit, poses,
                    ref mut fut) => {
                    match Pin::as_mut(fut).poll(cx) {
                        Poll::Ready(()) => *this = PatrolFuture::Start(
                            unit.clone(), *poses),
                        Poll::Pending => return Poll::Pending,
                    }
                }
                // In the done state (which is unreachable), return ready
                PatrolFuture::Done => return Poll::Ready(()),
            }
        }
    }
}

```

Part F Closures and Async/Await

```

    }
  }
}

// The original async function is equivalent to creating a new
// PatrolFuture instance
fn patrol(unit: UnitRef, pos: [i32; 2]) -> impl Future<Output = ()> {
    PatrolFuture::Start(unit.clone(), pos)
}

```

We see in the preceding code that the `patrol` function is desugared into a state machine that consists of three key states: `Start`, `WaitingToReachPosition0`, and `WaitingToReachPosition1`.

- The `Start` state is the initial state, creating a new `goto` future and moving to the `WaitingToReachPosition0` state.
- The `WaitingToReachPosition0` state polls the `goto` future and moves to the `WaitingToReachPosition1` state if the future is ready.
- The `WaitingToReachPosition1` state polls the `goto` future and returns to the `Start` state if the future is ready. The move back to `Start` implements the infinite loop behavior of the `patrol` function.

💡 Learn More: What Is the Pin in the Signature of the `poll` Method?

The `Pin` type in Rust guarantees that the data at a particular memory location will not move and its memory will not get invalidated. This guarantee is crucial in asynchronous programming, mainly when working with `Futures`.

The definition of the `poll` method is as follows:

```
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>
```

Note that `self` is wrapped in `Pin` as a `Future` might be self-referential, potentially referencing another part of itself. Moving such a future could invalidate that reference, leading to undefined behavior. The `Pin` wrapper prevents this by ensuring that the data inside cannot move, allowing safe referencing. Regarding the `poll` method, this setup effectively promises that neither the future nor anything it references will move during the poll's duration.

💡 Learn More: How Can Closures Create Self-Referential Structures in Async Rust?

In Rust, closures capture their environment, allowing them to use variables from the scope in which they are defined. The closure stores this captured data within itself, a key aspect since Rust treats closures as values that can be passed around and have lifetimes like any other value.

In async programming, you often chain futures together, frequently using closures within combinators like `map`, `and_then`, or in `async` blocks. These closures capture variables from their surrounding environment, which can result in self-referential structures.

Consider this example:

```
let future = async {
    let data = get_some_data().await;
    let reference = &data;

```

```
do_something_with(reference).await;
};
```

Here, `reference` is a reference to data. But data lives within the `Future` created by the `async` block - it is part of the future's state. So, this `Future` contains a reference to its internal state.

If this future were to move, the location of data in memory would change, and `reference` would still point to the old area, leading to undefined behavior. By using `Pin`, we can ensure that once the future is created, it can no longer move, preserving the validity of `reference`.

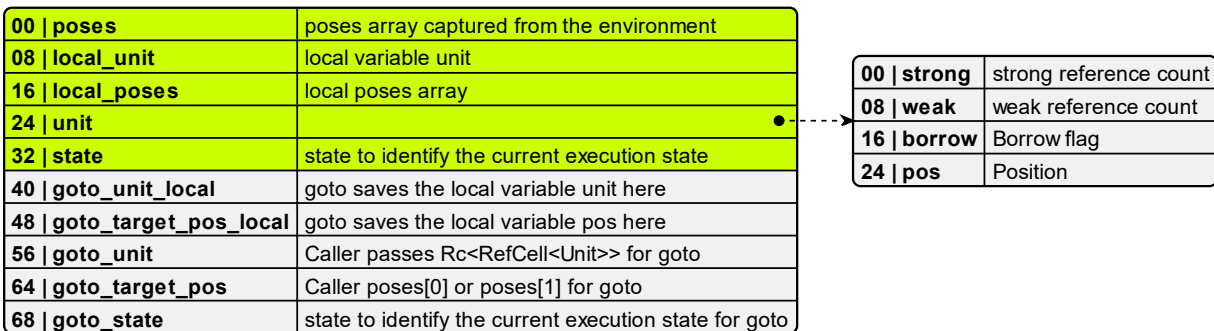
Understanding Async: Closures, State Machines, and Async Runtime

Closures and State Machines

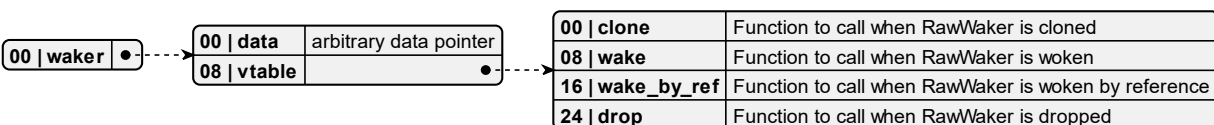
We have looked at the desugared code for the `patrol` function. The generated assembly of the `async` code uses closures to implement the state machine. We explored the mapping of `async` functions to closures when we looked at the `goto` function in the previous `async` deep dive.

The compiler converts the `patrol` function into a closure that takes two inputs: the closure environment and the task context. The closure environment is a struct containing the state of the state machine. The task context is a struct that holds the task's state executing the future. The closure returns a `Poll` value, either `Poll::Pending` or `Poll::Ready`. The `Poll::Pending` value indicates that the future is not ready, and the `Poll::Ready` value indicates that the future is ready.

The following diagram shows the closure environment for the `patrol` function. The environment is a struct that contains the environment of the `patrol` closure. The environment also includes the closure environment for the `goto` closure. The environment is passed to the closure as a mutable reference.



The second input to the closure is the task context used to schedule waking up the task when the future is ready. An example of a task context follows. We will not explore the task context as the `patrol` and `goto` functions do not use it.

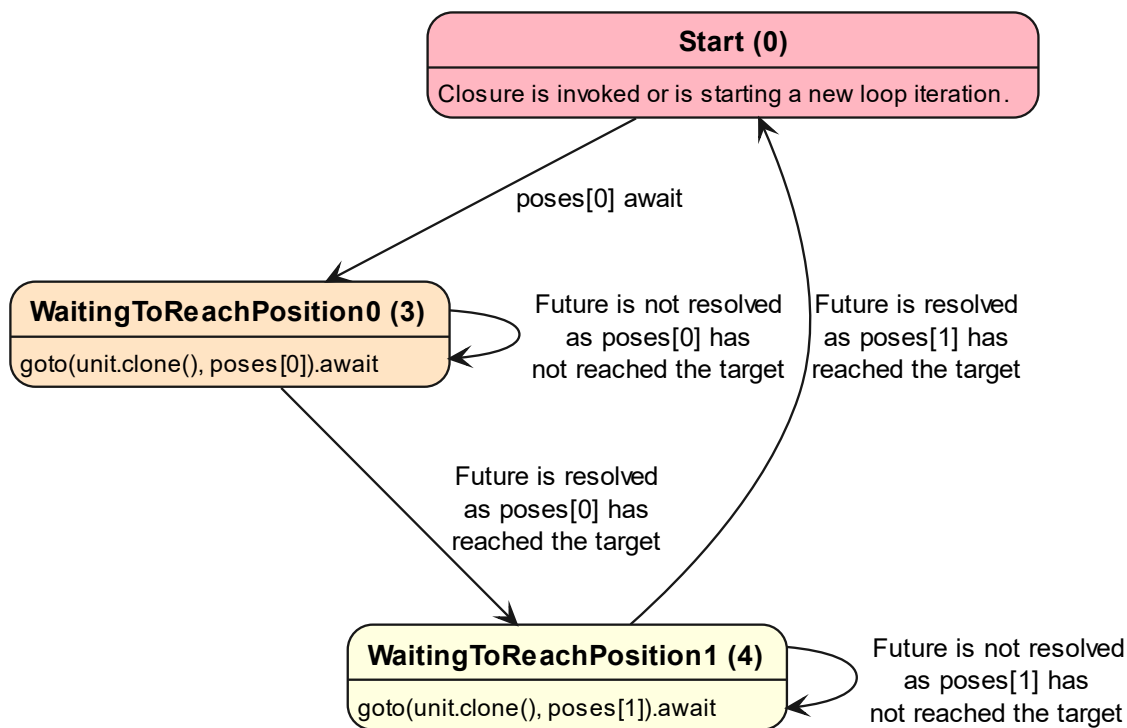


The closure also keeps track of the state of the state machine in the `state` field. The closure updates the `state` field whenever the state machine moves to a new state. As we saw with the desugared code, the state machine starts in the `Start` state and moves to the `WaitingToReachPosition0` state on the creation of the `goto` future. The state machine then moves to the `WaitingToReachPosition1` state when the `poses[0]` `goto` future is ready. The state machine returns to the `Start` state when the `poses[1]`

Part F Closures and Async/Await

`goto` future is ready. The move back to `Start` implements the infinite loop behavior of the `patrol` function.

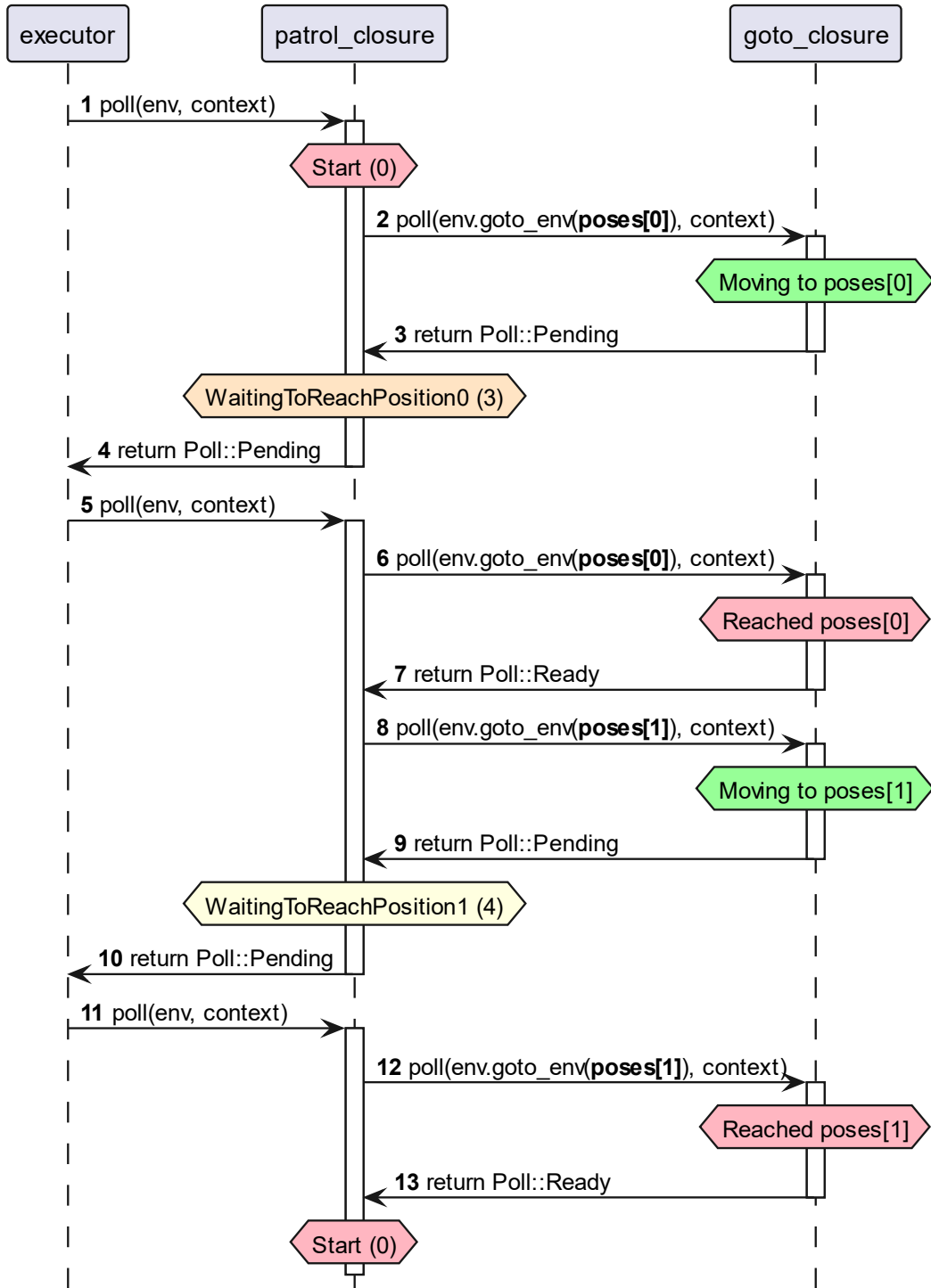
The following diagram shows the state machine for the `patrol` function.



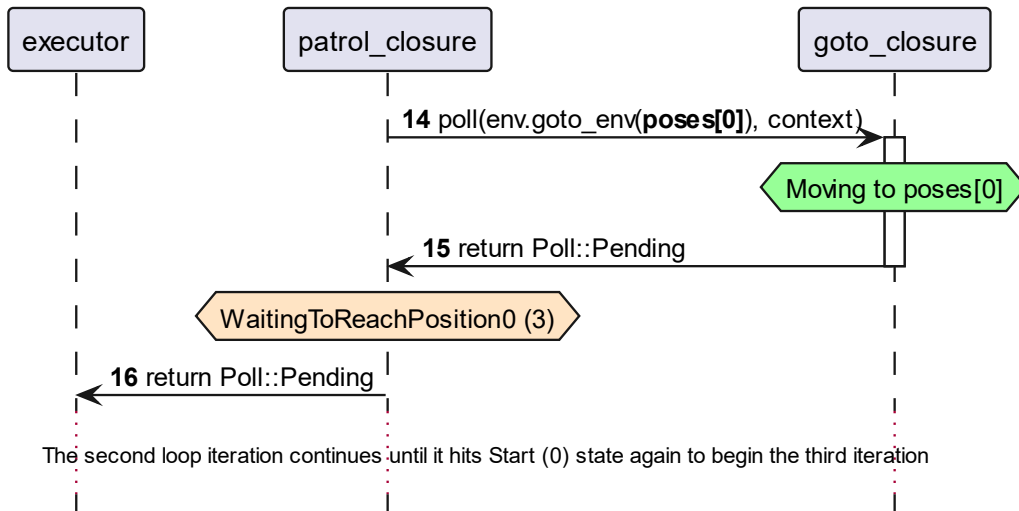
Role of the Async Runtime and Infinite Async Loops

The following sequence diagram shows an example of interactions between three participants: an executor, a `patrol` closure, and a `goto` closure. We will look at the first two loop iterations of the infinite loop in the `patrol` function.

First loop iteration:



Second loop iteration:



The process, as described in the sequence diagram, is enumerated here with numbers that match the sequence diagram.

1. The Executor invokes the `patrol` closure with the `poll` method, passing the closure environment and context as arguments.
2. `patrol` is now in the 🚀 `Start` state. The `patrol` closure then invokes the `goto` closure with the `poll` method, passing the `goto` environment with `poses[0]` and context as arguments, signaling the unit to move to the first position.
3. The `goto` closure is in the moving state and returns `Poll::Pending` to the `patrol` closure.
4. `patrol` enters the 🕒 `WaitingToReachPosition0` state and returns `Poll::Pending` to the executor.
5. The Executor again invokes the `patrol` closure with the `poll` method.
6. The `patrol` closure invokes the `goto` closure with the `poll` method, passing the same arguments.
7. Upon reaching `poses[0]`, `goto` returns `Poll::Ready` to the `patrol` closure.
8. The `patrol` closure then invokes the `goto` closure with the `poll` method, passing `poses[1]` to signal the unit to move to the second position.
9. `goto` moves to the second position, returning `Poll::Pending` to `patrol`.
10. `patrol` enters the 🕒 `WaitingToReachPosition1` state and returns `Poll::Pending` to the executor.
11. The Executor invokes `patrol` with the `poll` method.
12. `patrol` invokes `goto` with the `poll` method, passing the same arguments.
13. Upon reaching `poses[1]`, `goto` returns `Poll::Ready` to `patrol`, which re-enters the 🚀 `Start` state.
14. The process repeats in the second loop iteration, starting with `patrol` invoking `goto` to move to the first position again.
15. The `goto` closure is in the moving state and returns `Poll::Pending` to the `patrol` closure.
16. `patrol` enters the 🕒 `WaitingToReachPosition0` state and returns `Poll::Pending` to the executor.

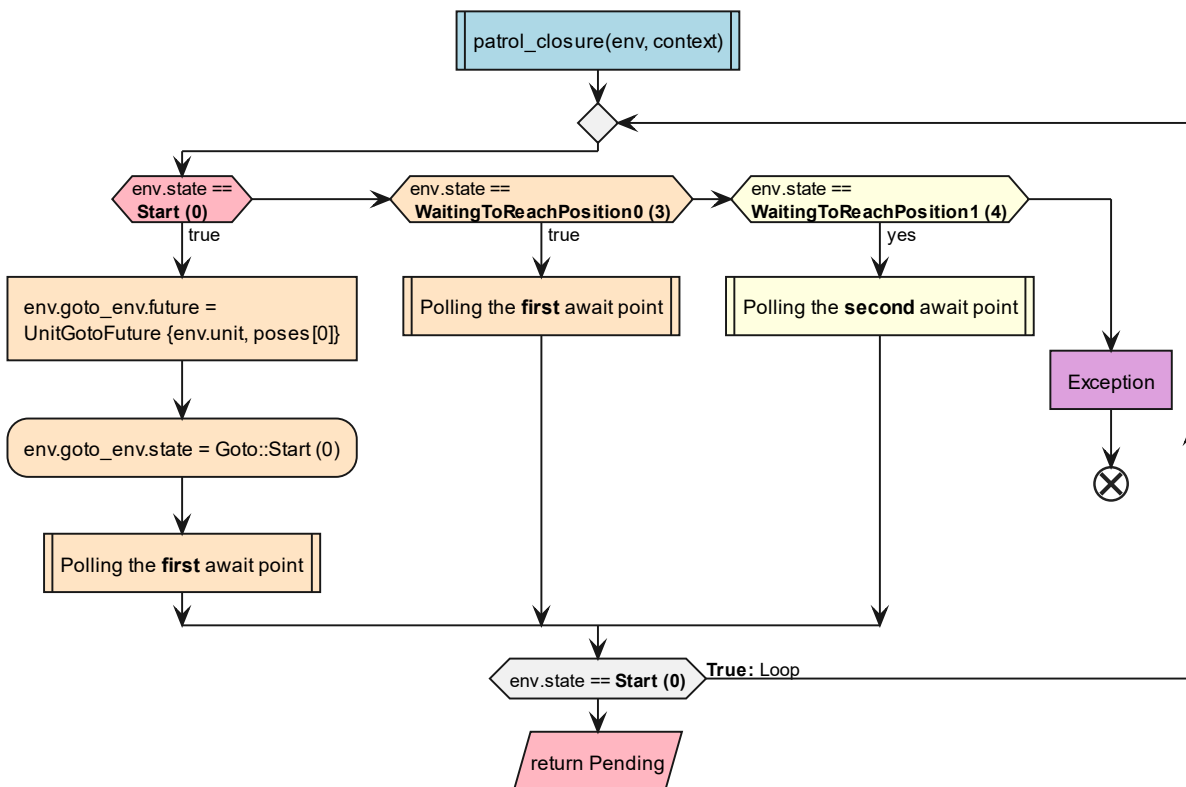
This process will continue indefinitely, signifying the infinite loop behavior of the `patrol` function. The executor is only responsible for invoking the `patrol` closure and does not need to have any knowledge about the infinite loop behavior.

Flowchart of the patrol Async Function State Machine

This section looks at a high-level flowchart of the `patrol` function. The flowchart shows the states of the `patrol` function and the transitions between the states. Note that the flowchart is a simplified representation that shows the states and transitions relevant to this chapter's discussion.

The `patrol` state machine has the following states:

- 🚀 **Start (0)**: This is the `patrol` state machine's initial state. The state machine prepares the `goto` closure environment and polls the first `await` point. The [next section](#) describes the polling procedure.
- ⌚ **WaitingToReachPosition0 (3)**: The state machine is in this state when the first `await` point is polled (described in the [next section](#)).
- 🕒 **WaitingToReachPosition1 (4)**: The state machine is in this state when the second `await` point is polled (described in the [second await point polling](#) section).



Polling the First Await Point

1 marks the first `await` point in the `patrol` function.

```

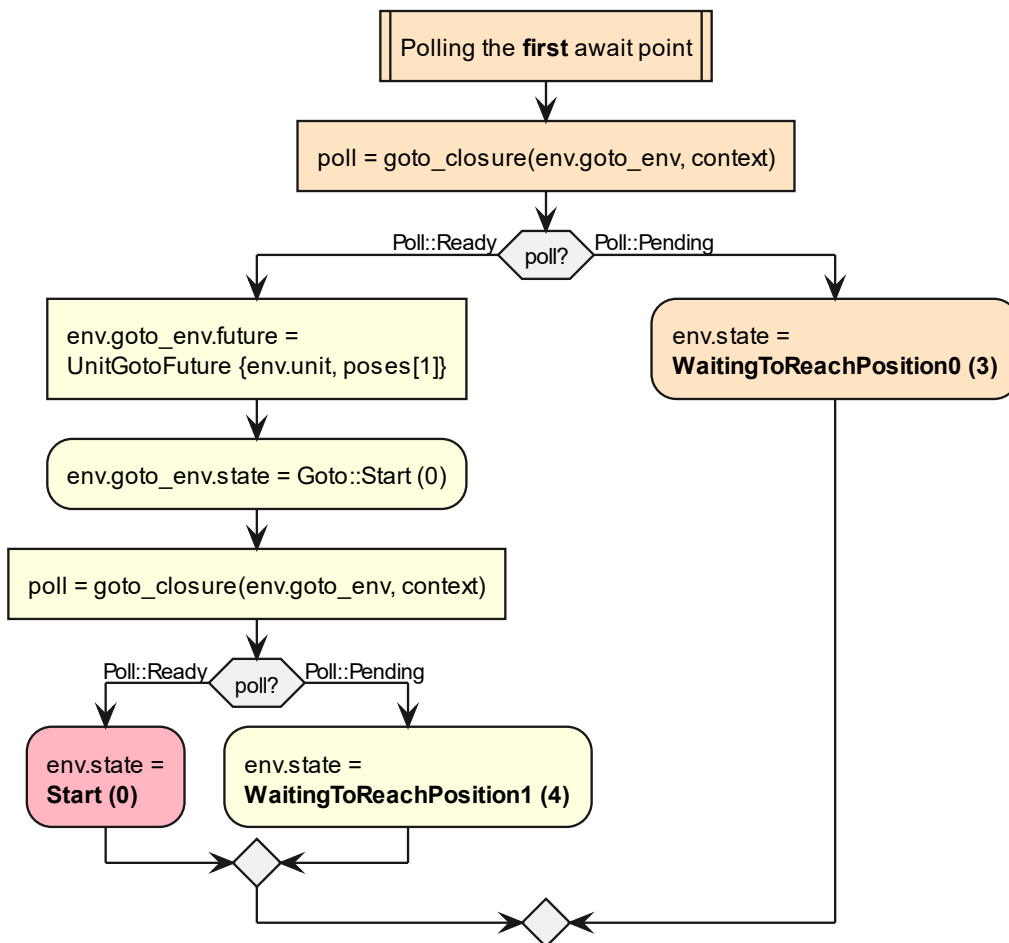
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
  loop {
    goto(unit.clone(), poses[0]).await; // 1
    goto(unit.clone(), poses[1]).await;
  }
}

```

Part F Closures and Async/Await

The goto closure environment has already been initialized within the patrol closure environment for the first await point. The goto closure is polled. The result of the first await point poll determines the further action.

- If the first await point returns `Poll::Ready`, the patrol function prepares the environment for the `poses[1]` await point and invokes the goto closure for the second await point.
- If the first await point returns `Poll::Pending`, the patrol function sets the state to 🕒 `WaitingToReachPosition0 (3)` and returns `Poll::Pending`.
- If the second await point returns `Poll::Pending`, the patrol function sets the state to 🕒 `WaitingToReachPosition1 (4)` and returns `Poll::Pending`.
- If the second await point returns `Poll::Ready`, the patrol function sets the state to 🚀 `Start (0)` and jumps to continue the infinite loop.



Polling the Second Await Point

2 marks the second await point in the patrol function.

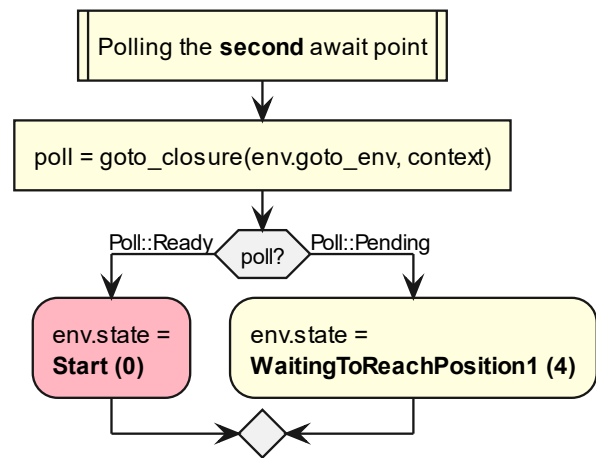
```

async fn patrol(unit: UnitRef, poses: [i32; 2]) {
    loop {
        goto(unit.clone(), poses[0]).await;
        goto(unit.clone(), poses[1]).await; // 2
    }
}

```


At this stage, the 'goto' closure environment is already initialized within the patrol closure environment for the second await point. Upon polling the goto closure, the outcome of this second await point determines the further course of action.

- If the second await point returns Poll::Ready, the patrol function sets the state to 🚀 Start (0) and jumps to continue the infinite loop.
- If the second await point returns Poll::Pending, the patrol function sets the state to ⌚ WaitingToReachPosition1 (4) and returns Poll::Pending.



Understand Nested Async/Await Generated Assembly

We now understand the high-level flow of the async code. Let us look at the generated assembly code.

Input and Output Registers

This assembly code, generated from a patrol closure, takes two inputs: the closure environment stored in rdi, and the task context stored in rsi. The output is stored in rax and can be Poll::Pending or Poll::Ready.

The code starts by saving the callee-saved registers and then moving the task context and closure environment to r14 and rbx, respectively. The code then loads the state from the closure environment and uses a jump table to determine which block of code to execute based on the state.

State Machine Jump Table

The compiler splits the generated assembly code into five code blocks, as shown in the following table:

State Id	Code Block	Code Block Behavior
0	🚀 Start	Executes before the first future is polled
1	🔴 Error handling 1	Throws an exception
2	🔴 Error handling 2	Panics
3	⌚ WaitingToReachPosition0	Waits for the first future to resolve
4	⌚ WaitingToReachPosition1	Waits for the second future to resolve

At the start of the function, the code uses the current state of the closure to select the code block for execution. It employs a state-indexed jump table to make this selection. Here follows the definition of the patrol function's jump table:

.LJTI58_0:

```
.long    .LBB58_1-.LJTI58_0 ; 🚀 Start (0): Code executed before the
                                ; first future is polled
.long    .LBB58_3-.LJTI58_0 ; 🔴 State 1: Throw an exception.
.long    .LBB58_2-.LJTI58_0 ; 🔴 State 2: Panic.
.long    .LBB58_6-.LJTI58_0 ; ⌚ WaitingToReachPosition0 (3)
.long    .LBB58_16-.LJTI58_0 ; ⌚ WaitingToReachPosition1 (4)
```

Start State

The 🚀 Start state performs the following actions:

1. Load the captured unit and poses from the closure environment into registers. Also, save an additional copy of these values in the closure environment.
2. Increment the reference count for the captured unit.
3. Save the unit and poses[0] in the goto closure environment.

States WaitingToReachPosition0 and WaitingToReachPosition1

For a clearer understanding of the code, let us break down the sequences executed within the patrol function for states 3 (⌚ WaitingToReachPosition0) and 4 (⌚ WaitingToReachPosition1):

1. When executing state 3 or state 4, increment the reference count for the captured unit.
2. If the reference count becomes zero, trigger an undefined behavior and halt the process. Proceed to the following step only if the reference count is greater than zero.
3. Prepare UnitGotoFuture and store the address of the goto closure environment within the closure environment.
4. Invoke the goto closure.
5. If the future is not ready, save the current state and return the function.
6. In case the future is ready, check the state of the goto closure.
7. Accordingly, decrement the reference count for the captured unit.
8. If the reference count drops to zero, deallocate the unit.

The function updates the state in the closure environment, and the function returns with either Poll::Pending or Poll::Ready in rax.

Annotated Patrol Closure Assembly Code

Now, we are ready to dig into the generated assembly code.

The Rust code for the patrol function is repeated here for easy cross-referencing with the assembly code.

```
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
    loop {
        goto(unit.clone(), poses[0]).await;
        goto(unit.clone(), poses[1]).await;
    }
}
```

Before diving into the assembly code, let us examine the input and output registers. The closure environment resides in rdi, while the task context is in rsi. The output, stored in rax, can be Poll::Pending or Poll::Ready. The following figure displays the memory the rdi register pointed to,

with annotations indicating each field's offset within the closure environment. It is important to note that the closure environments for `patrol` also encompass the closure environment for the `goto` closure.

00 poses	poses array captured from the environment
08 local_unit	local variable unit
16 local_poses	local poses array
24 unit	•
32 state	state to identify the current execution state
40 goto_unit_local	goto saves the local variable unit here
48 goto_target_pos_local	goto saves the local variable pos here
56 goto_unit	Caller passes Rc<RefCell<Unit>> for goto
64 goto_target_pos	Caller poses[0] or poses[1] for goto
68 goto_state	state to identify the current execution state for goto

00 strong	strong reference count
08 weak	weak reference count
16 borrow	Borrow flag
24 pos	Position

Input:

- `rdi`: The closure environment
- `rsi`: The `task_context`

Output:

- `rax`: `Poll::Pending` or `Poll::Ready`

```
playground::patrol::{{closure}}:
push    r15                ; Save the callee-saved registers
push    r14
push    rbx

mov     r14, rsi            ; Store the task_context in r14.
mov     rbx, rdi           ; Store the closure environment in rbx.
```

Jump to code block based on closure state

Index into the jump table using the state in the closure environment and jump to the code block corresponding to the state.

```
movzx   eax, byte ptr [rdi + 32] ; Load the state to select the block to
                                ; execute. The 32nd byte stores the state
                                ; of the closure environment.

lea     rcx, [rip + .LJTI58_0]   ; Load the address of the jump table in rcx.
                                ; The jump table is a list of offsets from
                                ; the start of the jump table to each block.

movsxd  rax, dword ptr [rcx + 4*rax] ; Get the jump offset from the entry
                                ; corresponding to the state. The index
                                ; in rax is multiplied by 4 because the
                                ; jump table is an array of 32-bit jump
                                ; offsets indexed by the state.

add     rax, rcx                ; Add the jump offset to the jump table to
                                ; determine the address of the block to execute.

jmp     rax                     ; Jump to the correct block.

.LBB58_1:
```

Part F Closures and Async/Await

 **Start (0):** Code executed before the first future is polled. At this point, `rbx` contains the closure environment, and `r14` contains the context.

```

mov rcx, qword ptr [rbx]      ; Load the captured poses array from
                               ; the closure environment.

mov rax, qword ptr [rbx + 24] ; Load the captured Rc<RefCell<Unit>>
                               ; from the closure environment.

mov qword ptr [rbx + 8], rax   ; Store the captured Rc<RefCell<Unit>>
                               ; in a local variable saved in the
                               ; closure environment.

mov qword ptr [rbx + 16], rcx  ; Store the captured poses array in a
                               ; local variable saved in the closure
                               ; environment.

jmp .LBB58_4                  ; Jump to the next block.

```

 **State (2):** Panic.

```

.LBB58_2:
lea rdi, [rip + str.1]
lea rdx, [rip + .L__unnamed_24]
mov esi, 34
call qword ptr [rip + core::panicking::panic@GOTPCREL]

```

;  **State (1):** Throw exception.

```

.LBB58_3:
ud2

```

 **Start (0):** Getting ready to wait for `poses[0]`.

At this point, `rbx` contains the closure environment, and `rax` points to `Rc<RefCell<Unit>>`. Increment the `Rc<RefCell<Unit>>` reference count.

```

.LBB58_4:
inc qword ptr [rax] ; Increment the strong reference count for
                    ; Rc<RefCell<Unit>>. The strong reference count is
                    ; stored in the first 8 bytes of the Rc<RefCell<Unit>>.

je .LBB58_28        ; If the reference count is 0, jump to error handling.

mov ecx, dword ptr [rbx + 16] ; Load poses[0] from the closure environment
                               ; Clone the Rc<RefCell<Unit>> to get a new
                               ; reference to the Unit stored in the
                               ; closure environment

mov qword ptr [rbx + 56], rax ; Copy the Rc<RefCell<Unit>> to the
                               ; goto::{{closure}} environment

mov dword ptr [rbx + 64], ecx ; Save the poses[0] to the
                               ; goto::{{closure}} environment

```

```
mov byte ptr [rbx + 68], 0 ; Set the initial state for the
                           ; goto:{{closure}} to 🚀 Goto::Start (0)
```

🕒 **WaitingToReachPosition0 (3):** Waiting to reach poses[0].

```
.LBB58_6:
lea r15, [rbx + 40] ; Load the address of the environment for
                   ; the goto:{{closure}}

mov rdi, r15 ; Load the address of the environment to the
             ; first argument register

mov rsi, r14 ; Load the task_context to the second
             ; argument register

call    playground::goto:{{closure}} ; Call the goto:{{closure}}

test    al, al ; Check if the future is ready
jne .LBB58_25 ; The Future is not ready.
           ; Save state and return
```

★ The future is ready. Continue executing the patrol closure.

```
movzx    eax, byte ptr [rbx + 68] ; Load the state of the goto closure

test     eax, eax ; Check if the goto's state is 0

je .LBB58_11 ; Goto's state is 0. Proceed to the next
             ; goto closure after decrementing the
             ; reference count for goto's unit at
             ; offset 56 in patrol:{{closure}}'s
             ; environment

cmp eax, 3 ; Check if the goto's state is 3

jne .LBB58_14 ; The goto's state is not 0 or 3, proceed
              ; to the next goto closure without
              ; decrementing the reference count
              ; for goto's unit
```

★ goto is in state (3). Decrement the reference count using the goto:{{closure}} environment's unit at offset 40 in patrol:{{closure}}'s environment.

```
mov rdi, qword ptr [r15] ; Load the Rc<RefCell<Unit>> from the
                          ; goto:{{closure}} environment

dec qword ptr [rdi] ; Decrement the reference count for unit

jne .LBB58_14 ; The reference count is not 0. Do not
              ; deallocate the unit at offset 40 in
              ; patrol:{{closure}}'s environment

jmp .LBB58_12 ; Reference count is 0. Jump to the next
              ; block to deallocate the unit
```

Part F Closures and Async/Await

```

.LBB58_11:

mov rdi, qword ptr [rbx + 56] ; Load Rc<RefCell<Unit>> from environment

dec qword ptr [rdi]          ; Decrement the reference count for unit

jne .LBB58_14                ; The reference count is not 0. Do not
                             ; deallocate the unit

.LBB58_12:

dec qword ptr [rdi + 8]      ; Decrement the reference count for unit
jne .LBB58_14                ; Reference count is not 0.
                             ; Jump to the next block

mov esi, 32                  ; Load the size of the allocation
mov edx, 8                   ; Load the alignment of the allocation

call    qword ptr [rip + __rust_dealloc@GOTPCREL] ; Deallocate the allocation

.LBB58_14:

mov rax, qword ptr [rbx + 8] ; Load Rc<RefCell<Unit>> from the environment

inc qword ptr [rax]          ; Increment the reference count for unit
je .LBB58_28                 ; The reference count is 0. Jump to
                             ; undefined behavior.

```

Prepare to wait for goto::`{{closure}}` environment.

```

mov ecx, dword ptr [rbx + 20] ; Load poses[1] from the environment

mov qword ptr [rbx + 56], rax ; Store the Rc<RefCell<Unit>> in the
                             ; goto::{{closure}} environment

mov dword ptr [rbx + 64], ecx ; Store the poses[1] in the
                             ; goto::{{closure}} environment

mov byte ptr [rbx + 68], 0    ; Store the goto closure's initial
                             ; state - 🚀 Goto::Start (0)

```

⌚ **WaitingToReachPosition1 (4):** Waiting to reach poses[1].

```

.LBB58_16:
lea r15, [rbx + 40] ; Get the address of the goto closure environment

mov rdi, r15        ; Load the address of the goto closure environment
                   ; to the first argument register

mov rsi, r14        ; Load the task_context to the second argument register

call    playground::goto::{{closure}} ; Call the goto closure

test    al, al      ; Check if the future is ready

```

```

jne .LBB58_26          ; The future is not ready yet. Save state and return

movzx  eax, byte ptr [rbx + 68] ; Load the goto closure state

test   eax, eax        ; Check if the state is 0
je     .LBB58_21        ; The state is 0. Jump to the next block

cmp    eax, 3          ; Check if goto's state is 3
jne    .LBB58_24        ; The state is not 3. Jump to the next block

mov    rdi, qword ptr [r15] ; Load the goto closure environment unit
dec    qword ptr [rdi]      ; Decrement the reference count for unit
jne    .LBB58_24          ; Reference count is not 0, jump to the next block

jmp    .LBB58_22        ; Reference count is 0. Jump to the next block

.LBB58_21:

mov    rdi, qword ptr [rbx + 56] ; Load Rc<RefCell<Unit>> from the environment
dec    qword ptr [rdi]          ; Decrement the strong reference count for unit
jne    .LBB58_24                ; The reference count is not 0. Skip
                                   ; deallocating the unit

.LBB58_22:

; Strong reference count is 0, decrement the weak reference count
dec    qword ptr [rdi + 8]      ; Decrement the weak reference count for unit

jne    .LBB58_24                ; The weak reference count is not 0. Jump to
                                   ; the next block

; The weak reference count is 0. Deallocate the unit
mov    esi, 32                  ; Size of unit
mov    edx, 8                   ; Alignment of unit

call   qword ptr [rip + __rust_dealloc@GOTPCREL] ; Deallocate the unit

.LBB58_24:

mov    rax, qword ptr [rbx + 8] ; rax = Rc<RefCell<Unit>> from the environment

jmp    .LBB58_4 ; Jump to the beginning of the loop to the code handling
                ; 🚀 Start (0). Note that the compiler does not explicitly
                ; update the state variable to 0. It just jumps to the
                ; beginning of the loop.

.LBB58_25:

mov    al, 3                    ; Set the state to "🕒 WaitingToReachPosition0 (3)" for the
                                ; next call to the function.
jmp    .LBB58_27                ; Jump to return

```

Part F Closures and Async/Await

.LBB58_26:

```
mov al, 4 ; Set the state to "🕒 WaitingToReachPosition1 (4)" for
; the next call to the function.
```

← The function is about to return. Store the updated state so that in the next call, it will continue from the correct code block.

.LBB58_27:

```
mov byte ptr [rbx + 32], al ; Store the updated state
mov al, 1 ; Return Poll::Pending

pop rbx
pop r14
pop r15
ret
```

✗ The reference count is 0 after an increment, jump to undefined behavior. This condition should never happen.

.LBB58_28:

```
ud2 ; Undefined instruction
ud2
jmp .LBB58_31 ; Drop memory and unwind the stack
```

.LBB58_31:

```
mov r14, rax
mov rdi, r15
call core::ptr::drop_in_place<playground::goto::{{closure}}>
mov rdi, qword ptr [rbx + 8]
call core::ptr::drop_in_place<playground::UnitGotoFuture>
mov byte ptr [rbx + 32], 2
mov rdi, r14
call _Unwind_Resume@PLT
ud2
```

🌂 Jump table for the patrol closure.

.LJTI58_0:

```
.long .LBB58_1-.LJTI58_0 ; 🚀 Start (0): Code executed before
; the first future is polled.
.long .LBB58_3-.LJTI58_0 ; 🛑 State 1: Throw exception.
.long .LBB58_2-.LJTI58_0 ; 🛑 State 2: Panic.
.long .LBB58_6-.LJTI58_0 ; 🕒 WaitingToReachPosition0 (3)
.long .LBB58_16-.LJTI58_0 ; 🕒 WaitingToReachPosition1 (4)
```

📁 Vtable for the patrol closure.

.L__unnamed_25:

```
; Destructor for the FnOnce trait object
```



```
.quad    core::ptr::drop_in_place<playground::patrol::{closure}>

; Size of object: 72 bytes (H). Alignment of object: 8 bytes (\b).
.asciz   "H\000\000\000\000\000\000\000\b\000\000\000\000\000\000"

; call_once method of the FnOnce trait
.quad    playground::patrol::{closure}}
```



Experiment: Executing Async Functions on `.await`

Code

Let us work with the following example in the [Rust playground](https://bit.ly/3rI1Hso) (<https://bit.ly/3rI1Hso>):

```
use std::time::Duration;
use tokio::time::sleep;

async fn async_fn() {
    println!("Async function execution started.");
    sleep(Duration::from_secs(2)).await;
    println!("Async function execution finished.");
}

#[tokio::main]
async fn main() {
    println!("Main function execution started.");
    let future = async_fn(); // Save the future returned by async_fn
    println!("Main function before awaiting the async function.");
    future.await; // Here we start the execution of async_fn
    println!("Main function execution finished.");
}
```

Run

Use the **RUN** button to run the code (top-left button in the Rust playground). You should see the following output:

```
Main function execution started.
Main function before awaiting the async function.
Async function execution started.
Async function execution finished.
Main function execution finished.
```

Explanation

This example demonstrates an essential aspect of Rust's asynchronous programming model: an `async` function does not start executing immediately upon being called. Instead, it returns a `Future`, a value representing a computation that will be completed in the future.

The key learning points are:

- Calling an `async` function returns a `Future` without executing the function's body at that moment.
- The returned `Future` symbolizes the ongoing computation of the `async` function, which does not initiate until you `await` the `Future`.

Part F Closures and Async/Await

- To start the execution of the `Future`, you use the `.await` keyword, as demonstrated in the `main` function with the `future.await` call.
- The `async` function's code, such as `print` statements and the delay in `async_fn`, remains unexecuted until the `Future` is awaited. This behavior is observable in the console output, where "Async function execution started" appears only after "Main function before awaiting the async function".

Explore the Assembly Code

1. Now set the **Optimization level** to **RELEASE** using the second drop-down menu from the left.
2. Switch the leftmost button to **ASM** to see the assembly code.

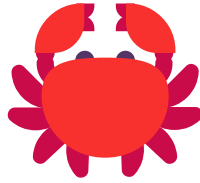
Some interesting searches to navigate the assembly code:

Search for	Description
<code>playground::main:</code>	The main function calls <code>main::{{closure}}</code> , which emerges from the expansion of <code>#[tokio::main]</code> . Search for a <code>playground::main::{{closure}}</code> call. Look for a call to <code>playground::main::{{closure}}</code>
<code>playground::main::{{closure}}</code>	The main function invokes a closure, <code>main::{{closure}}</code> , which begins with a jump table. Search for the call to <code>tokio::time::sleep</code> and trace the execution paths following the <code>sleep</code> function.



Key Takeaways

- The compiler generates a state machine for each nested `async` function. The state machine is a struct with a state variable and a closure environment. The state variable keeps track of the current state of the function. The closure environment stores the arguments and local variables of the function.
- The closure environment of the outer `async` function contains the closure environment of the inner `async` function. A large number of local variables and arguments can result in a notably larger closure environment.
- Even an `async` function with an infinite loop will return `Poll::Pending` when a future in the loop is pending. When the future becomes ready, the executor calls the function again, and the loop continues when the polled future returns `Poll::Ready`. The loop continues executing until it reaches a pending future. At this point, the function returns `Poll::Pending`. The executor will call the function again when the future becomes ready.
- The execution of an `async` function does not commence immediately upon its invocation. Instead, the function returns a `Future`, representing a computation that will be completed later. The returned `Future` embodies the ongoing computation of the `async` function, which only begins when you `await` it. We saw this behavior in the Rust playground example.



Chapter 20

Async Executors: Task Scheduling Insights

This chapter continues the previous chapters on Rust `async/await`. If you have not read them yet, we recommend you read them first:

- “*Async/Await: Internal State Machine Dynamics*” - `goto`
- “*Nested Async/Await: Desugaring to Loops*” - `patrol`

In this chapter, we will learn how the async executor schedules tasks. We will explore the workings of a basic async executor and its implementation in Rust. We will analyze the `async` scheduling of the `goto` and `patrol` functions from the previous chapters.

Async Executor Overview

Let us recap what we have learned about async executors in general and the simple async executor in particular.

💡 Learn More: Single Threaded Async Executor

A single-threaded async executor manages the execution of tasks in an asynchronous context using only a single thread. It operates by multiplexing multiple tasks onto the same thread and switching between them whenever a task voluntarily suspends its execution—usually because it is waiting for an I/O operation or a timer.

A single-threaded executor internally operates with an event loop and a task queue. It places a scheduled future for execution in the task queue. Entering the event loop, the executor repeatedly retrieves and polls futures from the queue. If a future is not ready—usually due to a pending I/O operation—it relinquishes control to the executor, allowing it to proceed to the next task. When a future becomes ready, the executor resolves it, enabling the caller of the async function to retrieve its result.

💡 Simple Async Local Executor

We will use the [simple async local executor](#) in our study of the async internals. The following table links the source code for both the executor and the example asynchronous code.

GitHub Link	Description
https://github.com/enlightware/simple-async-local-executor	enlightware's single-threaded polling executor.
https://github.com/eventhelix/simple-async-local-executor	eventhelix's fork of enlightware's single-threaded polling executor.
https://github.com/eventhelix/simple-async-local-executor/blob/main/examples/game-units.rs	game-units.rs example from eventhelix's fork of the single-threaded polling executor.

The code in the preceding links is dual licensed under the MIT and Apache 2.0 licenses.

goto and patrol Functions

To recap, here are the goto and patrol functions from the previous chapters:

```
#[derive(Default)]
struct Unit {
    /// The 1-D position of the unit. The unit can only move along this axis.
    pub pos: i32,
}
type UnitRef = Rc<RefCell<Unit>>;

/// A future that will move the unit towards `target_pos` at each step,
/// and complete when the unit has reached that position.
struct UnitGotoFuture {
    unit: UnitRef,
    target_pos: i32,
}

impl Future for UnitGotoFuture {
    type Output = ();
    fn poll(self: Pin<&mut Self>, _cx: &mut Context) -> Poll<Self::Output> {
        let unit_pos = self.unit.borrow().pos;
        if unit_pos == self.target_pos {
            Poll::Ready(())
        } else {
            self.unit.borrow_mut().pos
                += (self.target_pos - unit_pos).signum();
            Poll::Pending
        }
    }
}

/// Helper async function to write unit behavior nicely
async fn goto(unit: UnitRef, pos: i32) {
    UnitGotoFuture {
        unit,
        target_pos: pos,
    }
    .await;
}

/// Let a unit go back and forth between two positions
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
```

```
loop {  
    goto(unit.clone(), poses[0]).await;  
    goto(unit.clone(), poses[1]).await;  
}
```

Async Executors: Scheduling and Task Management

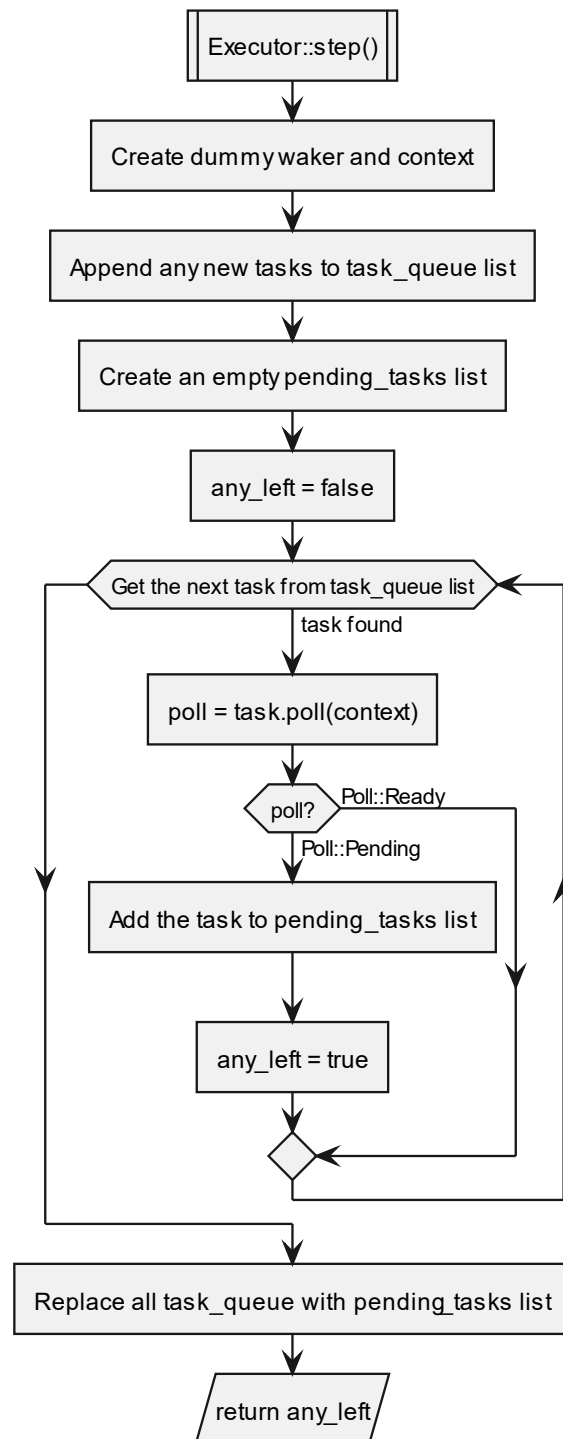
An async executor is responsible for running async tasks. It schedules the tasks and wakes them when they are ready to proceed.

Let us look at a simple async executor. In the following code, the `main` function spawns async tasks and then starts executing the tasks. Typically, an async executor runs the tasks until they are complete. In our example, we simulate the executor by calling the `Executor::step()` method in a loop. We limit the number of steps to 30 to run the example in the Rust playground.

```
fn main() {  
    let executor = Executor::default();  
    let units: [UnitRef; 2] = Default::default();  
    executor.spawn(patrol(units[0].clone(), [-5, 5]));  
    // executor.spawn(patrol(units[1].clone(), [-1, 1]));  
    for _ in 0..30 {  
        executor.step();  
    }  
}
```

The `Executor::step()` method schedules async tasks. When the function starts, it will look for any new spawned tasks and add them to the `task_queue`. Then, it will loop through the `task_queue` and execute the tasks. The executor will add a task to the `pending_tasks` list if it is not ready. The next step will use the `pending_tasks` list as the `task_queue`. The `Executor::step()` method returns true if any tasks await completion. The flowchart of the `Executor::step()` method follows:

Part F Closures and Async/Await



The Rust code for the `Executor::step()` method follows:

```

pub fn step(&self) -> bool {
    // Dummy waker and context (not used as we poll all tasks)
    let waker = dummy_waker();
    let mut context = Context::from_waker(&waker);
    // Append new tasks created since the last step into the task queue
    let mut task_queue = self.inner.task_queue.borrow_mut();
    task_queue.append(&mut self.inner.new_tasks.borrow_mut());

    // Loop over all tasks and poll them. If a task is not ready,

```

```
// add it to the pending tasks.
let mut pending_tasks = Vec::new();
let mut any_left = false;
for mut task in task_queue.drain(..) {
    match task.poll(&mut context) {
        Poll::Ready(()) => {} // task done
        Poll::Pending => {
            pending_tasks.push(task);
            any_left = true;
        }
    }
}
// Keep pending tasks for the next step
*task_queue = pending_tasks;
any_left // Return true if there are still tasks to run
}
```

Scheduling and Execution of `goto` and `patrol` Tasks in the Async Executor

Now that we have seen how the async executor works, let us learn how the executor schedules the `goto` and `patrol` functions. We will use the following sequence diagrams to understand how the executor schedules the async tasks.

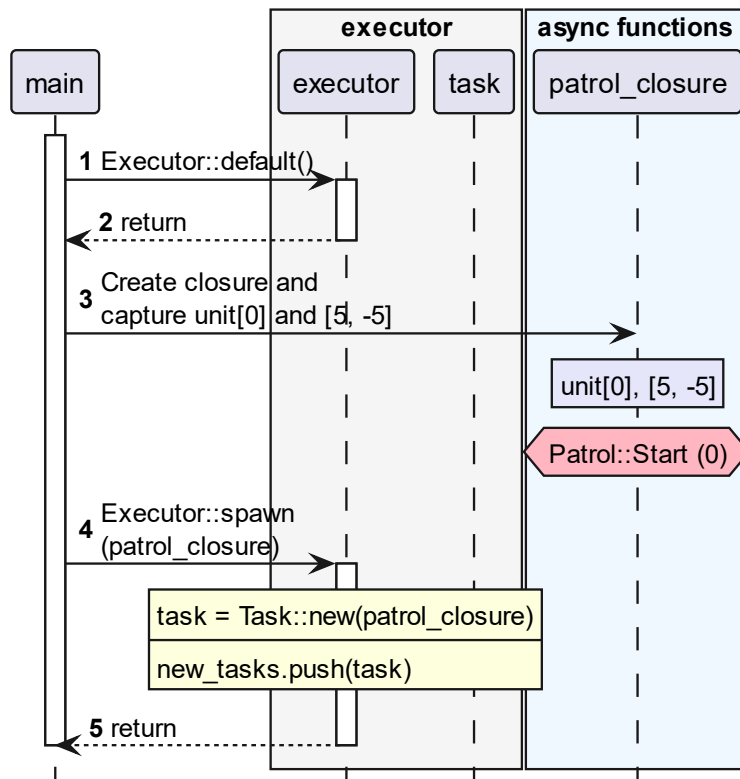
The following parts comprise the complete flow:

1. [Spawn an Async Task](#)
2. [Start Executing Async Tasks](#)
3. [Continue Execution of Async Tasks](#)

Spawn an Async Task

The following sequence diagram illustrates an executor spawning a task and executing it. An explanation of the numbered steps follows the diagram.

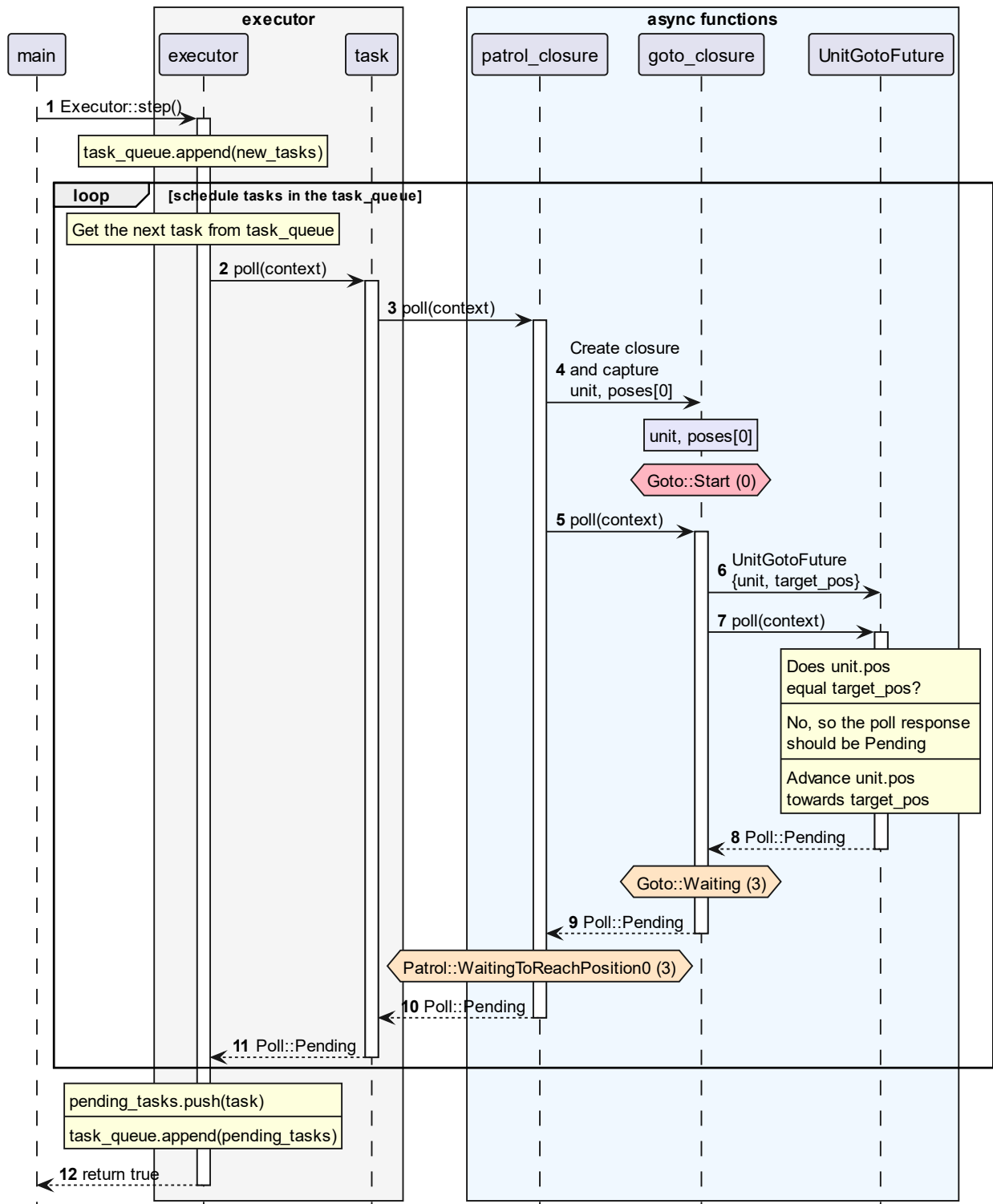
Part F Closures and Async/Await



1. The `main` function creates the executor using the `default` method.
2. The executor returns to the `main`.
3. The `main` function creates the `patrol_closure`, capturing `unit[0]` and `[5, -5]` in the closure environment and sets its initial state to 🚀 `Patrol::Start(0)`.
4. The executor creates a new task from the closure and stores it in the `new_tasks` list.
5. The executor returns to the `main` function.

Start Executing Async Tasks

The following sequence diagram shows how the executor executes the `patrol` task. A detailed explanation of the numbered steps follows the diagram.



1. main calls `Executor::step()` to execute the first task.
 - The executor looks for new spawned tasks and adds them to the `task_queue`.
2. The executor cycles through the `task_queue`, running the tasks it finds. In this case, it discovers a single task within the `task_queue`. Subsequently, it invokes the `poll` method on this task.
3. The task scheduled here contains the `patrol_closure`. The task involves the `poll` method of the `patrol_closure`.

Part F Closures and Async/Await

4. The `patrol_closure`, in the 🚀 `Patrol::Start` state, begins from the function's start. The closure environment captures `unit` and `poses[0]` to create the `goto_closure`, setting its initial state to 🚀 `Goto::Start`.

```

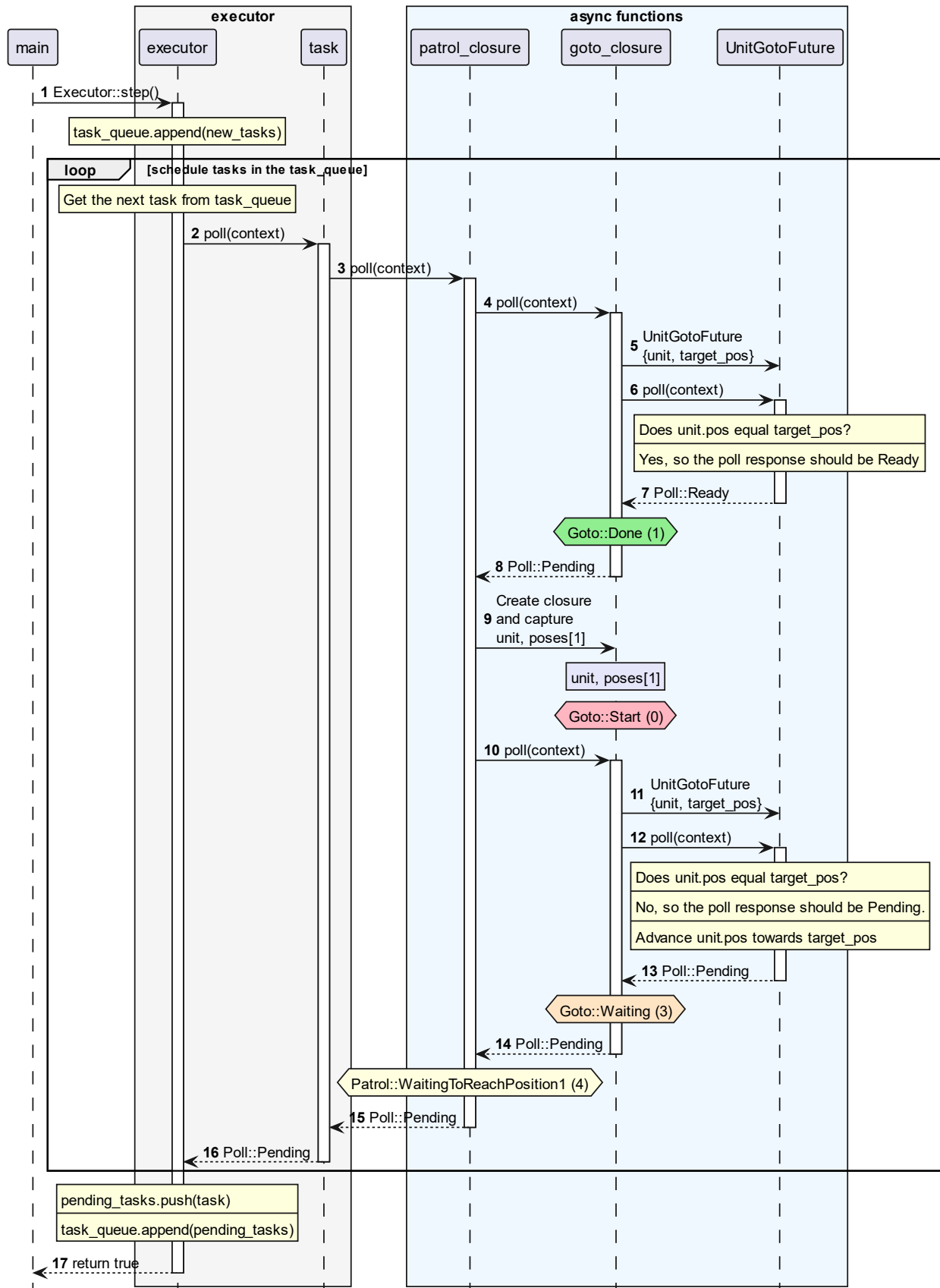
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
  loop {
    ➡ goto(unit.clone(), poses[0]).await;
      goto(unit.clone(), poses[1]).await;
  }
}

```

5. The `patrol_closure` polls the `goto_closure` to find if it has reached the first position specified in `poses[0]`. The poll method call of the `goto_closure` results from the `.await` marked with the ➡ arrow in the preceding code snippet.
6. The `goto_closure` creates a `UnitGotoFuture`.
7. The `goto_closure` polls `UnitGotoFuture`. The poll method of `UnitGotoFuture` checks if the `unit` has reached the `pos`. In this case, the `unit` has yet to reach the `pos` position.
8. The poll method returns `Poll::Pending` to the `goto_closure`. The `goto_closure` sets its state to ⌚ `Goto::Waiting`.
9. The `goto_closure` returns `Poll::Pending` to the `patrol_closure`. The `patrol_closure` sets its state to ⌚ `Patrol::WaitingToReachPosition0`.
10. The `patrol_closure` returns `Poll::Pending` to the task.
11. The task returns `Poll::Pending` to the executor.
12. Since the task is not ready, the executor adds the task to the `pending_tasks` list for future scheduling. The executor returns to the `Executor::step()` caller.

Continue Execution of Async Tasks

The following sequence diagram shows how the executor continues the execution of the `patrol` task. A detailed explanation of the numbered steps follows the diagram.



The executor continues to run if any events are scheduled. In our example, the scheduler calls `Executor::step()` in a loop. At each step, polling of the `goto_closure` moves the unit closer to the pos. Now we will see how the `goto_closure` returns `Poll::Ready` to the `patrol_closure` when the unit reaches the pos. We start at step 6 in the preceding sequence diagram.

Part F Closures and Async/Await

- Once the unit reaches the pos, the `goto_closure` returns `Poll::Ready` to the `patrol_closure`. The `patrol_closure` then polls the `goto_closure` again. The  shows the `.await` point in the `patrol_closure`.

```
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
    loop {
         goto(unit.clone(), poses[0]).await;
        goto(unit.clone(), poses[1]).await;
    }
}
```

- The `UnitGotoFuture` returns `Poll::Ready` to the `goto_closure`.
- The `goto_closure` returns `Poll::Ready` to the `patrol_closure`. The `goto_closure` sets its state to  `Goto::Done`.
- The `Poll::Ready` return helps us advance the execution until we hit the next `.await` point ( in the following code snippet). The `goto_closure` captures the unit and `poses[1]` in the closure environment. The initial state of the closure is set to  `Goto::Start`.

```
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
    loop {
        goto(unit.clone(), poses[0]).await;
         goto(unit.clone(), poses[1]).await;
    }
}
```

- The `patrol_closure` triggers the following poll to move the unit to the next pos position.

The `patrol_closure`, now at the next `.await` point, enters the  `Patrol::WaitingToReachPosition1` state. It then returns `Poll::Pending` to the task, which returns `Poll::Pending` to the executor. Subsequently, the executor schedules the task for later by adding it to the `pending_tasks` list and then returns to the `Executor::step()` caller.

The `patrol` function has an infinite loop; returning `Poll::Ready` from the `goto_closure` does not terminate the `patrol_closure`. The `patrol_closure` will continue to poll the `goto_closure` until the unit reaches the next pos position.



Experiment: Spawning Two Async Tasks; Infinite Loops in Async Functions

We have created a [Rust playground](https://bit.ly/45geQYh) (<https://bit.ly/45geQYh>) to experiment with the code. You can modify the code and run it to see the trace output.

Here is the main function that implements the executor. We see that the `units` array has two units. We have spawned a task corresponding to the first unit. We have commented out the code to spawn the second task.

```
pub fn main() {
    let executor = Executor::default();
    let units: [UnitRef; 2] = Default::default();
    executor.spawn(patrol(units[0].clone(), [-5, 5]));
    // executor.spawn(patrol(units[1].clone(), [-1, 1]));
    let print_poses = || {
```

```
println!(
    "Unit poses: {}",
    units
        .iter()
        .map(|unit| unit.borrow().pos.to_string())
        .collect::

```

Now, let us conduct the following experiments:

- [Spawning Two Tasks](#)
- [Confirming That the patrol Future Never Returns](#) poll::Ready

Spawning Two Tasks

1. Click the **RUN ►** button on the top left of the [Rust playground](#) window. You will see the following output. The units[0] unit is moving between -5 and 5. The units[1] unit is not moving because we have not yet spawned the second async task.

```
Unit poses: 0, 0
Unit poses: -1, 0
Unit poses: -2, 0
Unit poses: -3, 0
Unit poses: -4, 0
Unit poses: -5, 0
Unit poses: -4, 0
Unit poses: -3, 0
Unit poses: -2, 0
Unit poses: -1, 0
```

2. Now scroll to the end of the code in the [Rust playground](#) and uncomment the following line to spawn the second async task.

```
executor.spawn(patrol(units[1].clone(), [-1,1]));
```

3. To run the code, click the **RUN ►** button on the top left. The units[1] unit moves between -5 and 5. The units[1] unit moves between -1 and 1. You will see the following output.

```
Unit poses: 0, 0
Unit poses: -1, -1
Unit poses: -2, 0
Unit poses: -3, 1
Unit poses: -4, 0
Unit poses: -5, -1
Unit poses: -4, 0
Unit poses: -3, 1
```

Part F Closures and Async/Await

```
Unit poses: -2, 0
Unit poses: -1, -1
```

This example shows that the two spawned tasks run independently in a single thread. The excerpt from the `Executor::step` method shows how the executor runs multiple tasks. The executor calls each task's `poll` function in a loop. If the task is not ready, the executor pushes the task to the end of the queue. If the task is ready, the executor discards the task.

```
pub fn step(&self) -> bool {
// ⚡--- snip ---
    let mut pending_tasks = Vec::new();
    let mut any_left = false;
    for mut task in task_queue.drain(..) {
        match task.poll(&mut context) {
            Poll::Ready(()) => {} // task done
            Poll::Pending => {
                pending_tasks.push(task);
                any_left = true;
            }
        }
    }
}
// ⚡--- snip ---
}
```

Confirming That the `patrol` Future Never Returns `Poll::Ready`

In our example, the `patrol` function never returns `Poll::Ready` because when an `await` on `goto()` returns `Poll::Ready`, we change the target position and `await` the `goto()` future for the new target position.

Comments in the following code snippet show the state of the `patrol` function when the `goto` future returns `Poll::Ready`.

```
async fn patrol(unit: UnitRef, poses: [i32; 2]) {
    loop {
        goto(unit.clone(), poses[0]).await;
        // We reach here only when the unit reaches the target position
        // poses[0] and the goto future poll returns Poll::Ready. If the
        // goto future poll had returned Poll::Pending we would have
        // returned from the patrol closure to the executor after saving
        // the state ⌚ WaitingToReachPosition0 in the closure.

        goto(unit.clone(), poses[1]).await;
        // We reach here only when the unit reaches the target position
        // poses[1] and the goto future poll returns Poll::Ready. If the
        // goto future poll had returned Poll::Pending we would have
        // returned from the patrol closure to the executor after saving
        // the state ⌚ WaitingToReachPosition1 in the closure.
    }
}
```

We can confirm this by adding prints in the `step` function.

```

pub fn step(&self) -> bool {
// ⌘--- snip ---
    let mut pending_tasks = Vec::new();
    let mut any_left = false;
    for mut task in task_queue.drain(..) {
        match task.poll(&mut context) {
            Poll::Ready(()) => {
                println!("Poll::Ready\n");
            } // task done
            Poll::Pending => {
                pending_tasks.push(task);
                println!("Poll::Pending\n");
                any_left = true;
            }
        }
    }
}
// ⌘--- snip ---
}

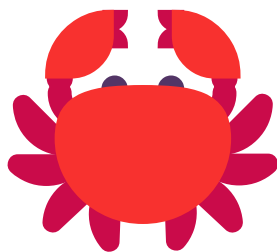
```

You can run the code in the [Rust playground](https://bit.ly/3YqcWSv) (<https://bit.ly/3YqcWSv>). You will see that `Poll::Ready` is never printed.

Key Takeaways

- Spawning an async task has nothing to do with the operating system. It is just a closure scheduled to be executed by the executor.
- A `.await` suspends the execution of an async task if the future is not ready. The executor will call the `poll` function again to check if the future is ready.
- The method returns `Poll::Pending` if the future is not ready. It returns `Poll::Ready` if the future is ready.
- If `await` is called inside an infinite loop, the function will never return `Poll::Ready` as it continues in the loop. Even a function with an infinite loop will return `Poll::Pending` if the future is not ready.

Note: Unlike the real-world async executors, the simple async scheduler in this chapter calls the `poll` method of a future at every step. In a practical scenario, an executor will invoke the `poll` method only when the future is ready to progress.



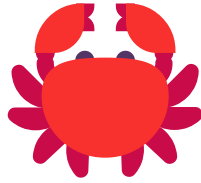
Part G

Transformations and Takeaways



Rust Under the Hood Companion (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews



Chapter 21

Transforming into String Slice Vector

We are nearing the end of our exploration into Rust internals. This chapter and the next reinforce the concepts learned throughout the book.

This chapter builds on our earlier discussion on how vectors translate into assembly (*“Vector Iteration in Assembly”* chapter). This time, we will explore the generated assembly for mapping a `bool` vector to a string slice vector.

Example Code: Map a Vector of Bools to a Vector of String Slices

The following code shows two functions:

- `convert<A,B>` function accepts a vector of type `A` and returns a vector of type `B`. The function accepts a closure that accepts an element of type `A` and returns an element of type `B`. Calling the `map` function on the vector iterator performs the conversion. Note that `convert<A,B>` is a generic function that generates no code.
- `convert_bool_vec_to_static_str_vec` function uses the `convert<A,B>` function to convert a vector of bools to a vector of string slices. The function accepts a vector of bools and returns a vector of string slices. The function calls the `convert<A,B>` function with a closure that converts a bool to a string slice.

```
pub fn convert<A,B> (v: Vec<A>, f: impl Fn(A) -> B) -> Vec<B> {
    v.into_iter().map(f).collect()
}

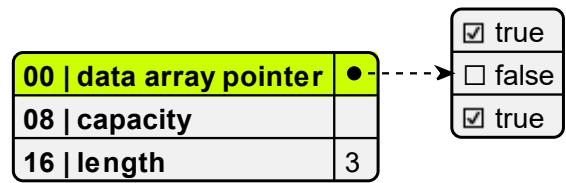
pub fn convert_bool_vec_to_static_str_vec(v: Vec<bool>) -> Vec<& 'static str> {
    convert(v, |n| if n {"true"} else {"false"})
}
```

Visualizing the Input and Output Vectors

To better comprehend the generated assembly code, let us first understand the input and output vectors of the `convert_bool_vec_to_static_str_vec` function.

Part G Transformations and Takeaways

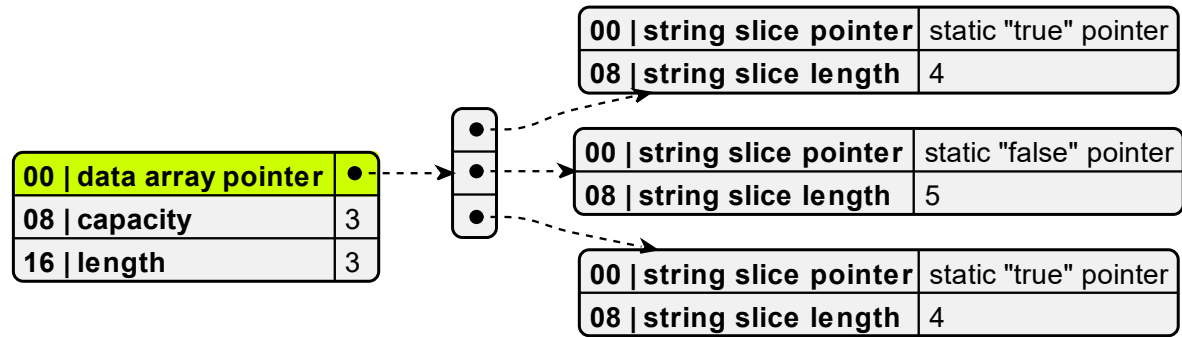
The input vector passed to the `convert_bool_vec_to_static_str_vec` function is a vector of bools. The following figure shows the memory organization of the input vector.



The input vector is a struct with three fields:

- An **8-byte data array pointer** points to the start of a bool array on the heap, where each bool occupies a single byte.
- An **8-byte capacity field** contains the length of the data array on the heap.
- An **8-byte length field** contains the number of elements in the vector.

The output vector of the `convert_bool_vec_to_static_str_vec` function is a vector of string slices. The following figure shows the memory layout of the output vector.

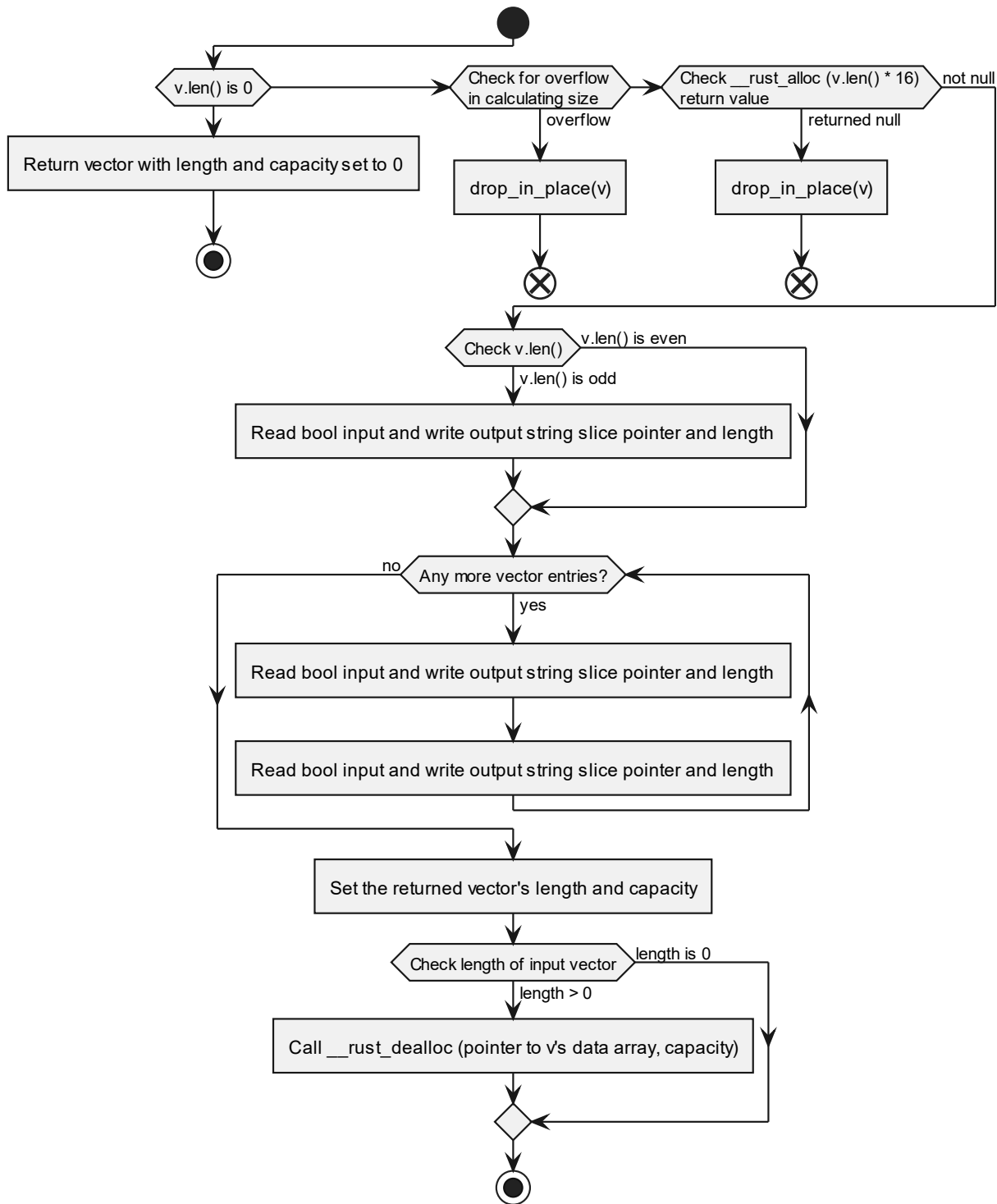


The three fields of the output vector are:

- An **8-byte data array pointer** points to the start of a data array on the heap, leading to an `&'static str` array. This `&'static str` represents a string slice with a static lifetime. The `&str` comprises a pointer to the string and the byte length of the `str`.
- An **8-byte capacity field** contains the length of the data array on the heap.
- An **8-byte length field** contains the number of elements in the vector.

Assembly Code Flowchart: Bool Vector to Vector of String Slices

Before we dive into the assembly code, let us first understand the high-level flow of the generated code. The following flowchart shows the high-level flow of the generated assembly code.



Overview of the Assembly Code

We are now ready to dive into the assembly code. Here is an overview of what we will see in the generated assembly code:

Overflow and Memory Allocation Failure Handling

The compiler will generate a panic and drop the input vector if there is an overflow in the size computation or if the allocation fails.

Length-Based Optimization

The generated code takes the following input vector length-based decisions:

- When the input vector length is 0, the output vector also has a length of 0, eliminating the need for heap allocation.
- If the input vector length is non-zero, the output vector matches this length and allocates its array on the heap.
 - If the input vector length is odd, the generated code handles one iteration and then falls through to the code for even length handling.
 - The generated code handles two iterations per loop entry if the input vector length is even. This loop unrolling reduces the branching penalty of the loops.

Eliminating the Need for a Branch

When examining the code, the initial expectation might be that it evaluates each `bool` in the input array. Based on this evaluation, the code should generate a pointer to the corresponding string slice (“true” or “false”) and subsequently update the slice’s length.

However, closer inspection shows the code omits the direct `bool` condition check within its loop body. Instead, it adopts an alternative, more optimized approach as detailed below:

1. **Conditional Movement:** The process of copying the “true” or “false” string slice state to the output vector array leverages the `cmove` (conditional move) instruction.
2. **Length Computation:** An XOR operation between the number 5 (represented in binary as `0b101`) and the value of the `bool` (0 or 1) determines the length of the string slice. This computation might seem a tad non-intuitive. The table below offers a breakdown:

bool value	XOR with <code>0b101</code> (equals 5)	Resulting String slice length
0	<code>0b101</code> (5)	5 (corresponding to “false”)
1	<code>0b100</code> (4)	4 (corresponding to “true”)

Cleaning up the Input Vector on Exit from the Function

The compiler generates a call to `__rust_dealloc` to free the input vector. Note that the function owns the input vector, so it is responsible for freeing up the memory allocated for the input vector array just before the function returns.

Annotated Assembly: Bool Vector to a Vector of String Slices

We are now ready to dive into the assembly code.

12
34 Input:

- `rdi` points to the output vector. The caller is responsible for reserving space for the output vector struct. The called function allocates memory for the output vector’s data array.
- `rsi` points to the input vector `v`

12
34 Output:

- `rax`: Address of the output vector

```
example::convert_bool_vec_to_static_str_vec:  
push    rbp                ; Save rbp
```

```

push    r15                ; Save r15
push    r14                ; Save r14
push    r13                ; Save r13
push    r12                ; Save r12
push    rbx                ; Save rbx
sub     rsp, 40             ; Allocate 40 bytes for local variables

```

★ Setting up registers based on the input vector v.

```

mov     rbp, rdi            ; rbp = address of the output vector
mov     r12, qword ptr [rsi] ; Load the address of the input
                                ; vector v's data array into r12
mov     r10, qword ptr [rsi + 8] ; Load the capacity of the input vector v
                                ; into r10
mov     r14, qword ptr [rsi + 16] ; Load the length of the input vector v
                                ; into r14
lea     r15, [r12 + r14]    ; Load the address that points after the
                                ; last element.

```

★ Save the input vector v's data array address, capacity, and length into the stack.

```

mov     qword ptr [rsp + 8], r12 ; Store the address of vector v's data
                                ; array into the stack
mov     qword ptr [rsp + 16], r10 ; Store the capacity of vector v into the
                                ; stack frame
mov     qword ptr [rsp + 24], r12 ; Store the address of vector v's data
                                ; array into the stack
mov     qword ptr [rsp + 32], r15 ; Store the address of the end of vector
                                ; v's data array into the stack

```

0 Check the length of vector v; if it is 0, set the output vector length to 0 and return from the function.

```

test    r14, r14           ; Test if the length of vector v is 0
je      .LBB1_1             ; Jump if the length of vector v is 0

```

● Check if this iteration will result in a capacity overflow.

```

xor     ebx, ebx           ; Set ebx to 0
mov     rax, r14           ; Set rax to the length of vector v
shr     rax, 59            ; The length of an array should not
                                ; exceed 2^59 bits
                                ; Move the uppermost nibble of the
                                ; length into lower bits
sete    al                ; Set al to 1 if the uppermost nibble in
                                ; the address is 0

```

Part G Transformations and Takeaways

```
jne    .LBB1_3           ; Jump if the length of vector v is
                        ; greater than 2^59 bytes
```

0 No capacity overflows. Now check if the length is 0.

```
mov     qword ptr [rsp], r10 ; Store the capacity of vector v on the stack
mov     r13, r14             ; Set r13 to the length of vector v
shl     r13, 4               ; Convert the length of vector v to bytes

mov     bl, al               ; Set bl to al (which happens to be 1 if the
                        ; length of vector v is less than 2^59 bytes)

shl     rbx, 3               ; Shift rbx to the left by 3 bits so that rbx
                        ; is set to 8.

test    r13, r13             ; Check if the number of bytes needed for the
                        ; output vector is 0

je      .LBB1_6              ; Jump to .LBB1_6 if no bytes are needed
```

★ Vector is greater than 0 bytes. Allocate memory for the output vector.

```
mov     rdi, r13             ; Set rdi = number of bytes needed for the output vector
mov     rsi, rbx             ; Set rsi to 8 (the alignment of the output vector)

call    qword ptr [rip + __rust_alloc@GOTPCREL] ; Call __rust_alloc

test    rax, rax             ; Test if the allocation failed
je      .LBB1_9              ; Jump to .LBB1_9 if rax is 0 (allocation failed)
```

★ Allocation succeeded. Now, set the address of the output vector's data array to rax.

★ Check if the length of vector v is odd. Handle one entry before entering the even loop.

```
.LBB1_10:
mov     qword ptr [rbp], rax ; Address of the output vector = rax

mov     qword ptr [rbp + 8], r14 ; Set the capacity of the output vector to
                        ; the length of vector v

mov     qword ptr [rbp + 16], 0 ; Set the length of the output vector to 0

test    r14b, 1              ; Test if the length of vector v is odd

mov     r10, qword ptr [rsp] ; Set r10 to the length of vector v

jne     .LBB1_12              ; Jump to .LBB1_12 if the length of
                        ; vector v is odd
```

★ The capacity of vector v is even.

```
mov     rdx, r12             ; Set rdx to the address of vector v's data array
lea     r8, [rbp + 16]       ; Set r8 = address of the output vector's length
```

★ Check if the vector length is 1.


```

cmp     r14, 1      ; Compare the length of vector v to 1
jne     .LBB1_14    ; Jump to .LBB1_14 if the length of vector v is not 1
jmp     .LBB1_17    ; Jump to .LBB1_17 if the length of vector v is 1
                        ; Length of vector v is 1, so the loop is not entered

```

0 The length of vector v is 0.

```

.LBB1_1:
mov     qword ptr [rbp], 8      ; Set the address of the output
                                   ; vector to 8

lea     r8, [rbp + 16]         ; Set r8 to the address of the length
                                   ; of the output vector

xorps   xmm0, xmm0            ; Set xmm0 to 0

movups  xmmword ptr [rbp + 8], xmm0 ; Set the length and capacity of the
                                   ; output vector to 0

jmp     .LBB1_17              ; Jump to .LBB1_17

```

★ The length of the output vector is odd, so one loop entry is performed before entering the loop that handles an even number of entries.

```

.LBB1_12:
movzx   ecx, byte ptr [r12]    ; Set ecx to the value of the first element of
                                   ; input vector v's bool data array

and     ecx, 1                 ; Mask of the upper bits of ecx to ensure that
                                   ; the value is 0 or 1

```

★ Initialize the length and pointer in the output vector entry. The compiler uses an XOR trick to determine the length of the slice.

- if true: $1 \text{ XOR } 5 = 4$ (length of "true").
- If false: $0 \text{ XOR } 5 = 5$ (length of "false").

The compiler uses this trick to avoid a branch.

```

mov     rsi, rcx              ; Set rsi to ecx

xor     rsi, 5                ; Set rsi to 5 if rsi is 0, otherwise set
                                   ; rsi to 4

test    rcx, rcx              ; Test if the first bit of vector v's
                                   ; data array is 0

lea     rcx, [rip + .L__unnamed_1] ; Set rcx to the address of "false"
lea     rdi, [rip + .L__unnamed_2] ; Set rdi to the address of "true"

```

★ The following conditional move makes rdi point to "true" if the bool value is 1. Otherwise, rdi points to "false".

```

cmov     rdi, rcx              ; rdi points to "false" if input
                                   ; element's entry is 0, otherwise,

```

Part G Transformations and Takeaways

```

                                ; it points to "true"
lea    rdx, [r12 + 1]          ; Set rcx = address of the next element
                                ; of input vector v
mov     qword ptr [rax], rdi    ; Set the address of the mapped string in
                                ; the string slice
mov     qword ptr [rax + 8], rsi ; Set the length of the string slice
add     rax, 16                ; Point to the next slice in the output
                                ; data array
lea     r8, [rbp + 16]         ; Set r8 to the address of the output
                                ; vector's data array
cmp     r14, 1                 ; Compare the length of vector v to 1
je      .LBB1_17               ; Jump to .LBB1_17 if the length of
                                ; the vector v is 1 (we are done)

```

🌀 ● **Begin the loop** to process two entries per iteration, targeting an even number of entries.

```

.LBB1_14:
mov     r9, rbp                ; Set r9 to the address of output vector
lea     rsi, [rip + .L__unnamed_1] ; Set rsi to the address of "false"
lea     rbp, [rip + .L__unnamed_2] ; Set rbp to the address of "true"

```

🏷️ Processing of the first entry within the iteration.

```

.LBB1_15:
movzx   ebx, byte ptr [rdx]    ; Set ebx to the first byte of input
                                ; vector v's data array
and     ebx, 1                 ; Mask off the upper bits to get the bool value

```

★ The code must initialize the length and pointer in the output vector entry. The compiler uses the XOR trick we saw earlier to determine the length of the output string slice.

- if true: $1 \text{ XOR } 5 = 4$ (length of "true").
- If false: $0 \text{ XOR } 5 = 5$ (length of "false").

```

mov     rcx, rbx               ; Set rcx to the bool value mapped to 0 or 1
xor     rcx, 5                 ; Set rcx to 5 if rcx is 0. Otherwise, set rcx to 4
test    rbx, rbx               ; Set the condition code bits based on the
                                ; content of bool value
mov     rdi, rbp               ; Set rdi to point to "true".

```

★ The following conditional move makes rdi point to "false" if the bool value is 0. Otherwise, rdi points to "true".

```

cmov     rdi, rsi           ; Set rdi to point to "false" if bool value
                                ; is 0. Otherwise, set rdi to point to "true"
mov      qword ptr [rax], rdi ; Copy the address of "true" or "false"
                                ; to the string slice
mov      qword ptr [rax + 8], rcx ; Set the length of the string slice

```

 Processing of the second entry within the iteration

```

movzx    ecx, byte ptr [rdx + 1] ; Set ecx to the next element of the input
                                ; vector v's data array

and      ecx, 1                ; Mask off the upper bits to convert bool
                                ; into 0 or 1

mov      rdi, rcx               ; Move the bool's value to rdi
xor      rdi, 5                 ; Convert the bool's value into the length
                                ; using the XOR trick

test     rcx, rcx               ; Set the condition codes based on the
                                ; bool's value

mov      rcx, rbp               ; rcx points to "true"

cmov     rcx, rsi               ; rcx points to "false" if the bool's
                                ; value is 0

mov      qword ptr [rax + 16], rcx ; Copy the address of "true" or "false"
                                ; to the string slice

mov      qword ptr [rax + 24], rdi ; Set the length of the string slice

```

 Moving to the next iteration.

```

add      rdx, 2                 ; Move the input vector past the two
                                ; entries we have just processed.

add      rax, 32                ; Move the output vector pointer past the
                                ; two entries we have just processed.

cmp      rdx, r15               ; Check if we have processed all the
                                ; entries of the input vector

jne      .LBB1_15               ; Jump to .LBB1_15 if more entries need
                                ; to be processed

```

  **End the loop** for an even number of entries.

```

mov      rbp, r9                ; Set rbp to the address of the output vector

```

★ Returning when the length is 0 or 1.

```

.LBB1_17:
mov      qword ptr [r8], r14    ; Set the length of the output vector to the
                                ; length of vector v

```

Part G Transformations and Takeaways

```
test    r10, r10          ; Test if the length of vector v is 0
je      .LBB1_19          ; Jump if the length of vector v is 0
```

💡 **Aside:** The compiler uses the following three instructions to set `rdx` to 1 as the call to `__rust_dealloc` expects the alignment of the vector to be in `rdx`. The alignment of the vector is 1 as the vector's data array is a `bool` array. The compiler does this by setting `rdx` to the bitwise-not of the length of the vector and then shifting `rdx` to the right by 63 bits. This approach works as the most significant bit of the length in `rdx` is guaranteed to be 0 as the compiler has already checked for length overflow.

```
mov     rdx, r10          ; Set rdx to the length of vector v
not     rdx               ; Set rdx to the bitwise not of rdx
shr     rdx, 63           ; Set rdx to the uppermost bit of rdx
```

♻️ Free the input vector `v`'s buffer, as this function owns the vector and `v` is going out of scope.

```
mov     rdi, r12          ; Set rdi to the address of input vector v's data array
mov     rsi, r10          ; Set rsi to the length of input vector v

call    qword ptr [rip + __rust_dealloc@GOTPCREL] ; Call __rust_dealloc
```

← **BACK** Return the output vector.

```
.LBB1_19:
mov     rax, rbp          ; Set rax to the address of the output vector
add     rsp, 40           ; Set rsp to the address of the first byte of the output
                        ; vector's data array

pop     rbx               ; Restore rbx
pop     r12               ; Restore r12
pop     r13               ; Restore r13
pop     r14               ; Restore r14
pop     r15               ; Restore r15
pop     rbp               ; Restore rbp
ret
```

0 The following code handles the case when the vector `v`'s length is 0.

```
.LBB1_6:
mov     rax, rbx          ; Set rax to rbx

test    rax, rax          ; Test if rbx is 0
jne     .LBB1_10          ; Jump to .LBB1_10 if rbx is not 0
```

✗ Memory allocation for the output vector has failed, so signal an error.

```
.LBB1_9:
mov     rdi, r13          ; Set rdi to r13
mov     rsi, rbx          ; Set rsi to rbx
call    qword ptr [rip + alloc::alloc::handle_alloc_error@GOTPCREL]
jmp     .LBB1_4           ; Jump to .LBB1_4
```

● Capacity overflow has occurred, so signal an error.

```
.LBB1_3:
call    qword ptr [rip + alloc::raw_vec::capacity_overflow@GOTPCREL]
```

💀 Common logic for exception handling.

```
.LBB1_4:
ud2                ; Halt execution
mov     rbx, rax    ; Set rbx to rax
lea     rdi, [rsp + 8] ; Set rdi to the address of the first
                    ; byte of the output vector's data array
call    core::ptr::drop_in_place ; Call core::ptr::drop_in_place
mov     rdi, rbx    ; Set rdi to rbx
call    _Unwind_Resume@PLT ; Call _Unwind_Resume
ud2                ; Halt execution
```

★ The compiler stores static string slices in the `.rodata` section of the binary. The following code shows the static string slices generated for the `true` and `false` strings.

```
.L__unnamed_2:
.ascii  "true"

.L__unnamed_1:
.ascii  "false"
```



Experiment: Try Out Different Vector Transformations

You can experiment with this code in [Compiler Explorer](https://godbolt.org/z/h5nf5j4qY) (<https://godbolt.org/z/h5nf5j4qY>). Here are a few things to try:

Convert a Vector of u64 to a Vector of bool

The following code converts a vector of `u64` to a vector of `bool` by checking whether the value is 0. Copy this code into Compiler Explorer and look at the generated assembly code.

```
pub fn convert_u64_vec_to_bool_vec(v: Vec<u64>) -> Vec<bool> {
    convert(v, |n| n != 0)
}
```

Here are some observations about the generated assembly code:

- The compiler initializes many constants for efficient SIMD (Single Instruction Multiple Data) operations.
- Memory is allocated for the new boolean vector using the Rust allocator (`__rust_alloc`).
- A loop iterates over the `u64` values in the input vector, checking each `u64` value to determine if it is zero. Based on this check, it then writes the corresponding boolean value (`true` or `false`) to the new vector.
- The generated code employs different looping strategies depending on the input vector's size, using SIMD instructions for large input vector lengths.
- The compiler optimizes the assembly to use SIMD instructions, allowing it to process multiple data points simultaneously. It simultaneously processes multiple `u64` values, employing SIMD instructions like `pcmpeqd`, `pshufd`, and `packuswb` for comparing, shuffling, and packing values.
- After populating the output vector, the generated code deallocates the input vector's heap memory (`__rust_dealloc`).

Convert a Vector of (f32, f32) Tuple to a Vector of f32

The following code converts a vector of (f32, f32) tuple to a vector of f32 by multiplying the two f32 values. Copy this code into Compiler Explorer and look at the generated assembly code.

```
pub fn convert_f32_f32_vec_to_f32_vec(v: Vec<(f32, f32)>) -> Vec<f32> {
    convert(v, |n| n.0*n.1)
}
```

Let us break down the assembly code:

- There is logic that checks the size of the vector to handle memory allocation (`call __rust_alloc`).
- As expected, the code goes over the input vector's elements, takes each (f32, f32) tuple, multiplies the two f32 numbers, and then collects them in the output vector.
- The generated code adopts different looping strategies based on the input vector's size, utilizing SIMD instructions for large input vector lengths.
- The compiler uses SIMD operations optimized for parallel floating-point operations (`movss`, `mulss`, etc.).
- After creating and populating the output vector, the code deallocates the input vector's heap memory (invoking `__rust_dealloc`).

AVX-512 Instruction Set

Change the compiler options to use the AVX-512 instruction set using the flag setting (*change in the edit box on the top right*):

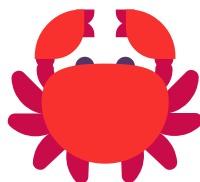
```
-C opt-level=3 -C target_cpu=skylake-avx512
```

The generated assembly code uses a lot of advanced SIMD instructions. Just hover over the assembly instructions in Compiler Explorer to learn more about them.

Can you spot SIMD patterns that we have encountered in earlier chapters?

Key Takeaways

- The functional style code in Rust is a zero-cost abstraction. The compiler can optimize the code to be as efficient as handwritten loops.
- An owned input vector will be deallocated when it goes out of scope. The compiler will generate code to deallocate the vector's buffer.
- The compiler will also generate code to deallocate the vector's buffer when an error occurs.
- The Rust compiler checks for address space overflow when allocating memory for a vector.
- If a function returns a vector, the compiler will generate the code to allocate memory for the vector's data buffer. The caller passes a "hidden" pointer to the function with the address of the returned vector's struct.
- The function will panic if it cannot allocate memory for a vector.
- The compiler uses different looping strategies based on the size of the input vector.
 - When working with `convert_bool_vec_to_static_str_vec`, the compiler defined a main loop for even entries and processed an additional entry when the input vector length was odd.
 - Our experiments with `convert_u64_vec_to_bool_vec` and `convert_f32_f32_vec_to_f32_vec` showed that the compiler uses SIMD instructions to process multiple data points simultaneously when the input vector length is large.



Chapter 22

Transforming into Owned String Vector

Welcome to the penultimate chapter of our book. This chapter extends the discussions from previous chapters, “*Transforming into String Slice Vector*” and “*Vector Iteration in Assembly*”. Our focus now shifts to vectors of owned strings. We will investigate how the compiler represents a vector of owned strings using two levels of heap allocation. Additionally, the chapter will address critical aspects like handling overflow and memory allocation failures in the output vector.

Example Code: Map a Vector of bools to a Vector of Owned Strings

- The `convert<A,B>` function, a generic function that generates no code, takes a vector of type `A` and returns a vector of type `B`. It requires a closure that takes an element of type `A` and returns an element of type `B`. The function converts by calling the `map` function on the vector iterator.
- `convert_bool_vec_to_owned_string_vec` function uses the `convert<A,B>` function to convert a vector of bools to a vector of owned strings. The function accepts a vector of bools and returns a vector of owned strings. It calls the `convert<A,B>` function with a closure that converts a bool to a string.

```
pub fn convert<A,B> (v: Vec<A>, f: impl Fn(A) -> B) -> Vec<B> {
    v.into_iter().map(f).collect()
}

pub fn convert_bool_vec_to_owned_string_vec(v: Vec<bool>) -> Vec<String> {
    convert(v, |n| (if n {"true"} else {"false"}).to_owned())
}
```

Before we proceed, let us review string representation in Rust.

String Representation in Rust

Rust’s two types represent strings: `str` and `String`. Here is a breakdown of their representations:

- `str`:

Part G Transformations and Takeaways

- This is a string slice and is immutable.
- It is a reference to a sequence of UTF-8 bytes.
- A string slice maintains a fixed size and cannot grow or shrink. It typically appears in its borrowed form, `&str`.
- It is a view into another string, represented by a pointer to the first byte and a length.
- **String:**
 - This is a growable, heap-allocated string type.
 - Internally, it is a wrapper around a vector of bytes (`Vec<u8>`) and ensures that the bytes are valid UTF-8.
 - It consists of three components:
 1. A pointer to the data (which is on the heap).
 2. A length (total used data).
 3. A capacity (total space allocated on the heap).

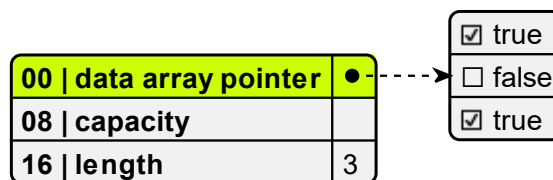
Visualizing the Input and Output Vectors

Now that we better understand strings in Rust let us visualize the input and output vectors of the `convert_bool_vec_to_owned_string_vec` function.

The input vector passed to the `convert_bool_vec_to_dynamic_str_vec` function is a vector of bools. The memory organization of this vector follows:

- An **8-byte data array pointer** points to the start of the data array on the heap. The heap address points to a `bool` array.
- An **8-byte capacity field** contains the length of the data array on the heap. The vector can grow to this size without reallocating.
- An **8-byte length field** contains the number of elements in the vector.

The following diagram shows the memory organization of the input vector of type `Vec<bool>`:



The output vector of the `convert_bool_vec_to_owned_string_vec` function is a vector of strings. The memory organization of this vector involves two levels of heap allocation. The first level is for the vector array. The second level of heap allocation is for each string's byte array.

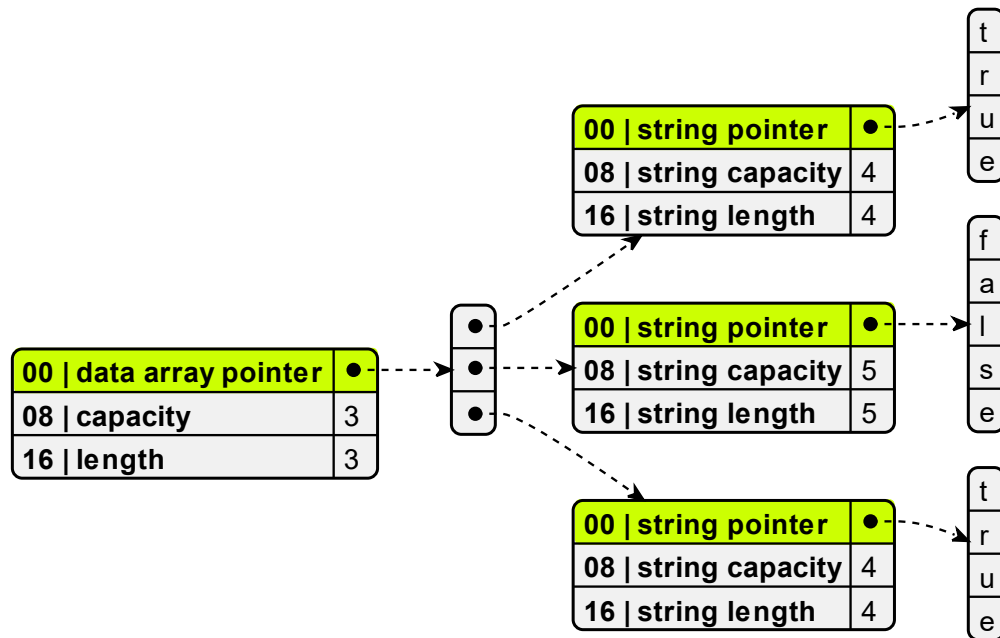
Vec<String> - The memory organization of the output vector of type `Vec<String>` is as follows:

- An **8-byte data array pointer** points to the start of a `String` array on the heap.
- An **8-byte capacity field** contains the length of the `String` array on the heap. The vector can grow to this size without reallocating.
- An **8-byte length field** contains the number of elements in the vector.

String - The memory organization of each `String` in the vector is as follows:

- An **8-byte data array pointer** points to the start of the byte array on the heap.
- An **8-byte capacity field** contains the length of the data array on the heap. The string can grow to this size without reallocating.
- An **8-byte length field** contains the number of bytes in the `String`.

The following diagram shows the memory organization of an output vector of type `Vec<String>`. Note the two tiers of heap allocation.

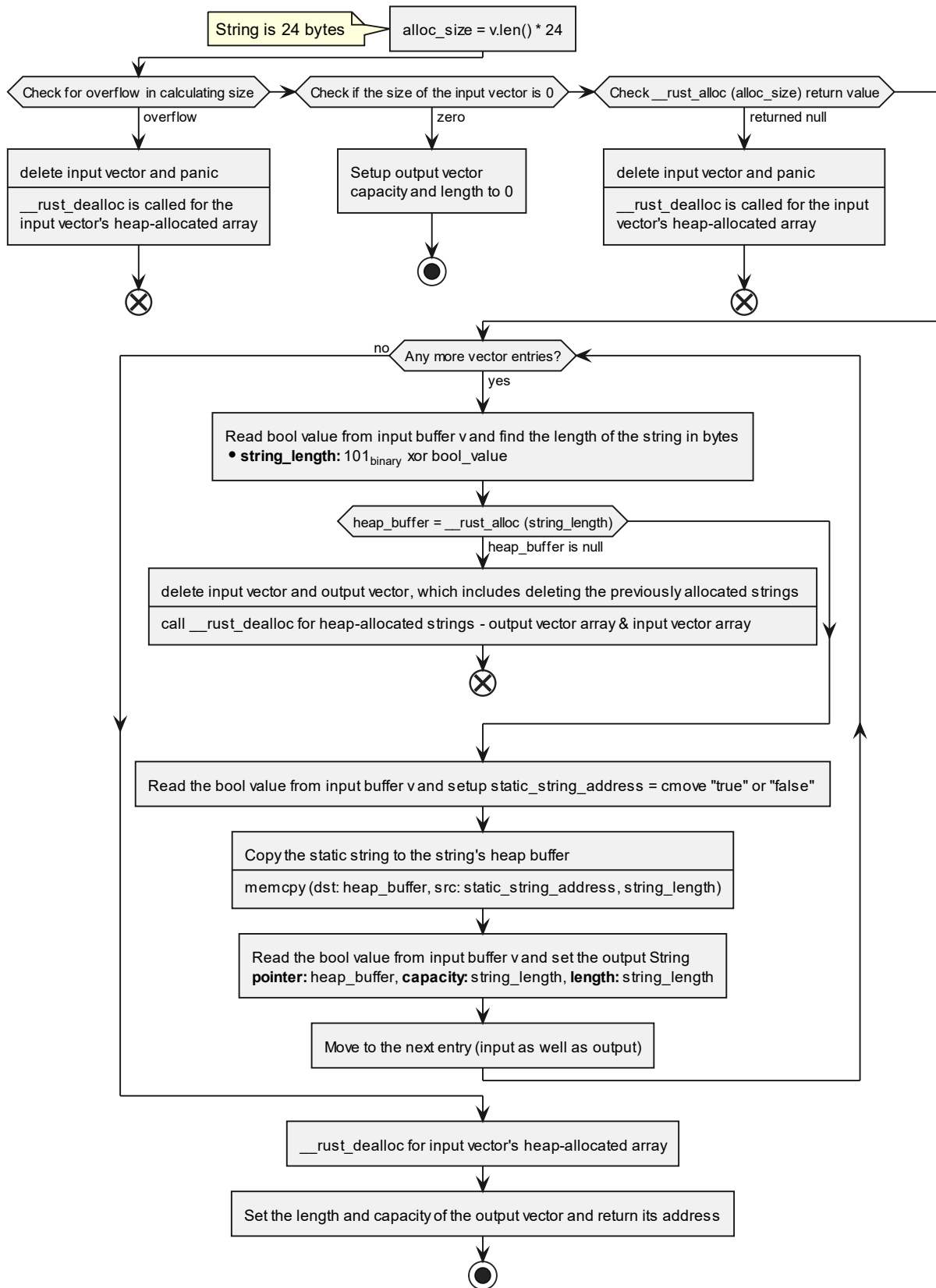


String Vector Generation Overview

This section will give us a high-level view of the generated code for the `convert_bool_vec_to_owned_string_vec` function.

Let us start by looking at the flowchart of the generated code.

Part G Transformations and Takeaways



Let us look at the critical parts of the flowchart:

- [Overflow and Memory Allocation Failure Handling for the Output Vector](#)
- [Preparing the Output Vector with Owned Strings](#)
- [Removing the If Condition for Checking the Boolean Value in the Input Vector](#)

- [Cleaning up the Input Vector on Exit from the Function](#)

Overflow and Memory Allocation Failure Handling for the Output Vector

The code calculates the output vector's size by multiplying the string's size (24 bytes) by the number of entries. If there is an overflow in the size computation, the code will generate a panic and drop the input vector.

The compiler generates code to handle the following scenarios:

- If the size calculation does not overflow, allocate an output vector of the required size.
- Should the allocation fail, trigger a panic and dispose of the input vector.

Preparing the Output Vector with Owned Strings

Loop through all entries in the input vector. For each entry:

- Allocate a new owned string.
 - If the allocation fails, generate a panic and drop the input vector and previously allocated strings.
 - If the allocation succeeds, copy the string from static "true" or "false" to the heap-allocated string memory. Set the capacity and length of the string to the length of the static string ("true" or "false").

Removing the If Condition for Checking the Boolean Value in the Input Vector

The generated code removes the check for true or false from the loop body using the following techniques:

- Use the conditional move (cmov) to copy a pointer to static "true" or "false" into a register.
- Compute the length using an XOR between 5 (101 binary) and the bool value (0 or 1).

Cleaning up the Input Vector on Exit from the Function

The compiler generates a call to `__rust_dealloc` to free the input vector. Note that the function owns the input vector, so it is responsible for freeing the heap allocation for the input vector.

Annotated Assembly: Bool Vector to a Vector of Owned Strings

We have completed the high-level overview of the generated code. Now, look at the annotated assembly code for the `convert_bool_vec_to_owned_string_vec` function.

1. The function takes a vector of bools (address stored in the `rsi` register) and converts it into a vector of owned strings. The address of the output vector is passed in the `rdi` register as a "hidden" parameter. The function returns the address of the output vector in the `rax` register.
2. The function allocates a data array for the owned strings with the same length as the original vector. It then initializes the output vector struct with the newly allocated owned string data array.
3. The function then iterates over the elements of the original vector and converts each bool into an owned string. It then returns the resulting vector of owned strings.

Let us look at the input and output registers for the function before we proceed:

 Input:

Part G Transformations and Takeaways

- **rdi**: The caller sends the address of the output vector struct as a “hidden” variable via **rdi**. This struct does not include the data array; instead, the called function will allocate the data array on the heap.
- **rsi**: Address of the input vector

Output:

- **rax**: Address of the output vector

example::convert_bool_vec_to_owned_string_vec:

```
push    rbp
push    r15
push    r14
push    r13
push    r12
push    rbx
sub     rsp, 104 ; Allocate 104 bytes on the stack
```

★ Load the input vector’s address and length into registers.

```
mov     rbp, rdi ; Copy the output vector's address to rbp
mov     rax, qword ptr [rsi] ; Get input vector's address from stack
mov     rcx, qword ptr [rsi + 8] ; Get input vector's capacity from stack
mov     r15, qword ptr [rsi + 16] ; Get the input vector's length

lea     r14, [rax + r15] ; Compute the address past the
                        ; last element in the input vector
```

★ Store the input vector’s address, capacity, and length on the stack.

```
mov     qword ptr [rsp + 40], rax ; Save input vector's address
mov     qword ptr [rsp + 48], rcx ; Save input vector's capacity
mov     qword ptr [rsp + 8], rax ; Save input vector's address
mov     qword ptr [rsp + 56], rax ; Save input vector's address

mov     qword ptr [rsp + 64], r14 ; Address past the last element in
                                ; the input vector in the stack

test    r15, r15 ; Check if the input vector's length is zero
je      .LBB3_1 ; Jump to code that sets the output
                ; vector's length to 0

movabs  rax, 384307168202282326 ; rax = 0x0555_5555_5555_5556
xor     r13d, r13d ; Set r13 to zero
```

★ Compute the size of the output vector’s data array.

The following code computes the size of the output vector’s data array by multiplying the string size (24 bytes) by the number of entries. If there is an overflow in the size computation, the compiler will generate a panic and drop the input vector.

```
cmp     r15, rax ; Compare input vector's length with
                ; 0x0555_5555_5555_5556
```

```

setb    al        ; Set the least significant bit of al to 1 if the input
                  ; vector's length is greater than 0x0555_5555_5555_5556

jae     .LBB3_3    ; If the input vector's length is greater than
                  ; 0x0555_5555_5555_5556, jump to the code that will
                  ; throw an exception for address overflow.

```

O Check if the output vector's length is zero.

```

lea     rdx, [8*r15] ; rdx = input vector's length times 8

mov     r13b, al      ; Copy al as it happens to contain 1
                  ; (due to the setb instruction).

shl     r13, 3        ; Multiply r13 by 8 so that it contains 8 for the
                  ; alignment of the output vector's data array

lea     r12, [rdx + 2*rdx] ; Multiply rdx by 3 and store it in
                  ; r12 (this is the size of the output
                  ; vector's data array)

test    r12, r12      ; Check the size of the output vector's data array
je      .LBB3_6        ; If the size of the output vector's data array is
                  ; zero, jump to the code that will set the output
                  ; vector's length to zero

```

NEW Allocate memory for the output vector's data array.

The `__rust_alloc` function allocates memory for the output vector's data array. It takes two arguments: the allocation's size and alignment. The function returns the address of the allocated memory in `rax`.

```

mov     rdi, r12      ; Copy size of output vector's data array to rdi
mov     rsi, r13      ; Copy alignment of output vector's data array to rsi
mov     rbx, rcx      ; Preserve rcx in rbx (used later)

```

★ Call the `__rust_alloc` function to allocate memory for the output vector's data array. The `__rust_alloc` function returns the address of the allocated memory in `rax`.

```

call    qword ptr [rip + __rust_alloc@GOTPCREL]

mov     rcx, rbx      ; Restore the rcx value from rbx
mov     rbx, rax      ; Copy address of output vector's data array to rbx

test    rbx, rbx      ; Check if the address allocation has failed
                  ; (returns NULL)

je      .LBB3_9        ; If the address allocation has failed, jump to the
                  ; code that will throw an exception for alloc failure

```

★ Prepare to map the input vector's data array to the output vector's data array.

```

.LBB3_10:
mov     qword ptr [rbp], rbx ; Store the address of the output
                  ; vector's data array in the output vector

```

Part G Transformations and Takeaways

```

mov     qword ptr [rbp + 8], r15 ; Store the input vector's length in
                                   ; the output vector's capacity

lea     rax, [rbp + 16]          ; rax = address of output vector's length

mov     qword ptr [rsp + 16], rax ; Store the address of the output
                                   ; vector's length on the stack

mov     qword ptr [rsp + 24], rbp ; Store output vector's address on
                                   ; the stack

mov     qword ptr [rbp + 16], 0   ; Set output vector's length to zero

mov     rax, qword ptr [rsp + 8]  ; Get the input vector's address
                                   ; from the stack

```

- Store the input vector's address and capacity on the stack.
- Store address past the last element in the input vector on the stack.
- Set the loop index in rbp to zero.

```

mov     qword ptr [rsp + 72], rax ; input vector's address to stack
mov     qword ptr [rsp + 32], rcx ; input vector's capacity to stack
mov     qword ptr [rsp + 80], rcx ; input vector's capacity to stack

mov     qword ptr [rsp + 96], r14 ; Save address past the last element in
                                   ; the input vector to stack

xor     ebp, ebp                  ; Set ebp to zero

```

🌀 ● The mapping loop begins here

★ The following code is the loop that will copy the input vector's elements to the output vector's data array.

```

.LBB3_11:
mov     rax, qword ptr [rsp + 8]    ; Get input vector's address
                                   ; from the stack

movzx   r13d, byte ptr [rax + rbp]  ; Get next input vector's element
mov     r12d, r13d                  ; Copy the next input vector's element
and     r12d, 1                     ; Map the bool value to 0 or 1

```

★ Compute the length of the string representation of the current element. Set the arguments for the `__rust_alloc` function call.

```

xor     r12, 5                      ; Set ebp to ebp xor 5 to get the correct length of string
                                   ; representation of the current element
                                   ; 101 xor 001 = 100 (4): Length of "true"
                                   ; 101 xor 000 = 101 (5): Length of "false"

mov     esi, 1                      ; Set the byte alignment of the output vector's
                                   ; data array to 1.

mov     rdi, r12                    ; Copy length of the string representation of the
                                   ; current element to rdi

```

NEW Allocate memory for the string buffer for the current element. If the allocation fails, jump to the code that will throw an exception for allocation failure.

```
call    qword ptr [rip + __rust_alloc@GOTPCREL]

test    rax, rax ; Check the status of the address allocation
                ; (address or NULL)

je      .LBB3_12 ; On memory allocation failure, jump to the code
                ; that will throw an allocation failure exception
```

★ Prepare arguments for the memcpy function call. cmove copies the address of the static string "true" or "false" to the rsi register.

```
mov     r14, rax ; Copy the address of the string buffer to r14

test    r13b, 1  ; Check if the current element is true

lea     rsi, [rip + .L__unnamed_1] ; Get the address of the static
                ; string "true"

lea     rax, [rip + .L__unnamed_2] ; Get the address of the static
                ; string "false"

cmov     rsi, rax ; Set rsi to the address of the static string "true" if the
                ; current element is true. Set rsi to the address of the
                ; static string "false" if the current element is false

mov     rdi, r14 ; Copy the address of the string buffer to rdi

mov     rdx, r12 ; Copy the length of the string representation
                ; of the current element to rdx
```

★ Copy the string representation of the current element to the string buffer and restore saved registers after the call to memcpy.

```
call    qword ptr [rip + memcpy@GOTPCREL]

mov     qword ptr [rbx], r14      ; Store the address of the string buffer in
                                ; the output vector's data array
mov     qword ptr [rbx + 8], r12  ; Store the capacity of the string
                                ; buffer in the output vector's data array
mov     qword ptr [rbx + 16], r12 ; Store the length of the string buffer in
                                ; in the output vector's data array
```

↺ Move to the next element in the input and output vectors, and check if you have reached the end of the input vector.

```
inc     rbp                ; Increment the input vector's index
add     rbx, 24            ; Increment the output vector's byte index
                        ; (each entry is 24 bytes long)
cmp     r15, rbp          ; Check if the input vector's index is less than the input
                        ; vector's length
jne     .LBB3_11          ; If less, jump for the next iteration of the loop
```

Part G Transformations and Takeaways

The mapping loop ends here

★ Restore the saved registers and check if the input vector's length is zero. If the input vector's length is not zero, jump to the code that will deallocate the input vector's data array.

```

mov    rbp, qword ptr [rsp + 24] ; Get the output vector's address from stack
mov    rcx, qword ptr [rsp + 32] ; Get the input vector's capacity from stack

mov    rax, qword ptr [rsp + 16] ; Get the address of the output vector's
                                ; length from the stack

mov    qword ptr [rax], r15      ; Store input vector's length in output
                                ; vector's length

test   rcx, rcx                ; Check if the input vector's capacity is zero

je     .LBB3_17                ; If it is zero, jump to the code that will
                                ; deallocate the input vector's data array

```

 Deallocate the input vector's data array.

```

.LBB3_16:
mov    rdx, rcx ; Copy the input vector's capacity to rdx

not    rdx      ; Flip bits of input vector's capacity (the most
                ; significant bit of the input vector's capacity is set
                ; to 0). Now, the most significant bit of the
                ; input vector's capacity is set to 1

shr    rdx, 63  ; Shift to bring the most significant bit of the input
                ; vector's capacity to the least significant bit
                ; (i.e., set it to 1) This sets the alignment to 1

mov    rdi, qword ptr [rsp + 8] ; Get the input vector's address from stack

mov    rsi, rcx ; Copy the input vector's capacity to rsi

; Deallocate the input vector's data array
call   qword ptr [rip + __rust_dealloc@GOTPCREL]

```

 Return the address of the output vector.

```

.LBB3_17:
mov    rax, rbp ; Copy the output vector's address to rax

add    rsp, 104 ; Deallocate the stack space allocated for the function

pop    rbx
pop    r12
pop    r13
pop    r14
pop    r15
pop    rbp
ret    ; Return from the function

```


✍ If the input vector's length is zero, set the output vector's length to zero and return the address of the output vector. If the input vector's length is not zero, jump to the code that will deallocate the input vector's data array.

```
.LBB3_1:
mov     qword ptr [rbp], 8      ; Set the output vector's address to 8
lea     rax, [rbp + 16]        ; rax = address of output vector's length

xorps   xmm0, xmm0            ; Set xmm0 to 0

movups  xmmword ptr [rbp + 8], xmm0 ; xmm0 is used to set the output
                                ; vector's capacity and length to 0

mov     qword ptr [rax], r15    ; Store the input vector's length in the
                                ; output vector's length

test    rcx, rcx              ; Check if the input vector's capacity is zero

jne     .LBB3_16 ; If the input vector's capacity is not zero, jump to the
                ; code that will deallocate the input vector's data array

jmp     .LBB3_17 ; If the input vector's capacity is zero,
                ; jump to the exit function

.LBB3_6:

mov     rbx, r13 ; Copy 8 to be used as the address of the
                ; output vector's data array

test    rbx, rbx ; Check if the address of the output vector's
                ; data array is zero

jne     .LBB3_10 ; Jump back to exit the function
```

✗ Handle the case where the output vector's data array allocation has failed.

```
.LBB3_9:
mov     rdi, r12 ; Copy the output vector's capacity to rdi
mov     rsi, r13 ; Copy the output vector's address to rsi

; Call function to handle the memory allocation failure
call    qword ptr [rip + alloc::alloc::handle_alloc_error@GOTPCREL]

jmp     .LBB3_4 ; Jump back to exit the function

.LBB3_12:
```

✗ Handle the case where the string buffer allocation has failed.

```
mov     rax, qword ptr [rsp + 8] ; Get the input vector's address from stack

add     rax, rbp ; Add the input vector's index to the input vector's address
inc     rax      ; Increment the input vector's index

mov     qword ptr [rsp + 88], rax ; Store the address of the current element
```

Part G Transformations and Takeaways

```

; in the input vector's data array
; in the stack

mov     esi, 1           ; Set the byte alignment to 1
mov     rdi, r12          ; Copy the output vector's capacity to rdi

```

✗ Call function to handle the memory allocation failure.

```

call    qword ptr [rip + alloc::alloc::handle_alloc_error@GOTPCREL]

jmp     .LBB3_4

```

✗ Call function to handle the case when the input vector's length has overflowed.

```

.LBB3_3:
call    qword ptr [rip + alloc::raw_vec::capacity_overflow@GOTPCREL]

```

✗ Handle memory allocation failure (the code frees memory allocated to the input and output vectors).

```

.LBB3_4:
ud2                      ; Throw an invalid instruction exception
mov     rbx, rax          ; Copy the output vector's address to rbx
lea     rdi, [rsp + 40]    ; Get the address of the input vector's length

```

🔪 Call the `drop_in_place` function to drop the input vector.

```

call    drop_in_place<Map<IntoIter<bool>>>

jmp     .LBB3_19          ; Jump to the code that will deallocate the output
                          ; vector's data array

mov     rbx, rax          ; Copy the output vector's address to rbx

mov     rdi, qword ptr [rsp + 16] ; Get the address of the output
                          ; vector's length from the stack
mov     rsi, rbp          ; Copy the output vector's capacity to rsi

call    core::ptr::drop_in_place<for_each> ; Cleanup for_each's state

lea     rdi, [rsp + 72]    ; Get the address of the output vector's
                          ; length from the stack

```

🔪 Call the `drop_in_place` function to drop the `into_iter` adapter.

```

call    core::ptr::drop_in_place<Map<IntoIter<bool>>>
mov     rdi, qword ptr [rsp + 24] ; Get address of output vector's
                          ; capacity from the stack

```

🔪 Call the `drop_in_place` function to drop the output vector's data array.

```

call    core::ptr::drop_in_place<Vec<String>>

.LBB3_19:
mov     rdi, rbx          ; Copy the output vector's address to rdi

```

```

call    _Unwind_Resume@PLT ; Call _Unwind_Resume function to resume unwinding
ud2                                ; Throw an invalid instruction exception
call    qword ptr [rip + core::panicking::panic_no_unwind@GOTPCREL]
ud2                                ; Throw an invalid instruction exception

```

✓ ✗ Saved strings for true and false. The code uses these strings to copy the string representation of the current element to the string buffer.

```

.L__unnamed_1:
    .ascii  "true"

.L__unnamed_2:
    .ascii  "false"

```

Memory Cleanup Code

The compiler must handle numerous corner cases and ensure memory cleanup in all scenarios. We will look at the cleanup code in this section.

Deallocate a String Vector

This function iterates through the vector and frees the string buffer. After the iteration, it deallocates the vector that held the now-deleted strings.

The `drop_in_place` function deallocates the memory for a `Vec`. It does this by deallocating the memory for each element of the `Vec` and then deallocating the memory allocated to the `Vec` itself.

An interesting thing to note here is that the compiler uses tail call optimization to avoid the overhead of the last `__rust_dealloc` call. The code jumps to the `__rust_dealloc` instead of making a call. The return from the `__rust_dealloc` function returns to the caller of the `drop_in_place` function.

12 34 Input: `rdi` contains the address of the output vector.

```

core::ptr::drop_in_place<Vec<String>>:
push    r15
push    r14
push    r13
push    r12
push    rbx

mov     r14, rdi                ; rdi contains the address of the
                                ; vector that needs to be destroyed.

mov     rax, qword ptr [rdi + 16] ; rax contains the size of vector.
test    rax, rax                ; Check if vector size is 0.
je      .LBB2_5                 ; Skip dealloc if size is zero.

mov     r12, qword ptr [r14]     ; r12 = pointer to first element
                                ; of vector

shl     rax, 3                  ; rax = size of the vector array in bytes

lea     r15, [rax + 2*rax]       ; r15 = three times the size of the
                                ; vector array in bytes.

```

Part G Transformations and Takeaways

```
xor    ebx, ebx                ; Set ebx to 0 to use it for calculating
                                ; the byte index of the vector array.
```

🔪 r13 contains the pointer to the deallocation function. The code saves the address in the register to avoid fetching the address in a loop.

```
mov    r13, qword ptr [rip + __rust_dealloc@GOTPCREL]

jmp    .LBB2_2                ; Jump to the deallocation loop.

.LBB2_4:

add    rbx, 24                ; Add the size of the vector array element.
cmp    r15, rbx                ; Check if we have reached the end of the vector array.
je     .LBB2_5                ; Reached the end of the vector array.
                                ; Break out from the loop.

.LBB2_2:

mov    rsi, qword ptr [r12 + rbx + 8] ; Get the length of the string
                                ; buffer that needs to be
                                ; deallocated

test   rsi, rsi                ; Check if the length is zero.
je     .LBB2_4                ; Skip deallocation if the length is zero.

mov    rdi, qword ptr [r12 + rbx] ; Get the pointer to the string buffer
                                ; that needs to be deallocated.

mov    edx, 1                  ; Set the byte alignment to 1 byte.
call   r13                     ; Call the deallocation function.
jmp    .LBB2_4                ; Continue the loop.

.LBB2_5:

mov    rax, qword ptr [r14 + 8] ; Get the capacity of the vector array.
test   rax, rax                ; Check if the capacity is zero.
je     .LBB2_7                ; Skip deallocation if the capacity is zero.

mov    ecx, 24                 ; Set the size of the individual vector array
                                ; elements to 24 bytes.

mul    rcx                     ; Multiply the size of the individual vector array
                                ; elements by the capacity.

test   rax, rax                ; Check if the result is zero.
je     .LBB2_7                ; Skip deallocation if the result is zero.
mov    rdi, qword ptr [r14]     ; Get the vector array pointer.
mov    edx, 8                  ; Set the byte alignment to 8 bytes.
mov    rsi, rax                ; Set the size of the vector array to rax.
pop    rbx
pop    r12
pop    r13
pop    r14
```

```

pop     r15

; Tail call optimized deallocation function.
jmp     qword ptr [rip + __rust_dealloc@GOTPCREL]

.LBB2_7:

pop     rbx
pop     r12
pop     r13
pop     r14
pop     r15
ret

```

Free the into_iter Owned Bool Vector

The code created the Map iterator from a vector of bools and a closure that maps bools to owned strings. The `drop_in_place` function deallocates the memory for the Map iterator.

```

core::ptr::drop_in_place<Map<IntoIter<bool>>>:
mov     rsi, qword ptr [rdi + 8] ; Get the capacity of the vector.
test    rsi, rsi                ; Check if the input vector is empty.
je      .LBB0_1                 ; Skip deallocation if empty.
mov     rdi, qword ptr [rdi]    ; Get a pointer to the input vector's
                                ; data array.

```

✏ The following three lines set the alignment to 1 byte before the call to the deallocation function.

```

mov     rdx, rsi ; Copy the input vector length to rdx.

not     rdx      ; Invert the bits of rdx. The objective is to
                ; set the MSB to 1.

shr     rdx, 63  ; Shift the bits of rdx to the right by 63 bits.
                ; This will set the register to 1.

```

✏ The code passes the following parameters to the deallocation function:

- `rdi` = pointer to the input vector's data array.
- `rsi` = length of the input vector. `rdx` = alignment.

```

; Tail call optimized deallocation function.
jmp     qword ptr [rip + __rust_dealloc@GOTPCREL]

.LBB0_1:
ret

```

Cleanup Iteration-Related Exceptions

✏ The code calls this function to clean up iteration-related exceptions. It limits the output length to the length of completed iterations.

```

core::ptr::drop_in_place<for_each>:
mov     qword ptr [rdi], rsi ; Set the length of the output vector to the

```

```
ret                                     ; number of completed iterations.
```



Experiment: Returning a Vec<char> and String Manipulation

You can view the code used in this chapter on [Compiler Explorer](https://godbolt.org/z/4Gn7covKM) (<https://godbolt.org/z/4Gn7covKM>). Hover over the code in Compiler Explorer to see the corresponding assembly code in the window on the right.

Let us also experiment with a few other functions to better understand the handling of strings in Rust.

- [Emoji Vector](#)
- [In-Place String Operations](#)
- [Join a Vector of &str to a string](#)

You can save some typing by using the following link that includes the preceding experiments:

- [Compiler Explorer](https://godbolt.org/z/G8j74fzrh) (<https://godbolt.org/z/G8j74fzrh>).

Emoji Vector

Add the following function to Compiler Explorer window. This function converts a vector of bools to a vector of emojis.

```
pub fn convert_bool_vec_to_emoji_vec(v: Vec<bool>) -> Vec<char> {
    convert(v, |n| if n { '😄' } else { '😞' })
}
```

A few items stand out in the generated code:

- In Rust, the char type represents a single Unicode scalar value, enabling it to represent any character in the Unicode standard, encompassing a range far beyond ASCII characters. The size of the char type in Rust is 4 bytes.
- The compiler uses SIMD instructions to copy the emojis to the output vector. It stores four copies of the emojis to facilitate SIMD operations. Look for .LCPI4_1 and .LCPI4_2 labels in the generated code.

```
.LCPI4_1:
    .long    128512 ; 😄 Grinning face emoji unicode code point
    .long    128512
    .long    128512
    .long    128512
.LCPI4_2:
    .long    128542 ; 😞 Disappointed face emoji unicode code point
    .long    128542
    .long    128542
    .long    128542
```

- Look at the loop immediately following the loading of the emojis. Follow along the logic of the loop that starts after the following lines of assembly. How many entries does the loop copy to the output vector in each iteration?

```
movdqa    xmm2, xmmword ptr [rip + .LCPI4_1]
movdqa    xmm3, xmmword ptr [rip + .LCPI4_2]
```

In-Place String Operations

Add the following code to Compiler Explorer window. This code in-place converts a vector of strings to lowercase.

```
pub fn in_place_string_replace(vec: &mut Vec<String>) {
    for text in vec.iter_mut() {
        *text = text.to_lowercase();
    }
}
```

To understand the generated assembly, let us look at the function signature of `to_lowercase`:

```
fn to_lowercase(&self) -> String
```

The function accepts a reference to `self` (a string slice `&str`) and returns a new `String` as the lowercase equivalent of the original string. It calls the `to_lowercase` function on each vector element within the `in_place_string_replace` function. This process returns a new `String` that the function then copies into the mutable vector passed to it. Consequently, each loop iteration allocates a new `String` and places it in the vector while dropping the original `String` stored in it.

Here are a few interesting things to note in the generated code:

- To optimize performance, the compiler copies the address of the `to_lowercase` function to a register, thereby avoiding fetching the function's address in each loop iteration.

```
mov     r13, qword ptr [rip + alloc::str::<impl str>::to_lowercase@GOTPCREL]
```

- Every loop iteration calls `__rust_dealloc` to deallocate the memory allocated to the original `String`.

```
call    qword ptr [rip + __rust_dealloc@GOTPCREL]
```

- The memory allocation for the new string is not seen in the code as the `to_lowercase` function handles it.

Join a Vector of `&str` to a String

Add the following code to Compiler Explorer window. This code joins a vector of `&str` to a `String`.

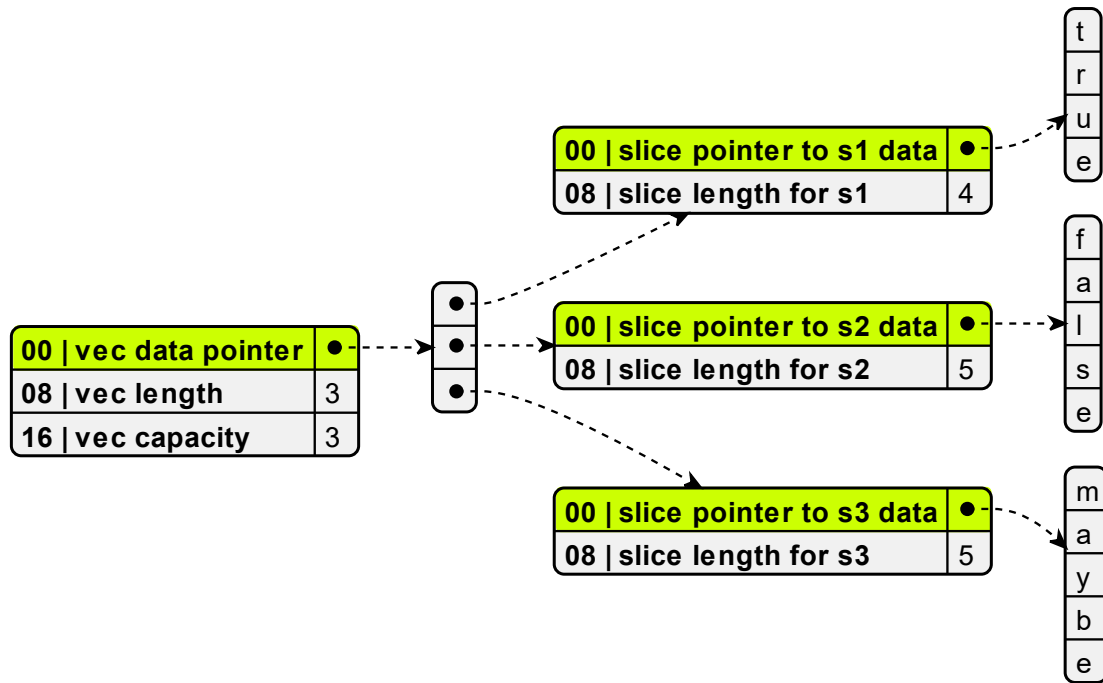
```
pub fn combine_strings(strings: Vec<&str>) -> String {
    strings.join(" ")
}
```

Before you look at the generated assembly code, let us look at the memory layout of the input vector of `&str`.

Consider the following code:

```
let s1 = String::from("true");
let s2 = String::from("false");
let s3 = String::from("maybe");
let vec: Vec<&str> = vec! [&s1, &s2, &s3];
```

The following figure shows the memory layout of `vec`:



Look for the following in the generated assembly code:

- Memory allocation for the output string.

```
call    qword ptr [rip + __rust_alloc@GOTPCREL]
```

- The output string grows as the code copies the input vector's elements to the output string.

```
call    alloc::raw_vec::RawVec<T,A>::reserve::do_reserve_and_handle
```

- Copying the input string vector's individual &str elements to the output string.

```
call    qword ptr [rip + memcpy@GOTPCREL]
```

- The strings vector is dropped at the end of the function.

```
call    qword ptr [rip + __rust_dealloc@GOTPCREL]
```

- Strings for error handling give us a clue about the possible errors that can occur. Search for the labels for the string to find the corresponding error handling code.

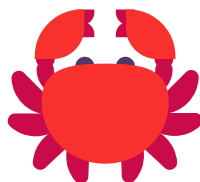
```
.L__unnamed_5:
    .ascii  "assertion failed: mid <= self.len()"

.L__unnamed_3:
    .ascii  "attempt to join into collection with len > usize::MAX"
```

Key Takeaways

- Creating a vector of owned strings from a vector of bools results in two tiers of memory allocations. The first tier is the allocation of the vector of strings, and the second tier is the allocation of individual strings.
- The program will panic if any of these memory allocations fail.

3. The program will also panic if the input vector's length has overflowed.
4. When the program panics, it will free the memory allocated to the input and output vectors.
5. The compiler generates considerable code for error handling when allocations are involved. This code has performance and memory implications. For instance, the additional overflow checks may impact performance in the inner loop of a hot function.



Chapter 23

Key Insights

This book has been a journey through the intricacies of Rust. We have covered diverse topics, from memory management to compiler optimizations. This chapter summarizes the key insights gained from this exploration.

1. **Smart pointer usage and memory management:** Rust's smart pointers, such as `Box` for heap allocation, `Rc` for reference counting, and `Arc` for atomic reference counting, are pivotal in managing ownership and borrowing in Rust. Each has its use case: `Box` for single ownership, `Rc` and `Arc` for shared ownership, with `Rc` being non-thread-safe and `Arc` thread-safe. These pointers manage memory automatically, but their overuse can lead to performance bottlenecks due to frequent heap allocations and reference counting overhead. Understanding the trade-offs and choosing the right smart pointer for the task is crucial for efficient memory management in Rust.
2. **Implications of implicit heap operations:** Rust's memory safety guarantees often involve implicit heap operations, especially in scenarios involving data structures like `Vec` or `Box`. These operations, while safe, can lead to runtime errors if the heap allocation fails. This aspect of Rust necessitates careful error-handling strategies, especially in memory-constrained environments or when dealing with large data allocations. Developers must be mindful of potential panics due to memory allocation failures and design their error handling accordingly.
3. **Efficient data structure handling by the compiler:** The Rust compiler optimizes the memory layout and access patterns of data structures like enums, structs, and tuples. This includes optimizations like field reordering to minimize padding, compact enum representation, and efficient pattern-matching code generation. These optimizations are crucial for high-performance applications, ensuring minimal memory footprint and optimal access speed.
4. **Equivalence of functional code and handwritten loops:** In Rust, the compiler's advanced optimization techniques ensure that functional programming constructs, like iterators and higher-order functions, often match the efficiency of traditional for-loops. This is a testament to Rust's ability to optimize iterator chains, effectively transforming them into loop-like performance under the hood. This insight empowers developers to use functional paradigms without the usual performance trade-offs in other languages.
5. **Trade-offs between static and dynamic dispatch:** The choice between static and dynamic dispatch in Rust has significant performance implications. Static dispatch allows the compiler to inline function calls, leading to faster execution but potentially larger binary sizes. On the other hand, dynamic dispatch provides flexibility and polymorphism but at the cost of indirect function

Part G Transformations and Takeaways

calls and potential runtime overhead. Understanding these trade-offs is crucial for making informed interface design and performance optimization decisions in Rust applications.

6. **Optimized closure handling in Rust:** Rust efficiently manages closures by storing their environment in a struct. When returning closures as `impl Fn`, the environment is often encapsulated within the caller's stack frame, with a thin pointer returned to the caller. This design allows for potential inlining by the compiler. In contrast, closures returned as `Box<dyn Fn>` utilize heap storage, resulting in a fat pointer. This pointer includes the environment and vtable pointers. The vtable points to the concrete type's destructor, size, alignment, and call methods. For optimal efficiency, it is advisable to prioritize `impl Fn` over `Box<dyn Fn>`. Doing so leverages stack storage, promotes inlining, and reduces reliance on heap allocation and `Box` dropping.
7. **State machine transformation of async functions:** Rust internally converts `async` functions into state machines. Each `await` in an `async` function leads to a `poll` call on a future, and these `await` points act as state transitions within the state machine. This approach allows for efficient management of asynchronous operations, with the state machine's structure enabling the function to pause and resume at these points, depending on the readiness of futures.
8. **Async closure environment and memory considerations:** In Rust's `async` paradigm, the closure environment plays a crucial role. It stores local variables and the state of the `async` function. However, this can lead to a large closure environment, especially with nested `async` functions or functions with numerous local variables. Understanding and managing the size of the closure environment is essential for efficient memory usage in asynchronous programming.
9. **Execution and polling mechanism of async tasks:** Rust defers the execution of `async` functions until the returned `Future` is awaited. When the future is awaited, it suspends the execution of the `async` task if the future is not ready, handing control back to the executor. The executor calls the `poll` method to check the readiness of the future, resuming the `async` task when it is ready. This mechanism ensures efficient concurrency handling, allowing multiple tasks to progress without blocking each other.
10. **Optimization with fixed-size arrays:** Rust's compiler is particularly efficient when dealing with fixed-size arrays. In such scenarios, the compiler can make assumptions about the data size and layout, leading to more optimized loop unrolling and vectorization. This contrasts with dynamic arrays or slices, where the compiler must account for variable lengths and potential runtime changes. Understanding this can guide developers in choosing the appropriate data structures for performance-critical code sections, favoring fixed-size arrays where possible for loop-intensive tasks.
11. **Looping strategies and SIMD instructions:** The Rust compiler adapts its looping strategy based on the size of the input vector. It uses a simple loop for small vectors, while for larger vectors, it employs SIMD (Single Instruction, Multiple Data) instructions to process multiple data points simultaneously. This adaptive strategy ensures optimal performance across different data sizes and types.
12. **Compiler efforts to avoid branches:** The Rust compiler employs various strategies to minimize branching. It optimizes code to reduce conditional branching, which can be costly in terms of CPU pipeline efficiency. This optimization includes pattern matching, loop unrolling, conditional move instructions, and even leveraging SIMD (Single Instruction, Multiple Data) instructions where appropriate. By minimizing branches, the compiler ensures more predictable and efficient execution paths, which is especially important in performance-critical applications.
13. **UTF-8 vector representation of strings:** Rust manages `String` and `&str` as vectors of UTF-8 bytes, ensuring efficient memory usage. `String` includes a pointer to a heap-allocated buffer, capacity, and length, while `&str` is a struct with a pointer to a string slice and its length. This

representation is crucial for efficient string handling in Rust, especially in scenarios involving large strings.

14. **Optimizing match expressions in Rust:** Leveraging compiler strategies like jump tables for large match sets, reordering based on frequency for range-based matches, and utilizing tuple match arms enhances performance and efficiency in code generation.
15. **Efficiency of tail call optimization:** Tail Call Optimization (TCO) in Rust reduces stack overhead by optimizing away the last function call, stack preparation, and return. This optimization enhances performance and is particularly effective when functions prepare their return values in the last function call. This optimization plays a vital role in recursive functions, as it can transform a recursive call into a loop.
16. **Inlining of functions:** The Rust compiler opportunistically inlines functions, especially when the function body is small. This inlining is crucial for performance optimization, as it reduces the overhead of function calls and enables further optimizations like loop unrolling.

As we wrap up this book, we hope it has been instrumental in deepening your understanding of Rust. Thank you for embarking on this journey to discover Rust's full potential. May the knowledge gained here serve as a solid foundation, enabling you to avoid pitfalls and craft efficient, elegant solutions in your future endeavors.

Let us continue the conversation at the [Rust Under the Hood Companion](https://eventhelix.com/ruth/) (<https://eventhelix.com/ruth/>) website. We are looking forward to your feedback and suggestions.



[Rust Under the Hood Companion](https://eventhelix.com/ruth/) (<https://eventhelix.com/ruth/>)

- Compiler Explorer and Rust Playground links
- Report and see Errata
- Feedback and Reviews